

残酷共学 · 增量笔记

自 2026-08-08 23:00 BJT 后新增
137 篇

本次增量 · 带索引

点击目录条目可跳转

数据来源：intensivecolearn.ing 活动公开打卡

生成时间：2026-08-09 23:01 (BJT)

目录

主题章节 → 笔记索引 (点击可跳转)

01 AMM机制 (26 篇)	8
笔记 001: Day 4 笔记 : The Graph —— 学会自己从链上查数据	8
笔记 002: 2026-08-08 打卡 从「能成交」走到「有利润」\	11
笔记 003: D3 学习进展 : CPMM、手续费、价格冲击与滑点	11
笔记 004: 2026-08-08 学习记录	12
笔记 005: 8.8 / 链上套利共学	13
笔记 006: Day 2 AMM、Swap Pricing、Slippage 与 Price Impact	17
笔记 007: 理解 DEX 价格与流动性	24
笔记 008: 今天上午终于把 Robinhood 这条套利为什么会突然开窗往前追了一层。昨晚群里一直在争一个问题，	25
笔记 009: UniSwap v3	25
笔记 010: Day 05 : V4 架构与套利系统学习记录	26
笔记 011: 2026-08-09 打卡 第一次把 LI.FI Quote 接进保守门槛\	32
笔记 012: Day 4 打卡 链上套利残酷共學	32
笔记 013: 今日工作总结	33
笔记 014: 今天研究了价格差的来源。	33
笔记 015: 1/6	34
笔记 016: Day 5 - 订单簿、报价与区块状态	35
笔记 017: 今日研究主题 今天主要研究 AMM、流动性深度和滑点之间的关系。	35
笔记 018: 链上套利学习 Day 5 笔记	36
笔记 019: 今日学习心得 : 从批量数据查询到用 AI 建立验证流程	45
笔记 020: 今日行动	45
笔记 021: Day 5 2026.08.09 Solana 通用 2-4 跳套利: executor v	46
笔记 022: 2026-08-09 · Day 5 / 21 · DEX 间套利——第一次完整双池计算	47
笔记 023: Uniswap V2 / V3 數學與運作筆記	53
笔记 024: Day 5 bps、前端报价口径与 UniswapX 路由观察	72

笔记 025: 在真的去 Polymarket 自动挂单之前, 先要解决行情数据的问题。目前找到这几个数据源:	72
笔记 026: 昨天周六, 请假休息一天。今天周日主要回顾上周进度和本周计划。	72
02 LI.FI与跨链 (39 篇)	73
笔记 027: Day 4 · 从“报价差”转向“可执行边际”	73
笔记 028: Day 3 续: 深挖 FP/IMD 合约源码, 揭示跨链 OFT 套利本质	74
笔记 029: 链上套利学习笔记 Day 4: 跨链与 LI.FI —— 真正的时间窗口	75
笔记 030: \=====	76
笔记 031:	79
笔记 032: Day 4 - 2026-08-08: 确定研究边界	80
笔记 033: Hermes Agent 研究与信号工作流学习笔记	81
笔记 034: 今天是我的第二天学习, 我阅读了《套利共学 为什么我在熊市发起链上套利残酷共学? 顺便分享下我的学习大纲》.	87
笔记 035: 在 Web3 / 链上交易 (DeFi) 的语境下, “路由” (Routing) 指的是将你的交易订单从“	88
笔记 036: Day 5 打卡: V3 Tick 深度解析 + LI.FI API 实操 + 跨链 Route 评估	88
笔记 037: 	90
笔记 038: 8.8完全的小白先入门\	91
笔记 039: 链上套利残酷共学 · 打卡	91
笔记 040: 2026-08-09 共学 Day 5 投入约 1 小时	92
笔记 041: Day 5 — 实战部署与实时监控	93
笔记 042: 2026-08-09	93
笔记 043: 2026年8月9日 链上套利学习笔记	94
笔记 044: LI.FI 跨链套利与 AI Agent 集成学习笔记	94
笔记 045: 今日完成	101
笔记 046: day5	102
笔记 047: https://github.com/ZeroNullNone/onchain_arbitrage_	102
笔记 048: 共学第5天。今天从同学精华里系统补了跨链套利与MEV两块知识:	103
笔记 049: 今天开始尝试把前几天学习的跨链、路由和 Agent 这些东西, 往真正的套利策略上结合。	104

笔记 050: Day 5 链上套利残酷共学	104
笔记 051: 一、核心架构：MCP Server 深度集成	105
笔记 052: 昨天用 LI.FI API 完成了 Base USDC → Arbitrum USDC → Base	106
笔记 053: 【Day 5 2026-08-09】LI.FI 定位	107
笔记 054: 2026.08.09	108
笔记 055: 案例分析：SK 海力士链上永续合约跨市场套利	109
笔记 056: Day 5 同一交易意图下，比较 CHEAPEST 与 FASTEST	109
笔记 057: 任务：写脚本调 /quote + /advanced/routes，解析多路由，按净收益排序输出 T	110
笔记 058: 链上套利共学 · Day5 打卡笔记 (08/09)	111
笔记 059: 1. debug 并完善先前的 RPC 脚本，获取不同链的延迟/archive 深度/额度限制	112
笔记 060: Day 5 打卡素材	113
笔记 061: Hermes Agent 研究与信号工作流学习笔记	113
笔记 062: 价格的影响到藏在toAmount里是一笔吃掉池深后的真实到手已扣的冲击，决定了池子深度加上我的下单量	122
笔记 063: \# 跨链快 6 秒，值多少钱？我用 9 次真实报价做了个小实验	123
笔记 064: Day 4：TUT 代币 Bitget vs Binance 价差 & 期现套利分析	124
笔记 065: 参考	125

03 MEV与抢跑 (10 篇) 126

笔记 066: Day 4 / 21 · Gas 与交易生命周期。	126
笔记 067: Day 03 · Gas、区块确认、执行延迟与 MEV 基础	129
笔记 068: Solana top 套利机制研究	130
笔记 069: Day 4 打卡：从「看懂交易」到「看懂攻击」——失败、原子性与 MEV 实操推演	131
笔记 070: Day 4 学习笔记	131
笔记 071: Day 5：全自动套利系统架构 v0.4	133
笔记 072: 搞懂了 MEV 里 staticcall 的作用和套利的整体流程。	136
笔记 073: 研究 MEV 生态	136
笔记 074: 链上套利学习笔记 Day 5：清算套利与 MEV	137
笔记 075: 今日共学打卡 Robinhood Chain 套利研究	138

04 做市与LP (5 篇)	138
笔记 076: Day 3 币股 LP 实操与自动化工具	138
笔记 077: 关于LP做市的总结	139
笔记 078: 什么是「针」和「接针」	141
笔记 079: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践, Agent 仅负责	143
笔记 080: D03 DEX、订单簿与 AMM	143
05 其他 (3 篇)	144
笔记 081: tradepools LBP的五个状态	144
笔记 082: 学习了robinhoodchain (https://robinhoodchain.blocksco	145
笔记 083: 	145
06 套利框架 (31 篇)	146
笔记 084: Day 4 算出利润之后, 交易为什么还是抢不到?	146
笔记 085: 套利核心原理与判断框架	147
笔记 086: 今日打卡	151
笔记 087: Day 4 — 周末 60 分钟价差监控 + 跨时段总汇	153
笔记 088: 第三天 跨链套利、清算与最大可提取价值	155
笔记 089: 资金费套利: 从 Funding Rate 排行榜到可执行交易策略的理解	157
笔记 090: 前几天理解的 DEX 间套利、跨链套利和三角套利, 本质上都比较接近“不同地方价格不一样, 所以买低卖高 157	
笔记 091: 链上套利残酷共学 · Day 3-4 打卡	157
笔记 092: 2026-08-09 學習打卡	158
笔记 093: 2026-08-09 套利学习日志	159
笔记 094: D5 (8/9) 跨链套利基础 + MEV 军备竞赛 (互动推演 + 真实数据)	159
笔记 095: 最近两天主要围绕套利机会的判断框架和两个案例做了整理。	160
笔记 096: 链上套利共学 Day 5 毛价差不等于利润: 真实报价验证闸门 + 注册表统一修复	160
笔记 097: 今天主要研究并实际操作了两种玩法。	161
笔记 098: 第五天学习记录: 今天继续围绕 Robinhood Chain 上 0xd712 套利体系, 把“对方机	161
笔记 099: 链上套利残酷共学 2026-08-09 打卡	162

笔记 100: Robinhood套利	163
笔记 101: Day 5 : 刷新价差并加入 Gas——昨天的候选为何失效。	163
笔记 102: 测算套利策略的失效阈值	164
笔记 103: 今天的主题是：把三池套利评估从“单次实时 Quote”升级为“可复跑的固定快照 + 更保守的路径级	164
笔记 104: 2026-08-09 链上套利共学打卡：从历史打卡提炼可执行研究模板	166
笔记 105: Day 5 · 研究数据的完整性 (look-ahead 与 cutoff 对齐)	166
笔记 106: 今年上半年在体验了从双币赢到期权的实践，在复盘，中可以认为合适的期权策略也可以认为是一种套利。	167
笔记 107: 看到群友发的0x等121208的调研报告，我作为小白不太理解。所以用ai帮我再次简单翻译了一下。	168
笔记 108: Day 5 打卡 (2026-08-09)	168
笔记 109: 学习笔记：链上套利 —— 从「发现价差」到「判断可执行」	169
笔记 110: Day 4 手算 AMM 價格，實測「無套利帶」	170
笔记 111: Day 5 · 成本敏感分析 (agent 开发工程师视角)	171
笔记 112: Day 5 - 2026-08-09 : 从双链库存套利切换到 Polymarket	173
笔记 113: 今日打卡	174
笔记 114: 链上套利残酷共学 · Day 4 心得	176
07 学习计划 (5 篇)	177
笔记 115: 最近因为 ColdCard 弱随机数问题 尝试去设计一个熵混合系统	177
笔记 116: 完成容量实验，对 Arbitrum USDC → Optimism DAI 同一条路线在 100/1	187
笔记 117: Day 4 / 2026-08-09	187
笔记 118: https://www.evm-from-scratch.app/content/ (https://	188
笔记 119: Day 05 · 2026-08-09 · 环境搭建：RPC、数据接口、钱包	191
08 实盘与数据 (13 篇)	192
笔记 120: 今天我把之前的历史 Flash Swap 案例，推进到了实时三池只读报价 → 保守 PnL → mi	192
笔记 121: 今日学习总结 MEV 套利、成本模型与策略验证	194
笔记 122: 研究了昨天群里老哥发的那个机器人。	194
笔记 123: Day 04 : Uniswap V3 Swap、报价验证与真实交易复现	194
笔记 124: 今天最大的收获是重新确认了一个判断原则：链上套利不是找“价格不一致”，而是验证“扣完所有成本后是否还	

笔记 125: Day 5 (8/9 周日) 打卡	201
笔记 126: Day 5 打卡 套利工具链搭建与风控体系落地	202
笔记 127: 2026-08-08\链上套利共学群日报.md	204
笔记 128: Day 5 链上套利-WETH/USDG 双池调研：为什么同一个币对有两个池子	206
笔记 129: 今日学习主题：从单一跨链价差，扩展到多链、多资产的结构套利雷达。	206
笔记 130: Solana LP 机会分析与对比	207
笔记 131: Day 5 (8/9) 打卡	207
笔记 132: Day 5 : Robinhood Chain 套利机器人近期交易与盈利能力分析	208
09 风险与安全 (5 篇)	211
笔记 133: 使用 Codex 开发实盘交易系统的阶段性学习笔记	211
笔记 134: ERC-20、Decimals、Allowance、Permit2 与授权风险	215
笔记 135: Day 5 学习笔记	215
笔记 136: 今天學了什麼	217
笔记 137: 补充基础知识	217

01 AMM机制 (26 篇)

笔记 001: Day 4 笔记 : The Graph —— 学会自己从链上查数据

作者 : GeoGu0 | 时间 : 2026-08-08 15:13 UTC | 字数 : 6401 | 评分 : 78

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 笔记 : The Graph —— 学会自己从链上查数据

日期 : 2026 年 8 月 8 日

进度 : 理解了 The Graph 是什么 + 跑通了三条 GraphQL 查询 + 学会了用 subgraph 查 swap 历史和池子价格 + 设计了找价差的查询思路

一、为什么需要 The Graph ?

前三天我用 DEX Screener 看实时报价,用 Etherscan/Arbiscan 查单笔交易详情,用 LI.FI API 拿跨链 Quote。这三个工具覆盖了"此刻的价格"和"单笔交易的成本",但有一个共同的盲区:它们不擅长回答"过去发生了什么"。

比如我想知道"昨天下午 3 点到 4 点之间,Arbitrum 上 Uniswap 和 Sushiswap 的 ETH 报价差了多少次超过 0.3%?"这类问题,DEX Screener 只能看当前、Etherscan 只能看单笔交易、LI.FI 只能查当时时刻的 Quote。我需要一个能批量查询历史数据的东西。

直接从区块链 RPC 查是可以的——但极慢。区块链的账本是一个区块一个区块、一笔交易一笔交易地线性存储的。要查"最近 1000 笔 swap"这个看似简单的需求,RPC 需要从最新区块开始逐个往前翻,找到每一个 swap 事件。每次 RPC 请求还有调用次数限制(Alchemy 免费层每天 3 亿 CU 听着多,但实际上以太坊主网一个 archive 查询就可能用掉几千 CU)。大规模历史查询在 RPC 上既慢又贵。

The Graph 解决了这个问题。它是一个去中心化的链上数据索引协议。核心思路是:提前定义好"我要给哪些合约的哪些事件建索引"(这就是 Subgraph),然后 The Graph 的节点持续扫描新区块,把匹配到的事件提取出来、转化成结构化的数据(Entity),存到数据库中。查询的时候直接用 GraphQL 语言去查这个数据库——毫秒级返回,不需要遍历区块。

RPC 是翻原始账本,一页一页翻。The Graph 是已经建好的 Excel 表格,你可以直接用筛选和排序。这就是为什么做链上数据分析和回测离不开 The Graph。

二、三个核心概念: Subgraph、Entity、GraphQL Query

Subgraph (子图):一份索引规则。你告诉 The Graph:去监听 Uniswap V3 的 Factory 合约、Pool 合约、Router 合约,当上面发生 PoolCreated、Swap、Mint、Burn 这些事件的时候,帮我记下来。一个 Subgraph 就是一个完整的索引定义文件,写好之后部署到 The Graph 网络上,节点就开始按你的规则去扫描链。

Entity (实体):索引后存到数据库里的数据模型。比如 Uniswap V3 Subgraph 定义了几个核心 Entity:Pool (池子,包含 token0、token1、feeTier、liquidity 等字段)、Token (代币,包含 symbol、name、decimals 等)、Swap (每次换币事件,包含 amount0、amount1、amountUSD、timestamp、sender 等)、PoolHourData (每个池子每小时的统计,包含 open/high/low/close 价格和成交量)。

GraphQL Query (查询):和 The Graph 交互的语言。和 SQL 有点像,但更简洁——你精确地告诉你需要什么字段、怎么排序、取多少条。不会多返也不会少返。The Graph 的 Playground 里可以直接写、直接跑、直接看结果。不需要写 Python 代码。

和套利直接相关的三个 Subgraph (要记住 ID,后面会反复用):

Uniswap V3: ID 为 5zvR82QoaXYfYDEKLZ9t6v9adgnptxYpKpSbxtgVENFV。几乎所有主流链(Ethereum、Arbitrum、Base、Optimism、Polygon)上 Uniswap V3 的交易数据都在这个 subgraph 里。字段包括 swaps (每笔交易详情)、pools (池子配置和当前状态)、poolHourDatas 和 poolDayDatas (历史 OHLCV 数据)、tokens (代币元数据)。

SushiSwap: ID 为 6UqLXLGZ2ssoY3h7SL7VJR6KVpEKQh4mHJBcQ6bKmcHN。覆盖多链上的 SushiSwap V2/V3。做 DEX 间价差对比时需要同时查 Uniswap 和 SushiSwap 的报价。

Aave V3: ID 为 Cd2gEDVeqnjBn1hSeqFMitQ8kZkzMAwBTLxTJMJAfHRA。用于查借贷协议的清算机会——监控哪些用户的 health factor 接近 1。

三、跑通三条 GraphQL 查询

在 The Graph Playground (thegraph.com/explorer) 里,选好 Subgraph ID,粘贴查询,点运行按钮。

查询一: Uniswap V3 最近 10 笔 Swap

```
{
  swaps(first: 10, orderBy: timestamp, orderDirection: desc) {
```



```

id
amount0
amount1
amountUSD
timestamp
pool {
  token0 { symbol }
  token1 { symbol }
}
}
}

```

这条查询返回 10 行数据。每行包含一次 swap 的金额（以 token0/token1 的最小单位计）、美元估值、时间戳、以及它发生在哪个池子（哪种币换哪种币）。timestamp 是 Unix 时间戳（秒数），可以用在线转换工具转成北京时间看是什么时候发生的。这些数据的价值在于：你可以批量拉取 swap 历史，计算某个时间段内的平均价差、最大价差、价差超过某个阈值的次数。

查询二：查 ETH/USDC 0.05% 池子最近 24 小时的价格

```

{
  poolHourDatas(
    first: 24,
    orderBy: periodStartUnix,
    orderDirection: desc,
    where: { pool: "0x88e6a0c2ddd26feeb64f039a2c41296fcb3f5640" }
  ) {
    periodStartUnix
    high
    low
    open
    close
  }
}

```

这个池子地址 0x88e6...3f5640 是 Ethereum 主网上 Uniswap V3 的 ETH/USDC 0.05% 费率池（要查 Arbitrum 上的需要换对应的 pool 地址）。返回每小时的开盘价、收盘价、最高价、最低价。有了这个数据，你就可以画 K 线图，或者计算“过去 24 小时里每小时价格变动的标准差”——这个数字对于估算延迟成本很有用。Day 3 我说延迟成本需要“历史波动率”，poolHourDatas 里的 high-low 差值就是一个可以拿来用的数据源。

查询三：查两个不同 DEX 的同一个交易对的最新 swap 价格

```

{
  uniswap: swaps(
    first: 1,
    orderBy: timestamp,
    orderDirection: desc,
    where: { pool: "0x88e6a0c2ddd26feeb64f039a2c41296fcb3f5640" }
  ) {
    amountUSD
    timestamp
    pool { token0 { symbol } token1 { symbol } }
  }
  sushiswap: swaps(
    first: 1,
    orderBy: timestamp,
    orderDirection: desc,
    where: { pool: "对应的 SushiSwap 池子地址" }
  ) {
    amountUSD
    timestamp
  }
}

```

```
pool { token0 { symbol } token1 { symbol } }  
}  
}
```

这条查询同时在两个 subgraph (或者同一个 subgraph 的不同池子) 上拉最新一笔 swap 的价格。如果两个价格在同一时刻 (或非常接近的时刻) 有差距, 那就是 DEX 间价差信号。

关键难点在于跨 subgraph 查询。Uniswap 和 SushiSwap 的数据在不同的 Subgraph 里, 无法在一条 GraphQL 查询里同时查两个 Subgraph。你需要分别调两次 API (或者用 The Graph 的 GraphQL Client 在代码里并发请求), 然后在代码层面做对比。这就是为什么后面 Day 16 写监控脚本的时候需要 Python——Python 负责并发请求两个数据源, 然后在代码里算价差。

四、设计“找价差”的查询方案

给定价差监控的需求——同一条链上两个不同 DEX、同一个交易对的实时报价对比——我从数据源到判断的完整思路是：

第一步, 确定两个池子的合约地址。Uniswap V3 的池子地址可以通过 token0、token1、feeTier 三个参数唯一确定 (Uniswap V3 Factory 的 CREATE2 地址推导)。通常直接查 The Graph 的 pools 实体, 用 token0、token1 和 feeTier 筛选出来, 比硬编码地址更灵活。

第二步, 分别取两个池子最新一笔 swap 的价格。两个 subgraph 分开查, 用 first: 1 和 orderBy: timestamp desc。

第三步, 在 Python 脚本里把两次查询的结果放在一起: 算价差 = $\text{abs}(\text{价格A} - \text{价格B}) / \min(\text{价格A}, \text{价格B})$ 。对比阈值 (比如 10 basis points = 0.1%), 大于阈值的打印出来。

第四步 (进阶), 不只看最新一笔 swap, 而是看池子当前状态。Uniswap V3 的 pools 实体里有一个字段叫 sqrtPrice (当前价格的平方根), token0Price 和 token1Price 也是从 sqrtPrice 直接算出来的。这些字段是实时的 (基于池子的当前 tick), 不需要等一笔 swap 发生。所以更高效的做法不是查 swaps, 而是直接查 pools 实体的 token0Price 字段。两边的 token0Price 直接对比, 算出实时价差。

查 pools 比查 swaps 更快、更准。因为 swaps 可能好几秒前刚发生过一笔, 价格的时效性不如 pools 的实时状态。但 pools 里的价格是 AMM 曲线上的现货价, 不包含滑点考虑。而查最新 swap 的价格是实际成交价——更真实但稍有时延。两种方案各有适用场景。

五、The Graph 在套利全流程里的位置

现在我的数据工具有了四件, 各司其职：

DEX Screener：快速肉眼看价格, 适合手动扫一眼“现在有没有大价差”。但没法自动化, 没法批量处理。

Etherscan/Arbiscan：查单笔交易细节。查 Gas、看 revert 原因、还原交易序列。

LI.FI API：跨链最优路由 + 实时报价。给你 toAmount、toAmountMin、executionDuration、route steps, 适合跨链套利发现。

The Graph：历史数据批量查询。适合回测、案例分析、统计价差分布、估算延迟成本和失败概率。

前三天的学习让我“能看懂一个 Quote”, 今天的 The Graph 让我“能自己从链上把数据挖出来”——不再只依赖第三方 API 喂到我嘴边。对于套利研究来说, 这是一个重要的分界线：依赖别人整理好的数据, 你能看到的和别人看到的一样；自己从链上拉数据, 你才有可能发现别人还没注意到的模式。

一个具体的例子：假设我想做 Arbitrum 上 Uniswap vs SushiSwap 的 DEX 间套利。用 The Graph 我可以拉取过去 7 天两个 DEX 各自池子的小时级价格数据 (poolHourDatas), 算出差价的时间序列——价差在一天里的什么时段最大？周末和工作日有区别吗？价差的分布是尖峰 (大部分时间很小, 偶尔很大) 还是均匀的？这些问题的答案直接影响策略设计：如果价差 99% 的时间都小于 5bp, 那我设触发阈值为 5bp 的话, 可能一天就抓到一两次机会——够不够付资金占用成本？

这个分析, 就是 Day 14 复盘和 Day 19 回测的基础。

六、今天的三个核心收获

第一, The Graph 是 RPC 的加速层。RPC 翻原始账本, The Graph 查建好的索引表。做历史分析和回测必须用 The Graph, 否则等你用 RPC 遍历完过去一个月的 swap 事件, 行情都换了好几轮了。

第二, 查 pools 实体的 token0Price 比查 swaps 更快更准。pools 给你当前实时报价 (基于 tick), swaps 给你实际成交价 (有时延但更真实)。做实时价差监控应该优先查 pools, 做历史成交分析才用 swaps。

第三, 知道了怎么跨 subgraph 找价差。两个 DEX 的数据在不同名字空间里, 没法一条 GraphQL 搞定——需要 Python 并发请求两个 endpoint, 然后代码层面做对比。这就是 Day 16 写监控脚本的逻辑基础。

七、今天的三个问题

1. The Graph 的 free tier 请求频率和每月 CU 额度是多少？我写监控脚本多久轮询一次不会被限流？
2. poolHourDatas 里的 high 和 low 不是真实成交价而是 AMM 曲线上的理论价格——这个差异在做回测时会有多大影响？
3. 除了 Uniswap 和 SushiSwap, 还有哪些 DEX 有维护良好的 Subgraph？PancakeSwap、Curve、Balancer 的数据能不能也在 The Graph 上查到？

八、从今天开始, 我的数据工具箱齐了

价差发现：DEX Screener（肉眼）+ The Graph pools（代码）

成本估算：Etherscan（单笔 Gas）+ LI.FI（跨链综合成本）+ The Graph swaps（历史成交价）

路径分析：LI.FI route steps（拆 swap 和 bridge）

回测分析：The Graph poolHourDatas（历史价格序列）

前三天的学习是“理解每一行数据的含义”。今天学的是“自己去把数据拿出来”。Day 5 要学的 RPC 和 eth_call 是最后一块拼图——直接和链对话，不经过任何中间层，模拟交易执行。四件工具聚齐之后，后续两周的深挖和脚本编写就有了数据底座。

笔记 002: 2026-08-08 打卡 | 从「能成交」走到「有利润」\

作者：mirrornight2026 | 时间：2026-08-08 15:15 UTC | 字数：1239 | 评分：64

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-08 打卡 | 从「能成交」走到「有利润」\

\

今天继续用同一组两池 AMM 示例研究交易金额和执行风险。我之前容易把「原子执行」「两条腿都成交」和「套利成功」混在一起，现在知道它们是三件不同的事。\\

\

先看投入金额。利润函数可以写成：\\

\

净利润 $\pi(I)$ = 第二腿最终到账 $O(I)$ - 本金 I - Gas G \\

\

固定 Gas 会把整条利润曲线向下移动。在交易路径和 gasUsed 不变的简化模型里，它通常不会改变理论最优投入金额，但会压缩可盈利区间。沿用池 A 100,000 USDC / 1,000 Token、池 B 105,000 USDC / 1,000 Token、两边手续费 0.3%、Gas 1 USDC 的教学例子，结果是：\\

\

\\- 下方保本点约 23.13 USDC；\\

\\- 理论最优投入约 1,085.93 USDC；\\

\\- 上方保本点约 2,171.21 USDC。\\

\

这也解释了两个看似相反的亏损。投入 10 USDC 时，扣 Gas 前还能赚约 0.435 USDC，但 1 USDC 的 Gas 无法覆盖，最后亏约 0.565 USDC。投入 3,000 USDC 时，第二腿只能拿回约 2,954.65 USDC，扣 Gas 前已经亏约 45.35 USDC；即使 Gas 为零也会亏，主因是两边的价格冲击。\\

\

我今天还弄清楚了「两条腿都能成交」的意思：第一腿在便宜池用 USDC 买 Token，第二腿在贵池把 Token 卖回 USDC。两次兑换都成功，只说明路径完成了，不说明赚钱。3,000 USDC 的例子里，两条腿可以都成功，但最终资金变少，所以成交成功不等于套利成功。\\

\

原子执行只保证两条腿一起完成或一起回滚，也不会自动保证利润。合约还需要检查：\\

\

最低最终到账 = 本金 + 预计 Gas + 最低目标利润 + 风险缓冲\\

\

例如本金 1,000 USDC、预计 Gas 1 USDC、最低目标利润 5 USDC，最低最终到账就是 1,006 USDC。执行时如果只能拿回 1,004 USDC，最终检查应让整笔交易回滚。这样不会留下中间 Token，但失败 Gas 仍然要付。\\

\

Quote 也不是锁价。扫描器读取的是某个区块的池子储备，交易真正进入后续区块时，其他交易可能已经改变储备。最终应以执行区块里的真实 Fill 为准。多个机器人同时发现机会时，排在前面的机器人可能先把价差吃掉；后面的交易即使有原子保护，也可能因为利润门槛不再满足而回滚，并留下失败 Gas。\\

\

今天形成的判断是：先问两条腿是否能完成，再问它们能否原子地一起完成，最后用真实到账检查是否覆盖本金、全部成本、最低利润和风险缓冲。不能把「交易成功」当成「套利成功」，也不能只记录抢到机会的交易。下一步我想把候选、提交、成交和回滚都记录下来，用真实失败率和 Gas 消耗估算策略期望值。

笔记 003: D3 学习进展：CPMM、手续费、价格冲击与滑点

作者：Qun Wang | 时间：2026-08-08 15:32 UTC | 字数：1707 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D3 学习进展：CPMM、手续费、价格冲击与滑点

今日问题

$xy = k$ 怎样决定一笔有限规模交易的输出？如何区分池手续费、价格冲击、状态变化造成的滑点，以及滑点容忍度？

使用的一手资料或区块/数据

Uniswap V2 whitepaper、Library 与 Pair 合约源码。

简化池：100,000 USDC / 50 WETH，手续费 0.3%。

Ethereum mainnet 固定区块 0x18809d4，区块哈希

0xcc96fdb5d9807ef919f5ddc04f40f003e867731a854bb27e38c727464398904f。

Uniswap V2 USDC/WETH pair：0xB4e16d0168e52d35CaCD2c6185b44281Ec28C9Dc。

做了什么

从 $xy = k$ 推导交易前边际价格、有限交易平均成交价格 and 交易后边际价格。

在同一简化池中比较 100 USDC 与 1,000 USDC 输入，拆分无手续费价格冲击与 0.3% 手续费的输出影响。

理解 997/1000 只用于有效输入，而池储备增加完整的 amountIn。

用两个状态区分价格冲击与滑点，并用 amountOutMin 判断 0.5% 容忍度下是否回滚。

解码固定区块的 raw reserves 与 ABI 返回值，用整数公式复算 Router 报价。

结果或失败

简化池的交易前边际价格是 2,000 USDC/WETH。100 USDC 的可执行输出为 0.049800349051995160 WETH；1,000 USDC 的可执行输出为 0.493579017198530649 WETH，说明规模增大时价格冲击非线性增加。若一笔 5,000 USDC 交易先改变池状态，同样 100 USDC 的输出降至 0.045178981251284187 WETH，相对原报价滑点约 9.2798%，低于 0.5% 容忍度对应的最小输出 0.049551347306735184 WETH，因此会回滚。

固定区块中，pair reserves 为 8,861,915.877947 USDC 与 4,645.090378877195088753 WETH；输入 100 USDC，本地整数公式和 Router eth_call 均得到 52258485296111725 raw WETH，即 0.052258485296111725 WETH。

本次曾出现价格单位遗漏、WETH decimals 小数点错一位、混淆缩放分母，以及不理解手续费输出损失为何不能直接用 $3/2000$ 计算的问题；经逐步提示后完成。Worksheet 的退出卡是辅导整理稿，不是无提示闭卷作答，因此这次只记录进展，不宣称 D3 已完全掌握。

证据链接或本地路径

evidence/0003-cpmm-quote/worksheet.md

evidence/0003-cpmm-quote/README.md

evidence/0003-cpmm-quote/pair-reserves.json

evidence/0003-cpmm-quote/router-quote.json

reference/0004-cpmm-quote-anatomy.html

下一假设

在同一固定区块中选取两个 V2 池，对 USDC $\rightarrow$ WETH $\rightarrow$ USDC 路径做整数报价；只有把两跳池手续费、价格冲击、Gas、失败损失和未回收资产统一换算成 USDC 后，才能判断是否越过利润阈值。

风险认识的变化

相同边际价格不代表相同规模下的可执行输出；eth_call 报价不证明交易已执行，也不证明盈利；amountOutMin 是执行边界而不是额外成本；报价已经包含池手续费，不能再重复扣除。

笔记 004: 2026-08-08 | 学习记录

作者：capromka | 时间：2026-08-08 15:46 UTC | 字数：486 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-08 | 学习记录

今天学习了 LP 第2课：池子中的价格形成机制，进一步理解了 AMM 的基本原理。

主要收获

AMM 不需要人工挂买卖价格，池内两种资产的数量比例决定了池内价格。

学习了经典 AMM 的核心公式： $x \times y = k$ ，也就是池中两种资产数量的乘积保持不变。

买 ETH 的人越多，池里的 ETH 越少，ETH 就会越来越贵；所以买得越多，成交价格越高。

也理解了为什么很难把池子里的某种资产直接买空：越接近买空，价格会越来越贵，需要付出的另一种资产越来越多。

初步认识了 Price Impact 和 Slippage：前者主要是自己的交易把池子价格推偏了，后者主要是预期成交和最终成交之间的差异。

今日总结

AMM 的价格不是人为设定的，而是由池子里的资产比例自动形成；交易本身又会改变资产比例，从而推动价格变化。
今天算是把“LP 为什么会影响价格、交易为什么会产生滑点/价格影响”这块基础逻辑搞明白了。

笔记 005: 8.8 / 链上套利共学

作者：richardxv | 时间：2026-08-08 15:47 UTC | 字数：7337 | 评分：76
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8.8 / 链上套利共学

链上套利零基础学习版

从“看到调用栈”进阶到“判断利润在哪里计算、交易为什么成功或回滚”

资料范围：群聊正文 + 13 张图片 | 版式：compact_reference_guide | 读者：完全零基础

今天最重要的升级：不要再把“链上读了状态”“链上计算了利润”“链下模拟过”“交易最终回滚”当成同一件事。成熟套利系统通常同时使用链下数据与计算、RPC 读取或模拟、交易内状态校验和链上原子执行。调用栈只能告诉你执行了什么，不能单独还原完整策略。

本文将群聊中的技术争论、交易追踪图和收益截图拆开验证。群聊观点用于发现问题；官方文档用于校正概念；真正的策略结论仍要靠交易回执、状态变化和可复现实验。

一、今日主线：套利机器人到底在做什么

状态获取 → 机会发现 → 路径与金额计算 → 构造交易 → 排序/提交 → 链上校验与执行 → 回执核算

群聊围绕一个	Robinhood	Chain
机器人样本展开。有人认为它在链上实时获取状态并计算利润，有人认为利润已在链下算好，链上只读取		slot0
等状态做校验。更准确的结论是：仅凭调用栈无法二选一。		

看到 token0、token1、slot0、fee、extsload，能确认执行过程读取了这些池状态。

看到 swap、unlock、callback、transfer、withdraw，能确认交易走过哪些合约动作。

但调用栈不展示机器人的链下图搜索、缓存、触发条件、模拟结果和提交策略。

因此，链上追踪适合验证执行逻辑，不足以单独证明完整机会发现算法。

二、四个容易混淆的执行环境

环境

发生在哪里

会不会产生链上费用

能说明什么

本地计算

机器人进程/内存

本身不产生 Gas

可做图搜索、报价和最优金额计算。

eth_call

RPC 节点本地执行

不创建交易，用户不付链上 Gas

读取或模拟给定区块状态下的结果。

交易内 STATICCALL

真实交易的 EVM 调用栈

会计入该交易 Gas used

证明交易执行时读取了合约状态。

状态修改调用

真实交易上链执行

会计入 Gas；失败也会消耗已执行部分

产生 swap、transfer、利润或回滚结果。

一句话校正：“查链为什么要	Gas？”要先看是哪一种查法。链下	eth_call	不向链上付
Gas；真实交易里调用合约读取状态，则会增加这笔交易的 Gas used。			

三、怎样读懂交易追踪图

1. 图里的关键节点

CALL：普通外部调用，可进入代理、池管理器或代币合约。

DELEGATECALL：在当前合约存储上下文中执行另一份代码，常见于代理结构。

STATICCALL：只读调用；截图中用于读取 token0、token1、slot0、fee、余额或扩展存储。

PoolManager.unlock / unlockCallback：与 Uniswap v4 的解锁回调和内部余额结算流程有关。

swap / transfer / withdraw：真正改变资产或完成结算的动作。

2. 一张追踪图不能证明什么

不能只凭 gas used 判断利润计算在链上还是链下。

不能只凭出现 STATICCALL 就断定是“盲套利”或“实时全量计算”。

不能只看绿色转入数字认定净利润；还要计算输入、输出、Gas、借款归还和前后余额。

不能把地址余额曲线直接当利润曲线；转入、充值、桥接和估值变化都会改变余额。

四、V2、V3、V4 为什么越往后越难算

1. V2：单池数学简单，竞争不简单

$x \times y = k$

Uniswap v2 的核心池使用恒定乘积模型。给定储备、手续费和输入量，可以在本地算出输出；三角或多跳套利则要把多个池组成路径，再求扣费后的最优输入量。数学并不神秘，但状态必须新鲜，提交必须足够快。

2. V3：可以算，但不是只看一个储备数

V3 把流动性集中在价格区间，并用 Tick 划分价格空间。当前价格、当前流动性、Tick 边界和跨 Tick 后新增/减少的流动性都会影响报价。说“V3 完全算不了”是错的；说“和 V2 一样拿一个公式就够”也不完整。

3. V4：单例 PoolManager、Hooks 与 Flash Accounting

V4 使用 PoolManager 管理池，并通过 unlock / unlockCallback 和内部 delta 完成复杂操作的结算。Flash Accounting 让多跳过程中只在最终结算时发生必要的代币转移，降低转账开销；Hooks 又允许池加入自定义逻辑。因此路径计算、状态读取和风险判断会更复杂。

工程结论：简单池里速度可能更突出；多协议、多跳、V3/V4

路径里，算法、状态同步、Gas

优化和提交延迟缺一不可。把“算法”和“速度”对立起来没有意义。

五、所谓“盲发”到底盲在哪里

群聊里的“盲发”不一定意味着随机发交易。更常见的含义是：机器人无法等到完整链下模拟后再决定，而是根据触发信号快速构造多笔候选交易，并在交易中设置状态或利润条件；不满足条件就回滚。

一种可理解的研究流程

1. 订阅新区块、日志或公共交易，识别最近发生变化的池。
2. 维护 hot pool 列表，避免每次扫描全部池子。
3. 批量读取关键状态，在本地更新池图和报价。
4. 搜索单池、三角、多跳或跨协议路径，估算最优输入与最低利润。
5. 构造带状态断言和最小利润保护的交易所。
6. 通过公共 RPC、私有通道或 bundle 提交，并监控是否落块。
7. 按交易回执、真实余额和全部费用复盘，而不是只数成功笔数。

大量发送的代价：并发候选交易可能提高抢到机会的概率，也会增加失败成本和被限流的风险。每笔只赚很少时，任何成本误差都会吞掉全部利润。

Gas、nonce

管理、RPC

六、Gas、回滚和截图里的失败原因

1. Gas limit 不等于 Gas used

Gas limit 是你允许执行消耗的上限，Gas used 是实际执行消耗。回滚会撤销状态变化，但已经执行过的计算和调用通常仍消耗 Gas；未使用的上限并不会全部收费。因此“回滚一定耗尽 Gas limit”和“失败交易几乎没有成本”都不能一概而论。

2. 本次失败图真正告诉我们的事

失败追踪明确显示 ERC20: transfer amount exceeds balance。它更像余额、授权或计算金额约束失败，不能直接解释成“机器人发现无利润后主动取消”。要判断主动利润保护，应找自定义 revert reason、利润断言代码或可复现实验。

3. Calldata 与额外读取

把稳定不变或已知参数放进 calldata，可能省掉部分外部读取；但对价格、流动性、Tick 等会变化的状态，必须保证输入与执行时状态一致。Gas 优化不能以使用过期状态为代价。

七、路径发现、延迟和私有通道

可执行优势 = 数据新鲜度 + 路径质量 + 金额优化 + 构造速度 + 提交通道 全部成本

简单池的机会容易被复制，往往更依赖延迟和基础设施；多协议、多跳策略可能靠更好的路径搜索获得优势。私有提交通道可以减少公开传播和被复制的风险，但不保证交易一定被包含，也不能消除状态变化。

Flashbots 的 `eth_sendBundle` 接口把签名交易列表作为原子 bundle 提交，并允许配置哪些交易可以回滚；`eth_callBundle` 可在指定状态上模拟 bundle。由此可见，“bundle 是否全有或全无”必须结合具体参数，而不是只看 bundle 这个词。

八、闪电贷：没有本金，不等于没有成本

闪电贷或 flash swap 允许在同一原子交易中先取得资产，并在交易结束前归还、支付费用或满足池的不变量。若条件不满足，整笔交易回滚。它降低的是前置资本门槛，不会替你解决 Gas、竞争、错误报价、合约风险和机会稀缺。

闪电贷套利净利润 = 套利输出 借款归还 贷款/池费用 Gas 小费 失败成本

ETH 本位是否需要对冲

如果目标是增加 ETH 数量，赚到的 ETH 可以按 ETH 本位记录；如果你的最终目标是美元购买力，持有 ETH 仍承担价格波动。对冲解决的是资产价格风险，不是套利交易本身是否赚钱。要把“策略利润”和“资金库计价风险”分开。

九、CEX 价差与资金费率套利

1. 跨交易所价差

不同 CEX 的订单簿价格可能暂时不同。真正执行时通常要预先先把资金放在两边，同时买低卖高，再通过后续调仓恢复库存。若等买完再提现到另一家，价差往往已经消失。

净价差 = 卖出成交额 买入成交额 双边手续费 滑点 调仓/借币/提现成本

2. 资金费率套利

永续合约通过资金费率帮助合约价格贴近现货指数。资金费率为正时，多头向空头支付；常见中性结构是买入现货、做空等值永续，尽量抵消价格方向，只赚取资金费和基差收敛。

预期净收益 = 收到的资金费 + 基差变化 开平仓费 滑点 借币/资金成本 对冲误差

资金费率会变，甚至反向；截图中的年化不是保证。

现货和合约数量、乘数、结算币种不一致会产生对冲误差。

杠杆与保证金管理不当仍可能爆仓。

交易所停机、限提、风控和对手方风险无法靠对冲消除。

任何 Cross 或跨所工具都只能当执行工具研究，不能当收益承诺。

十、LP 高 APR 与 Meme：为什么不属于无风险套利

1. LP 收益的来源

LP 把两种资产提供给交易池，收益可能来自交易手续费和代币激励。价格变化会让仓位逐渐变成其中一种资产；集中流动性离开价格区间后还会停止赚取手续费。因此 LP 是带库存风险的做市/流动性策略，不是简单锁定价差。

2. 高 APR 截图怎样读

图片中的 AOB/USDT 案例声称界面 APR 约 32,000%，实际体验约 7,200%，并最终清仓退出。它至少说明三件事：短时 APR 年化容易夸大；激励和成交量会快速变化；持有高波动代币可能让本金损失超过挖矿收益。数字本身没有链上仓位和完整现金流，不能独立验证。

3. 名人发帖与 Meme 上涨

社交平台内容引发 Meme 价格上涨属于事件驱动和方向性交易。它依赖市场情绪、注意力和后续接盘，不具备套利的双边锁定结构。钱包里突然出现的广告币或垃圾币也可能制造虚假的收益和余额展示。

十一、怎样评价本次图片证据

图片证据

可以支持

不能支持

调用栈

读了哪些状态、调用哪些池和函数。

完整路径搜索和利润算法放在哪里。

成功/失败历史

交易频率、回滚存在、界面显示的金额和费用。

未经逐笔核算的长期净利润。

余额曲线

某地址某时点的资产余额变化。

变化全部来自套利。

LP APR 截图

界面当时展示的年化值和发帖者说法。

可持续收益、真实成本和回撤。

赛道推荐页

一种研究假设和排序。

99.5%成功率、单笔收益等结论可复现。

资料质量提醒：原文末尾有一段 AI 总结，提到等本次正文和图片中没有直接证据的对象。为避免把二手总结当事实，本文没有采纳这些说法。

Astro、QQQ.Bot

证据优先级

官方协议文档 > 可复现代码/模拟 > 交易回执与状态变化 > 调用栈 > 浏览器统计 > 截图 > 群聊观点 > AI 二手总结

十二、七天学习任务

1. 第 1 天：选一笔成功交易，只画出调用树，不判断利润。
2. 第 2 天：从回执中找 Gas used、状态变化、代币转入转出，算真实现金流。
3. 第 3 天：用纸面例子学会 V2 的 $xy=k$ 和含手续费报价。
4. 第 4 天：学习 V3 的价格区间、Tick、当前流动性和跨 Tick。
5. 第 5 天：用 `eth\call` 只读查询一个池子的状态，理解区块参数和返回值。
6. 第 6 天：建立 hot pool 监控思路，只记录机会，不签名、不广播。
7. 第 7 天：复盘一周数据，比较理论毛利润、模拟净利润和真实可执行利润。

学习目标：当你能够解释“状态在哪里读、路径在哪里算、交易在哪里校验、失败为什么收费、利润如何从回执核算”，你才真正看懂了今天这场争论。

十三、每次分析机器人都问这十个问题

1. 机会由什么触发：日志、新块、mempool、报价 API，还是固定扫描？
2. 机器人维护哪些池和代币，状态多久更新一次？
3. 路径和最优输入量在哪里计算？
4. 交易内读取了哪些状态，只是校验还是重新计算？
5. 最低利润、最大滑点、最大 Gas 和超时条件是什么？
6. 提交走公共 RPC、私有通道还是 bundle？
7. 失败原因是利润不足、状态变化、余额不足、授权还是合约错误？
8. 成功交易的真实输入、输出、借款归还、Gas 和小费是多少？
9. 地址余额变化是否混入转账、充值或未实现估值？
10. 样本覆盖多少天、多少笔交易、多少失败，能否独立复现？

十四、术语速查

术语

小白解释

`eth\call`

让节点立即执行一次调用并返回结果，不创建链上交易；常用于读取和模拟。

STATICCALL

EVM 内部只读调用语义；在真实交易的执行路径中出现时仍会消耗该交易的 Gas。

slot0

Uniswap v3 池的关键状态入口之一，常包含当前 `sqrtPriceX96`、tick 等紧凑状态。

Tick

集中流动性中的离散价格边界；跨越活跃 Tick 时，可用流动性会变化。

Calldata

交易调用携带的输入参数。把稳定参数直接传入可减少某些额外读取，但会增加 calldata。

Revert

交易执行回滚，状态修改不保留；已经执行消耗的 Gas 通常仍要支付。

Gas used / limit

used 是实际消耗，limit 是允许上限；不能把上限当成本，也不能认为回滚一定耗尽上限。

Hot pool

近期活跃或状态频繁变化的池子集合，用于缩小实时扫描范围。

Bundle

按指定顺序提交的一组签名交易；是否原子、允许哪些交易回滚，要看具体接口参数。

Flash swap/loan

同一原子交易中先借用资产，结束前归还并满足费用或不变量。

Funding rate

永续合约多空双方周期性交换的费用，用于帮助合约价格贴近现货指数。

APR

把当前收益率年化后的展示值；高波动池的短时 APR 很容易失真。

十五、官方核查入口

以下资料用于核对概念和接口，不代表任何策略收益：

Ethereum JSON-RPC 与 eth_call：<https://ethereum.org/developers/docs/apis/json-rpc/>

Uniswap

Tick：<https://developers.uniswap.org/docs/get-started/concepts/liquidity-providers/concentrated-liquidity>

Uniswap v4 Flash Accounting：<https://developers.uniswap.org/docs/protocols/v4/concepts/flash-accounting>

Uniswap v2 白皮书与 Flash Swaps：<https://docs.uniswap.org/whitepaper.pdf>

Flashbots Bundle RPC：<https://docs.flashbots.net/flashbots-auction/advanced/rpc-endpoint>

OKX 永续资金费率说明：<https://www.okx.com/en-us/help/funding-fees-for-perpetual-contracts-faq>

十六、最后记住这十句话

1. 看到状态读取，不等于看到完整利润算法。
2. eth_call 不上链；交易内 STATICCALL 会计入 Gas。
3. V3/V4 可以计算，但状态和路径比 V2 复杂得多。
4. 算法与延迟不是二选一，成熟系统两者都要。
5. 回滚撤销状态，不会抹掉已经消耗的执行成本。
6. Gas limit 不是最终费用，Gas used 才接近实际执行消耗。
7. 闪电贷降低本金门槛，不消除费用和竞争。
8. 余额增长、绿色转入和 APR 都不等于经审计净利润。
9. LP 与 Meme 交易承担方向和库存风险，不是无风险套利。
10. 小白先学会读交易和算净利润，再研究自动发送。

笔记 006: Day 2 | AMM、Swap Pricing、Slippage 与 Price Impact

作者：HerschelLi-31 | 时间：2026-08-08 15:51 UTC | 字数：9567 | 评分：79

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 | AMM、Swap Pricing、Slippage 与 Price Impact

1. 今天的核心问题

Day 1 我已经知道：

Spread > 0

并不代表：

NetProfit > 0

集中流动性与

今天进一步研究其中最重要的一项成本：

【 Price Impact (价格影响) 】

在 AMM 中，屏幕上看到的 Price 通常只是当前 Pool State 下的边际价格。真正交易时，我自己的 Swap 会改变 Pool：

$(x, y) \rightarrow (x', y')$

因此真正需要关心的是：

【 Executable Price(Q) 】

而不是：

Displayed Price

其中 (Q) 是我的 Trade Size (交易规模)。

1. 什么是 AMM？

AMM 全称：Automated Market Maker (自动做市商)

传统 Order Book (订单簿) 市场依赖：

Buy Orders + Sell Orders

完成撮合。

而 AMM 使用 Liquidity Pool (流动性池) 和数学规则决定 Swap Price。

最经典的模型是 Constant Product AMM (恒定乘积做市商)：

【 $x \setminus y = k$ 】

其中：

x：Token X 的 Pool Reserve (储备量)

y：Token Y 的 Pool Reserve (储备量)

k：Constant Product (常数)

例如 ETH/USDC Pool：

$x = 100 \text{ ETH}$

$y = 400,000 \text{ USDC}$

因此：

$k = 100 \setminus 400,000 = 40,000,000$

只要没有考虑 Fee、Liquidity Addition 等其他操作，Swap 前后需要满足：

$x \setminus y = k$

1. AMM 中 Price 从哪里来？

假设：

$x = 100 \text{ ETH}$

$y = 400,000 \text{ USDC}$

最简单地看，当前 ETH Price：

$P = y / x$

所以：

$P = 400,000 / 100 = 4000$

即：

$\text{ETH} \approx 4000 \text{ USDC}$

但这里的 4000 不能理解成：

"Pool 愿意无限量以 4000 USDC/ETH 卖 ETH。"

它只表示：

当前 Pool State 附近的 Marginal Price (边际价格)。

如果我的 Trade Size 越来越大，我就会不断改变 x 和 y，所以后续每一小部分 ETH 的价格都会不同。

1. 为什么买 ETH 会让 ETH 价格上涨？

假设用户使用 USDC 买 ETH。

原来：

$x = 100 \text{ ETH}$

$y = 400,000 \text{ USDC}$

用户把 USDC 放入 Pool，因此：

$y \uparrow$ (增加)

同时从 Pool 拿走 ETH：

$x \downarrow$ (减少)

因为：

$$x \setminus y = k$$

所以新的 Pool State 满足：

$$x' \setminus y' = k$$

而 ETH 的 Pool Price：

$$P' = y' / x'$$

由于 $y' > y$ 并且 $x' < x$ ，因此：

$$P' > P$$

也就是说：

【 Buy ETH $\rightarrow$ ETH Price $\uparrow$ 】

反之：

【 Sell ETH $\rightarrow$ ETH Price $\downarrow$ 】

这就是 AMM 自己完成价格调整的基本机制。

1. 手推一次 Swap

继续使用：

$$x = 100 \text{ ETH}$$

$$y = 400,000 \text{ USDC}$$

暂时忽略 Trading Fee。

假设我投入：

$$\Delta y = 4000 \text{ USDC}$$

购买 ETH。

Swap 后：

$$y' = 400,000 + 4000 = 404,000$$

Constant Product 要求：

$$x' \setminus y' = k$$

因此：

$$x' = k / y'$$

代入：

$$x' = 40,000,000 / 404,000 \approx 99.0099$$

所以我从 Pool 得到的 ETH 为：

$$\Delta x = 100 - 99.0099 \approx 0.9901 \text{ ETH}$$

因此我不是用 4000 USDC 买到了 1 ETH，而是只得到大约 0.9901 ETH。

我的 Average Execution Price (平均执行价格)：

$$P_{\text{avg}} = 4000 / 0.9901 \approx 4040$$

虽然最初看到：

$$\text{Spot Price} = 4000$$

但真正平均成交价格约为：

$$4040$$

这就是 Trade Size 对 Executable Price 的影响。

1. General Swap Formula (通用 Swap 公式)

假设：

x 是我要买的 Token Reserve，

y 是我要投入的 Token Reserve。

我投入：

$$\Delta y$$

Swap 以后：

$$y' = y + \Delta y$$

由于：

$$x \setminus y = x' \setminus y'$$

所以：

$$x' = (x \setminus y) / (y + \Delta y)$$

因此得到的 Token 数量：

$$\Delta x = x - x'$$

代入：

$$\Delta x = x - \sqrt{(x \setminus y) / (y + \Delta y)}$$

整理后得到：

$$\left[\Delta x = (x \setminus \Delta y) / (y + \Delta y) \right]$$

这个公式非常重要。它告诉我：

AmountOut 并不是 AmountIn \ SpotPrice

而是一个关于 Trade Size 的 nonlinear function (非线性函数)。

1. 加入 Trading Fee (交易手续费)

现实中的 AMM 通常会从 Amount In 中收取一定比例的 Trading Fee。

假设 Fee Rate = f

那么真正参与 Swap Pricing 的输入是：

$$\Delta y_{\text{effective}} = \Delta y \setminus (1 - f)$$

于是：

$$\left[\Delta x = \sqrt{x \setminus \Delta y \setminus (1 - f)} / \sqrt{y + \Delta y \setminus (1 - f)} \right]$$

因此 Fee 有两层影响：

第一层：TradingFee 直接减少我的有效输入。

第二层：我的实际 Amount Out 进一步改变，因此也影响最终 Execution Price。

1. Spot Price 与 Average Execution Price

这是 Day 2 最重要的区别之一。

Spot / Marginal Price (现货/边际价格)

Swap 前：

$$P_{\text{spot}} = y / x$$

表示非常小的一笔额外交易在当前 State 附近面对的 Price。

Average Execution Price (平均执行价格)

如果我实际投入 Δy 获得 Δx ，那么：

$$\left[P_{\text{avg}} = \Delta y / \Delta x \right]$$

真正套利的时候，我关心的不是 P_{spot} ，而是：

$$\left[P_{\text{avg}}(Q) \right]$$

因为我的利润取决于整个订单的平均成交价格。

1. 什么是 Price Impact ?

Price Impact (价格影响) 是：

我的交易本身对 Market Price 造成的影响。

例如：

Swap 前： $P_{\text{spot}} = 4000$

由于我的 Buy Order 改变 Pool State，Swap 后： $P_{\text{spot}} = 4080$

那么 $4000 \rightarrow 4080$ 就是我的交易导致的 Price Movement。

Trade Size 越大，相对于 Pool Liquidity 越大：

$(Q / \text{Liquidity}) \uparrow$

通常：

Price Impact $\uparrow$

因此可以建立非常重要的直觉：

$$\left[\text{Trade Size} \uparrow \Rightarrow \text{Price Impact} \uparrow \right]$$

而且通常不是简单的 linear relationship (线性关系)。

1. Price Impact 与 Slippage 不完全一样

这两个概念很容易混淆。

Price Impact：

主要描述：自己的订单改变 Pool State 所导致的价格变化。

它来自：TradeSize 相对于 Liquidity 太大。

Slippage (滑点)：

Slippage 更广泛地表示：

【 Expected Execution Price $\neq$ Actual Execution Price 】

(预期执行价格 $\neq$ 实际执行价格)

例如我提交交易时预计：

1 ETH = 4000 USDC

但交易真正进入 Block 之前，其他人先进行了 Swap。于是我实际成交：

1 ETH = 4015 USDC

这也是 Slippage。

因此简单理解：

Price Impact 更偏向：我自己造成的价格移动。

而 Slippage 更偏向：Expected Price 与最终 Actual Execution Price 的差异。

Price Impact 可以成为 Slippage 的来源之一，但二者不应该完全当成同一个概念。

1. Slippage Tolerance 又是什么？

交易界面经常出现：

Slippage Tolerance = 0.5% (滑点容忍度)

它并不是：“我一定会损失 0.5%。”

而是：我允许最终收到的数量最多比 Quote 差多少。

例如当前 Quote：

1 ETH $\rightarrow$ 4000 USDC

Slippage Tolerance：0.5%

那么我可以设置：

Minimum Amount Out = $4000 \times (1 - 0.005) = 3980$ USDC

如果最终 execution 只能得到 3970 USDC，那么交易应当：

Revert (回滚失败)

这实际上是一个 Risk Control (风险控制)。

Slippage Tolerance 太小： $\rightarrow$ Failed Transaction $\uparrow$ (失败率上升)

Slippage Tolerance 太大： $\rightarrow$ Bad Execution / MEV Risk $\uparrow$ (被夹击风险上升)

因此需要权衡。

1. 为什么 Screen Spread 会消失？

假设：

DEX A：ETH = 4000

DEX B：ETH = 4040

屏幕显示：

Spread = $(4040 - 4000) / 4000 = 1\%$ (即 100 bps)

看起来非常赚钱。

但我要交易：Q = 20 ETH

在 DEX A 买 20 ETH 会推高价格。

在 DEX B 卖 20 ETH 又会压低价格。

于是：

Executable Buy Price(Q) > 4000

同时：

Executable Sell Price(Q) < 4040

真正应该计算：

【 Executable Spread(Q) = $P_{\text{sell,avg}}(Q) - P_{\text{buy,avg}}(Q)$ 】

而不是直接算 4040 - 4000。

这也是为什么：

【 Displayed Spread $\neq$ Executable Spread 】

1. Arbitrage 与 Trade Size 的关系

假设套利利润是： $\Pi(Q)$

随着 Q 增加：

Gross Revenue 可能增加：Gross Profit(Q) $\uparrow$

但同时：

Price Impact(Q) $\uparrow$

Slippage(Q) ↑

Trading Fee(Q) ↑

所以利润不会无限增长。

一种典型情况可能是：

\\ Trade Size | Gross Spread Profit | Price Impact + Fee | Net Profit |

\\ \$100 | \$1 | \$0.50 | \$0.50 |

\\ \$1,000 | \$10 | \$3 | \$7 |

\\ \$10,000 | \$100 | \$50 | \$50 |

\\ \$20,000 | \$200 | \$190 | \$10 |

\\ \$30,000 | \$300 | \$350 | -\$50 |

因此存在一个非常重要的问题：

【 $Q^* = \operatorname{argmax}_Q \text{NetProfit}(Q)$ 】

也就是：什么 Trade Size 可以使 Net Profit 最大？

套利研究不是简单问："有没有价差？"

而应该继续问："这个机会的 Capacity (容量) 有多大？"

1. Liquidity 为什么如此重要？

假设两个 Pool 都显示：ETH = 4000

Pool A : 100 ETH + 400,000 USDC

Pool B : 10,000 ETH + 40,000,000 USDC

它们的 Spot Price 完全一样。

但是如果我要买 10 ETH，两者的执行结果会非常不同。

因为相对 Trade Size：

Pool A : $10 / 100 = 10\%$

Pool B : $10 / 10000 = 0.1\%$

因此 Pool A 的 Price Impact 会远远更大。

所以：

【 Price without Liquidity is incomplete information 】

(没有流动性数据的价格是不完整的信息)

以后看到任何套利市场，都不能只记录 Price，还应该记录 Liquidity 和 Depth。

1. Arbitrage 本身为什么会让价格重新收敛？

假设：

DEX A : ETH = 4000

DEX B : ETH = 4040

套利者在 DEX A 买入 (Buy_A) 会导致 $P_A \uparrow$

套利者在 DEX B 卖出 (Sell_B) 会导致 $P_B \downarrow$

因此：

$P_A \uparrow$ 同时 $P_B \downarrow$

最终：

$P_A \approx P_B$

套利者其实在主动消耗 Price Dislocation (价格错位)。

这说明：

【 Arbitrage Opportunity has finite capacity 】

(套利机会容量有限)

你交易得越多：Opportunity Remaining ↓

最终直到：Marginal Profit ≈ 0 (套利空间被吃完)。

1. 一个完整 Arbitrage Example

假设：

Pool A : 100 ETH / 400,000 USDC (所以 $P_A = 4000$)

Pool B : 100 ETH / 404,000 USDC (所以 $P_B = 4040$)

屏幕上：Spread = 40 USDC (即 1%)

现在我不能直接说：1 ETH $\Rightarrow$ \$40 Profit

而应该分别计算：

Leg 1 :

在 Pool A 使用 USDC 买 ETH (Buy ETH) , 得到 AmountOut_A
由此计算 Average Buy Price

Leg 2 :

把 ETH 在 Pool B 卖出 (Sell ETH) , 得到 AmountOut_B
由此计算 Average Sell Price

然后计算 :

Gross Profit = USDC_received - USDC_spent

再减去 Gas 和其他执行成本 :

【 Net Profit = Gross Profit - Gas - Other Costs 】

这才是真正的 Arbitrage Calculation。

1. Day 2 Python Experiment

今天应该建立 : notebooks/amm.ipynb

首先写一个最基础的 Constant Product Swap Function :

```
def amount_out(reserve_in, reserve_out, amount_in, fee):
```

```
    effective_in = amount_in \ (1 - fee)
```

```
    output = (
```

```
        reserve_out \ effective_in
```

```
        / (reserve_in + effective_in)
```

```
    )
```

```
    return output
```

输入 : reserve_in, reserve_out, amount_in, fee

输出 : amount_out

然后继续计算 : spot_price, average_execution_price, price_impact

1. 第二个 Experiment : 改变 Trade Size

固定 Pool : 100 ETH / 400,000 USDC

然后测试 : \$100, \$500, \$1,000, \$5,000, \$10,000, \$25,000, \$50,000

记录 :

\	Amount In	ETH Out	Average Price	Price Impact
---	-----------	---------	---------------	--------------

\	\$100	...	...	...
---	-------	-----	-----	-----

\	\$500	...	...	...
---	-------	-----	-----	-----

\	...	...	...	...
---	-----	-----	-----	-----

最后画图 :

Q 作为 x-axis , PriceImpact(Q) 作为 y-axis。

我要亲眼观察 :

Trade Size ↑ ⇒ Price Impact ↑

1. 第三个 Experiment : 改变 Liquidity

固定 : Trade Size = \$10,000

分别测试 :

Pool 1 : 100 ETH / 400,000 USDC

Pool 2 : 1,000 ETH / 4,000,000 USDC

Pool 3 : 10,000 ETH / 40,000,000 USDC

三者初始 Price 都是 : 4000

但是比较 AmountOut 和 PriceImpact , 应该能够观察到 :

Liquidity ↑ ⇒ Price Impact ↓

这说明 : 同一个 Price 并不代表同一个 Executability。

1. Day 2 最重要的公式回顾

2. Constant Product (恒定乘积公式):

【 $x \setminus y = k$ 】

3. Spot Price (现货价格):

【 $P = y / x$ 】

1. Effective Input (有效输入):

$$\text{AmountIn_effective} = \text{AmountIn} \setminus (1 - f)$$

2. Amount Out (输出数量):

$$\text{AmountOut} = (\text{ReserveOut} \setminus \text{AmountIn_effective}) / (\text{ReserveIn} + \text{AmountIn_effective})$$

3. Average Execution Price (平均执行价格):

$$P_avg = \text{AmountIn} / \text{AmountOut}$$

4. Arbitrage Profit (套利净利润):

$$\text{NetProfit}(Q) = \text{ExecutableSpread}(Q) - \text{Fees}(Q) - \text{Gas} - \text{OtherCosts}$$

5. Optimal Trade Size (最优交易规模):

$$Q = \arg\max_Q \text{NetProfit}(Q)$$

6. Day 2 验收题

Q1: 为什么一个 Pool 显示 ETH = 4000 USDC, 并不意味着可以用 \$400,000 买 100 ETH?

Answer: 因为 4000 只是当前 Pool State 附近的 Marginal Price。大量买入 ETH 会导致 ETH Reserve ↓, USDC Reserve ↑, 使 ETH Price 不断上涨, 因此 Average Execution Price 会高于初始 Spot Price。

Q2: 为什么 Pool Liquidity 越大, 通常 Price Impact 越小?

Answer: 因为同样的 Trade Size 占 Pool Reserve 的比例更小 (TradeSize / Liquidity ↓), 因此交易造成的 Reserve Relative Change 更小, Price Movement 也更小。

Q3: Price Impact 和 Slippage 有什么区别?

Answer: Price Impact 主要指: 自己的交易改变 Pool State 导致的价格变化。Slippage 指: Expected Execution 与 Actual Execution 之间的差异。Price Impact 可以造成 Slippage, 但外部市场变化、其他交易抢先执行等因素也会造成 Slippage。

Q4: 为什么套利利润不能简单按照 Trade Size 线性放大?

Answer: 因为 Trade Size ↑ 同时会导致 Price Impact ↑, 并增加 Trading Fee 等成本。因此 Profit(100Q) ≠ 100 * Profit(Q), 甚至可能出现 Profit(Q) > 0 但 Profit(100Q) < 0。

Q5: 套利研究中为什么 Executable Price 比 Spot Price 更重要?

Answer: 因为真正 PnL 来自实际可以完成的交易 (BuyPrice_actual 和 SellPrice_actual), 而不是交易前屏幕显示的两个价格。因此, 【Executable Spread】才是真正决定套利是否存在的变量。

1. Day 2 总结

Day 1 我学到:

Spread ≠ Profit

Day 2 进一步理解:

$$\text{Displayed Price} \neq \text{Executable Price}$$

原因是:

Trade → Pool State Change → Price Change

因此以后分析 DEX Arbitrage, 必须把 Price 升级成:

$$\text{Price}(Q, \text{Liquidity}, \text{Fee}, \text{State})$$

也就是说: 一个价格离开 Trade Size 和 Liquidity 几乎没有完整意义。

今天建立的研究链条是:

Pool State → Spot Price → Trade Size → Amount Out → Price Impact → Executable Price → Net Profit

最终最重要的问题不再是: "两个 DEX 相差多少?"

而是:

$$\text{在给定的 Trade Size } Q \text{ 下, 两边真正可以成交的 Average Price 相差多少?}$$

这也是接下来使用真实 Quote API 时最重要的基础。

笔记 007: 理解 DEX 价格与流动性

作者: Marco | 时间: 2026-08-08 15:57 UTC | 字数: 709 | 评分: 54

理解 DEX 价格与流动性

第四天的学习重点是理解 DEX 中的价格和流动性，认识“页面上看到的价格”与“实际交易能够获得的价格”之间为什么会存在差异。

在 DEX 中，常见的价格概念至少包括中间价、池子现货价格、交易报价和最终成交价。中间价通常只是买卖两侧价格的参考值；池子现货价格反映当前储备比例；交易报价则根据具体输入金额计算；最终成交价还会受到区块状态、交易排序、滑点保护和执行时机影响。因此，直接使用网页显示的价格判断套利，容易高估真实收益。

流动性是影响实际成交结果的核心因素。同一个交易对，在不同 DEX 或不同池子中可能拥有完全不同的资金规模、手续费档位和价格区间。输入金额较小时，报价可能接近中间价；输入金额变大后，交易会改变池子储备比例，产生更明显的价格影响。金额越大、流动性越低，滑点和价格冲击通常越明显。

进行套利分析时，至少应比较同一交易对在两个池子中的实际报价，并测试小、中、大三种输入金额。同时记录池子流动性、手续费、预计输出、价格影响和报价时间。只有在固定金额和相近时间条件下比较，才能判断价差是否真实存在。

今天的关键知识点是：流动性决定了价格能否被执行，报价金额决定了价格会如何变化。一个看似很大的价差，如果无法承载目标交易规模，或者扣除滑点和手续费后收益转负，就不属于可执行的套利机会。

下一步可以选择一个交易对，比较两个

DEX

或两个流动性池的报价，并建立“输入金额—输出金额—价格影响”的对照表。第五天将进一步学习如何使用

LI.FI

获取和拆解跨链或跨协议路线报价。

今日小结：套利分析不能只看价格，还要同时看池子的深度、交易规模 and 实际成交结果。

笔记 008: 今天上午终于把 Robinhood

这条套利为什么会突然开窗往前追了一层。昨晚群里一直在争一个问题，

作者：fanchen621 | 时间：2026-08-08 17:30 UTC | 字数：1670 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天上午终于把 Robinhood 这条套利为什么会突然开窗往前追了一层。昨晚群里一直在争一个问题，这类 bot 到底是在链上算利润，还是链下先算好，再用 slot0、tick 之类的状态做最后校验。我顺着这个问题继续拆，发现之前只盯池子的 Swap 还是太浅了，真正影响报价的东西藏在 V4 hook 里面。我先翻了 Uniswap v4 的 hooks 源码。V4 会直接用 hook 地址的低 14 位表示回调权限，我们这条路径上的 hook 0x28388c...0088，低位正好是 0x88，对应 beforeSwap 和 beforeSwapReturnDelta。后一个权限允许 hook 在 swap 真正进入池子之前改变实际交换量。这个细节一下把昨天一个怪现象解释通了，套利有时会在目标池完全没有 Swap 的情况下突然从正变负，因为报价逻辑还会吃 hook 自己依赖的外部状态。X 上今天也看到有人在讨论把 arbitrage capture 直接写进 V4 hook。我没有采信里面的项目收益宣传，只拿了一个思路，hook 本身可能就是价格形成和套利分配的一部分。Ethereum Research 最近一篇关于 MEV 来源归因的研究也很有启发，他们在 Polygon 的大样本里发现绝大多数原子套利都能追到一个明确的 source transaction。于是我直接照这个方法做反事实回放，开始找我们的 Robinhood 机会到底是被哪一笔状态变化打开的。今天抓到的新窗口在 block 31553392。用 1.2 ETH 做只读原子模拟，毛收益大约 0.00002726 ETH，把 gasPrice 按 60m wei、gas 再加 20% buffer 后，还剩大约 0.00000644 ETH。数字不大，但至少成本后还是正的。这些都是 shadow 结果，没有签名和广播，我没有把它算成实盘收益。接着我把区块一块块往前切。31551895 时间一条路径还是 -0.00001764 ETH，到了 31551896 突然跳成 +0.00004631 ETH，下一块又扩大到 +0.00016154 ETH。最有意思的是，31551896 里面没有目标 V3、V4 PoolManager 或 hook 的 Swap 日志，这次机会的来源显然不在目标池的一笔直接 Swap 上。我把这一块里的 13 笔交易全部翻出来，发现第 11 笔 0x29bc...c5fbb 调用了 0x3a804e...09429。Phalcon 直接把这笔调用解成了 updatePrice(newPrice)。然后我又回头拆一笔真实成功走过同一路径的交易，Phalcon 一共有 932 个 trace node，里面目标 hook 被标成 BuffetAMM。它在 beforeSwap 过程中会主动调用同一个 0x3a804e...09429 合约读取状态。两边终于接上了，一边有人更新价格，另一边 BuffetAMM 在成交前读取这个价格源，而我们的套利窗口就在价格更新的那个 block 从负翻正。今天还把这个发现写成了一个只读触发器。监控现在收窄到 0x3a804e...09429 的 updatePrice，价格源更新时才立刻重跑 WETH → USDG → BuffetAMM → ETH 的精确模拟和 gas gate。历史那笔更新已经能被触发器准确识别；实时又跑了 91 个新块，这段时间没有出现下一次 update，所以我没有硬凑一个“命中”。这轮让我对机会发现的理解又具体了一点。以后看到一个套利窗口，我会继续往前问是谁让这个窗口出现的。对这条 Robinhood 路线，监控对象已经从大量 Swap 收窄到了真正参与定价的外部价格源。下一步就盯这个价格源每次更新后，edge 能活多少毫秒、什么规模最优、竞争者多久进场，这比继续全链无差别扫池子更接近真正能用的触发器。

笔记 009: UniSwap v3

作者：xsh | 时间：2026-08-08 18:10 UTC | 字数：254 | 评分：51

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

UniSwap v3

Deploy <https://github.com/Uniswap/v3-periphery/blob/main/deloys.md>

合约 : UniswapV3Pool SwapRouter UniswapV3Factory

Arch & SourceCode

Core

Periphery 辅助功能

SDK

SubGraph

Info

Interface

Event

Swap

Mint/Burn

笔记 010: Day 05 : V4 架构与套利系统学习记录

作者 : FireLoveInIce | 时间 : 2026-08-08 19:00 UTC | 字数 : 12662 | 评分 : 90

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 05 : V4 架构与套利系统学习记录

note链接 : <<https://github.com/FireLoveInIce/21DaysCoLearnNote/tree/main/day05>>

今日学习结论

今天的主线不是继续堆叠 V4 接口，而是把 V2、V3、V4 的演进和套利系统的实际分工串起来：

text

V2 Pair：每个币对一个独立合约，使用全范围 $xy=k$

→ V3 Pool：每个币对/费率一个独立合约，引入集中流动性和 Tick

→ V4 PoolId：不再为每个 Pool 部署合约，全部逻辑 Pool 由 PoolManager 管理

V4 删除的不是 Pool 这个经济概念，而是“每个 Pool 必须对应一个独立合约”的部署模式。V4 Pool 仍然有独立的价格、Tick、流动性、Position 和费用状态，只是通过 PoolId 在 Singleton PoolManager 中索引。

套利系统也不是 Python 和 Solidity 二选一：

text

Python：发现 Pool、维护状态、搜索循环、筛选和模拟

Solidity：原子执行、Callback 付款、还款和 minProfit 保护

Uniswap Pool/PoolManager：执行最终权威的 Swap 数学

一、Day04 前置知识回顾

sqrtPriceX96 与 Tick

sqrtPriceX96 是 Pool 当前精确价格的 Q64.96 表示。

Tick 是当前价格所在的离散对数坐标，用于定位流动性区间和搜索边界。

快速比较 Pool 的边际价格应使用 sqrtPriceX96 并处理 Token 顺序和 decimals。

计算指定数量能否套利，不能只看 sqrtPriceX96 或 Tick，必须处理手续费、价格影响、跨 Tick 流动性和 Gas。

text

$\text{rawPrice}(\text{token1}/\text{token0}) = (\text{sqrtPriceX96} / 2^{96})^2$

$\text{tickPrice}(\text{token1}/\text{token0}) = 1.0001^{\text{tick}}$

新 V3/V4 Pool 的 Core 初始化接口必须接收 sqrtPriceX96。普通用户通常只在前端输入或确认人类价格、Token 数量和价格范围，前端/SDK 再计算底层整数；已有 Pool 添加流动性时不重新初始化价格。

当前 Tick 对谁“当前”

当前 Tick 属于某个具体 Pool 在某个区块状态下的价格坐标，不属于某个 LP 或某笔交易：

Swap 从当前 Tick 开始，执行中移动价格，成功后写回新 Tick。

添加/移除流动性读取当前 Tick 判断 Position 是否活跃，通常不改变价格和 Tick。

Quoter 模拟会计算 Swap 后 Tick，但 eth_call 结束后不持久化。

V3 Observation

V3 slot0 中的 observationIndex、observationCardinality 和 observationCardinalityNext 属于 Pool 内置的历史累计值环形缓冲区，用来计算 TWAP。它记录本 Pool 的 tickCumulative 和 secondsPerLiquidityCumulativeX128，不是 Chainlink 等外部喂价。

二、V2 Pair、V3 Pool、V4 PoolId 的演进

维度	V2 Pair	V3 Pool	V4 逻辑 Pool
身份	排序后的 Token 对	Token 对 + fee	完整 PoolKey 的哈希 PoolId
是否独立合约	是	是	否，共用 PoolManager
创建方式	Factory 部署 Pair	Factory 部署 Pool	PoolManager 初始化内部状态
价格状态	reserves	sqrtPriceX96、Tick、L	V3 类状态 + 动态 LP Fee
流动性范围	全范围	集中流动性	集中流动性
Token 托管	Pair 合约	Pool 合约	PoolManager 集中托管
多跳结算	Pair 间转 Token	每个 Pool Callback 付款	unlock 内累计 Delta，最后净额结算
扩展方式	Fork/外围合约	Fork/外围合约	Hooks
原生 ETH	通常使用 WETH	通常使用 WETH	Currency(address(0))

为什么 V2 升级到 V3

V2 流动性分布在价格从 0 到无穷大的全部范围，资金利用率低。V3 允许 LP 选择 tickLower, tickUpper)，让资金集中在更可能成交的价格范围，并允许同一币对存在多个费率 Pool。

代价是引入 sqrtPriceX96、Tick、Tick Bitmap、逐 Tick Swap、NFT Position 和 Callback，报价与集成明显更复杂。

为什么 V3 升级到 V4

V3 仍然为每个 Token 对和费率部署独立 Pool 合约，存在三个主要限制：

1. 每个新 Pool 都要重复部署 Core 字节码。
2. 多跳需要在多个 Pool 合约间转移 Token，并逐池 Callback 结算。
3. 动态费率、限价单、特殊 Oracle 等功能通常需要 Fork Core 或增加复杂外围合约。

V4 通过 Singleton、Flash Accounting、Hooks 和原生 ETH 支持降低创建与多跳成本，并把自定义逻辑从 Core Fork 转为生命周期 Hook。

三、PoolKey 与 PoolId

V4 PoolKey 是逻辑 Pool 的完整配置：

```
solidity
struct PoolKey {
    Currency currency0;
    Currency currency1;
    uint24 fee;
    int24 tickSpacing;
    IHooks hooks;
}
```

PoolId 是 PoolKey 的标准 ABI 编码哈希：

```
text
PoolId = keccak256(abi.encode(PoolKey))
```

关键规则：

currency0 < currency1，按底层地址值排序。

fee、tickSpacing 或 Hook 任一不同，都是不同 Pool。

PoolId 是 bytes32 状态索引，不是合约地址。

实际 swap 仍传完整 PoolKey；PoolManager 从中计算 PoolId，同时使用 Currency、Fee、Tick Spacing 和 Hook。

相同 PoolKey 在不同链或不同 PoolManager 下可能得到相同 PoolId，因此全局唯一键应为：

```
text
chainId + PoolManager + PoolId
```

V4 没有每池独立合约，核心状态概念上保存在：

```
solidity
mapping(PoolId => Pool.State) internal pools;
```

发现 V4 Pool 应监听 PoolManager 的 Initialize 事件，保存 PoolId 和构造 PoolKey 所需的全部字段。

四、V3 与 V4 多跳的区别

假设路径为：

text

USDC → WETH → DAI

V3

text

Router

→ USDC/WETH Pool.swap

→ Callback 支付 USDC

→ WETH/DAI Pool.swap

→ Callback 支付 WETH

→ 得到 DAI

两跳调用两个独立 Pool 合约，每个 Pool 独立持有 Token、更新 Storage 并检查 Callback 付款。

V4

text

Router/执行合约

→ PoolManager.unlock

→ unlockCallback

→ PoolManager.swap(PoolKey A)

→ PoolManager.swap(PoolKey B)

→ settle USDC

→ take DAI

→ 所有 Currency Delta 必须归零

PoolManager 不会自动搜索路径。Router、套利合约或 Python 必须明确决定每一跳并依次调用 swap。

中间 Currency 可以通过会计抵消：

text

第一跳：USDC -1000，WETH +0.5

第二跳：WETH -0.5，DAI +990

净额：USDC -1000，WETH 0，DAI +990

最终只需支付 USDC、提取 DAI，中间 WETH 通常不需要在 Router 和 PoolManager 间重复转移。

五、Unlock、Delta 与 Hooks

Unlock 和 Flash Accounting

text

PoolManager.unlock(data)

→ 调用 msg.sender.unlockCallback(data)

→ Callback 中执行 swap/modifyLiquidity 等操作

→ 累计调用者的 Currency Delta

→ settle 负 Delta，take 正 Delta

→ Callback 返回

→ 非零 Delta 数不为零则整笔交易回滚

| Delta | 含义 | 处理方式 |

| :--- | :----- | :----- |

| > 0 | PoolManager 欠调用者 | take 或继续净额结算 |

| < 0 | 调用者欠 PoolManager | 转入资产并 settle |

| = 0 | 已结清 | 可以结束 unlock |

ERC-20 一般先 sync(currency) 记录余额基准，再转入 Token 并 settle()；原生 ETH 使用 payable settle(value: amount)()。Callback 必须验证：

solidity

```
require(msg.sender == address(poolManager));
```

Hooks

Hook 可以在 Initialize、增减流动性、Swap 和 Donate 前后执行。权限由 Hook 地址低位标志决定，但权限位只说明“哪些入口可能被调用”，不能说明业务一定安全。

Hook 可能：

动态覆盖 LP Fee。

返回额外 Delta，改变最终输入或输出。

根据 hookData、Oracle 或自己的 Storage 改变行为。

调用外部协议、消耗额外 Gas 或 revert。

是代理或依赖可升级合约。

第一版自动执行策略：

text

无 Hook → 可进入 Quoter 和完整模拟

已审查 Hook → 使用真实 hookData 模拟后白名单接入

未知 Hook → 只发现和监控，不自动执行

六、闪电贷与套利原子性

闪电贷资金来自 Aave、Balancer、Uniswap Pool/PoolManager 等协议已有流动性，不是凭空创建。借款、使用、偿还必须发生在同一笔交易；无法偿还本金和费用时整笔交易回滚，但已经执行的计算仍然消耗 Gas。

| 协议 | 闪电能力 |

| :----- | :----- |

| Aave V3 | flashLoan / flashLoanSimple |

| Uniswap V2 | Flash Swap，通过 uniswapV2Call 结算 |

| Uniswap V3 | flash() / uniswapV3FlashCallback |

| Uniswap V4 | unlock 内 take 资产并在结束前 settle Delta |

同链多步套利需要在一笔 EVM 交易中执行，通常由自定义 Solidity 合约负责：

text

请求闪电流动性

→ 执行多个 Swap

→ 偿还本金和费用

→ 检查 $\text{finalBalance} \geq \text{initialBalance} + \text{minProfit}$

→ 不满足则 revert

Python Bot 不能替代链上原子性，它负责发现、模拟、构造和发送交易。合约不能主动发起顶层交易，仍需要 EOA/Bot 触发。CEX-DEX、跨链和延迟赎回不在同一 EVM 状态机中，无法获得相同的全流程原子性。

七、池发现与循环套利系统评估

池发现是必要基础

输入必须包含 chainId + tokenAddress。在已登记的 DEX 部署范围内：

V2：扫描 Factory PairCreated，或已知另一 Token 时调用 getPair。

V3：扫描 Factory PoolCreated，或已知另一 Token 和 fee 时调用 getPool。

V4：扫描 PoolManager Initialize，保存 PoolKey 和 PoolId。

Factory/Router 地址还不够。每个部署还应配置：

text

chainId、协议族、Factory/PoolManager、deploymentBlock

Quoter、StateView、Router、Multicall、wrappedNative

费用规则、ABI 版本和允许的 Hook

Curve、Balancer、DODO 等不是 V2/V3/V4 的简单别名，需要独立 Adapter 和报价数学。

路径搜索适合 Python

按链建立有向图：

text

Token = 节点

Pool 的每个方向 = 一条边

循环示例：

text

USDC → WETH → DAI → USDC

第一版应限制在同链 2~4 跳，禁止重复 Pool，过滤低流动性、危险 Token 和未知 Hook。跨链路径不是原子循环，应独立建模。

报价采用混合方案

Pool 类型	第一版报价策略
标准 V2	Python 使用 reserves、fee 和整数公式精确计算
V3	Python 做边际价和粗筛，候选路径交给 QuoterV2
无 Hook V4	本地粗筛 + V4 Quoter
Hook V4	必须使用真实 PoolKey、hookData 和 Hook 状态执行 Quoter/eth_call
其他 DEX	每个协议单独 Adapter

最终必须对真实套利执行合约的完整 calldata 做：

```
text
eth_call
+ estimateGas
+ minProfit
```

利润口径：

```
text
netProfit
= finalAmount - initialAmount
- flashLoanFee
- gasCost
- builderPayment
- safetyBuffer
```

八、主网套利的现实边界

以太坊主网热门池的简单 DEX-DEX 原子套利已经高度专业化。竞争优势更多来自本地节点、Pending 状态模拟、私有订单流、Builder 连接和竞价，而不只是会写价格公式。

“长尾”指交易量、流动性和关注度较低的大量资产、Pool 或协议。长尾机会可能持续更久、竞争更少，但通常伴随低流动性、转账税、黑名单、恶意 Hook、无法卖出和管理员权限等风险。长尾不等于无风险机会。

当前合理目标是先运行影子模式：记录理论机会、精确模拟利润、下一块存活率、实际竞争者和失败成本，再决定是否投入真实资金和低延迟基础设施。

九、Python 与 Solidity 的职责

工作	Python	Uniswap/DEX 合约	自定义 Solidity 执行器
扫描 Factory/Manager 事件	是	发出事件	否
维护 Token/Pool 图	是	否	否
搜索循环和输入规模	是	否	否
快速边际价格计算	是	提供状态	否
V2 本地报价	是	最终复算	否
V3/V4 Quoter	发起 eth_call	EVM 中执行	否
最终完整模拟	发起 eth_call	被调用	提供执行入口
逐 Tick 最终成交	可预测	是	不重复实现
Callback 鉴权和付款	否	发起 Callback	是，直接调用 Pool 时
闪电贷还款	否	检查还款	是
原子 minProfit	事前估算	否	是
历史交易和事件解析	是	发出事件	否

推荐系统结构：

```
text
Pool Indexer
→ State Reader
→ Quote Adapters
→ Route Graph
→ Opportunity Evaluator
→ Execution Simulator
→ Solidity Executor
```

十、自测问题与答案

1. 为什么 V4 不能只用 PoolManager 地址标识一个 Pool？

一个 PoolManager 同时管理大量逻辑 Pool，所有 Swap 日志也可能由同一 Manager 发出。必须使用 PoolId 区分内部状态；跨链数据库中应使用 chainId + PoolManager + PoolId。

2. PoolKey 中为什么必须包含 Hook 和 tickSpacing？

Tick Spacing 决定可初始化边界和流动性网格，Hook 决定生命周期扩展、动态费率和额外 Delta。相同 Currency 和 fee、但 Tick Spacing 或 Hook 不同的市场，执行行为和状态都不同，必须是不同 Pool。

3. PoolId 如何计算，为什么执行 Swap 时仍需完整 PoolKey？

text

```
PoolId = keccak256(abi.encode(PoolKey))
```

PoolId 只适合索引状态。执行时还需要 PoolKey 中的 Currency、fee、tickSpacing 和 Hook 来完成结算、逐 Tick Swap 和 Hook 调用，所以 PoolManager.swap 接收完整 PoolKey。

4. unlock 结束前为什么所有 Currency Delta 必须归零？

unlock 允许调用者暂时欠 PoolManager 资产或拥有应收资产。如果允许非零 Delta 离开交易，就会形成无抵押坏账或未领取权益。归零检查使 Flash Accounting 只在本次原子交易内有效。

5. 正 Delta 和负 Delta 分别代表谁欠谁？

以调用者为观察主体：正 Delta 表示 PoolManager 欠调用者，可 take；负 Delta 表示调用者欠 PoolManager，必须转入资产并 settle。

6. ERC-20 结算为什么通常要先 sync，原生 ETH 有何不同？

ERC-20 转账不会把到账数量作为函数返回给 PoolManager，sync 先保存余额基准，转账后 settle 才能用余额差确认到账。原生 ETH 可以随 payable settle{value: amount} 一起发送，因此不走相同的 ERC-20 基准流程。

7. V3 和 V4 的 amountSpecified 符号语义有什么差异？

| 入口 | Exact Input | Exact Output |

| :----- | :----- | :----- |

| V3 Pool.swap | 正数 | 负数 |

| V4 PoolManager.swap | 负数 | 正数 |

方向 zeroForOne 的含义不变，但共用执行代码时必须显式隔离数量符号。

8. Hook 地址权限位说明什么，为什么仍然需要阅读 Hook 代码？

权限位只说明 PoolManager 会调用哪些 Hook 生命周期函数，以及是否允许返回 Delta；它不说明费率算法、外部依赖、管理员权限、代理升级、重入行为和失败条件。自动执行前必须审查实际代码和状态依赖。

9. 为什么带 Hook Pool 的报价必须使用真实 hookData 执行 Hook？

Hook 可能根据 hookData 选择费率、用户规则或自定义 Delta。使用空数据或不同数据得到的报价不一定对应最终交易，可信报价必须复现相同 PoolKey、调用者、hookData 和区块状态。

10. 为什么 Swap 事件可能不足以解释 Hook Pool 的最终资产变化？

PoolManager 的 Swap 事件主要描述基础 Pool 变化和 Swap 后状态。Hook 可能返回额外 Delta、转移 Token 或执行外部调用，所以最终调用者余额还要结合 Hook 事件、trace、Router 返回值和实际 Token Transfer。

11. StateView、Hook 状态和 Quoter 为什么必须固定在同一 blockTag？

如果价格来自区块 N、流动性来自 N+1、Hook 费率来自 N+2，就会构造出链上从未同时存在的虚假状态和利润。相同 blockTag 保证所有输入属于同一状态快照。

12. 未知 Hook 为什么不能直接进入自动套利执行器？

未知 Hook 可能改变费率和输入输出、依赖可操纵状态、恶意转账、升级、消耗异常 Gas 或直接 revert。完整模拟只能证明某个区块状态下成功，不能替代代码审查和白名单边界。

十一、今日总结

1. V4 仍然有 Pool，但它是 PoolManager 内部由 PoolId 索引的逻辑状态，而不是独立合约。

2. V3 多跳跨多个 Pool 合约逐池结算；V4 在一次 unlock 中修改多个 PoolId 状态，并用 Currency Delta 抵消中间资产。

3. PoolManager 不负责寻找路径，路径搜索仍由 Router、执行合约或 Python 完成。

4. 闪电贷和原子执行必须依赖同一笔 EVM 交易；失败时状态回滚，但 Gas 不返还。

5. Python 适合做 Pool 索引、图搜索、报价筛选和 eth_call 编排；Solidity 负责 Callback、还款和最终 minProfit。

6. 第一版系统应先支持单链 Uniswap V2/V3：V2 本地精确报价，V3 使用 Quoter；稳定后再接 V4 无 Hook Pool、白名单 Hook 和其他 DEX。

7. 主网热门池简单套利竞争激烈，学习目标应先是准确发现、复现和模拟，而不是直接假设可以稳定盈利。

官方源码与资料

[Uniswap V2 Core (<https://github.com/Uniswap/v2-core>)
Uniswap V3 Core (<https://github.com/Uniswap/v3-core>)
Uniswap V3 Periphery (<https://github.com/Uniswap/v3-periphery>)
Uniswap V4 Core (<https://github.com/Uniswap/v4-core>)
PoolManager (<https://github.com/Uniswap/v4-core/blob/main/src/PoolManager.sol>)
IPoolManager (<https://github.com/Uniswap/v4-core/blob/main/src/interfaces/IPoolManager.sol>)
PoolKey (<https://github.com/Uniswap/v4-core/blob/main/src/types/PoolKey.sol>)
PoolId (<https://github.com/Uniswap/v4-core/blob/main/src/types/PoolId.sol>)
Hooks (<https://github.com/Uniswap/v4-core/blob/main/src/libraries/Hooks.sol>)
StateLibrary (<https://github.com/Uniswap/v4-core/blob/main/src/libraries/StateLibrary.sol>)
Aave V3 Flash Loan Receiver
(<https://github.com/aave-dao/aave-v3-origins/blob/main/src/contracts/misc/flashloan/interfaces/IFlashLoanSimpleReceiver.sol>)

笔记 011: 2026-08-09 打卡 | 第一次把 LI.FI Quote 接进保守门槛\

作者: mirrornight2026 | 时间: 2026-08-09 03:58 UTC | 字数: 1361 | 评分: 79
来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-09 打卡 | 第一次把 LI.FI Quote 接进保守门槛\

\

今天没有继续停留在静态 AMM 公式里,而是第一次实际请求了 LI.FI 的只读 API,把前两天学的 Quote、最低到账、Gas 和成本门槛接到了真实返回值上。

\

我最开始会下意识去看 `\toAmount\`:只要这个预计到账量高于门槛,就觉得路线可以接受。实际应该在执行前用 `\toAmountMin\` 做保守判断,成交后再看真实 Fill。`\toAmount\` 和 `\toAmountMin\` 都是目标 Token 的数量,不是时间;时间是单独的 `\executionDuration\` 和报价采集时间。

\

第一条真实报价是 BSC 上 `\10 USDT → USDC\`,允许滑点 0.3%。LI.FI 返回:

\

- \- `\toAmount ≈ 9.969217 USDC`
- \- `\toAmountMin ≈ 9.939309 USDC`
- \- LI.FI 平台费约 0.025 USDT\
- \- 预计 Gas 约 0.0166 美元,以 BNB 支付\

\

这笔 LI.FI 平台费已经体现在路线和最终报价里,不能在利润计算时再扣一次;Gas 则属于外部成本,需要另外核算。随后扫描了 10、100、1,000 和 3,000 USDT 四个金额。本批报价里的 LI.FI 费用都是投入的 0.25%,但 Gas 基本保持在 0.0166–0.0169 美元。它再次说明:按金额变化的平台费和近似固定的执行 Gas 不能混成一项。

\

为了不拿两种稳定币的美元标价硬凑利润,我又做了一个只读闭环:

\

`\100 USDT → USDC → USDT`

\

第二腿严格使用第一腿的 `\toAmountMin\` 作为输入。两腿结束后的最低最终到账只有约 99.136284 USDT,扣 Gas 前已经比本金少约 0.863716 USDT,两腿预计 Gas 还要约 0.0435 美元,因此直接判定为 `\REJECT\`。这只是顺序请求的两条 Quote,不是原子成交,也没有虚构实际 Fill。

\

最后比较了 Single-step Quote 和 Advanced Routes。前者直接选出 nordstern;后者返回 nordstern、fly、bitget、okx、1inch 和 sushiswap 六条候选路线。Sushiswap 的 Gas 比 nordstern 少约 0.0015 美元,但最低到账少约 0.108346 USDC,综合结果反而更差。我现在的判断是:选择 LI.FI 路线不能只看 Gas,因为 Gas 节省了,不一定到账量会变多。

\

今天全程只读,没有连接用户钱包,没有授权、签名或广播,也没有调用 `\steps/transaction\`; Single-step Quote 返回的交易请求和 `calldata` 已经丢弃。下一步准备把 Quote 和 Route 的链、Token、decimals、时间、步骤、included/external/unknown 成本统一成可复算的数据结构,再考虑建立可复用采集器。

笔记 012: Day 4 打卡 | 鏈上套利殘酷共學

作者: roy328line | 时间: 2026-08-09 06:31 UTC | 字数: 761 | 评分: 69

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 打卡 | 鏈上套利殘酷共學

今日實作

新增 tycho_scanner.py (全池掃描版)

- 移除 USDC/WETH 限制, 接受所有幣對
- PoolRegistry 改用 (token_a, token_b) 分組, 同幣對跨 DEX 才互比
- 新增 exchange : uniswap_v2 + sushiswap_v2 + uniswap_v3
- 預填 ~30 個主流 token decimals, 避免 RPC 查詢

新增 tri_scanner.py (三角套利掃描器)

- 建構 TokenGraph : token 鄰接池列表
- 枚舉所有三角路徑 $A \rightarrow B \rightarrow C \rightarrow A$, 去重後呼叫 optimal_size_tri()
- 目前限制 : 只跑 v2-style AMM, v3 集中流動性 Day 5 擴展

三角路徑枚舉邏輯

對每個 token_a :

找所有 pool_ab $\rightarrow$ token_b

找所有 pool_bc $\rightarrow$ token_c ($c \neq a$)

找 pool_ca 使 $c \rightarrow a$ 閉環

去重 (相同三池不同起始 token 只算一次)

掃描結論

- tycho_scanner.py (全池兩池) : net_star > gas = 0 次
- tri_scanner.py (三角套利) : net_star_usd > \$5.5 = 0 次

根本原因 : v2 AMM 三池合計費率約 90 bps (三個 0.3%) , 吃掉所有邊際 spread。

明日待辦 (Day 5)

- tri_scanner.py 加入 v3 低費率池 (5 / 30 bps)
- v3 集中流動性的 price_impact 計算
- TOKEN_USD_PRICE 改成動態抓取 (現在寫死)

笔记 013: 今日工作总结

作者 : wangche | 时间 : 2026-08-09 08:42 UTC | 字数 : 535 | 评分 : 64

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日工作总结

阅读并核对了 Uniswap、PancakeSwap 官方文档。

确认 Uniswap 支持 BSC 和 Robinhood Chain , PancakeSwap 先按计划只监控 BSC。

配置了 uniswap_api_key 和 the_graph_api_key。

新增本地 DEX 监控 SQLite 数据库 : data/uniswap.sqlite。

设计并实现池子快照字段 : 交易量、价格、TVL、流动性、排名、抓取时间和数据来源。

增加 Uniswap/PancakeSwap 的 DEX 区分和查询接口。

增加数据库迁移和线程安全处理。

重启并验证本地项目 : <http://127.0.0.1:8767/>。

完整测试通过。

待完成

接入 Uniswap Robinhood Chain 的 Subgraph endpoint。

实现 Uniswap/PancakeSwap 每小时 Top 10 采集器。

增加源数据时效判断, 避免使用过期交易量。

把实际采集结果写入数据库并展示到页面。

今天完成的是数据存储和接口底座, 尚未进行任何真实交易或资金操作。

笔记 014: 今天研究了价格差的来源。

作者 : wang wanlu | 时间 : 2026-08-09 09:40 UTC | 字数 : 396 | 评分 : 60

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

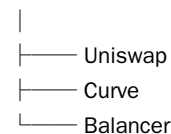
今天研究了价格差的来源。

链上资产并不存在一个绝对统一的价格。

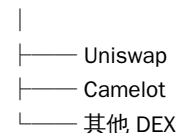
不同 DEX、不同链、不同流动性池实际上都在形成自己的局部价格。

例如：

Ethereum



Arbitrum



每个市场都有自己的：

流动性
交易深度
LP
用户
交易顺序
Gas 环境

因此价格会短暂偏离。

尤其是在：

大额交易
新币上线
市场剧烈波动
跨链资金迁移
流动性突然撤出

的时候，更容易出现局部价格失衡。

所以套利机会本质上可以理解成：

不同市场之间的价格发现速度不一致。

笔记 015: 1/6

作者：alex | 时间：2026-08-09 11:00 UTC | 字数：635 | 评分：63

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1/6

整理了一个最近完善的链上套利执行合约，主要面向新池启动时刻。

它不负责预测价格，而是在机会出现后，将多段兑换、资产结算和收益校验放进一笔原子交易中完成。

2/6

整个项目最重要的原则不是“尽量成交”，而是“只在确定盈利时成交”。

交易执行完成后，合约会统一检查最终资产和最低收益。如果任意步骤失败或收益不足，整笔交易直接回滚，不留下中间状态。

3/6

执行层兼容多种主流 AMM 池型，可以在一条路径中组合不同协议和不同版本。

链下程序负责发现机会、计算路径和评估收益；链上合约只负责严格执行与验证。把复杂计算放在链下，可以减少链上和执行延迟。

Gas

4/6

针对新池启动场景，系统重点处理了：

启动时间与区块状态变化
单边流动性和初始价格状态
多池连续兑换

原生资产与包装资产转换
滑点、Gas 和最低利润保护
任意步骤失败时的原子回滚

5/6

这个项目让我更明确了一点：

套利系统真正困难的部分，通常不是找到一个价格差，而是保证链下看到的机会，在交易真正进入区块时仍然成立。

报价只是候选答案，最终必须由链上状态和收益不变量确认。

6/6

目前整体架构已经完成，下一步主要优化：

更快的区块状态更新
更准确的执行前模拟
更稳定的交易提交
极端流动性状态测试
更低的链上执行成本

笔记 016: Day 5 - 订单簿、报价与区块状态

作者：MountainE | 时间：2026-08-09 11:20 UTC | 字数：476 | 评分：55

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 - 订单簿、报价与区块状态

今天对比了订单簿和

AMM

的成交机制。订单簿通过买卖委托形成买卖盘和深度，成交通常受价格优先、时间优先和可用委托影响；AMM 则根据储备量和曲线直接计算出成交价格。

对套利执行来说，“看到报价”不等于“可以交易”。报价需要同时标记来源、时间、交易规模、有效期、滑点、手续费和 Gas。

我记录了区块状态对执行的影响：

- 在待确认时，报价和流动性可能已经变化。
- 新区块可能包含其他交易，从而改变 AMM 储备和可执行价格。
- 报价过期、区块重组或交易失败都可能让预期收益消失。

今日产出：完成了“报价出现 → 验证 → 执行”流程图笔记：

1. 报价出现：记录交易对、时间和来源。
2. 验证：重新查询深度、滑点、报价有效期、区块状态和全部成本。
3. 执行：只在净收益仍为正且风险可接受时执行，并记录实际结果。

今日结论：套利系统必须同时关注价格、深度和区块状态，并把报价当作短暂且需验证的信号。

明日计划：把 Gas、协议费、桥费、滑点、延迟、失败交易和资金占用统一放进净收益模型。

笔记 017: 今日研究主题 今天主要研究 AMM、流动性深度和滑点之间的关系。

作者：fishCookie | 时间：2026-08-09 13:01 UTC | 字数：1116 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日研究主题 今天主要研究 AMM、流动性深度和滑点之间的关系。

我想弄清楚一个问题：> 如果两个 DEX 的同一资产价格不同，为什么不能直接在低价平台买入，再到高价平台卖出？

一、AMM 的价格不是固定报价

在传统订单簿中，交易者会看到不同价格上的买单和卖单。而很多 DEX 使用 AMM 管理流动性。以常见的恒定乘积模型为例，池子中的两种资产需要大致满足： $x \times y = k$ 其中：
- x 是池子中资产 A 的数量；
- y 是池子中资产 B 的数量；
- k 是相对稳定的乘积。当用户从池子里买走资产 A 时，资产 A 的储备减少，资产 B 的储备增加，池内价格也会随之改变。因此，DEX 页面显示的价格通常只是当前状态下的边际价格，并不代表整笔交易都能按照这个价格成交。

二、什么是价格影响

假设我看到某个 DEX 上的 ETH 比另一个 DEX 便宜。如果我只买入很小的数量，对池子储备的影响可能不明显；但如果交易金额较大，就会改变池子中两种资产的比例，后续部分会以更差的价格成交。这就是价格影响。影响价格变化的主要因素包括：

交易金额； - 池子流动性； - 当前资产储备比例； - 池子的手续费等级； - 是否经过多跳路由。
同样一笔交易，在深度较大的池子中可能只有很小的价格影响，在深度较小的池子中却可能直接吃掉全部价差。

三、滑点与价格影响的区别

今天我进一步区分了两个容易混淆的概念。 ### 价格影响 价格影响是自己的交易对池内价格造成的改变。
交易金额越大、池子越浅，价格影响通常越明显。 ### 滑点 滑点是预期成交结果与实际成交结果之间的偏差。
除了自己的交易造成的价格影响，滑点还可能来自： - 报价后市场价格发生变化； - 其他交易在自己之前成交； -
区块拥堵导致执行延迟； - 路由发生变化； - MEV 搜索者抢先交易； - 跨链期间目标链价格改变。
所以，即使获取报价时显示盈利，真正成交时也可能变成亏损。

四、为什么交易金额会改变套利结果

套利机会不能脱离交易金额进行讨论。 一个价差可能在小金额时能够盈利，但利润不足以覆盖
Gas；增大交易金额后，虽然理论毛利润增加，价格影响和滑点也可能迅速扩大。 因此需要寻找的不是“最大价差”，而是： >
在考虑流动性、手续费、Gas 和风险之后，预期净收益最高的交易规模。
后续分析报价时，我会至少测试几个不同的输入规模，而不是只测试一个金额。

笔记 018: 链上套利学习 Day 5 笔记

作者：Harper | 时间：2026-08-09 13:32 UTC | 字数：5120 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习 Day 5 笔记

今日主题

今天开始从「知道链上数据从哪里来」进入：

如何真正查询 DEX 数据，以及套利监控工具需要哪些数据。

今天学习的核心概念：

GraphQL

Pool

Token

Swap

TVL

Volume

Contract Address

Decimals

The Graph 如何把这些数据提供给程序

一、前几天的知识路线

目前的学习路线：

Day 1：

DEX / 智能合约 / 流动性池 / AMM

↓

Day 2：

滑点 / 流动性深度 / Gas / MEV

↓

Day 3：

链上数据 / RPC / API

↓

Day 4：

The Graph / Event / Subgraph

↓

Day 5：

GraphQL / Pool / Token / Swap / TVL / Volume

二、什么是 GraphQL？

The Graph 已经把大量链上数据整理好了。

但是程序需要告诉 The Graph :

我具体想要什么数据？

这时候可以使用：

GraphQL

简单理解：

GraphQL 是一种向 API 查询自己需要的数据的方式。

可以把它理解成「点菜」。

数据库里可能有：

Pool

Token

Swap

Transaction

Liquidity

Volume

Fee

但如果我只需要：

Pool 地址

TVL

Volume

就只查询这些字段。

三、GraphQL 查询示例

例如：

```
{
  pools(first: 5) {
    id
    totalValueLockedUSD
    volumeUSD
  }
}
```

暂时不需要会写代码，只需要能看懂。

翻译：

pools

\= 查询池子

first: 5

\= 给我前 5 个池子

id

\= 池子的 ID / 地址

totalValueLockedUSD

\= 池子的 TVL

volumeUSD

\= 池子的交易量

整段查询的意思：

给我 5 个池子，并告诉我它们的地址、TVL 和交易量。

四、GraphQL 和 API 的关系

API：

程序获取数据的接口。

GraphQL：

设计和查询 API 的一种方式。

所以 GraphQL 和 API 不是完全对立的两个东西。

GraphQL API 本身就是 API 的一种形式。

普通 REST API

可能请求：

GET /pool/ETH-USDC

服务器返回：

Price

TVL

Volume

Token

Fee

创建时间

交易数量

其他信息

即使我只需要其中两个数据，也可能全部返回。

GraphQL

可以主动指定：

我只需要：

Price

TVL

服务器就只返回需要的数据。

五、什么是 Pool？

Pool：

流动性池。

例如：

ETH / USDC Pool。

从数据角度看，一个 Pool 可能包含：

Pool

├── id

├── token0

├── token1

├── liquidity

├── TVL

├── volume

├── fee

└── price

例如：

ETH / USDC

Pool Address:

0x123...

TVL:

\$20,000,000

24h Volume:

\$5,000,000

Fee:

0.3%

这些数据以后都会成为套利监控工具的重要输入。

六、什么是 TVL？

TVL :

Total Value Locked

中文 :

总锁仓价值。

简单理解 :

这个流动性池里面总共有多少钱。

例如 :

Pool A :

TVL:

\$50,000,000

Pool B :

TVL:

\$2,000

说明 :

Pool A 的资金规模远大于 Pool B。

七、为什么套利机器人关心 TVL ?

因为 :

有价格，不代表这个价格可以承载我的交易。

例如 :

Pool A :

ETH = 3000 USDC

TVL = \$50,000,000

Pool B :

ETH = 3300 USDC

TVL = \$2,000

表面价差 :

10%

看起来是巨大的套利机会。

但 Pool B :

只有约 \$2,000 流动性。

如果投入几千美元 :

交易本身就可能大幅改变价格。

所以 :

Price 必须和 Liquidity / TVL 一起看。

不能只看价格。

八、什么是 Volume ?

Volume :

一段时间内实际发生的交易金额。

例如 :

24h Volume:

\$5,000,000

意思 :

过去 24 小时，这个池子发生了大约 500 万美元的交易。

Volume 可以帮助判断 :

这个池子 :

是否活跃

是否有人交易

当前价格是否具有市场参考价值

九、TVL 和 Volume 的区别

TVL :

池子里面现在有多少钱。

Volume :

一段时间内有多少钱发生了交易。

例如 :

TVL:

\$10,000,000

24h Volume:

\$4,000,000

表示 :

池子里锁着约 1000 万美元。

过去 24 小时发生约 400 万美元交易。

两者不是一个概念。

十、什么是 Token 数据 ?

套利程序不能只记录 :

ETH

USDC

还需要记录 :

Token

Name

Symbol

Contract Address

Decimals

例如 :

Name:

USD Coin

Symbol:

USDC

Contract Address:

0x...

Decimals:

6

十一、为什么 Contract Address 很重要 ?

因为 :

Token 名字和 Symbol 可以重复甚至伪造。

例如 :

两个 Token 都可能叫 :

USDC

但是 :

Contract Address 不同。

它们可能完全不是同一个资产。

所以套利机器人不能判断 :

名字一样

\=

同一个Token

应该判断：

Chain

\+

Contract Address

十二、什么是 Decimals？

链上 Token 通常使用整数记录数量。

Decimals 告诉程序：

这个整数应该移动多少位小数点，才能变成人类看到的 Token 数量。

例如：

USDC 常见：

Decimals = 6

ETH 常见表示：

Decimals = 18

以后程序读取链上 Token 数量时，需要根据 Decimals 转换。

这个概念后面继续深入。

十三、什么是 Swap？

Swap：

一次真实的 Token 兑换。

例如：

10000 USDC

↓

Uniswap

↓

3.3 ETH

就是一次 Swap。

Subgraph 可以记录：

Swap

|—— 时间

|—— Pool

|—— Token In

|—— Token Out

|—— Amount In

|—— Amount Out

|—— Transaction

十四、为什么 Swap 数据对套利重要？

因为：

Swap 是真实发生的交易。

通过历史 Swap，可以研究：

某个池子真实成交情况

价格什么时候发生变化

某个价差出现后发生了什么

一段时间的成交数量

历史套利机会是否真的存在

所以后面做：

历史分析

回测

Paper Trading

都会使用 Swap 数据。

十五、The Graph 如何把数据交给程序？

完整流程：

区块链

↓

智能合约产生 Event

↓

The Graph 监听 Event

↓

Subgraph 按规则整理数据

↓

建立索引

↓

GraphQL 查询

↓

返回 Pool / Token / Swap 数据

↓

套利监控程序分析

十六、套利监控工具的数据结构

以后我的监控工具至少需要：

Pool 数据

Pool Address

Token0

Token1

Price

Liquidity

TVL

Volume

Fee

Token 数据

Name

Symbol

Contract Address

Decimals

Chain

Swap 数据

Timestamp

Pool

Token In

Token Out

Amount In

Amount Out

Transaction

十七、一个套利机会应该怎么看？

假设：

Pool A：

ETH价格：

3000 USDC

TVL：

\$10,000,000

24h Volume：

\$4,000,000

Pool B：

ETH价格：

3300 USDC

TVL：

\$1,500

24h Volume：

\$80

表面价差：

10%

不能马上认为：

「发现巨大套利机会。」

因为 Pool B：

TVL只有 \$1,500

24h交易量只有 \$80

说明：

流动性非常低

交易非常不活跃

很小的交易就可能造成巨大价格变化

3300 USDC 的价格未必具有真实可执行性

所以应该首先标记：

可疑价格

然后进一步模拟真实交易。

十八、目前的套利判断逻辑

第一阶段：

发现价格差

↓

第二阶段：

确认 Chain + Contract Address

↓

第三阶段：

检查 TVL / Liquidity

↓

第四阶段：

检查 Volume

↓

第五阶段：

模拟真实 Swap

↓

第六阶段：

计算滑点

\+

DEX Fee

\+

Gas

↓

第七阶段：

计算预计净利润

↓

最后：

满足条件

↓

套利监控工具发送提醒

十九、今日核心概念表

概念

一句话理解

GraphQL

向 API 指定并查询自己需要的数据

Pool

两种资产组成的流动性池

TVL

池子当前锁定的总资产价值

Volume

一段时间内发生的交易金额

Token

链上的代币资产

Contract Address

判断 Token 身份的重要标识

Decimals

链上整数转换成人类可读 Token 数量的精度规则

Swap

一次真实的 Token 兑换

二十、今天最大的认知升级

以前：

DEX A价格

vs

DEX B价格

现在：

Price

\+

TVL

\+

Volume

\+

Token Contract

\+

Pool

\+

Swap

一起判断。

所以：

一个看起来非常大的价格差，反而可能意味着这个池子的流动性极差、数据异常或者价格根本不可执行。

套利监控工具的任务不是寻找「最大的价差」。

而是寻找：

真实、可成交、扣除成本后仍然存在的价差。

Day 5 一句话总结

The Graph 通过 Subgraph 整理链上数据，我们可以通过 GraphQL 查询 Pool、Token、Swap、TVL、Volume 等信息。套利监控不能只比较价格，而需要同时确认资产身份、流动性、交易活跃度和真实成交能力，再计算最终净利润。

笔记 019: 今日学习心得：从批量数据查询到用 AI 建立验证流程

作者：Chic | 时间：2026-08-09 13:39 UTC | 字数：1095 | 评分：59

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习心得：从批量数据查询到用 AI 建立验证流程

今天的学习重点是继续推进工具层内容，主要围绕 The Graph、池子价格变化、Backrun 识别，以及如何用 AI/Agent 辅助套利研究。

首先学习了 The Graph 的基础概念。之前更多是用 Etherscan 一笔一笔看交易，而 The Graph 更适合批量查询链上历史数据。我的理解是：Etherscan 适合回答“这笔交易发生了什么”，The Graph 适合回答“过去一段时间这类交易反复发生了什么”。其中 Subgraph 可以理解为某个协议整理好的链上数据集，GraphQL 是向这个数据集提问的语言。

接着学习了如何读一个简单的 swaps 查询。比如通过 `swaps(first: 10, orderBy: timestamp, orderDirection: desc)` 可以查询最近 10 笔 Swap，并返回 `id`、`sender`、`recipient`、`amount0`、`amount1` 等字段。这里我理解到，`amount0` 和 `amount1` 不是简单的买入卖出价格，而是池子里两种 Token 的数量变化，可以用来观察一次 Swap 对池子状态造成的影响。

然后学习了如何用 The Graph 看池子价格变化。AMM 池子的价格由池子里两种资产的比例决定，大额 Swap 会改变池子资产比例，从而造成价格冲击。如果某一笔大额 Swap 之后，马上出现反向交易并把价格拉回，就可能是 Backrun。这里我进一步理解了 Backrun：它不是抢在别人前面，而是跟在大额交易后面，利用大额交易造成的价格偏差获利。

今天还学习了 AI/Agent 在套利研究中的正确用法。AI 不应该被当成“喊单工具”或“预测赚钱机会的工具”，而更适合作为研究助理，用来拆交易、列验证清单、补充成本项、检查风险和生成观察表备注。一个模糊问题，比如“两个链上 ETH 价格不一样是不是套利机会”，应该改写成结构化问题，补充源链、目标链、资产、投入金额、`toAmount`、`toAmountMin`、Gas、桥费、执行时间、Route 等字段，再让 AI 帮助计算净收益和判断安全边际。

今天最大的收获是：我开始把“机会判断”转化为“数据验证流程”。看到价格差异时，不再直接问能不能做，而是先问需要哪些数据、缺哪些成本、保守情况下是否仍有净收益、是否存在滑点、延迟、MEV、失败交易和桥风险。后续学习中，我希望继续练习完整案例复盘，把单笔交易、跨链 Quote 或疑似 Backrun 案例整理成结构化研究笔记。

笔记 020: 今日行动

作者：xsl | 时间：2026-08-09 13:49 UTC | 字数：1062 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日行动

今天复盘了已有的双腿套利研究框架，重点检查一个经常被忽略的问题：两种价格相关的资产，是否真的存在可以迫使价差收敛的经济机制。

我对照阅读了两组 Hynix 资产识别案例：

Binance SKHYNIXUSDT 与 Hyperliquid xyz:SKHY

Hyperliquid xyz:SKHY 与 Binance SKHYUSDT

复盘时依次检查资产权利、单位换算、交割合约、定价来源、收敛机制和可证伪条件，而不是看到同一发行人或相似就直接认定存在套利。

ticker

证据与结果

第一组分别参考海力士普通股和 ADS。虽然 $10 \text{ ADS} = 1 \text{ ordinary}$ ，两者也属于同一发行人的权益暴露，但两条 perpetual 都是现金结算合约，彼此不能交割或转换，因此只能归类为 relative value，不能称为严格套利。

已有历史回放也支持这个边界：

归一化溢价范围为 4.03% 至 41.31%

30% 溢价入场、持有 8 小时的收敛策略收益为 -4.45%

validation 最终选择的是溢价扩张方向

holdout 虽为 +0.78%，但只有 2 笔交易，不足以证明正期望

第二组虽然都参考同一个 ADS，但仍是不同平台的现金结算合约。历史溢价范围只有 -0.318% 至 +0.951%，validation 没有产生交易，holdout 为 -0.150%，因此不具备前向开仓资格。

真实公开 L1 smoke test 中，两组都没有出现新的合格阈值穿越，最终为 0 paper fills。

失败与结论

今天修正的核心认识是：同一发行人、确定换算比例或相似 ticker，都不能单独证明价差一定收敛。

只有存在可执行的转移、赎回、到期结算、共同 oracle 或其他强制收敛机制，屏幕价差才可能接近严格套利。缺少这些机制时，本质上是在交易相关性和市场行为，价差完全可能继续扩大。

所以，研究套利候选时，第一步不是计算收益，而是先证明两腿的资产身份和收敛机制。

下一步

1. 把资产身份、单位换算、结算方式、收敛机制和证伪条件固化为候选机会的前置检查。
2. 从 DEX 或跨链场景中选择一个存在明确赎回或结算闭环的候选。
3. 先用小窗口公开数据验证价差能否覆盖手续费、Gas、滑点、失败腿和资金占用成本，再决定是否进入 Paper Trading。

笔记 021: Day 5 | 2026.08.09 | Solana 通用 2-4 跳套利: executor v

作者：xzone911 | 时间：2026-08-09 13:55 UTC | 字数：1731 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 | 2026.08.09 | Solana 通用 2-4 跳套利: executor v3 主网升级

今日目标

把 Solana StonkFun 套利从固定三角路线扩展为通用 2-4 跳闭环系统。executor 主网原地升级至 v3，新增四类 protocol ABI，搜索引擎改为增量触发 + 按 hop 分 lane。

Executor v3: 七类 protocol ABI

在原 Program ID 上原地升级，不换地址：

- 新增 4 类: Raydium AMM v4、Raydium CPMM、Pump AMM、Pump bonding curve
- 现有 3 类: Orca Whirlpool、Raydium CLMM、Meteora DAMM v2/DLMM
- 链上 ProgramData 核对通过, 108,232 bytes
- 不维护 pool-address allowlist，靠 program owner / discriminator / mint / vault / PDA 做结构校验
- 缺失 ATA、WSOL 补款、SyncNative 均在 executor 内原子完成。任一 swap、逐跳最低输出、slot deadline、最终利润或残余 token 检查失败时全部回滚。

搜索引擎重构

增量触发替代全量轮询

- PairKey → route index 反向索引: 账户变化只重算受影响路线
- 不可变 StateFrame: 同一路线多跳共用同一 revision, 不混 slot
- 轮转抽样 + 定时全量审计: 增量漏报为 0

六个搜索组件

| 组件 | 做法 |

|-----|-----|

| 路由模板 | 9 条 2-4 跳闭环: 1 条跨 venue 两跳 + 4 条三跳 + 4 条四跳 |

| 增量选择 | changed pair → 反向索引 → 只报价受影响模板 |

| 状态帧 | 同一 revision 下所有 hop 用同一 processed snapshot |

| 公共腿缓存 | (revision, pair, direction, amount) 不复算 |

| stale 取消 | 新 revision 到达时取消旧 state/build/optimize/simulate/submit |

| 分 lane | state + fast-two + USDC-two + three + four, 各维护独立 revision queue |

三个踩坑

1. 并行 tick 加载炸 RPC

公共 RPC ~10 calls/s, 48 个 V3 池并行 → 全部 403。改为串行逐池 + 按 spacing 自适应缩小范围(spacing=10 → ±200bps, spacing=50 → ±500bps)。

2. 同一 pair 的伪两跳消耗报价预算

WSOL → T → WSOL 是同一个 pair, 即使只有一个 pool 也走 100 档报价、最后逐档失败。修复: 入队前按 PairKey 计数检查 distinct-pool 容量。

3. 通用路线被两跳挤饿

混合 general worker 中三跳入队 1,086 次有 378 次被替换。修复: 拆成 state + fast/USDC-two/three/four 四个独立 lane, 各维护自己的 revision 和 pending queue。

安全边界

- 亏损机会在解码私钥前即被拒绝
- ProfitTooLow 由 executor 链上最终断言
- 每个 eligible 先 simulateTransaction → 通过才 sendBundle
- 同一时间最多一个 in-flight bundle
- 私钥和 RPC 通过 tmpfs credential 注入, 不写磁盘不写日志

笔记 022: 2026-08-09 · Day 5 / 21 · DEX 间套利——第一次完整双池计算

作者: clare ma | 时间: 2026-08-09 14:24 UTC | 字数: 9730 | 评分: 89

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-09 · Day 5 / 21 · DEX 间套利——第一次完整双池计算

验收:

在 Ethereum block 25716638 的池子储备量与两跳报价基础上, 完成 100 USDC → SushiSwap V2 → WETH → Uniswap V2 → USDC 的只读双池 Paper Trading。两跳后、尚未扣 Gas 的余额为 99.397014 USDC; 扣除 Day 4 已测量的教学 Gas 参数 0.0103 USDC 后, 余额为 99.386714 USDC, 最终净收益为 0.613286 USDC。该 Gas 参数不代表该区块的实时 Gas 成本。结论仅限本次样本: 本次样本不可行。

学了什么

边际价差只是收益线索, 不是实际利润

本次统一使用 USDC per ETH 作为价格方向。池子的边际价格公式为:

$$P = \text{raw_reserve_USDC} / \text{raw_reserve_WETH} \times 10^{12}$$

在 Ethereum block 25716638 的池子储备量下:

Uniswap V2: 1917.5995303 USDC/ETH

SushiSwap V2: 1916.1032693 USDC/ETH

边际价差为:

$$P_{\text{Uniswap}} / P_{\text{SushiSwap}} - 1$$

$$= 0.0007808875$$

$$= 0.07808875\%$$

0.07808875% 只是理论收益线索, 不是实际利润。它只能说明 SushiSwap V2 的边际价格较低、Uniswap V2 的边际价格较高, 因此值得继续计算以下路径:

USDC → SushiSwap V2 买入 WETH

WETH → Uniswap V2 卖回 USDC

实际结果必须使用包含协议费和确定性价格冲击的两跳 Router 报价计算, 不能把边际价差直接乘以本金后当作最终利润。

V2 报价必须使用含手续费公式

两跳都使用 Uniswap V2 类型池的含手续费公式:

amountOut =

$$(\text{amountIn} \times 997 \times \text{reserveOut})$$

÷

$$(\text{reserveIn} \times 1000 + \text{amountIn} \times 997)$$

代码形式:

$$\text{amountOut} = (\text{amountIn} \times 997 \times \text{reserveOut}) / (\text{reserveIn} \times 1000 + \text{amountIn} \times 997)$$

Solidity 使用整数除法，所以最终输出会向下取整。公式分子只是中间值，不能直接当成 amountOut。

raw token amount 必须按照 decimals 转换

本次涉及的代币精度：

USDC : 6 decimals

WETH : 18 decimals

因此：

100000000 raw USDC

= 100 USDC

51990635358449635 raw WETH

= 0.051990635358449635 WETH

99397014 raw USDC

= 99.397014 USDC

raw token amount 没有先按照代币 decimals 转换时，不能直接当成人类可读数量。

第二跳必须交换输入和输出储备

第一跳方向是：

USDC → WETH

因此：

reserveIn 是 USDC 储备；

reserveOut 是 WETH 储备。

第二跳方向变成：

WETH → USDC

因此必须交换：

reserveIn 是 WETH 储备；

reserveOut 是 USDC 储备。

不能因为两个池子的 token0 都是 USDC、token1 都是 WETH，就在第二跳继续使用第一跳的储备方向。

费率模型和实际 Router 报价是两种不同的核算口径

单次 V2 swap 的有效手续费项为：

$3 / 997$

= 0.0030090271

= 0.30090271%

双边手续费为：

$2 \times 3 / 997$

= 0.0060180542

= 0.60180542%

两跳价格冲击合计为：

$0.0008112481 + 0.0000112207$

= 0.0008224688

= 0.08224688%

双边手续费加价格冲击为：

$0.0060180542 + 0.0008224688$

= 0.0068405230

= 0.68405230%

相对边际价差的有效价格缺口为：

$0.0068405230 - 0.0007808875$

= 0.0060596355

= 0.60596355%

这套费率模型用于解释为什么边际价差不足以覆盖已知成本。

实际 Router 报价已经包含协议费和确定性的价格冲击。因此，得到两跳回款 99.397014 USDC 后，不能再次扣除 0.60180542% 的双边协议费和 0.08224688% 的价格冲击，否则会重复计算成本。

价格冲击是交易规模沿

曲线造成的确定性结果，已经包含在报价里。滑点则是报价之后池子状态变化造成的报价到执行结果偏离。本次固定区块 Trading 没有量化真实执行滑点。

AMM

Paper

做了什么

1. 固定池子储备量与两跳报价的链上快照

池子储备量与两跳报价统一取自 Ethereum block 25716638。Gas 使用 Day 4 已测量的教学参数 0.0103 USDC，不代表该区块的实时 Gas 成本。执行模式为只读链上数据 + Paper Trading。

代币：

USDC : 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48

WETH : 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2

代币精度：

USDC : 6 decimals

WETH : 18 decimals

池子：

Uniswap V2 WETH/USDC Pair : 0xB4e16d0168e52d35CaCD2c6185b44281Ec28C9Dc

SushiSwap V2 WETH/USDC Pair : 0x397FF1542f962076d0BFE58eA045FfA2d347ACa0

两个池子的 token0 都是 USDC，token1 都是 WETH。

本次没有执行 approve、swap 或 bridge，没有签名，也没有发送任何真实交易。

2. 读取两个池子的固定储备量

Uniswap V2 储备量读取命令：

```
cast call 0xB4e16d0168e52d35CaCD2c6185b44281Ec28C9Dc "getReserves()(uint112,uint112,uint32)" --rpc-url https://ethereum.publicnode.com --block 25716638
```

SushiSwap V2 储备量读取命令：

```
cast call 0x397FF1542f962076d0BFE58eA045FfA2d347ACa0 "getReserves()(uint112,uint112,uint32)" --rpc-url https://ethereum.publicnode.com --block 25716638
```

Uniswap V2：

USDC reserve : 8885093253567 raw

WETH reserve : 4633445676816826852274 raw

人类可读储备：

8885093.253567 USDC

4633.445676816826852274 WETH

SushiSwap V2：

USDC reserve : 123266859346 raw

WETH reserve : 64332054186403440077 raw

人类可读储备：

123266.859346 USDC

64.332054186403440077 WETH

3. 使用 Router02 纯函数核对报价

Uniswap Router02 的纯函数报价核对命令为：

```
cast call 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D "getAmountOut(uint256,uint256,uint256)(uint256)" <amountIn_raw> <reserveIn_raw> <reserveOut_raw> --rpc-url https://ethereum.publicnode.com
```

这个命令是只读调用，没有发送交易。

4. 计算边际价格并选择交易方向

固定储备量对应的边际价格为：

Uniswap V2 : 1917.5995303 USDC/ETH

SushiSwap V2 : 1916.1032693 USDC/ETH

因此选择：

100 USDC

→ SushiSwap V2 买入 WETH

→ Uniswap V2 卖回 USDC

选择原因：

SushiSwap V2 的边际价格较低，用相同的 USDC 可以换到更多 WETH；

Uniswap V2 的边际价格较高，相同数量的 WETH 可以换回更多 USDC。

这只是根据边际价格选择需要继续核算的路径，不代表该路径已经产生实际利润。

5. 计算第一跳：SushiSwap V2 买入 WETH

输入参数：

amountIn = 100 USDC

= 100000000 raw USDC

reserveIn = 123266859346 raw USDC

reserveOut = 64332054186403440077 raw WETH

使用公式：

amountOut =

$(\text{amountIn} \times 997 \times \text{reserveOut})$

÷

$(\text{reserveIn} \times 1000 + \text{amountIn} \times 997)$

计算过程中，约 6.4139E30 是接近公式分子的中间值，不是 amountOut。

经过整数除法后得到：

amountOut = 51990635358449635 raw WETH

转换为人类可读数量：

$51990635358449635 / 10^{18}$

= 0.051990635358449635 WETH

6. 计算第二跳：Uniswap V2 卖出 WETH

第二跳方向为 WETH → USDC，因此必须交换输入和输出储备：

amountIn = 51990635358449635 raw WETH

reserveIn = 4633445676816826852274 raw WETH

reserveOut = 8885093253567 raw USDC

未取整的连续结果约为：

99397014.328917029 raw USDC

Solidity 整数除法后的实际输出为：

99397014 raw USDC

USDC 有 6 decimals，转换为人类可读数量：

$99397014 / 10^6$

= 99.397014 USDC

99.397014 USDC 是两跳 Router 报价后的、尚未扣 Gas 的余额。

7. 计算回程结果和教学 Gas 参数下的净收益

初始本金：

100 USDC

两跳后、尚未扣 Gas 的余额：

99.397014 USDC

回程绝对损失：

$100 - 99.397014$

= 0.602986 USDC

回程损失率：

$0.602986 \div 100$

= 0.00602986

= 0.602986%

Gas 使用 Day 4 已测量的教学参数：

Gas_total = 0.0103 USDC

该参数不代表 Ethereum block 25716638 的实时 Gas 成本，也不是当天实时 Gas。后续仍需使用当天实时 gas price 重新验证。

扣除教学 Gas 参数后的余额：

99.397014 0.0103

= 99.386714 USDC

最终净收益：

99.386714 100

= 0.613286 USDC

最终净收益率：

0.613286 ÷ 100

= 0.00613286

= 0.613286%

得到什么结果

完整双池 Paper Trading 结果

| 项目 | 结果 |

|---|---|

| Ethereum block | 25716638 |

| 交易路径 | SushiSwap V2 → Uniswap V2 |

| 初始本金 | 100 USDC |

| SushiSwap V2 第一跳输出 | 51990635358449635 raw WETH |

| SushiSwap V2 第一跳人类可读输出 | 0.051990635358449635 WETH |

| Uniswap V2 第二跳未取整连续结果 | 约 99397014.328917029 raw USDC |

| Uniswap V2 第二跳整数输出 | 99397014 raw USDC |

| 两跳后、尚未扣 Gas 的余额 | 99.397014 USDC |

| 扣 Gas 前回程损失 | 0.602986 USDC |

| 扣 Gas 前回程损失率 | 0.602986% |

| Day 4 教学 Gas 参数 | 0.0103 USDC |

| 扣除教学 Gas 参数后的余额 | 99.386714 USDC |

| 最终净收益 | 0.613286 USDC |

| 最终净收益率 | 0.613286% |

其中：

0.07808875% 是边际价差，只是收益线索，不是实际利润；

99.397014 USDC 是两跳 Router 报价后的、尚未扣 Gas 的余额；

99.386714 USDC 是从 99.397014 USDC 中扣除 0.0103 USDC 教学 Gas 参数后的余额；

最终净收益是 0.613286 USDC；

Router 报价已经包含协议费和确定性的价格冲击，不能再次从 99.397014 USDC 中重复扣除；

真实执行滑点、延迟成本和真实失败交易预期成本尚未量化。

费率模型拆解

第一跳 SushiSwap V2 价格冲击：

amountIn / reserveIn

= 100 / 123266.859346

= 0.0008112481

= 0.08112481%

第二跳 Uniswap V2 价格冲击：

amountIn / reserveIn

= 0.051990635358449635

/ 4633.445676816826852274

= 0.0000112207

= 0.00112207%

两跳价格冲击合计：

0.0008112481 + 0.0000112207

= 0.0008224688

= 0.08224688%

双边手续费：

$2 \times 3 / 997$

= 0.0060180542

= 0.60180542%

双边手续费加价格冲击：

$0.0060180542 + 0.0008224688$

= 0.0068405230

= 0.68405230%

相对边际价差的有效价格缺口：

$0.0068405230 - 0.0007808875$

= 0.0060596355

= 0.60596355%

汇总：

| 项目 | 结果 |

|---|---|

| Uniswap V2 边际价格 | 1917.5995303 USDC/ETH |

| SushiSwap V2 边际价格 | 1916.1032693 USDC/ETH |

| 边际价差 | 0.07808875% |

| 双边有效手续费 | 0.60180542% |

| 第一跳价格冲击 | 0.08112481% |

| 第二跳价格冲击 | 0.00112207% |

| 两跳价格冲击合计 | 0.08224688% |

| 双边手续费加价格冲击 | 0.68405230% |

| 相对边际价差的有效价格缺口 | 0.60596355% |

边际价差只有 0.07808875%，而双边协议费和两边价格冲击形成的有效价格成本约为 0.68405230%。因此，该样本的边际价差不足以覆盖已知成本。

这里的费率模型用于解释结果。可执行 Router 报价得到的 99.397014 USDC 已经包含协议费和确定性的价格冲击，不能从该回款中再次扣除。

八项成本台账

| # | 项目 | 状态与结果 |

|---|---|---|

| 1 | 价差收益 | 已量化：边际价差为 0.07808875%，只是收益线索，不是实际利润 |

| 2 | Gas | 已量化教学参数：0.0103 USDC；该参数沿用 Day 4 已测量结果，尚未用当天实时 Gas 重新验证 |

| 3 | 协议费 | 已量化：双边有效费率为 0.60180542%；已包含在 Router 报价中，不能重复扣除 |

| 4 | 桥费 | 确实为 0，因为本次是同一条链上的 DEX 间交易，没有使用跨链桥 |

| 5 | 滑点和价格冲击 | 价格冲击已量化，约为 0.08224688%，并已包含在 Router 报价中；真实执行滑点尚未量化 |

| 6 | 延迟成本 | 尚未量化，不能写成 0 |

| 7 | 失败交易成本 | 本次 Paper Trading 没有真实失败交易，实际发生值为 0；真实执行的预期成本尚未量化 |

| 8 | 资金占用成本 | 本次没有投入真实本金，实际发生值为 0；真实执行的预期成本尚未量化 |

遇到什么失败 / 卡点

1. 一开始把约 6.4139E30 的公式分子中间值误当成了 amountOut。纠正后重新执行完整除法，第一跳实际输出为：

51990635358449635 raw WETH

= 0.051990635358449635 WETH

2. 混淆了 raw token amount 和 decimals。WETH 必须除以 10^{18} ，USDC 必须除以 10^6 ，不能直接把 raw 整数当成人类可读数量。

3. 第二跳方向从 USDC → WETH 变成了 WETH → USDC，最初忘记交换 reserveIn 和 reserveOut。纠正后使用 WETH 储备作为输入储备、USDC 储备作为输出储备。

4. 一度把未取整的连续结果：

99397014.328917029 raw USDC

当成 Solidity 实际输出。Solidity 使用整数除法，实际输出是：

99397014 raw USDC

= 99.397014 USDC

5. 混淆了小数和百分比：

0.0007808875

= 0.07808875%

两种形式不能直接混用。

6. 初步核算时曾把尚未量化的延迟成本、失败交易成本和资金占用成本直接写成 0。纠正后改为区分：

- 本次 Paper Trading 的实际发生值；
- 真实执行场景中的预期成本。

7. 当前使用的 Gas_total = 0.0103 USDC 是沿用 Day 4 已测量结果的教学参数，不代表 Ethereum block 25716638 的实时 Gas 成本，也不是当天实时 Gas。当天实时 gas price 及对应 Gas 成本仍待验证。

下一步验证什么

1. 使用当天实时 gas price 重新计算双边交易的 Gas 成本，替换当前的 Day 4 教学参数 0.0103 USDC。

2. 继续区分固定区块 Paper Trading 和真实执行：

- 固定区块 Router 报价已经包含协议费和确定性的价格冲击；
- 报价后池子状态变化造成的真实执行滑点尚未量化；
- 延迟成本尚未量化；
- 真实执行的预期失败交易成本尚未量化；
- 真实执行的预期资金占用成本尚未量化。

3. 保持相同核算口径：

- 先把边际价差作为收益线索；
- 再使用完整双跳 Router 报价判断实际回款；
- 不能把边际价差直接写成实际利润；
- 不能在最终回款上重复扣除已经包含的协议费和确定性价格冲击；
- 99.397014 USDC 是扣 Gas 前余额；
- 99.386714 USDC 是扣除 0.0103 USDC 教学 Gas 参数后的余额；
- 最终净收益为 0.613286 USDC。

4. 继续寻找并验证其他固定区块、交易路径或本金规模，但只把结论限定在对应样本，不能从本次结果泛化为“所有 DEX 套利都不可行”。

最终结论：

本次样本在 Ethereum block 25716638、SushiSwap V2 → Uniswap V2、100 USDC 本金和当前教学 Gas 参数 0.0103 USDC 下不可行。

两跳 Router 报价后的、尚未扣 Gas 的余额为 99.397014 USDC；扣除教学 Gas 参数后的余额为 99.386714 USDC；最终净收益为 0.613286 USDC。该结论不泛化为“所有 DEX 套利都不可行”。

笔记 023: Uniswap V2 / V3 數學與運作筆記

作者：corn15 | 时间：2026-08-09 14:29 UTC | 字数：12223 | 评分：84

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Uniswap V2 / V3 數學與運作筆記

1. Uniswap V2 核心：Constant Product AMM

Uniswap V2 的核心公式：

$$x \cdot y = k$$

其中：

(x)：token X reserve

(y)：token Y reserve

(k)：constant product

只要發生 swap，pool 會沿著這條曲線移動。

2. V2 的價格

假設：

```
\[
x = token0\
]
```

```
\[
y = token1\
]
```

則 token0 以 token1 計價的瞬時價格為：

```
\[
P=\frac{y}{x}\
]
```

例如：

```
\[
x=100\ ETH\
]
```

```
\[
y=200000\ USDC\
]
```

則：

```
\[
P=\frac{200000}{100}=2000\
]
```

即：

```
\[
1 ETH=2000 USDC\
]
```

3. V2 Swap Formula

假設使用者投入：

```
\[
\Delta x\
]
```

並拿走：

```
\[
\Delta y\
]
```

忽略 fee 時：

```
\[
(x+\Delta x)(y-\Delta y)=xy\
]
```

整理：

```
\[
y-\Delta y
=====
\frac{xy}{x+\Delta x}\
]
```

因此：

```
\[
\Delta y
=====
y-\frac{xy}{x+\Delta x}\
]
```

通分後：

$$\boxed{\Delta y = \frac{\Delta x}{x + \Delta x}}$$

4. 加入 V2 0.3% Fee

Uniswap V2 收取：

$$0.3\%$$

因此真正參與 swap 定價的 input 為：

$$\Delta x_{\text{eff}} = 0.997 \Delta x$$

代入 swap formula：

$$\boxed{\Delta y = \frac{y(0.997 \Delta x)}{x + 0.997 \Delta x}}$$

注意

如果投入：

$$10 \text{ ETH}$$

真正參與定價的是：

$$9.97 \text{ ETH}$$

但實際 pool reserve 最後仍增加：

$$10 \text{ ETH}$$

因為：

$$0.03 \text{ ETH}$$

fee 也留在 pool 中。

因此有 fee 時，交易後實際上：

$$k' > k$$

也就是 LP 的資產會隨 fee 增加。

Uniswap V3

5. V3 並沒有拋棄 ($xy=k$)

V3 的數學仍然建立在 constant product AMM 上。

最大的不同在於：

V2 的 liquidity 分布在整個價格範圍，而 V3 可以只在特定價格區間提供 liquidity。

例如：

V2

Liquidity



V3 :

Liquidity



這就是：

Concentrated Liquidity

6. V3 的價格 (P) 是什麼？

Uniswap protocol 定義：

$$P = \frac{\text{token1}}{\text{token0}}$$

也就是：

1 單位 token0 可以換多少 token1。

因此一定要先確認：

token0 = ?

token1 = ?

不能直接假設：

ETH = token0

USDC = token1

因為 token0 / token1 是按照 token address 排序。

另外鏈上使用的是 token raw units，因此實際顯示價格時還要處理 decimals。

7. 為什麼 V3 使用 $(\sqrt{P})$ ？

從 V2：

$$xy = k$$

開始。

定義：

$$L = \sqrt{k} = \sqrt{xy}$$

又因為：

$$P = \frac{y}{x}$$

]

所以：

$\backslash$

$\sqrt{P} = \sqrt{\frac{yx}{L}}$

]

可以推導：

$\backslash$

$\boxed{\backslash}$

$x = \frac{L}{\sqrt{P}}$

$\backslash$

]

以及：

$\backslash$

$\boxed{\backslash}$

$y = L \sqrt{P}$

$\backslash$

]

驗證：

$\backslash$

$\frac{L}{\sqrt{P}}$

=====

$\frac{\sqrt{xy}}{\sqrt{y/x}}$

$= x$

$\backslash$

]

另外：

$\backslash$

$L \sqrt{P}$

=====

$\sqrt{xy}$

$\sqrt{\frac{yx}{L}}$

$= y$

]

因此：

$\backslash$

$\boxed{\backslash}$

$x = \frac{L}{\sqrt{P}}$

$\backslash$

]

$\backslash$

$\boxed{\backslash}$

$y = L \sqrt{P}$

$\backslash$

]

是理解 V3 最重要的兩條公式。

8. 為什麼 $(\sqrt{P})$ 很方便？

假設價格由：

$\backslash$

$P_1 \rightarrow P_2$

]

因為：

```
\[
y=L\sqrt P\]
```

所以：

```
\[
y\_2-y\_1
=====
L(\sqrt{P\_2}-\sqrt{P\_1})\]
```

因此：

```
\[
\boxed{\Delta y}
=====
L\Delta\sqrt P\} \]
```

token1 與 $(\sqrt{P})$ 之間變成線性關係。

token0：

```
\[
x=\frac{L\sqrt P}\]
```

因此：

```
\[
\boxed{\Delta x}
=====
L\left(\frac{1}{\sqrt{P\_2}}-\frac{1}{\sqrt{P\_1}}\right)\]
```

所以使用：

```
\[
\sqrt P\]
```

可以讓 swap 數學變得非常方便。

9. Liquidity (L) 到底代表什麼？

V2 中：

```
\[
L=\sqrt{xy}\]
```

且：

```
\[
k=L^2\]
```

但不要單純把 (L) 理解成：

Pool 中有多少錢。

更好的理解是：

Liquidity 代表這段價格曲線的深度。

從：

$$\Delta y = L \Delta \sqrt{P}$$

可以看到：

相同價格變化：

$$\Delta \sqrt{P}$$

如果：

$$L$$

越大，就需要越多 swap amount 才能造成相同價格變化。

因此：

$$\boxed{L \text{ 越大} \rightarrow \text{Price Impact 越小}}$$

10. V3 Concentrated Liquidity

假設 LP 選擇價格範圍：

$$[P_a, P_b]$$

而目前價格：

$$P_a < P < P_b$$

則 LP 的實際 token 數量為：

$$\boxed{x = L \left(\frac{1}{\sqrt{P}} - \frac{1}{\sqrt{P_b}} \right)}$$

以及：

$$y = L \left(\sqrt{P} - \sqrt{P_a} \right)$$

$\right)\backslash$
 $\}\backslash$
 $]$

和 V_2 :

$\backslash\backslash$
 $x=\frac{L}{\sqrt{P}}\backslash$
 $]$

$\backslash\backslash$
 $y=L\sqrt{P}\backslash$
 $]$

相比，相當於扣掉區間外不需要提供的 liquidity。

11. Virtual Reserves

將 :

$\backslash\backslash$
 $x=\backslash$
 $L\backslash$
 $\left(\backslash$
 $\frac{1}{\sqrt{P}}\backslash$

$\frac{1}{\sqrt{P_b}}\backslash$
 $\right)\backslash$
 $]$

移項 :

$\backslash\backslash$
 $x+\frac{L}{\sqrt{P_b}}\backslash$
 $\frac{L}{\sqrt{P}}\backslash$
 $]$

另外 :

$\backslash\backslash$
 $y+L\sqrt{P_a}\backslash$
 $L\sqrt{P}\backslash$
 $]$

兩式相乘 :

$\backslash\backslash$
 $\boxed{\backslash$
 $\left(\backslash$
 $x+\frac{L}{\sqrt{P_b}}\backslash$
 $\right)\backslash$
 $\left(\backslash$
 $y+L\sqrt{P_a}\backslash$
 $\right)$
 $L^2\backslash$
 $\}\backslash$
 $]$

因此 V_3 range 中仍然可以理解為 :

$\backslash\backslash$
 $XY=k\backslash$
 $]$

只是使用了：

Virtual Reserves

也就是：

V3 可以只用較少實際資金，在特定價格區間模擬出更深的 V2 liquidity。

這就是 concentrated liquidity 提升資本效率的原因。

12. 價格碰到 Range Boundary 時會發生什麼？

假設 LP range：

$\backslash \backslash$

$\backslash [P_a, P_b] \backslash$

]

價格來到 (P_a)

$\backslash \backslash$

y

=

$L(\sqrt{P_a} - \sqrt{P_a}) \backslash$

$\backslash = 0 \backslash$

]

因此 position：

token0: 100%

token1: 0%

價格位於 range 中

token0 + token1

價格來到 (P_b)

$\backslash \backslash$

x

=

$L \backslash$

$\backslash \text{left}(\backslash$

$\frac{1}{\sqrt{P_b}} \backslash$

$\frac{1}{\sqrt{P_b}} \backslash$

$\backslash \text{right}) \backslash$

$\backslash = 0 \backslash$

]

因此：

token0: 0%

token1: 100%

可以記成：

Price < Pa

100% token0

Pa < Price < Pb

token0 + token1

Price > Pb

100% token1

13. Tick 是什麼？

V3 不允許 LP 在任意無限精度價格建立 boundary。

而是把價格切成離散的網格：

$\backslash \backslash$

$\backslash \boxed{\backslash$

$P(i) = 1.0001^i \backslash$

} \

]

其中：

\ \

$i = \text{tick}$ \

]

因此：

\ \

P_{i+1}

=====

$P_i \times 1.0001$ \

]

每增加一個 tick，價格約增加：

\ \

0.01% \

]

反過來：

\ \

$\boxed{\text{tick} = \log_{1.0001} P}$ \

]

} \

]

因此：

tick 本質上是價格的 logarithmic coordinate。

14. Tick 與 $(\sqrt{P})$

由：

\ \

$P = 1.0001^{\text{tick}}$ \

]

得到：

\ \

$\sqrt{P}$

=====

$\sqrt{1.0001^{\text{tick}}}$ \

]

所以：

\ \

$\boxed{\sqrt{P}}$ \

]

=====

$1.0001^{\text{tick}/2}$ \

} \

]

因此：

tick

↓

$P = 1.0001^{\text{tick}}$

↓

$\sqrt{P}$

它們其實都是同一個價格的不同表示方法。

15. sqrtPriceX96 是什麼？

Solidity 沒有一般 floating-point 運算，因此不能直接存：

$\sqrt{P} = 44.721359\dots$

Uniswap 使用 fixed-point representation：

```
\[
\boxed{
\sqrt{P} \times 2^{96}
}
=====
\sqrt{P \times 2^{96}}
]\]
```

所以讀回來：

```
\[
\boxed{
\sqrt{P}
}
=====
\frac{\sqrt{P} \times 2^{96}}{2^{96}}
]\]
```

再平方：

```
\[
\boxed{
P =
\left(
\frac{\sqrt{P} \times 2^{96}}{2^{96}}
\right)^2
}
]\]
```

因此：

sqrtPriceX96

本質上就是：

把 $(\sqrt{P})$ 放大 (2^{96}) 倍後，使用 integer 儲存。

16. Q64.96

sqrtPriceX96 使用 Q64.96 fixed-point representation。

概念上：

[64-bit integer][96-bit fractional]

總共：

```
\[
160 \text{ bits }
]\]
```

所以 Solidity 中使用：

```
uint160 sqrtPriceX96;
```

X96 代表：

```
\[
\times 2^{96}
]\]
```

17. 分段 Liquidity

假設三個 LP：

LP A

Range: [1800, 2200]

L = 100

LP B

Range: [1900, 2100]

L = 200

LP C

Range: [2000, 2300]

L = 300

價格區間：

1800 1900 2000 2100 2200 2300



A



B



C



因此 active liquidity：

1800 ~ 1900

只有 A：

$\backslash \backslash$

L=100\

]

1900 ~ 2000

A + B：

$\backslash \backslash$

L=100+200=300\

]

2000 ~ 2100

A + B + C：

$\backslash \backslash$

L=600\

]

2100 ~ 2200

A + C：

$\backslash \backslash$

L=400\

]

2200 ~ 2300

只有 C：

$\backslash \backslash$

L=300\

]

所以 V3 的 liquidity 是：

$\backslash \backslash$

$\backslash \boxed{\backslash$

Piecewise \ Constant \ Liquidity \

$\backslash \backslash$

]

18. liquidityNet

Uniswap 不需要在每一個價格都儲存完整 liquidity。

只需要在 LP range 的 boundary 記錄變化。

例如：

LP A: [1800, 2200], L=100

可以記成：

1800: +100

2200: -100

完整例子：

Tick liquidityNet

1800 +100

1900 +200

2000 +300

2100 -200

2200 -100

2300 -300

假設價格由低往高：

Before 1800

L = 0

cross 1800

L = 100

cross 1900

L = 300

cross 2000

L = 600

cross 2100

L = 400

因此 Uniswap V3 只需要保存：

1. current active liquidity

2. initialized ticks

3. 每個 initialized tick 的 liquidityNet

19. Swap 穿越 Tick 時怎麼運作？

假設：

Current Price = 1980

且：

1900 ~ 2000:

L = 300

做一筆讓價格上升的大 swap。

Step 1：先算到下一個 initialized tick

目前：

\

1980\rightarrow2000\

]

整段：

\

L=300\

]

所以需要的 token1 input：

\

\Delta y

=====

```

300\
(\sqrt{2000}-\sqrt{1980})\
]

```

Step 2 : 碰到 2000

2000 的 :

liquidityNet = +300

因此 :

```

\ \
L:\
300\rightarrow 600\
]

```

Step 3 : 繼續下一段

接著 :

```

\ \
2000\rightarrow 2100\
]

```

現在 :

```

\ \
L=600\
]

```

所以 :

```

\ \
\Delta y
=====

```

```

600\
(\sqrt{P\_2}-\sqrt{2000})\
]

```

Step 4 : 如果又碰到 2100

假設 :

liquidityNet = -200

則 :

```

\ \
L:\
600\rightarrow 400\
]

```

然後繼續處理剩餘 input。

20. V3 Swap 的核心演算法

可以把一筆 swap 理解成 :

Input Amount

|

▼

Current sqrtPrice

Current Tick

Current Liquidity

|

▼

尋找 swap 方向上的 next initialized tick

|

▼

計算 :

到 next tick 需要多少 input ?

|

```

      |
      | |
Input 不夠 Input 足夠
      | |
      ▼ ▼
算 final price 到達 tick
結束 |
      ▼
更新 liquidity
      |
      ▼
處理剩餘 input
因此大 swap 可能：
L=300
1980 —————→ 2000
      |
      | cross tick
      ▼
L=600
2000 —————→ 2100
      |
      | cross tick
      ▼
L=400
2100 —————→ 2117
      ↑
input 用完
21. 為什麼 V3 Quote 比 V2 複雜？
V2
只需要：
reserve0
reserve1
fee
然後：
\\
amountOut
=====
f(reserve0,reserve1,amountIn)\
]
V3
需要：
sqrtPriceX96
current tick
current liquidity
initialized ticks
liquidityNet
fee
Swap 過程：
Current Price
↓
Next Tick
↓

```

Compute Swap Step

↓

Cross Tick?

┌──────────┐

No Yes

| |

End Update L

|

└────────→ Next Tick

因此：

只知道 current price + current liquidity，不能精確計算任意大小的 V3 swap。

如果 swap 很小、完全不 cross tick，才可能只用目前的 (L) 計算。

22. V2 與 V3 的核心差異

| V2 | V3 |

| :----- | :----- |

| (xy=k) | 局部仍是 constant product |

| 全價格範圍 liquidity | Concentrated liquidity |

| reserve0 / reserve1 | sqrtPrice + liquidity |

| 一條連續 liquidity curve | Piecewise liquidity |

| LP 被動覆蓋 (0~∞) | LP 自選價格範圍 |

| Quote 相對簡單 | 需要逐 tick 計算 |

23. V3 核心概念關係圖

xy = k

|

| L = √k

▼

L = √xy

|

┌──────────┐

▼ ▼

x = L/√P y = L√P

| |

└──────────┐

|

▼

sqrt price

|

┌──────────┐

▼ ▼

sqrtPriceX96 tick

| |

√P × 2⁹⁶ P = 1.0001^{tick}

| |

└──────────┐

▼

Current Price

|

▼

Active Liquidity L

|

▼

Next Initialized Tick

|

▼

Swap Step

|
cross tick

|
▼

$L \pm \text{liquidityNet}$

24. 套利視角下要讀懂哪些資料？

看到一個 V3 pool，可以依序問：

1. token0 是誰？
2. token1 是誰？
3. sqrtPriceX96 是多少？
4. 轉換後目前 (P) 是多少？
5. current tick 是多少？
6. current active liquidity (L) 是多少？
7. swap 方向上的 next initialized tick 在哪？
8. 該 tick 的 liquidityNet 是多少？
9. 到 next tick 需要多少 amountIn？
10. cross tick 後新 liquidity 是多少？
11. 剩餘 input 是否需要繼續下一段？

這基本上就是實作：

V3 quoter

V3 simulator

arbitrage searcher

時需要處理的核心內容。

25. 最重要的心智模型

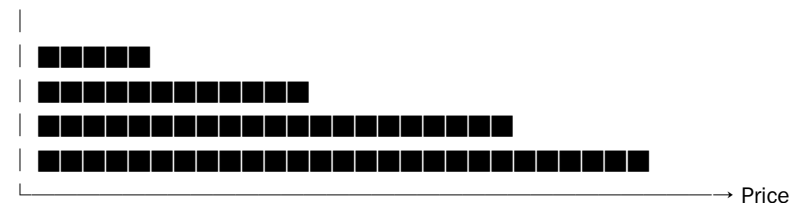
不要把 V3 pool 想成：

ETH reserve

USDC reserve

更適合想成：

Liquidity



↑
Current Price

目前價格所在位置決定：

Active Liquidity

swap 推動：

sqrtPrice

價格碰到：

initialized tick

就更新：

liquidityNet

再繼續下一段 swap。

26. 關鍵公式總表

V2

$\backslash\backslash$

$xy=k\backslash$

$\backslash$

$\backslash\backslash$

$P=\frac{yx}{\backslash}$

$\backslash$

無 fee :

$\backslash\backslash$

$\Delta y=\backslash$

$\frac{y\Delta x}{\backslash}$

$\{x+\Delta x\}\backslash$

$\backslash$

0.3% fee :

$\backslash\backslash$

$\boxed{\backslash}$

$\Delta y=\backslash$

$\frac{y(0.997\Delta x)}{\backslash}$

$\{x+0.997\Delta x\}\backslash$

$\backslash$

$\backslash$

V3 基礎

$\backslash\backslash$

$\boxed{\backslash}$

$L=\sqrt{xy}\backslash$

$\backslash$

$\backslash$

$\backslash\backslash$

$\boxed{\backslash}$

$x=\frac{L}{\sqrt{P}}\backslash$

$\backslash$

$\backslash$

$\backslash\backslash$

$\boxed{\backslash}$

$y=L\sqrt{P}\backslash$

$\backslash$

$\backslash$

V3 Swap

token1 :

$\backslash\backslash$

$\boxed{\backslash}$

Δy

=====

$L(\sqrt{P_2}-\sqrt{P_1})\backslash$

$\backslash$

$\backslash$

token0 :

$\backslash\backslash$

$\boxed{\backslash}$

Δx

=====

$L\backslash$

$\left(\backslash$

$\frac{1}{\sqrt{P_2}}\backslash$

$$\frac{1}{\sqrt{P_{-1}}}\left(\frac{1}{\sqrt{P}} - \frac{1}{\sqrt{P_{-1}}}\right)$$

V3 Concentrated Liquidity Position

$$\frac{1}{\sqrt{P}} \left(\frac{1}{\sqrt{P}} - \frac{1}{\sqrt{P_{-1}}} \right)$$

$$\frac{1}{\sqrt{P_{-b}}}\left(\frac{1}{\sqrt{P}} - \frac{1}{\sqrt{P_{-b}}}\right)$$

$$\frac{1}{\sqrt{P}} \left(\frac{1}{\sqrt{P}} - \frac{1}{\sqrt{P_{-a}}} \right)$$

Tick

$$P = 1.0001^{\text{tick}}$$

$$\text{tick} = \log_{1.0001} P$$

sqrtPriceX96

$$\sqrt{P \times 2^{96}}$$

$$\sqrt{P \times 2^{96}}$$

$$\frac{\sqrt{P \times 2^{96}}}{2^{48}}$$

笔记 024: Day 5 | bps、前端报价口径与 UniswapX 路由观察

作者：Prousty Wu | 时间：2026-08-09 14:51 UTC | 字数：986 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 | bps、前端报价口径与 UniswapX 路由观察

今天从“看见池间价格差”进一步走到“知道前端每个数字到底代表什么”。

1. 厘清 bps : 1 bps = 0.01%。0.66 bps 对 10,000 USDG 的无摩擦理论毛利只有约 0.66 USDG ; 边际价差不是指定规模的可执行利润。

2. 厘清 amountIn / amountOut : 在 Exact Input 报价中, 上方输入的 USDG 是 amountIn, 下方预计收到的 NVDA 就是前端格式化后的 amountOut。完整精度、区块号、Tick 和指定 Pool 身份仍需 API、RPC 或 Quoter。

3. 纠正 Price Impact 与 Max Slippage 的混淆 : Price Impact 是自己的订单规模造成、已经进入当前报价的执行恶化 ; Max Slippage 是报价生成后允许额外恶化的保护范围。1,000 USDG → 4.43981 NVDA、最大滑点 2.5% 时, 经典 AMM 路由的最低输出近似为 4.32881475 NVDA。2.5% 等于 250 bps, 远大于候选价差, 因此实盘必须使用 minProfit 做闭环保护。

4. 手动记录了 Robinhood Chain NVDA/USDG 的九组报价 : 100、1,000、5,000 USDG, 分别比较 UniswapX、V4 与 V3。100 USDG 时 V3 显示输出最高 ; 1,000 与 5,000 USDG 时 UniswapX 最高 ; V4 输出随规模扩大下降更明显。但报价跨越数分钟且没有固定区块和 Pool, 所以只能作为前端执行基准, 不能证明池间套利。

5. 理解了 UniswapX 是 Solver 竞争形成的 Intent 最优执行报价, 不是指定 AMM Pool ; 仅限制到 V3/V4 也仍可能多跳或拆单。聚合器最佳输出与指定 Pool Edge 必须分开验证。

下一步 : 先以 1,000 USDG 完成 V4 买→V3 卖、V3 买→V4 卖两个方向的手动闭环 ; 随后用 RPC 固定 blockTag, 显式指定 V3 Pool 和 V4 PoolKey, 扩展到 100 / 1,000 / 5,000 / 10,000 USDG 的只读报价阶梯。

笔记 025: 在真的去 Polymarket

自动挂单之前, 先要解决行情数据的问题。目前找到这几个数据源 :

作者：newraina | 时间：2026-08-09 14:55 UTC | 字数：726 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

在真的去 Polymarket 自动挂单之前, 先要解决行情数据的问题。目前找到这几个数据源 :

Binance 的 Spot direct bookTicker 和 Spot direct trade

Polymarket RTDS 提供的 Chainlink 和 Binance 数据

Chainlink 数据

在试图接入

Binance

数据源测试时, 碰到一个比较有意思的问题, 是关于「时间」的 : 原来电脑上的时间是「不准」的, 这个不准有两层意思 :

电脑里负责时间的石英晶振本身有误差, 大致的偏差值是 10-50 ppm, 也就是每过一天偏差1到5s。

很多电脑都会联网通过NTP自动校准时间, 但这个自动校准偏差的过程本身也会带来系统时间的波动, 因为

NTP

校准偏差有两种方式 :

slew : 稍微加快/放慢钟的走速, 用几分钟到几十分钟把偏差慢慢磨平。

step : 当偏差超过阈值 (通常 ~128ms), slew 来不及了, 系统直接把墙钟设成正确值。这一步可能让时间往回跳。

既然电脑 (还有服务器) 有上面的时间偏差问题, 那多个数据源里的时间信息就不能直接比较。在需要时序数据的时候, 就需要一个单一来源、稳定不能往回跳的时间, 可以用 process.hrtime.bigint() 拿到, 叫单调时钟。

但是单调时钟也是来自硬件的, 其实也有偏差问题, 只是它不会被 NTP 服务自动修改值, 保证了不会往回调。所以即使使用了它也不是一个精确的时间。不过, 因为硬件的时间偏差是比较稳定的, 所以可以通过定期算出本机时间和对方服务器时间的偏差值来校准

。

笔记 026: 昨天周六, 请假休息一天。今天周日主要回顾上周进度和本周计划。

作者：allen123wzh | 时间：2026-08-09 14:59 UTC | 字数：642 | 评分：52

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

昨天周六，请假休息一天。今天周日主要回顾上周进度和本周计划。

上周进度：

1. 完成针对某个pool的总体liquidity distribution监控
2. 完成针对某个address在某个pool的liquidity distribution监控
3. 完成某个pool在uni v3 active tick的liquidity详情计算
4. 把相应的查询命令封装成telegram bot，实现telegram直接查询
5. 尝试输出md网页，局域网访问打通，公网通过tcp 443方案无法访问，本周尝试cloudflare tunnel

本周计划：

1. 尝试用程序接管一个账户，之前从没用命令行操作过一个address，希望实现程序不停监控某矿池情况，发现某时间后自动出发buy/sell，实现睡觉时的仓位接管
2. 将pool+address查询功能做成网页，输入pool，输入某个address，就可以查询pool的任意tick流动性分布情况和address specific的流动性情况，也相当于学习一下前端设计
3. 查阅uni v3和option关系的论文和协议，尤其是panoptic和gamma swap将uni v3 LP转换为永续期权的方案。完成对于robinhood chain上的链上美股，在某个区间内（相当于sell区间下沿的put和区间上沿的call）和对应1-week 1-month的美股期权的收益比较。

02 LI.FI与跨链 （39 篇）

笔记 027: Day 4 · 从“报价差”转向“可执行边际”

作者：Junfan Chen | 时间：2026-08-08 15:00 UTC | 字数：1800 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 · 从“报价差”转向“可执行边际”

日期 2026-08-08

主题 把 Day 1、Day 2 的发现收敛成后续 collector 的判断框架

今天的整理

前两次实验解决的是两个不同层级的问题：

Day 1 验证了 LI.FI 报价接口，并发现同币种跨链不能直接用两端 priceUSD 计算价差，否则会把预言机偏差误认成交易收益。
Day 2 用整数公式和链上 Router 完成 AMM 定价校准，并确认 toAmount 只是乐观报价，真正的执行下界应看 toAmountMin。

今天没有急着继续堆更多交易样本，而是把这两条经验合并成一个更严格的机会定义：

看到价差不等于存在套利机会；只有在统一计价口径、扣除全部成本，并以最差可接受成交量计算后仍为正，才值得进入下一步验证。

1. 统一三种容易混淆的数字

后续采样需要明确区分：

1. 显示价差：来自价格页面或 priceUSD，适合观察市场，但可能混入预言机噪声。
2. 报价边际：按 toAmount 计算，是路由器给出的乐观结果。
3. 可执行边际：按 toAmountMin 计算，并扣除 gas、协议费、桥费等成本，才是策略筛选应使用的数字。

对于同币种跨链，主口径应是 token 数量：

text

$$\text{gross_token_bps} = (\text{toAmount} / \text{fromAmount} - 1) \times 10000$$

priceUSD 只用于单独记录 oracle_skew_bps，不能混入净收益。

2. 后续 collector 的最小字段

基于前两天踩过的坑，下一版采样至少要保留：

text

timestamp / chain / token / amount

route / quote_type / execution_duration

to_amount / to_amount_min

gas_cost_usd / protocol_fee / bridge_fee
gross_token_bps / executable_bps
slippage_band_bps / oracle_skew_bps

其中：

text

$$\text{slippage_band_bps} = (\text{toAmount} - \text{toAmountMin}) / \text{toAmount} \times 10000$$

如果：

text

$$\text{slippage_band_bps} > \text{gross_edge_bps}$$

说明机会已经被报价的不确定区间覆盖，应直接标红，而不是继续把它当候选套利。

3. 成本优先级重新排序

Day 2 的结果显示，在约 100 USDC 的资金规模下：

text

gas（绝对成本）> 协议/池手续费（固定比例）>> 曲线滑点

因此小资金策略不应先花大量精力优化滑点或复杂拆单。更有价值的问题是：

固定 gas 成本摊到多大本金后才不再主导；

LI.FI/桥/DEX 的固定费率是否已经高于常见价差；

报价有效期和 toAmountMin 是否会吞掉剩余边际；

RFQ/intent 路由与 AMM 路由是否应分组统计。

4. 当前判断

目前只有少量样本，还不能得出“某条路径长期不可套利”的结论；但已经可以排除两种假机会：

由不同链 priceUSD 偏差制造的假价差；

只按 toAmount 为正、按 toAmountMin 已为负的假边际。

下一步应进行多金额、多时间点采样，寻找

最小可行下单量，而不是继续围绕单笔报价做结论。重点记录失败和被过滤的原因，因为“为什么不能做”本身就是策略边界。

下一步

[] 扫描多个资金档位，测量 gas 占比下降曲线

[] 同时保存 toAmount 与 toAmountMin

[] 将 RFQ/intent 与 AMM 路由分组

[] 增加 band_over_edge 红灯判据

[] 用时间序列验证边际是否可重复，而不是只看单点

笔记 028: Day 3 续：深挖 FP/IMD 合约源码，揭示跨链 OFT 套利本质

作者：Zhou Junjie | 时间：2026-08-08 15:03 UTC | 字数：980 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 3 续：深挖 FP/IMD 合约源码，揭示跨链 OFT 套利本质

做了什么

1. 用 Basescan 和 Etherscan 分别查看了 FP (Base) 和 IMD (ETH) 的已验证合约源码
2. 发现 IMD 合约文件名为 BridgedFP.sol，继承 LayerZero OFT + Ownable，构造函数中原始名称为 "Fren Pet" / "FP"
3. 证实 FP 和 IMD 本质上是同一个代币：FP 是 Base 上的标准 ERC-20（带买卖税费），IMD 是 ETH 上通过 LayerZero OFT 协议的跨链映射版本
4. 归档完整案例到 Obsidian（Trade/参考/链上套利案例-FP跨链OFTAnalysis.md）

核心发现

|| FP (Base) | IMD (ETH) |

|---|---|---|

| 合约名 | FrenPetToken.sol | BridgedFP.sol |

| 继承 | ERC20 | LayerZero OFT |

| 原名称 | Fren Pet / FP | Fren Pet / FP（已改名） |

IMD 就是 FP 的 OFT 桥接版，后改名 Identity.md/IMD。LayerZero OFT 是连接两者的唯一机制——与 Uniswap v4 Hook 无关。

套利逻辑

Base 侧流动性薄 (\$365K)，持有人分散 (21K)，大跌 12.7%。ETH 侧流动性深 (\$477K)，持有人集中 (1K)，仅跌 2.99%。价差 ~3% 可通过 OFT 桥接套利。

教训

读源码之前的所有推断——Hook、V4、Robinhood、内部代币

vs

外部市场——全是错的。链上分析的核心纪律：先确认链，再看源码，后做推断。

工具收获

cast code + cast source 可读合约字节码和源码

Basescan/Etherscan 的合约页面直接提供已验证源码和 ABI

onchainos 的 token search 提供跨链搜索能力

将完整分析结构化归档到 Obsidian 参考目录

笔记 029: 链上套利学习笔记 Day 4：跨链与 LI.FI —— 真正的时间窗口

作者：倪响 | 时间：2026-08-08 15:06 UTC | 字数：1654 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习笔记 Day 4：跨链与 LI.FI —— 真正的时间窗口

2026-08-07 | 残酷共学 Day 4

参考：LI.FI 文档 (<https://docs.li.fi/>) | LI.FI API Overview (<https://docs.li.fi/api-reference/overview>)

为什么跨链值得看

前三天的结论是：同链 DEX 间/三角套利不是个人赛道——毫秒级竞争、MEV searcher 碾压。

跨链不一样。跨链天然有一个时间窗口——桥的 finality 时间至少几十秒到几分钟。MEV searcher 可以在一条链上毫秒级操作，但跨链必须等桥确认目标链上的资产到账。这段时间不需要拼手速。

LI.FI 是什么

一个跨链流动性聚合器。连接了桥 + DEX + solver 网络，对外只暴露一个 API。

你用 Day 1-3 的方式做跨链，需要自己：

1. 选桥 (Across / Stargate / Hop...)
2. 获取桥的 quote
3. 估算目标链 Gas
4. 处理 token 映射 (USDC vs USDC.e vs bridged USDC)
5. 构建两笔交易 (源链 + 目标链)

LI.FI 把上面全封装了，一个 GET 请求搞定。

实际测了一下

调了 LI.FI Quote API，从 Arbitrum 跨 1000 USDC 到 Ethereum：

fromChain: 42161 (Arbitrum)

toChain: 1 (Ethereum)

fromToken: USDC

toToken: USDC

fromAmount: 1000 USDC

结果：

输入: 1,000.00 USDC (Arbitrum)

到账: 997.50 USDC (Ethereum)

损耗: 2.50 USDC (0.25%)

其中：

LI.FI 固定费: ~\$1.00

桥费: ~\$1.50

Gas (Arb侧): ~\$0.024

1000U 跨链损失 \$2.50，比例 0.25%。如果跨链价差能超过这个数，才有套利空间。

跨链套利的成本结构

和同链不一样，跨链多了一层：

同链成本 (Day 1-3) :
Price Impact + DEX 手续费 + Gas

跨链成本:

Price Impact + DEX 手续费 + 桥费 + 跨链协议费 + 双链 Gas + 桥 finality 风险

但这个结构里最重要的是: 桥的费用和时间是固定的, 不和交易金额等比放大。这对套利意味着——存在一个"最小可盈利金额", 跨过去才划算。

这条路有前途吗

和前几天不一样, 我觉得跨链是个人可以碰的方向。原因:

1. 时间窗口: 桥的 finality 以分钟计, 不是毫秒
2. 信息不对称: 同一个 token 在不同链上的价格不是每个人都在同时监控的
3. 复杂度门槛: 跨链套利需要理解多条链、多个桥、多个 DEX, 这本身就是护城河
4. 工具还不成熟: DEX 间套利有成熟的 bot, 跨链套利还在早期

但竞争也在变激烈——LI.FI 自己就在做 intent-based 的跨链执行 (用户表达意图, solver 网络竞拍执行权)。这条路也在被基础设施化。

共学打卡: 调通了 LI.FI Quote API, 1000U 从 Arbitrum 跨到 Ethereum 损失 \$2.50 (0.25%)。跨链和同链的最大区别: 时间窗口长 (桥 finality 按分钟算), 个人有时间思考和执行。这是 Phase 2 值得深入的方向。

笔记 030: \=====

作者: hellingercheri-gif | 时间: 2026-08-08 15:17 UTC | 字数: 5296 | 评分: 78

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

\=====

Gate CrossEx / LI.FI / Backpack 三项目对比

整理日期: 2026-08-08

=====

【一句话结论】

它们名字里都带"交易 / 跨", 但根本不在同一层:

- Gate CrossEx = 中心化交易所之上的机构撮合与清结算层
- LI.FI = 链上流动性的路由中间件 (面向开发者的 B2B 基础设施)
- Backpack = 一家自己就是流动性场所的持牌交易所 + 自托管钱包

\=====

一、三者各自是什么

=====

1. Gate CrossEx —— 交易所原生的"加密 Prime Broker"

谁做的 / 何时

Gate (原 Gate.io) 推出, 2025 年 10 月 Beta 上线。

解决什么问题

机构 / 量化在多家 CEX 开户, 每家都得按最坏情况备足保证金, 抵押品无法跨所净额; 行情剧烈时靠链上转账调仓要 20 分钟以上, 仓位早爆了。

怎么做

- (1) 跨所统一账户: 一个账户接入多家主流交易所 (上线时为 Gate / Binance / OKX, The Block 研究口径为已扩到 5 家, 2026 年 3 月前后加入 Bybit), 无需在各所重复注册。
- (2) 保证金共享 + 跨所盈亏互抵: 在 Gate 的一笔资金可同时给多家所的仓位当保证金; A 所赚的钱直接抬高支撑 B 所仓位的整体保证金池。可用抵押资产 7 种: BTC / ETH / BNB / SOL / XRP / USDT / USDC。
- (3) 内部信用清结算: 抵押品在场所间靠内部信用划转 (毫秒级), 不走链上确认 —— Gate 在这里扮演信用中介。
- (4) 统一 API 格式、低延迟执行; 支持现货 / 全仓杠杆 / USDT 本位永续, 最高 20 倍杠杆; 4,200+ 交易对。
- (5) 多所交易量合并计 VIP, 更容易拿到低费率档。

谁用

专业投资者、量化团队、机构。不是自助开通，需走 Gate 机构渠道申请，基本是 API 主导。

数据

Gate 2026 年 Q2 财报口径：CrossEx 的 AUM 环比 +128.1%、交易量 +202.6%、交易用户 +76%。

注意

托管细节（资产究竟在谁名下、保证金层如何收费）官方未完全披露，The Block 的研究报告也点了这个信息缺口。

1. LI.FI —— 跨链流动性的"路由中间件"，卖给开发者

谁做的

Philipp Zentner + Max Klenk，2021 年创立。LI.FI 是公司 / 协议，不是交易所。

是什么

把 30 多个 DEX、20 多座跨链桥、以及意图 (intent) 结算系统聚合进一套 API / SDK / 可嵌入 Widget，自动算出"最便宜 + 最快 + 最安全"的跨链兑换路径。自我定位是"DeFi 基础设施与 dApp 之间的中间层"。

核心价值

- 冗余：某座桥宕机 / 拥堵 / 被黑，自动改路由，接入方的产品不跟着挂。
- 连通性：一次集成即接到 Uniswap、1inch、Stargate、Across 等全部主要流动性来源。

商业模式

纯 B2B（分层套餐 + 合作方门户）。客户是钱包和交易所本身 —— Robinhood、Binance、Kraken、MetaMask、Phantom、Ledger、Hyperliquid、Circle、支付宝等，接近 1,000 家 B2B 伙伴。

面向 C 端的门面

Jumper Exchange —— 自家跨链兑换前端，跑在同一套协议上。

数据 / 融资

- 月交易量从 2024 年 10 月的 11.5 亿美元涨到 2025 年 10 月的 80 亿美元 (+595%)。
- 2025 年 12 月完成 2,900 万美元 A 轮扩展 (Multicoin + CoinFund 领投)，累计融资 5,170 万美元；此前 2023 年 5 月 A 轮 1,750 万美元。
- 2026 年 Q1 计划推出开放 intent / solver 市场，并投入 AI agent 与稳定币基础设施。

风险实录

2024 年 7 月 16 日，新加的一个合约 facet 缺少校验、可被任意调用，约 1,000 万至 1,160 万美元被盗，波及 Ethereum / Arbitrum 上 153 个设置了"无限授权"的钱包（有限授权不受影响，且有限授权是 API / SDK / Widget 的默认值）。这已是同类问题第二次发生。

代币

目前没有上线流通的官方代币。空投农场圈流传的 "\$LZRD" 属未证实 / 未发布状态，不要当既成事实。

1. Backpack —— 垂直一体化的持牌交易所 + 自托管钱包

谁做的

Armani Ferrante，2022 年创立。他是 Solana 上 Anchor 框架的作者（约一半 Solana 程序基于它）。FTX 暴雷让他们损失 1,450 万美元，约占运营资金的 88%；"反 FTX"是这家公司的叙事底色。

产品三件套

- Backpack Exchange (CEX)
- Backpack Wallet（自托管多链钱包：Solana / Ethereum / Bitcoin / Sui / Monad），早期以 xNFT（可执行 NFT）出名
- Mad Lads NFT 系列

交易所能做什么

现货、永续合约、借贷、钱包内 swap 与跨链、自动化策略；另外上线了 7x24

代币化美股经纪业务（覆盖 150 多个国家，起手标的为 SpaceX、Micron、SanDisk，法币或稳定币即时结算）。Backpack Card 已进入私测。

合规是它的核心护城河

- 迪拜 VARA 的 VASP 牌照
- 2025 年 1 月以 3,270 万美元收购 FTX EU，拿到塞浦路斯 MiFID II 牌照
- 2025 年 9 月开出 Backpack EU，自称欧洲唯一受监管的永续合约提供方
- 2026 年 5 月 27 日拿到拉脱维亚央行 MiCA CASP + 支付机构双牌
- 2024 年 12 月成为日本 JVCEA 二类会员（Binance 之后首家新会员）

安全

MPC 托管 + 储备金证明；自 2022 年上线以来无已知被黑记录。

代币 BP

2026 年 3 月 23 日发行，总量 10 亿枚，26% 用于空投（积分持有者 2.4 亿 + Mad Lads 持有者 1,000 万）；质押 BP 进入 Backpack Participant Program 换取手续费与平台权益。

规模

永续合约 24 小时成交量约 1.16 亿美元、未平仓约 8,020 万美元（快照口径）；未平仓结构 BTC-PERP 占 47.7%、SOL 占 21.1%、ETH 占 15.6%；历史上单日永续成交量曾突破 10 亿美元。

\=====

二、相同点

=====

1. 都在打“碎片化”这一个战争，只是战场不同 —— 加密资产被切散在几十家交易所、上百条链、两种托管模式里，这三家都在做“把散的重新聚起来”。
2. 都是聚合 / 统一入口的思路：CrossEx 统一多所账户，LI.FI 统一多桥多 DEX，Backpack 统一钱包、现货、永续、美股、借贷到一个 App。
3. 都靠“路由 + 规模”赚钱：本质是通过量的汇聚拿到更好的执行价或更低的费率档（CrossEx 合并 VIP 计量、LI.FI 抽路由量、Backpack 靠自有量做市与费率）。
4. 都在 2025 至 2026 年明显扩张：CrossEx 的 AUM +128%、LI.FI 月量 +595% 并完成融资、Backpack 发币并连拿牌照。
5. 都以“专业 / 机构级基础设施”自我包装，不满足于做单一散户产品。

\=====

三、不同点（按维度逐项对照）

=====

【所处层级】

Gate CrossEx：CEX 之上的机构清结算 / 保证金层

LI.FI：链上流动性路由中间件

Backpack：场所本身（CEX）+ 钱包

【中心化程度】

Gate CrossEx：完全中心化，Gate 做信用中介

LI.FI：完全非托管，链上合约执行

Backpack：CEX 托管 + 钱包自托管，两条腿

【聚合的是谁的流动性】

Gate CrossEx：别家 CEX（Binance / OKX / Bybit 等）

LI.FI：别家 DEX 与跨链桥

Backpack：自家撮合簿 —— 它被聚合的一方

【主要客户】

Gate CrossEx：量化基金、做市商、机构交易台

LI.FI：开发者 / 企业（B2B），终端用户仅通过 Jumper 间接接触

Backpack：散户 + Solana 原住民 + 做市商项目

【接入方式】

Gate CrossEx：机构申请，API 为主，非自助

LI.FI : 接 API / SDK / Widget , 几分钟集成
Backpack : 注册即用, App 或网页

【赚钱方式】

Gate CrossEx : 交易费 + 保证金层 (费率模型未披露)
LI.FI : 分层套餐 / 路由抽成, B2B SaaS 型
Backpack : 交易费、经纪费、BP 代币经济

【监管姿态】

Gate CrossEx : 全球离岸交易所的机构业务
LI.FI : 非托管中间件, 本身不需牌照
Backpack : 牌照最重 —— VARA / MiFID II / MiCA / JVCEA

【主要风险类型】

Gate CrossEx : 交易对手与信用风险 (钱在 Gate, 且跨所信用敞口叠加)
LI.FI : 智能合约 + 底层桥风险 (2024 年已实证被盗约 1,160 万美元)
Backpack : CEX 托管风险 (用 MPC + 储备金证明 + 牌照对冲) + 代币经济风险

【有无代币】

Gate CrossEx : 无 (Gate 有 GT, 但非 CrossEx 专属)
LI.FI : 无上线代币
Backpack : 有, BP, 2026-03-23 发行

一句话切分 :

- 你是跑多所套利的量化团队 -> CrossEx 解的是"保证金锁死在各家所"
- 你是做钱包 / dApp / 交易所前端的开发者 -> LI.FI 省掉"自己接二十座桥"
- 你是在受监管的地方交易、又要自己拿私钥的用户 -> Backpack 是那个场所

\=====

四、一个值得注意的关系 : 它们更像上下游, 而非竞争对手

=====

LI.FI 的客户名单里已经有 Binance、Kraken、Robinhood、Phantom、MetaMask ——
也就是说, Gate 这类交易所的链上出入金与兑换、Backpack 这类钱包的 swap 与跨链, 正好是 LI.FI 在卖的东西。

真正互为直接对手的是 :

- CrossEx vs 传统加密 Prime Broker / 场外结算商 (Copper、Ceffu、FalconX 这一档)。CrossEx 的差异化是"交易所原生、毫秒级内部信用划转"。
- Backpack vs 其他持牌永续交易所
- LI.FI vs 其他桥聚合器 (Socket / Bungee、Rango 等)

风险提示 :

CrossEx 把多家交易所的信用敞口集中到 Gate 一个点上 —— 这既是它效率提升的来源, 也是它最大的单点风险 ; 而它的托管安排恰恰是公开信息里最薄的一块。

笔记 031:

作者 : LIQIHUEI | 时间 : 2026-08-08 15:31 UTC | 字数 : 3110 | 评分 : 59
来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

合约架构	
代理合约 (Proxy)	
0x1521027B665FA38Fa4a642991607ad708376dD7b	
类型: ERC1967 代理	

在 21 天内，怎样把“研究链上套利”收敛成一个能够采集数据、计算成本并被证伪的具体问题？

我做了什么

今天先重新梳理了共学目标和参与规则。这个共学没有固定课表，重点不是听完多少材料，而是持续行动并用真实证据验证假设。因此我没有同时研究 DEX 间、三角、清算、MEV 等所有方向，而是只保留一个候选：

研究 Base 与 Arbitrum 上 WETH/USDC 的双边库存型价格差。基准规模固定为 1,000 USDC，使用 LI.FI 等报价工具比较两侧可执行价格，并把 Gas、滑点、失败损失和库存再平衡成本全部计入。

策略假设两条链预先都有库存。信号出现时同时买低卖高，之后再计算跨链再平衡成本。我暂不采用“买入后跨链、到账后再卖”的顺序路径，因为桥接期间的价格变化会把它变成带方向风险的交易，不能仅凭屏幕价差称为套利。

证据与决策

| 维度 | 决定 |

| :----- | :----- |

| 链 | Base 与 Arbitrum |

| 资产 | WETH/USDC |

| 名义规模 | 1,000 USDC |

| 执行模式 | 双边库存、同时 Paper Trade |

| 主要指标 | 扣除全部成本后的 net_edge_usd 与 net_edge_bps |

| LI.FI 用途 | 路由/报价比较与库存再平衡成本估算 |

| 实盘 | 本阶段不做 |

我把下面这条作为可证伪假设：

两条链偶尔会出现可执行价差，但只有当价差大于两侧 Gas、滑点、再平衡、失败期望损失和安全缓冲之和时，才算候选信号。

零假设同样成立：加入全部成本和时间约束后，可能根本没有可重复的正净 Edge。

风险边界

只做公开报价查询和 Paper Trading，不连接主钱包、不签名、不授权、不广播交易。

私钥、助记词、Cookie 和 API 密钥不进入代码、笔记、截图或日志。

Token 必须核对链 ID、合约地址和 decimals，不能只按符号匹配。

两侧报价时间差超过 5 秒就不作直接比较。

保存负收益和失败样本，不只记录“看起来赚钱”的结果。

不把一次正样本当作策略，不在完成成本模型前讨论实盘。

今天的结论

今天完成了研究范围的第一次收敛。结营目标不是证明套利一定赚钱，而是回答一个更清楚的问题：在固定链、资产和规模下，是否存在扣除全部成本后仍可重复的净 Edge？

完整研究章程已经单独整理，后续所有数据字段、脚本和复盘都以这套口径为准。

明天只做一件事

画出套利机会地图，对比 DEX 内、DEX 间、跨链、三角、稳定币、清算和 MEV，解释为什么本期最终只保留“Base/Arbitrum + WETH/USDC + 双边库存”这一条路径。

笔记 033: Hermes Agent 研究与信号工作流学习笔记

作者：OxPrint | 时间：2026-08-08 15:40 UTC | 字数：6897 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Hermes Agent 研究与信号工作流学习笔记

1. Hermes Agent 在这套体系里的定位

Hermes 最适合承担的是研究自动化与机会证据管理。

整个流程可以设计成：

发现异常

↓

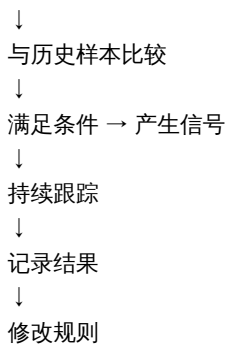
自动收集数据

↓

统一整理

↓

建立机会证据



Hermes 本身支持网页搜索、终端执行、文件操作、代码执行、子 Agent、持久记忆、Skills 和定时
Cron，因此很适合把一次研究逐渐固化成长期运行的工作流。

这里最重要的思想是：

Agent 负责发现、研究、解释和迭代；脚本负责精确采集和计算。

例如每 10 秒请求一次 LI.FI、Binance、OKX 报价，这类事情应该让 Python 做；Hermes
每隔一段时间读取数据，寻找异常、解释路径变化、整理候选机会。

2. 从“一次观察”变成“可重复研究”

假设某天看到：

Arbitrum USDC

→ Base USDC

LI.FI 路径突然便宜很多

低效做法是看一眼，然后记住“Across 好像便宜”。

正确研究方式是把它转换成一个固定实验：

每 30 秒：

查询

Arbitrum → Base

资产

USDC → USDC

金额

\$100

\$1,000

\$5,000

\$10,000

保存：

时间

桥

路径

输入

输出

Gas

费用

预计时间

运行几天以后，你得到的就从：

“Across 好像便宜。”

升级成：

“Arbitrum→Base 的 1,000–5,000 USDC 区间，Across 在过去 7 天约 68% 的时间提供最佳报价；超过 10,000 USDC 后最优路径经常切换。”

后者才具有交易研究价值。

3. Hermes 应负责的五层工作

| 层 | Hermes 做什么 | 结果 |

| :--- | :----- | :--- |

信息发现	搜索协议更新、桥异常、新池、新交易对	候选主题
研究拆解	把“这里可能有机会”拆成变量	实验计划
工具调用	调 Python、API、网页、终端	原始数据
分析整理	对比路径、金额、时间、成本	研究结论
自动监控	Cron + Skill 定期运行	信号/报告

Hermes 支持把多步骤流程保存为 Skill。官方建议重复出现的多步骤工作创建 Skill，以后 Agent 可以按相同程序再次执行。
这非常适合套利研究。

例如建立：

```
skills/  
├── lifi-route-research/  
├── cex-dex-spread/  
├── bridge-liquidity-watch/  
├── stablecoin-depeg/  
└── opportunity-review/
```

以后你只需要告诉 Hermes：

研究 Base → Arbitrum 的 USDC 跨链机会

它读取 lifi-route-research Skill，自动按照固定方法执行。

4. 最重要的东西：Opportunity Evidence

每一个候选机会都建立一条证据记录。

我建议采用这样的数据模型：

字段	示例
opportunity_id	LIFI-20260808-001
时间	2026-08-08 14:32:08 UTC
类型	cross_chain_spread
源链	Arbitrum
目标链	Base
源资产	USDC
目标资产	USDC
金额	10,000 USDC
路径	Across
源链 DEX	无
Bridge	Across
目标链 DEX	无
预计输出	10,013.2 USDC
Gas	\$0.41
Bridge Fee	\$1.84
价格优势	+0.15%
预计时间	42 秒
报价有效期	30 秒
状态	candidate
实际执行	否
失败原因	—
数据来源	LI.FI
原始响应	JSON 文件路径

一个机会从发现到结束可以有状态：

```
DISCOVERED  
↓  
VALIDATED  
↓  
SIGNAL  
↓  
EXECUTED
```

或者

REJECTED

EXPIRED

FAILED

以后研究“为什么套利策略不赚钱”时，这些数据特别重要。

5. 失败记录比成功记录更有价值

失败必须分类。

例如：

failure_reason 含义
:----- :-----
SPREAD_GONE 价差消失
QUOTE_EXPIRED 报价过期
GAS_TOO_HIGH Gas 激增
SLIPPAGE 滑点超过限制
BRIDGE_CAPACITY 桥库存不足
NO_ROUTE 聚合器无可执行路径
PRICE_IMPACT 冲击成本过高
INSUFFICIENT_LIQUIDITY 流动性不足
RPC_ERROR RPC 故障
TX_REVERT 链上交易回滚
EXECUTION_TOO_SLOW 执行时间过长
HEDGE_FAILED 对冲腿失败

几周以后可以统计：

发现 1,284 个候选机会

- └─ 823：价差消失
- └─ 181：流动性不足
- └─ 103：Gas 后无优势
- └─ 77：桥库存不足
- └─ 61：可执行
- └─ 39：其他

这时候你已经知道应该优化什么。

6. Hermes 的持续监控机制

Hermes 有原生 Cron，可以定时启动新的 Agent 会话，并且 Cron 可以加载指定 Skill。结果可以保存到本地文件，也可以发送 Telegram 等渠道。

例如：

每 5 分钟

↓

运行 cross-chain-opportunity Skill

↓

读取过去 5 分钟报价数据库

↓

寻找异常

↓

发现值得关注的机会

↓

生成 evidence

↓

Telegram 通知

这里有一个很重要的成本优化。

高频任务

例如：

每 10 秒查询 LI.FI

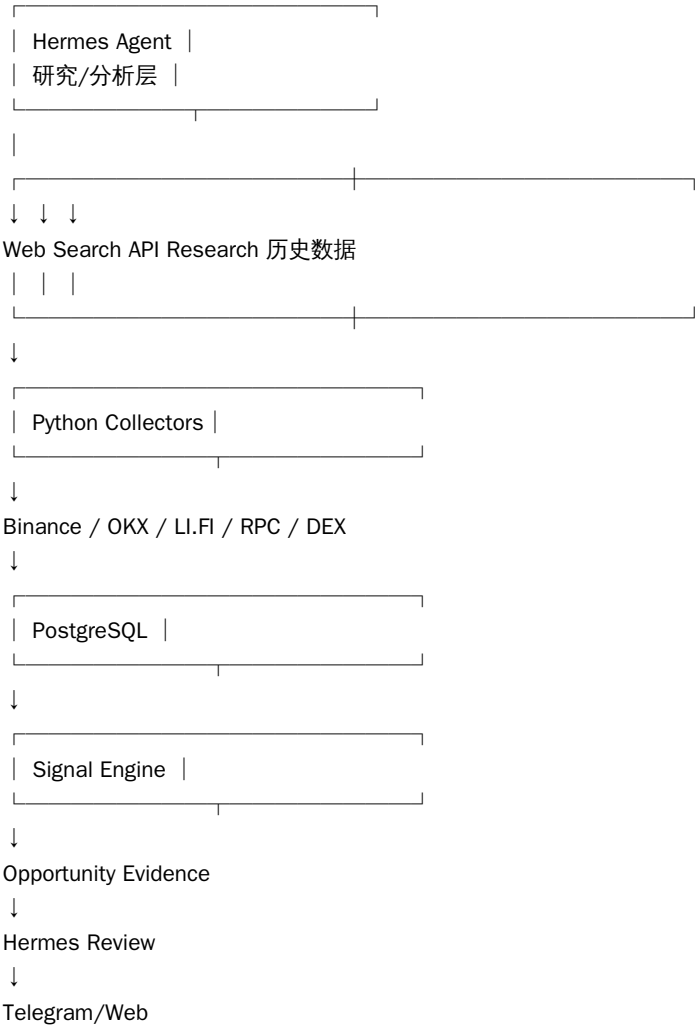
每 2 秒查询 Binance

计算价差
写 PostgreSQL
使用普通 Python 服务。
低频智能任务
例如：
每 15 分钟：
分析最近数据
识别异常
解释为什么路线变化
寻找新的金额区间
使用 Hermes。

Hermes Cron 甚至支持完全不调用模型的 no-agent 模式，让固定脚本按计划运行，stdout 直接作为结果发送，可以节省模型 Token。

7. 一个真正适合你的研究架构

我建议以后整体变成：



Hermes 放在上层做研究。
底层行情采集、计算和交易继续保持确定性程序。

8. Hermes 的 Memory 和 Skill 应该分别存什么

官方对此区分得很清楚：

Memory 保存事实，Skill 保存方法。

例如 Memory：

研究过程中发现：

Base → Arbitrum

USDC 大额路线经常切换。

Across 在小额区间表现较稳定。

Skill :

如何研究跨链路径：

1. 请求多个金额
2. 同时请求 FASTEST/CHEAPEST
3. 保存所有 route
4. 分离 Gas、Bridge、DEX Fee
5. 与过去 24h 中位数比较
6. 异常超过阈值生成 opportunity

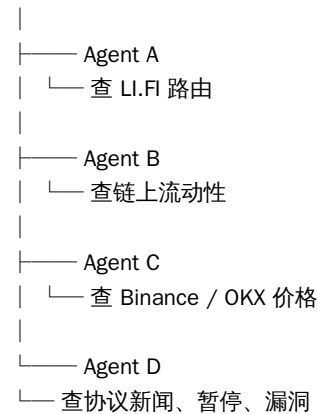
这两个结合起来，Hermes 才会越用越有价值。

9. 子 Agent 可以怎么用

Hermes 支持把任务委派给隔离上下文的子 Agent，完成以后只把结果返回主 Agent。

例如同时研究一个机会：

主 Agent



最后主 Agent 综合：

机会：

ETH Base → Arbitrum

价差：0.74%

预计跨链成本：0.19%

持续时间：4m32s

历史出现次数：17

流动性：充足

风险：中

结论：

值得继续监控

这比让一个 Agent 从头到尾连续查更适合复杂研究。

10. 最终要形成三个层次

第一层：Observation

我发现 Base 上 ETH 比 Arbitrum 贵。

价值最低。

第二层：Query

每分钟比较：

Base ETH

Arbitrum ETH

LI.FI 搬运成本

已经可以重复。

第三层：Signal

当：

两链价差 > 0.8%

AND

LI.FI 搬运成本 < 0.25%

AND

可交易金额 > \$10,000

AND

连续出现 3 次

生成信号。

这时候已经进入策略阶段。

再继续：

Signal

↓

Evidence

↓

人工确认

↓

Execution

↓

Result

↓

复盘

就形成完整研究闭环。

11. 我的学习结论

Hermes 真正值得利用的地方是把研究方法积累下来。它支持持久 Memory、Skills、会话搜索、子 Agent、终端/网页工具和 Cron，因此一项手工研究可以逐步演化成固定 Skill，再演化成定时任务和持续信号系统。

对于套利研究，最终应该形成：

人工发现

→ Agent 研究

→ 固定研究模板

→ Python 数据采集

→ Evidence 数据库

→ Signal

→ 自动监控

→ 执行结果

→ Agent 复盘

→ 修改规则

笔记 034: 今天是我的第二天学习，我阅读了《套利共学 | 为什么我在熊市发起链上套利残酷共学？顺便分享下我的学习大纲

作者：JackChang-01 | 时间：2026-08-08 15:42 UTC | 字数：401 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天是我的第二天学习，我阅读了《套利共学 | 为什么我在熊市发起链上套利残酷共学？顺便分享下我的学习大纲》这篇文章，我并且按照大纲的要求，并且让ai辅助我开始学习。

在开始学习之前我想记录一下我阅读的感悟，之前我理解的套利其实对于我价值观而言是有一些贬义的成分，我的认知让我了解这个词缺乏长期主义和缺少真实价值，我的偏见把这个词当作是机会主义的化身，短暂受打压的代表。但是今天我阅读了之后我的价值观有了很大的改变，现实中商人也同样是“把西边的东西卖给东边”赚取差价，这不就是套利么，所以世界充满了套利。我应该改变过去对于这个词的看法，并且去学习如何链上套利，跨链套利。这才是我这次学习的意义，通过这次残酷的学习经历，也让我对自己的学习能力清晰的认识。以下是我今天所学习的内容：

理解概念：DEX 内套利、DEX 间套利、跨链套利、三角套利、稳定币套利、清算和 MEV 分别为什么出现？

笔记 035: 在 Web3 / 链上交易 (DeFi) 的语境下, “路由” (Routing) 指的是将你的交易订单从“

作者 : bizzarespinach | 时间 : 2026-08-08 15:52 UTC | 字数 : 1136 | 评分 : 73
来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

在 Web3 / 链上交易 (DeFi) 的语境下, “路由” (Routing) 指的是将你的交易订单从“输入资产”匹配并兑换为“输出资产”的具体路径规划算法或机制。
简单来说, 路由就像是 “金融导航软件”。当你想从资产 A 换成资产 B 时, 路由负责为你寻找一条资金损耗最少、成交概率最高的路线。
为什么要使用“路由”?

在去中心化交易所 (DEX, 如 Uniswap、Curve、Balancer 等) 中, 资金流动是分散在成千上万个不同的流动性池 (Liquidity Pools) 里的。
很多时候, 你想要交易的两个代币之间没有直接的流动性池, 或者直接兑换的滑点 (Slippage) 极高。这时候就需要路由来帮忙打通路径。

路由的主要工作机制

路由主要通过以下三种方式帮你完成最佳交易路径的规划 :

1. 多跳路由 (Multi-hop Routing / 间接兑换)

如果你想用代币 \$X\$ 换代币 \$Z\$, 但市场上没有 \$X/Z\$ 的直接流动性池, 路由会自动将交易拆解为多个步骤 :

路径 : $\$X \rightarrow Y \rightarrow Z\$$

例子 : 先用 \$X\$ 换成流动性极佳的公用中介代币 \$Y\$ (如 \$ETH\$ 或 \$USDC\$), 再用 \$Y\$ 换成 \$Z\$。

2. 拆单路由 (Split-path Routing / 路径分割)

如果你交易的规模 (Amount/Size) 较大, 把所有资金砸进单一流动性池会导致剧烈的价格冲击 (Price Impact)。路由会把你的订单拆分成多份, 走不同的池子同时执行 :

路径 :

\$40\%\$ 的资金走 : 池子 1 ($\$X \rightarrow Z\$$)

\$35\%\$ 的资金走 : 池子 2 ($\$X \rightarrow Y \rightarrow Z\$$)

\$25\%\$ 的资金走 : 池子 3 ($\$X \rightarrow WETH \rightarrow Z\$$)

3. 跨链/聚合路由 (Cross-chain & DEX Aggregator Routing)

聚合路由 (如 1inch, ParaSwap) : 扫描整条链上数十个 DEX 的池子, 组合出一条最优的链上兑换路径。

跨链路由 (如 Stargate, Hop, Li.Fi) : 涉及不同链 (Chain) 时, 路由不仅要规划 DEX 路径, 还要计算跨链桥 (Bridge) 的延迟、手续费 (Gas) 和跨链流动的安全性, 将“跨链桥接 + 链上兑换”组合成一步操作。

笔记 036: Day 5 打卡 : V3 Tick 深度解析 + LI.FI API 实操 + 跨链 Route 评估

作者 : vera | 时间 : 2026-08-08 15:54 UTC | 字数 : 2915 | 评分 : 89
来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 打卡 : V3 Tick 深度解析 + LI.FI API 实操 + 跨链 Route 评估实验

今日学习内容

一、Uniswap V3 Tick 机制深度拆解

1. 为什么是 1.0001 ?

- $price = 1.0001^{tick}$, 相邻 tick 之间价格差 0.01%
- $1.0001^{10000} \approx e$, 每 10000 个 tick 价格翻 e 倍
- 对数刻度 : 价格变化是乘法关系 (涨2倍= $\times 2$), 不是加法关系
- tick 范围 : -887272 到 +887272, 覆盖 2^{-128} 到 2^{128}

2. TickSpacing : LP 区间边界的对齐规则

| 费率档 | tickSpacing | 含义 |

|-----|-----|-----|

| 0.01% | 1 | 每 tick 可做边界 (最精细) |

| 0.05% | 10 | 边界只能是 0, 10, 20... |

| 0.30% | 60 | 边界只能是 0, 60, 120... |

| 1.00% | 200 | 边界只能是 0, 200, 400... |

tickSpacing 越大，穿越边界频率越低，Gas 越省。类比：公交车站，0.01%池每站停，1%池大站快车。

3. LP 选区间的具体计算

- ETH \$2000 -> tick ≈ 76001 -> 对齐到 76000 (tickSpacing=10)
- 区间 \$1800-\$2200 -> tickLower=74940, tickUpper=77000
- 只有区间内的资金"干活"赚手续费，出区间后"休眠"

4. Active Tick 与流动性跳变

- 当前价格所在的 tick 叫 active tick
- 价格穿越 tick 边界时流动性突然增减 -> 阶梯状滑点的根源
- 类比：V2 是全程一样宽的公路，V3 有的路段8车道有的1车道

5. sqrtPriceX96

- 合约内部不存 price，存 $\text{sqrtPriceX96} = \sqrt{\text{price}} \times 2^{96}$
- 用平方根形式让交易计算只需一次加法
- 乘 2^{96} 是固定浮点数精度，可用 gas 高效的位运算

二、LI.FI API 安装与基础功能测试

参考 <https://docs.li.fi/agents/overview> 完成工具调研和测试。

LI.FI 三种集成方式：

- REST API：直接 HTTP 调用，无需安装（选择此方式）
- MCP Server：hosted 在 <https://mcp.li.quest/mcp>，需 MCP 客户端
- CLI：token 效率更高的文本输出

5 个基础测试全部通过：

1. /chains - 支持 69 条 EVM 链
2. /tokens - 确认 USDC 地址和 decimals (Ethereum: 0xA0b8...eB48, Arbitrum: 0xaf88...5831, 均 6 decimals)
3. /tools - 35 个桥 + 35 个 DEX 聚合器
4. /quote - 1000 USDC 跨链报价，选中 Eco 桥，7 秒到达
5. /advanced/routes (POST) - 返回 16 条候选路线对比

三、LI.FI 跨链 Route 评估实验（只读，不执行交易）

实验目标：用 1000 USDC 做 Ethereum $\rightarrow$ Arbitrum 跨链报价评估，计算 Break-even Spread，判断稳定币跨链套利是否可执行。

核心数据：

指标 数值
----- -----
输入 1,000 USDC (Ethereum)
最优到账 997.50 USDC (Arbitrum, Eco 桥)
总成本 \$2.50 (25.00 bps)
其中 LI.FI 服务费 \$2.50 (25 bps, 占 91%)
其中 Gas \$0.12 (1.22 bps)
链间实际价差 4.37 bps
Break-even Spread ~27.30 bps
缺口 -22.93 bps

16 条路线对比关键发现：

- 最便宜 Eco：25.00 bps（推荐）
- 最快 Across：2 秒到达，26.05 bps
- 最差 Symbiosis：47.57 bps
- 所有路线成本底线都是 ~25 bps (LI.FI 固定费不可绕过)

金额规模测试（U 型成本曲线）：

- 1 USDC：80.54 bps (Gas 占比高)
- 100 USDC：26.54 bps
- 1,000 USDC：25.00 bps (最优点)
- 10,000 USDC：35.48 bps (滑点开始显现)

结论：稳定币跨链套利不可行

- 链间价差 (4.37 bps) 远低于跨链成本 (27.30 bps)
- 主要卡在 LI.FI 0.25% 固定服务费 (占总成本 91%)
- 与知识总结中的 27.34 bps 门槛高度吻合

实验定位：属于套利全流程的 ②信息收集 + ④成本核算 环节，是花钱之前的可行性验证。

四、实验报告已归档

完整实验报告已保存到 Obsidian：D:\Obsidian\学习资料\链上套利残酷共学\LI.FI实验报告.md

包含：6 步实验流程、16 条路线完整对比表、Break-even 计算过程、金额规模测试、瓶颈分析、下一步建议。

今日核心认知

1. tick 是对数刻度的离散化：不是均匀价格轴，每 0.01% 画一道线，LP 只能在有“公交站”的地方上下车
2. 流动性跳变 = 套利机会：V3 阶梯状滑点的根源是 tick 边界的流动性增减，大单交易穿越低流动性 tick 时产生可预测的滑点
3. LI.FI 服务费是稳定币跨链套利的致命瓶颈：25 bps 固定费吃掉 91% 成本，链间价差只有 4 bps，根本不够覆盖
4. 成本随金额呈 U 型曲线：小额 Gas 占比高，大额滑点占比高，1000 USDC 是最优点
5. 套利可行性验证可以用只读 API 完成：不需要花钱就能算清 Break-even，这是套利全流程最前置的环节

明日计划

1. V3/V4 提问检验（已跟其他 AI 学过，需要查漏补缺）
2. 研究绕过 LI.FI 直接调桥合约的方案（省掉 25 bps 服务费）
3. 考虑转向非稳定币套利（ETH/WBTC 跨链有真实价格波动）

笔记 037:

作者：Cool-CoCo | 时间：2026-08-08 15:58 UTC | 字数：1602 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月7日（周五）—— 整合与总结

今天把之前的知识串起来了，也参考了其他同学的笔记，做了个完整总结。

一、套利策略全景图

根据其他同学的优秀笔记，链上套利策略可以归纳为九大类：

1. DEX内套利（同一个DEX的不同池子）
2. DEX间套利（不同DEX之间）
3. 跨所套利（DEX vs CEX）
4. 跨链套利（不同链之间）
5. 三角套利（三个交易对循环）
6. 脱锚套利（稳定币脱锚时）
7. 清算套利（参与链上清算）
8. MEV套利（抢跑、三明治等）
9. 统计套利（基于历史数据的策略）

二、成本结构详解

7类成本需要全部考虑：

1. 交易手续费 - DEX收取的百分比费用
2. Gas费 - 链上执行成本，对小额交易影响最大
3. 滑点 - 交易对池子价格的影响
4. 跨链成本 - 桥的费用
5. 融资成本 - 如果用了借贷
6. 失败成本 - 失败交易仍会消耗Gas
7. 机会成本 - 资金占用成本

Gas占比计算公式： $\text{Gas占比} = \text{Gas美元成本} \div \text{输入美元价值}$

关键发现：金额增大时，Gas占比通常下降，但流动性和价格冲击的重要性会提高。

三、执行中的关键点

报价过期问题：发现报价 → 判断机会 → 授权 → 签名 → 源链确认 → 跨链 → 目标链执行

时间差内价格可能已经变化，跨链套利尤其需要考虑两端价格漂移。

为什么路径会突然变化（相同参数几十秒后重新询价）：

某个桥目标链库存变化

DEX池价格变化

Gas价格变化

求解器更新报价

大额交易消耗了流动性

某个桥临时限额或暂停

四、可以发现的几种机会

1. 路径优化机会：同样的资产和金额，不同桥产生不同输出
2. 规模分层机会：大额拆分可能获得更好输出
3. 跨链市场价差：目标链资产价格较高，跨链成本低于价差
4. 再平衡机会：资金分布在两条链上，同时完成买入和卖出

五、风险意识

主要风险：

费用侵蚀（Gas、手续费、桥费）

竞争与抢跑（被其他机器人抢先）

执行风险（交易失败、价格变化）

技术风险（合约漏洞、桥被攻击）

安全建议：

合约代码必须经过多人审查

使用OpenZeppelin等安全模板

测试网充分测试后再上主网

私钥不得分享

六、实操建议

如何开始研究：

1. 固定测试矩阵，每次只改变一个变量
2. 连续采样（每30秒一次），寻找路径切换点和费用异常
3. 同时请求FASTEST和CHEAPEST两种排序
4. 记录每次报价的完整数据，统一换算成美元比较

需要记录的数据：采样时间 源链 / 目标链 源资产 / 目标资产 输入数量 预计输出 toAmount 最低输出 toAmountMin 输出美元价值
Gas美元成本 桥费用 / DEX费用 / LI.FI服务费 总费用 价格冲击 预计执行时间 使用的桥 / DEX 执行步骤数量 路径ID 报价有效时间

七、三天学习总结

从8月5日的小白状态，到8月6日开始理解工具和结构，再到今天能串起整个知识框架。链上套利是一个系统工程，不仅需要理解价格差异，还要掌握执行工具、成本计算和风险控制。

下一步计划：

动手跑一下LI.FI的API，获取真实报价数据

设计自己的测试矩阵，开始连续采样

学习Solidity合约编写，为闪电贷套利做准备

以上为8月5日-7日的三天的学习笔记，知识点略多，需要多消化。

笔记 038: 8.8完全的小白先入门\

作者：Krystal6633 | 时间：2026-08-08 15:58 UTC | 字数：158 | 评分：47

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8.8完全的小白先入门\

套利不是"看到差价就冲"，而是"算完所有成本还有利润才冲"。

你今天看到了一个真实数据：LI.FI 服务费 0.25% 是跨链的硬成本下限。这意味着：

如果两个市场之间的价差 小于 0.3%，做跨链套利基本是亏的

之前觉得"赚钱的机会"，可能 75% 以上都被 Gas 和服务费吃掉了

笔记 039: 链上套利残酷共学·打卡

作者：Oxlareina | 时间：2026-08-08 16:32 UTC | 字数：1061 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · 打卡

今日主题：数据源可靠性排查

凌晨发现一条 "+14bps 可吃" 信号，复核后确认是假象。切回 DexScreener 真实池价核

对，顺带把 ETH/ARB/OP/Polygon/Base/BSC/AVAX 六链稳定币跨 DEX 价差全扫了一遍，得出当前规模下不值得真实投入的结论。

核心发现：14bps 假象是怎么来的

根因：Base 上可用池子少、深度差，"最优路由"常是流动性不足的冷门池，报价虚高但不具备真实成交能力。

关键教训：套利信号必须用 真实池价验证。判断一个价差能不能吃，先看池子的真实流动性，而不是先看报价数字任何"两个数据源打架"的场景，先信能直接验证成交深度的一手数据源

全链扫描结论

[图片]

晚间用实时监控 dex_pool_monitor.py (每分钟扫 Arbitrum/Base/Optimism/Polygon) 复

核，最新一轮原始价差最高也只是 Polygon USDC/USDC 3.0bps，仍远低于覆盖滑点+gas 所需门槛。

结论：跨链/跨 DEX 稳定币套利在当前规模 (\$500~\$2000 试水档) 经济上不划算——桥费+时间成本+专业机器人的实时竞争，把绝大多数账面价差都吃掉了。这类微价差基本被 MEV/套利机器人实时抹平，人工介入没有窗口。

TODO 进度更新

T1 确定监控路径清单 (3-5 条稳定币跨链路径 × 3 档规模：\$1k / \$10k / \$100k)

T2 写报价抓取脚本 (LI.FI 版验证不可靠，已切换为 DexScreener 真实池价版)

T3 接入 cron 定时跑，落盘日志 (dex_arb_monitor 每10分钟 + dex_pool_monitor 每分钟)

T4 跑满 1-2 周，检查数据完整性

T5 写净收益计算逻辑

T6 产出净收益分析表与结论

T7 决策：进入执行层 or 复盘归档

今日进度：3 / 7 完成

下一步：T4，继续跑 dex_arb_monitor (净 >3bps 才 TRIGGER) 观察 1-2 周，当前判定持续全负，暂不动钱。

笔记 040: 2026-08-09 | 共学 Day 5 | 投入约 1 小时

作者：wuyuandao | 时间：2026-08-09 01:08 UTC | 字数：378 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-09 | 共学 Day 5 | 投入约 1 小时

今天学了什么：

用 LI.FI API 调了 5 条真实跨链 Quote——100 USDC Arb→OP、10,000 USDC Arb→BSC、ETH 跨链比价 (Arb vs OP 价差仅 0.08%)、USDC 三链对比 (Arb→OP/BSC/Polygon 损耗都在 0.25% 左右)。每条 Quote 拆解了路由步骤、费用项和损耗比例。

得到什么结果：

- ① LI.FI 费是跨链最大成本 (固定 0.25%)，稳定币跨链基本被吃光
- ② Gas 可忽略，不管金额大小都是几毛钱
- ③ 正常市场下主流币价差不到 0.1%，稳定币无脱锚，不存在套利空间
- ④ 真实机会转瞬即逝，手动查永远慢一步，必须上脚本监控

困难：无

下一步：The Graph 配置 + 开始接触数据采集脚本

笔记 041: Day 5 — 实战部署与实时监控

作者：pennchester | 时间：2026-08-09 02:08 UTC | 字数：1105 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 — 实战部署与实时监控

今日核心收获

1. 套利机器人架构设计

- 套利机器人不是“一个脚本”，是四层实时系统：监控层→分析层→执行层→控制层
- 监控层：多池子事件订阅 + Multicall 批量查询 + Mempool 监听三管齐下
- 分析层：价差检测→最优规模→Gas估算→MEV风险评估→净利润决策
- 执行层：交易构建 + EIP-1559 Gas策略 + Flashbots私有bundle
- 控制层：熔断器 + 资金热冷分离 + 多节点冗余 + 告警系统

2. 关键基础设施决策

- 监控 50-200 个池子用 Multicall + 事件驱动，不能逐池轮询
- RPC 节点选择直接影响延迟和可见性：公共RPC（免费但慢）→自建节点（极低延迟但\$500-1500/月）
- 2026年 Builder 集中度（Titan 51.5%+Quasar 22.6%=74%）决定 bundle 必须走对渠道

3. 熔断与风控机制

- 5类必备熔断：滑点超标/Gas异常/合约revert率/闪电贷失败/MEV攻击
- 资金结构：主钱包(冷,99%)↔套利钱包(热,1%)↔Gas钱包，物理隔离
- 交易状态机：Pending→等待→成功/超时→加速或取消

4. L1 vs L2 部署策略

- 模拟验证：1.5% 价差在L1(Gas \$46)亏损，在L2(Gas \$0.86)轻松盈利\$6-15
- 策略：L2做90%高频小额机会，L1只抓\$200+利润的大机会
- 跨链套利暂不可行——桥延迟太大

5. 模拟器实战验证

- 编写完整四层架构套利监控模拟器（monitor_simulator.py）
- 60个区块模拟：检测36次机会，执行30笔交易，100%成功率，净利润\$248.57
- 熔断器演示：3次连续失败→自动熔断→冷却→半开→恢复

核心认知

1. 熔断器不是“可能需要的”，是“必须有的”——没有熔断=机器人裸奔
2. 一个RPC节点=单点故障，最少主备两个
3. L2不是银弹——Gas省了但MEV竞争更激烈
4. 套利资金和持仓资金必须物理隔离
5. Builder关系不是玄学——不给他们发bundle凭什么打包你

产出

- 学习笔记：Day5_实战部署与实时监控.md（四层架构+基础设施+风控+多链+告警+十大坑）
- 代码：monitor_simulator.py（完整四层架构模拟器+熔断器+套利检测场景对比）

明日计划

Day 6：具体策略模拟与回测

笔记 042: 2026-08-09

作者：Sansho2 | 时间：2026-08-09 03:09 UTC | 字数：863 | 评分：69

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-09

1. 今天学/做了什么

Day5：把失败与延迟从「提醒」写成可记账项与否决条件。交付 `notes/failure-modes.md` + `notes/day5-plan.md`：八类失败模式（回滚耗 Gas、单腿、Quote TTL、桥在途边消失、冲击幻觉、费率/基差翻脸、余额/授权、资产身份）、延迟拆成机会衰减 vs 结算等待，并挂回 Day2 五步流程。用 Day4 cost_model.py 做失败敏感度对照。

2. 得到什么结果

- 关键洞见：失败若只是烧少量 Gas，期望成本常 <1bps；一旦变成「单腿砍仓」，同一名义下失败项可跳到 数十

bps，与毛利同级。例：跨链毛 40bps、名义 \$1000—— $p_{fail} 5\% \times \$0.4 \rightarrow fail \approx 0.2bps$ 、净仍负； $p_{fail} 20\% \times \$20$ 砍仓 $\rightarrow fail \approx 40bps$ 、净约 43bps。

- 结构否决：无库存的跨链裸搬、非原子第二腿、Quote 超 TTL，优先整笔否决，而不是把 delay 调大 1bps 假装覆盖。

- 同链路径差：验证最优路由 = 执行质量；误读成双边锁利 = 语义失败（立刻落入单腿风险）。

- 双腿纪律不变：不能同时完成两腿就不叫套利。

3. 失败或不确定点

p_{fail} 与砍仓损失仍是启发式，缺实盘拒单统计；未在本日重跑一批 LI.FI TTL 老化实验；Funding 出场细则仅本地，不公开。

4. 对净收益模型的更新

失败项必须区分「Gas 级」与「单腿级」；延迟分 (A) 报价老化 (B) 桥/最终性等待；五步第 4 步（双腿可行）升级为硬闸门。

5. 明天要验证的最小问题

Day6：对照 taoli.tools / QQQ.BOT，看产品如何处理执行、对冲与失败恢复，列可借鉴 vs 可忽略清单。

笔记 043: 2026年8月9日 | 链上套利学习笔记

作者：WikiToTheMoon | 时间：2026-08-09 06:25 UTC | 字数：499 | 评分：59

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026年8月9日 | 链上套利学习笔记

今天继续学习链上套利，对套利机会的理解有了一些新的认识。

之前理解套利，更多关注：

同一个资产在不同地方价格不同，通过低买高卖获得收益。

但进一步学习后发现，真正的套利并不是简单寻找价格差，而是分析：

为什么会产生差异，以及这种差异是否能够被执行。

在链上环境中，不同市场可能存在不同的规则：

不同 DEX 的流动性不同；

不同链的交易成本不同；

不同协议的执行路径不同；

不同市场参与者的信息和反应速度不同。

这些差异可能形成套利机会。

但是，发现差异只是第一步，还需要继续判断：

是否有足够流动性完成交易；

交易成本是否会吞掉收益；

执行过程中是否存在价格变化风险；

最终获得的收益是否真实可实现。

因此，我目前对链上套利的理解变成：

套利本质上是在不同市场规则、流动性和执行条件之间寻找暂时的不一致，并通过交易行为促使价格重新趋于一致。

下一阶段继续学习不同套利模式（DEX

内套利、DEX

间套利、跨链套利等），理解它们分别利用了什么市场差异，以及实际执行中的主要风险。

笔记 044: LI.FI 跨链套利与 AI Agent 集成学习笔记

作者：QQQ | 时间：2026-08-09 06:28 UTC | 字数：4632 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LI.FI 跨链套利与 AI Agent 集成学习笔记

一、LI.FI 在套利系统中的定位

很多人认为：

发现跨链价差 = 可以套利

实际上，发现价差只是第一步。

真正决定是否能够盈利的是：

Net Profit

=

Gross Spread

- Bridge Fee
- Swap Fee
- LI.FI Service Fee
- Gas
- Slippage
- Price Impact
- Latency Cost
- Failure Risk

因此：

LI.FI 不是套利策略，而是套利执行基础设施（Execution Layer）。

策略负责寻找机会，LI.FI 负责回答：

资金应该怎么走？

当前最佳 Route 是什么？

实际执行成本是多少？

如何构造交易？

当前跨链执行到哪一步？

二、跨链套利完整流程

标准流程：

发现价差

|



确认 Token / Chain

|



LI.FI Quote

|



计算真实成本

|



Profit Engine

|



模拟执行

|



交易

|



Status 跟踪

其中：

真正重要的是 Quote。

不能根据价格决定套利。

必须根据：

Quote + Cost Model

决定是否执行。

三、Quote 接口最重要的数据

LI.FI /quote 返回很多字段。

套利系统真正关心的是：

| 字段 | 作用 |

| :----- | :----- |

toAmount	预计收到数量
toAmountMin	最低保证收到数量
feeCosts	所有手续费
gasCosts	Gas 成本
executionDuration	跨链预计耗时
tool	实际 Bridge / Solver
includedSteps	Route 经过哪些步骤
transactionRequest	未签名交易

这些字段共同决定：

真实执行成本。

四、Break-even Spread (盈亏平衡价差)

套利系统真正需要计算：

最低需要价差

=
LI.FI Fee
+
Bridge Fee
+
Swap Fee
+
Gas
+
Price Impact
+
Slippage
+
Latency Cost
+
Failure Cost

例如：

Gross Spread

40 bps

但是：

LI.FI Fee

25 bps

Gas

3 bps

DEX Fee

5 bps

Slippage

8 bps

总成本：

41 bps

最终：

Profit

-1 bps

因此：

理论存在套利

≠

真正可以执行套利

五、不要发现套利以后才跨链

这是跨链套利最重要的思想之一。

很多人：

发现机会

↓

Bridge

↓

资金到账

↓

套利

问题：

跨链可能需要：

数秒

数十秒

数分钟

而套利窗口：

通常只有几秒钟。

因此成熟策略应该：

Ethereum

USDC

ETH

Arbitrum

USDC

ETH

Base

USDC

ETH

提前部署库存。

真正套利：

发现机会

↓

直接在目标链成交

随后：

利用 LI.FI 做：

Inventory Rebalancing

即：

Trade First

↓

Rebalance Later

而不是：

Bridge First

↓

Trade Later

六、LI.FI 更适合库存再平衡

实际生产环境：

LI.FI 更多承担：

资金调度

而不是：

套利抢速度

例如：

Ethereum 库存减少：

USDC

100k

Arbitrum 增加：

USDC

180k

此时：

利用 LI.FI

重新平衡：

Ethereum

140k

Arbitrum

140k

保持下一轮套利能力。

七、AI Agent 如何集成 LI.FI

LI.FI 已提供：

MCP Server

CLI

Agent Skills

SDK

API

因此：

Claude Code

Hermes

Codex

等 Agent

都可以直接调用：

get-chains

get-token

get-quote

get-status

get-allowance

Agent 不需要自己拼 HTTP 请求。

八、推荐的 Agent 架构

Scanner

↓

Opportunity Engine

↓

LI.FI Quote

↓

Cost Engine

↓

Simulator

↓

Risk Engine

↓

Wallet Executor

↓

Chain

其中：

AI Agent：

负责：

查询

分析

Route 比较

利润计算

Wallet：

负责：

Approve

Sign

Broadcast

不要让 Agent 持有私钥。

九、Wallet 与 Agent 解耦

推荐架构：

AI Agent

↓

Execution Request

↓

Wallet Service

↓

Blockchain

Wallet 层应设置：

最大交易金额

最大滑点

Allowlist Token

Allowlist Chain

Allowlist Bridge

Daily Loss Limit

Minimum Profit Threshold

保证 Agent 即使判断错误，也不会无限制交易。

十、LI.FI API 风控

未认证 API：

Quote 与 Route

调用次数有限。

生产环境建议：

使用 API Key

缓存 Chains

缓存 Tokens

缓存 Tools

批量查询

Debounce 请求

处理 HTTP 429

读取 RateLimit Header

避免：

把 LI.FI 当免费行情接口无限轮询。

十一、LI.FI 在套利系统中的职责

LI.FI 更适合作为：

Cross-chain Execution Layer

负责：

Bridge

Swap

Route

Quote

Status

Token Metadata

Gas

Composer

Inventory Rebalancing

而不是：

负责：

寻找套利机会

判断是否值得交易

这些应该由：

Opportunity Engine

和

Risk Engine

负责。

十二、推荐的套利 Agent 闭环

Market Scanner

↓

Opportunity Detection

↓

LI.FI Quote

↓

Route Compare

↓

Cost Model

↓

Simulation

↓

Risk Decision

↓

Execute

↓

Status Tracking

↓

Inventory Rebalancing

↓

Database

↓

AI Analysis

↓

继续优化策略

整个系统可以分为三个核心模块：

① Opportunity Layer（机会发现）

负责：

CEX

DEX

LP

Funding

MEV

发现套利机会。

② Cost & Risk Layer（成本与风控）

负责：

Quote

Fee

Gas

Slippage

Latency

Price Impact

Break-even Spread

决定是否真正执行。

③ Execution Layer（执行层）

负责：

LI.FI Route

Wallet

Sign

Broadcast

Status

Inventory Rebalancing

完成跨链执行与资金调度。

学习总结

LI.FI 的价值不在于“帮你发现套利”，而在于将理论套利机会转换为真实可执行的交易路径，并提供统一的跨链执行、成本评估和资金再平衡能力。

对于 AI Agent 而言，LI.FI 更适合作为 Execution Infrastructure（执行基础设施）；对于套利系统而言，应将 Opportunity Engine（机会发现）、Cost & Risk Engine（成本与风控）和 Execution Layer（执行层）解耦，形成可扩展的多链自动化套利架构。

笔记 045: 今日完成

作者：Louis lue | 时间：2026-08-09 07:22 UTC | 字数：1828 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日完成

学习与研究

- 完成 2026-08-08 与 2026-08-09 两份链上套利问题文档的系统回答，并逐项复核技术准确性。
- 掌握并修正了表面价差、毛利润、净利润、价格冲击、真实滑点和滑点容忍度之间的区别。
- 明确 `eth_call` 用于指定状态下的只读调用，`eth_estimateGas` 用于 Gas 估算，详细内部调用与 Gas 轨迹需要 tracing API 或专业模拟服务。
- 明确同链机会可省略多轮持续性确认，但不能省略即时重新报价、完整成本重算和模拟。
- 修正 LI.FI Quote 与 advanced Route 的关系：Quote 是可直接执行的最佳报价/Step；advanced routes 可返回多条复杂 Route，选中后需为相应 Step 获取交易数据。
- 明确原子执行只能避免永久留下单边库存，失败仍需支付执行到 Revert 点前实际消耗的 Gas。

编码与验证

- 新增 `packages/strategy/src/profit-model.ts`，实现毛利润、外部成本、净利润、ROI、ROI% 和 ROI bps 计算。
- 实现 `minNetProfitUsd` 与 `minRoiBps` 双阈值 AND 判断，并保留全部拒绝原因。

- 对 NaN、Infinity、零本金、负成本、负阈值和派生结果溢出进行输入保护。
- 发现普通 JavaScript 浮点比较会在 0.3 - 0.1 - 0.1 等边界误判；最终使用“number 十进制字符串 → bigint 精确分数 → 交叉相乘”的方式比较净利润和 ROI bps 阈值。
- 将学习案例接入只读 Dashboard，展示毛利润 \$12.00、外部成本 \$6.30、净利润 \$5.70 和 57.00 bps，并明确标注不是实时报价、自动执行关闭。
- 按要求在核心模型和测试中加入充分的中文审计注释，说明公式、费用口径、精度边界和安全假设。

关键结论

可执行 amountOut 若已包含池手续费、价格冲击、桥费或聚合器费，对应成本字段必须传 0，避免重复扣费；调用方必须逐数据源确认费用口径。

展示层可格式化小数，但策略阈值不能用舍入值判断。按数值尺度放大 EPSILON 容差会在大数值下错误放行，因此阈值比较改用 bigint 精确十进制分数。

minAmountOut 只保护单步最低输出；整条原子套利还需在合约末尾独立校验最终余额和最低利润。

当前系统仍是学习与监控原型，不包含真实行情、钱包、签名或自动执行。

证据

学习文档：docs/2026-08-08-research-questions.md、docs/2026-08-09-research-questions.md。

利润模型：packages/strategy/src/profit-model.ts。

自动化测试：tests/profit-model.test.ts、tests/dashboard-server.test.ts。

Git 提交：d771e8a [verified] feat: add audited profit model。

测试结果：3 个测试文件、15 项测试全部通过。

类型检查：npm run typecheck 通过。

依赖审计：npm audit 返回 0 vulnerabilities。

独立代码审查最终通过：无安全问题、无逻辑错误。

运行验证：GET http://127.0.0.1:3000/api/health 返回 status=ok、mode=read-only、executionEnabled=false。

未完成 / 下一步

尚未接入 LI.FI 真实 Quote / Route API。

尚未实现 DEX Quoter、RPC 报价采集、数据库持久化和 Telegram 机会告警。

当前 Dashboard 的利润数字来自静态学习案例，不代表真实套利机会。

下一步将接入 LI.FI 基础数据，保存链、Token、工具、路径步骤、费用口径、报价有效期和原始响应，并继续保持只读模式。

笔记 046: day5

作者：lunateng | 时间：2026-08-09 09:07 UTC | 字数：391 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

day5

LI.FI 是跨链流动性聚合器:同时查询多个 DEX/桥/路由并给出最优报价的组合器（如 LI.FI），核心价值是路由优化。

7种套利方式：

- 1.DEX 内套利：某池子因为一笔大额交易被砸出 1% 的偏离，你买回后等它回归。
- 2.DEX 间套利：同一条链上，两个 DEX 对同一资产报出不同价格。
- 3.跨链套利：同一资产在不同链上价格不同（链间隔离所致）。
- 4.三角套利：同一 DEX 内，三种资产的“两两汇率”绕一圈后不一致。USDC → ETH → DAI → USDC 转一圈，结束时钱变多了。
- 5.稳定币套利：稳定币（USDC/USDT 等）暂时脱锚，或在不同市场出现价差。
- 6.清算套利：借贷协议里有人抵押品跌破清算线，你可以打折吃下他的抵押品并代为还款。
- 7.MEV：矿工把一笔大额买单插到自己买单之后，推高价格再卖出。

\

笔记 047: https://github.com/ZeroNullNone/onchain_arbitrage_

作者：Nothing | 时间：2026-08-09 09:09 UTC | 字数：2600 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/docs/daily/day_05.md

状态与结论

状态：完成；实现、两小时 evidence run 与 offline verification 均通过。
Collector 只调用 LI.FI read-only GET /v1/quote，不签名、不广播交易。
Raw JSON 是 authoritative、append-only evidence；Parquet 与 DuckDB 是从 Raw 重建的 normalized projection。
当前固定 Universe 为 Day 4 的 3 routes × 3 exact USDC sizes，单进程、并发上限 2。

完成项

每次 attempt 使用独立 UUID request ID，并记录 UTC request start 与纯网络 latency。
30 秒 timeout、最多 3 次 attempt、指数 backoff；429 尊重 Retry-After。
全局 request-start rate limiter 默认每次间隔至少 1 秒。
response、HTTP error、transport error 均在任何解析前原子写入 Raw JSON。
Secret request headers 不落盘；响应的 cookie/auth 类 header 被 redacted。
成功解析后 append 一条 DuckDB row 与一个 immutable Parquet part。
Token/cost amount 使用 integer raw units 的十进制字符串并保留 explicit decimals；不使用 binary float 做金额语义。
collection_attempts 独立保存 success、parse failure、timeout、unavailable 与其他失败 outcome。
Collector 重启复用数据库但生成新 request ID/file，不覆盖已有 Raw 或 Parquet。

Storage Contract

Raw：data/raw/lifi/YYYY-MM-DD/<UTC>_<request_id>.json。
Parquet：data/normalized/lifi/quotes/part-<request_id>.parquet。
DuckDB：normalized_quotes 与 collection_attempts 两张 append-only table。
每条 normalized record 的 request_id 与 raw_ref 必须匹配原始 envelope；raw_ref 使用绝对路径。
Raw write 失败时不做 normalized write。Parse 失败保留 Raw，并写显式 parse_failure attempt。
Parquet/DuckDB write 失败不会被转换成成功，也不会用 stale/cache/fallback 数据替代。

Two-hour Evidence Run

Dataset：ignored local data/raw/lifi/day05_final、data/normalized/lifi/day05_final 与 data/normalized/day05_final.duckdb。
Command：uv run python scripts/collect_quotes.py --duration 7200
UTC observation window：2026-08-09 06:40:49.897176 → 08:40:58.410812，共 2:00:08.513636。
Request count：83 attempts / 83 Raw files。
Success：81 normalized quotes，success rate 97.59%。
Rate limited：2；LI.FI 返回 source code 1005 与约 2 小时 Retry-After，Collector 保存 Raw 后等待并成功恢复采集。
Parse failure：0。
Timeout：0。
p50 / p95 network latency：1,725.714 ms / 3,852.330 ms。
Unavailable routes：0。
Lineage audit：81 DuckDB rows、81 Parquet parts；81/81 normalized records 的 request_id 与存在的 Raw envelope 匹配。

验证

uv run pytest
Offline tests 覆盖 raw-first lineage、DuckDB/Parquet 内容、restart append、timeout retry、429 Retry-After、parse failure 与 polling interval 约束。
Live smoke round：9 requests、9 success、0 parse failure、0 timeout、0 unavailable。
下一步

Day 6 在每次 Quote 附近增加 read-only chain head/block context。
Structured concurrency、compression/partition tuning 与长期 scheduling 保留在 backlog。

笔记 048: 共学第5天。今天从同学精华里系统补了跨链套利与MEV两块知识:

作者：cjdaocoin | 时间：2026-08-09 09:53 UTC | 字数：584 | 评分：51

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

共学第5天。今天从同学精华里系统补了跨链套利与MEV两块知识:

1. 成本地板思维(peanutwen00):跨链价差能持续存在,是因为抹平它的动作很贵——桥费+延迟+滑点,价差本质是抹平成本的影子。选标的第一准则应是成本地板而非波动率,WBTC约2bps vs meme币480bps,价差大≠能赚。三种桥对比:Lock&Mint(当铺,锁仓池是黑客大靶子)、Burn&Mint(官方桥最安全但慢)、Liquidity Network(最快最贵,套利常用),大额资金须分批走。

2. 报价隐性损耗:池子汇率 $\neq$ 1:1(汇率损耗)+边际价格 $\neq$ 成交价(滑点)+报价时点 $\neq$ 结算时点,只认toAmount到手,不认费率百分比。
3. 区块构建与MEV根源(Azhan):以太坊区块有"进块市场"(基础费人人平等)和"位置市场"(优先费买执行顺序),套利者愿为"先执行"付几乎全部利润,这就是MEV的根源;订单流 $\rightarrow$ 路由 $\rightarrow$ 构建 $\rightarrow$ 确认四层链路,独家订单流是Builder的命根子。
4. 资金费率套利(loveyuanfandj):各所结算周期/方向/口径不同,必须统一换算为每小时费率再比较,且费率差只能当候选信号,还要验证行情时效、结算时间窗口、深度、基差等。
- 今天重点补了"价差与成本同源"和"MEV军备竞赛"两条主线,下一步按大纲推进LI.FI路径分析与报价采集。

笔记 049: 今天开始尝试把前几天学习的跨链、路由和 Agent 这些东西，往真正的套利策略上结合。

作者：Oxgen3 | 时间：2026-08-09 10:00 UTC | 字数：806 | 评分：66
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天开始尝试把前几天学习的跨链、路由和 Agent 这些东西，往真正的套利策略上结合。

之前更多是在了解各种工具和机制，比如 LI.FI 怎么找跨链路径、不同 Route 的成本有什么区别。今天开始思考一个更实际的问题：如果真的发现一个价差，到底怎么样才能判断它是不是一个可以执行的机会？

我先从公开案例和一些真实交易入手，把一个看起来存在的价差拆开来看，而不是看到价差就直接认为可以套利。

重点开始关注几个成本：

Gas
DEX / 协议手续费
Bridge 跨链费用
滑点
跨链和执行延迟
失败交易产生的成本
资金占用成本

以前看到两个地方有价差，第一反应可能是：

有价差 $\rightarrow$ 买低卖高 $\rightarrow$ 有利润。

但真正算下来，价差并不等于利润。

中间任何一个环节的成本增加，都可能把原本看起来不错的机会吃掉。

所以今天开始尝试建立一个比较简单的成本模型：

预期价差 Gas 手续费 Bridge 成本 滑点 其他执行成本 = 实际可获得空间

另外也开始意识到，策略不能只看某一次成功的交易。

如果以后真的要做，就需要拿历史数据去回测，或者先用模拟盘 / Paper Trading 验证，再考虑小规模测试，看看这个机会是不是长期存在，而不是刚好碰到一次。

今天最大的收获

以前更关注的是：

哪里存在价差？

现在开始转向：

这个价差扣掉所有成本之后还剩多少？这种机会能不能重复出现？

这也是从“发现机会”到“建立策略”比较重要的一步。

下一步准备把前面学到的 LI.FI 路由数据和 Hermes 的自动化能力结合起来，尝试做一个最小版本的监控流程，把符合条件的机会自动记录下来，再慢慢验证到底有没有稳定的 Edge。

先验证，再执行；先小规模测试，再考虑放大。

笔记 050: Day 5 | 链上套利残酷共学

作者：HashFlow | 时间：2026-08-09 11:07 UTC | 字数：802 | 评分：82
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 | 链上套利残酷共学

日期：2026.08.09 (周日)

主题：从假滑点到真实滑点 — LI.FI 报价接入

投入时间：\~3h

核心产出

之前脚本滑点用 V2 近似公式，对 V3 无效。对比 6 种方案后选 LI.FI。

调研对比

pool.swap() eth_call → 需 callback；Quoter V2 → Base RPC 不可用；LI.FI Quote API → 免费可用；Uniswap Trading API → 需 key；V3 SDK → 需切 JS

LI.FI Agent 栈

llms.txt — 入口索引

openapi.yaml — API 契约

ai-plugin.json — 工具注册

allowExchanges 参数可限定 DEX

脚本接入

GET /v1/tools → Base 链 14 DEX

GET /v1/quote?allowExchanges={dex} → 逐 DEX 报价

新增 analyze_via_lifi() 独立路径

真实验证

1 WETH on Base:

SushiSwap 1917.80 | KyberSwap 1914.82 | OKX 1914.69 | 1inch 1913.18

价差 \$4.62/ETH (0.24%)

报价含 toAmountMin (0.5% 滑点保护)

脚本三条路径

1. DEX Screener — 池子列表 + 参考价

2. Base RPC — pool.fee() + slot0() → 费率 + 现货价

3. LI.FI — /v1/quote → 含滑点成交价

下一步

Day 6 (周一)：周复盘 + 知识图谱 v1

笔记 051: 一、核心架构：MCP Server 深度集成

作者：Yolandatsai | 时间：2026-08-09 11:09 UTC | 字数：1420 | 评分：76

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、核心架构：MCP Server 深度集成

透過 MCP (Model Context Protocol) 協議，Hermes Agent 可以直接將 LI.FI 的功能視為自身的「原生技能 (Agent Skills)」，不需硬編碼繁瑣的 API。

專屬工具箱 (Agent Skills)：LI.FI 提供的 MCP Server 讓 Agent 具備 get_quote (獲取報價)、get_status (追蹤跨鏈狀態) 與 get_connections (查詢可用鏈與代幣) 等標準化工具。

CLI 與文件動態讀取：Hermes Agent 能即時檢索 LI.FI 的最新開發文件與 CLI 指令，當鏈上合約升級或參數調整時，AI 能自主修正調用參數，無需人工干預。

上下文感知：Agent 在對話或執行任務時，能自動將用戶的自然語言（例如：「幫我把 Arbitrum 的 1000 USDC 換成 Optimism 上的 ETH，要最便宜的路線」）轉譯為 LI.FI SDK 的精準參數。

二、核心大腦：智能套利與路徑分析

LI.FI 的多鏈流動性聚合能力，是 Hermes Agent 執行複雜金融策略（如跨鏈套利、資產再平衡）的實時數據庫。

價差訊號捕捉：Hermes Agent 持續監控各條公鏈（如 Ethereum, Base, Polygon, Arbitrum 等）的 DEX 價格。一旦發現某資產在 A 鏈與 B 鏈存在利潤空間，即刻觸發任務。

最優路徑精算：Agent 調用 LI.FI API，在數百個跨鏈橋和 DEX 中篩選出速度最快或成本最低的路由，並精確計算出滑點 (Slippage) 與 Gas 消耗。

動態淨利評估：Agent 會自動扣除雙邊鏈上的 Gas 費、跨鏈橋手續費及潛在滑點損耗，只有在預估「純利潤 > 設定閾值」時，才會批准執行，避免流動性損耗。

三、全流程自動化執行閉環

結合兩者可建立一個無需人工介入的自動化鏈上交互閉環：

[機會發現] ——> [路徑與成本驗證] ——> [策略構建與執行] ——> [訊號監控與復盤]
(Hermes) (LI.FI API) (智能合約/錢包) (AI 數據分析)

1. 機會發現 (Discovery)：Hermes Agent 透過鏈上數據流 (Data Feeds) 或預言機，24/7 監控流動性池與市場異常價差。
2. 路徑與成本驗證 (Verification)：AI 將具體代幣與金額輸入 LI.FI MCP 工具，獲取當前最即時的跨鏈路由報價，並完成成本效益分析。
3. 策略構建與執行 (Execution)：Hermes Agent 生成交易 hex 數據或調用帳戶抽象 (AA) 錢包，透過單筆交易 (Bundled Transactions) 一鍵授權並執行交換。
4. 訊號監控與復盤 (Post-Trade Analysis)：利用 LI.FI 的 TxStatus 追蹤跨鏈橋進度。交易完成後，AI 自動結算實際利潤，並將數據寫入日誌，動態優化下一次的滑點與路徑選擇。

笔记 052: 昨天用 LI.FI API 完成了 Base USDC → Arbitrum USDC → Base

作者：rakisa | 时间：2026-08-09 11:15 UTC | 字数：2067 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

昨天用 LI.FI API 完成了 Base USDC → Arbitrum USDC → Base USDC 的报价闭环。今天继续研究两个问题：把滑点从 0.1% 放宽到 0.5% 会不会得到更好报价，以及 CHEAPEST 和 FASTEST 到底会选择什么路线。

实验设置

我固定使用：

text

Base USDC → Arbitrum USDC

输入金额：1,000 USDC

preset：stablecoin

设置三组策略：

1. stablecoin preset 默认参数：CHEAPEST、0.1%滑点；
2. CHEAPEST、0.5%滑点；
3. FASTEST、0.1%滑点。

三种策略各请求三次，共九次。为了减少固定调用顺序造成的影响，每一轮都轮换请求顺序。每次保存请求时间、HTTP状态、API响应延迟、实际滑点、预计到账、最低到账、费用、Gas、桥和预计执行时间。

全程没有连接钱包、没有签名、没有发送交易。

实际结果

策略	成功	路线	预计到账	Gas	预计时间
默认参数	3/3	Fee Collection → Eco	997.500000 USDC	\$0.0072	7秒
CHEAPEST + 0.5%	3/3	Fee Collection → Eco	997.500000 USDC	\$0.0072	7秒
FASTEST + 0.1%	3/3	Fee Collection → Across	997.394808 USDC	\$0.0053	1秒

九次请求都成功。三轮中每个策略分别返回了相同的桥、到账量、Gas 和预计时间。整个采样窗口只有约20秒，所以只能说明当时结果稳定，不能代表长期成功率或长期最优路线。

今天验证的第一个问题：放宽滑点不一定改善报价

默认策略实际应用0.1%滑点，第二组实际应用0.5%，但两组都返回：

text

toAmount：997.5 USDC

toAmountMin：997.5 USDC

桥：Eco

预计时间：7秒

说明在本次 Eco 路线中，把滑点放宽到0.5%没有提高到到账，也没有改变路线。

这也再次说明：滑点容忍度不是额外扣除的手续费。它是一条成交保护条件。当前结果不能推广到包含 AMM swap 的路线，因为池状态变化时，滑点设置仍可能影响最低到账和成功率。

第二个问题：更快确实需要成本

FASTEST 选择 Across，把预计执行时间从7秒缩短到1秒，但预计到账量减少：

text

997.500000 - 997.394808 = 0.105192 USDC

相对于1,000 USDC 输入，大约少到账0.0105192%。Across 的 Gas 又比 Eco 低约0.0019美元，所以按1 USDC约等于1美元粗略计算，速度优先方案的额外综合成本约为0.103292美元。

可以把这次结果理解成：当时多付约0.103美元，换预计6秒提速。这个数字只适用于本次报价，不能当作固定桥费。

第三个发现：CHEAPEST不是只挑Gas最低

Across 的 Gas 估算比 Eco 更低，但 CHEAPEST 仍然选择 Eco，因为 Eco 的最终到账更多。说明路由器比较的是整体结果，不是单独比较Gas。

以后计算候选路线时，不能只写：

text

Gas最低 = 最便宜

而要比较：

text

预计到账 - Gas - 费用 - 时间惩罚 - 失败风险

其中时间惩罚和失败概率目前还没有足够数据，不能为了算出一个分数就随便填写。

第四个发现：API延迟和跨链时间不是一回事

本次API响应时间在602–2,589毫秒之间，这表示服务器返回报价的速度。Eco的7秒和Across的1秒是路线预计执行时间。两个字段不能相加，也不能混为同一个指标。

今天的结论

今天没有发现套利机会，但完成了一个有用的路由策略对照：默认/最便宜方案多到账约0.105

USDC，速度优先方案预计快6秒。哪一个更好取决于机会能维持多久，而不是永远选择“最快”或“最便宜”。

短窗口三次稳定也不能证明真实可靠性。下一步需要增加采样时间、金额和链组合，同时记录失败、无路线和报价变化，再建立有证据支持的路线评分器。

笔记 053: 【Day 5 | 2026-08-09】LI.FI 定位

作者：OxChainWorld | 时间：2026-08-09 11:40 UTC | 字数：666 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 5 | 2026-08-09】LI.FI 定位

投入：80 分钟

学了：

- 读 docs.li.fi 概览 + li.fi 官网
- LI.FI = 路由与执行层：一次集成对接多链 DEX/桥/solver 等流动性
- 优化目标：价格、Gas、成功率、速度 (+ token 映射/fallback)
- 关键边界：它不是策略信号源

做了：

- 写完 02-lifi-positioning.md
- 回看 Day3–4 的 Quote 用法，核对「显微镜 vs 雷达」有没有用错

结果：

- 聚合什么：执行侧路径组件（DEX/桥/intent 等），不是套利信号
- 优化什么：同一意图下的路径与成交可靠性
- 定位句：我主要用它来测量「完成同一意图」的执行成本与路径差异；不会用它来当机会雷达
- 关键发现：报价优 ≠ 有 edge；LI.FI 帮我证伪，不帮我「发现必赚」

失败/不确定：

- Intent/Composer/Earn 等延伸产品未深读（本周先盯 Quote 实验）

证据：

- 笔记：02-lifi-positioning.md
- 阅读：<https://docs.li.fi/>、<https://li.fi/>
- 前序报价证据继续有效：evidence/d4-2-cross-eth-arb-10k-usdc.json 等

净收益判断：今日无下单 → 不适用（定位日）

对研究流程的判断：成立——把 LI.FI 放在执行成本层

卡点：无（概念日）；下一步要开始记实验表

笔记 054: 2026.08.09

作者：Johnova | 时间：2026-08-09 12:04 UTC | 字数：1106 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026.08.09

Day 5：链上套利残酷共学

今日核心进展

2026.08.08

Day 4：跨链套利、稳定币套利

今日完成

- [x] 理解跨链套利分类：锁铸/销毁铸造/流动性池/意图求解器
- [x] 掌握跨链套利成本模型：Gas+桥费+滑点+最终性时间成本
- [x] 理解稳定币三大套利：去锚定/跨链价差/资金费率
- [x] 搭建 LI.FI 跨链路由对比原型
- [x] 编写稳定币去锚定+跨链价差监控原型
- [x] 建立跨链套利成本计算器

核心洞察

1. 跨链套利本质是「对冲最终性时间风险」而非单纯找价差 — 最终性时间内的价格波动是最大隐性成本
2. 稳定币跨链套利的核心是「桥费+Gas vs 价差」 — CCTP 免费但需对冲，Across 快但有费用
3. 去锚定套利的窗口期极短 — 需要秒级监控 + 预授权 + 极速执行
4. 跨链套利的盈亏平衡点通常在 1-5 bps — 只有大额或高频才能摊薄固定成本

技术决策落地

| 模块 | 方案 | 关键参数 |

|-----|-----|-----|

| 跨链路由 | LI.FI 聚合器 + CCTP 优先 | 滑点 0.5%, 最终性 15min |

| 稳定币监控 | DexScreener + Binance Web3 API | 偏离 >50bps 报警 |

| 成本模型 | Gas+桥费+滑点+时间成本 | 盈亏平衡 1-5 bps |

| 对冲策略 | 目标链开反向永续 | 资金费率 > 0.01%/8h |

遗留疑问

1. 如何量化「最终性时间成本」的统计分布？
2. Across 等意图桥的求解器竞价机制如何建模？
3. 稳定币去锚定时的流动性枯竭风险如何量化？

明日预习

- 学习清算套利机制：触发条件、奖励机制、竞争格局
- 研究 MEV 基础：抢跑、三明治、Flashbots/MEV-Share 使用
- 预习 LI.FI SDK 深度集成

共学第 4/21 天 | 链上套利残酷共学

关键洞察

- 待补充...

技术决策落地

| 模块 | 方案 | 关键参数 |

|-----|-----|-----|

| 待补充 || |

遗留疑问

- 1.
- 2.

明日预习

-

共学第 5/21 天 | 链上套利残酷共学

笔记 055: 案例分析：SK 海力士链上永续合约跨市场套利

作者：0x2077 | 时间：2026-08-09 12:35 UTC | 字数：520 | 评分：53
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

案例分析：SK 海力士链上永续合约跨市场套利

2026 年 7 月，SK 海力士（SK Hynix）股价因 AI 芯片需求增长受到市场关注。由于韩国股票市场存在地域限制，部分加密市场投资者无法直接购买韩国 SK 海力士股票，只能通过交易所提供的股票永续合约参与。

由于大量币圈用户看多 SK 海力士，在链上及加密交易平台的股票永续合约中集中开多，导致合约价格严重偏离韩国现货市场。一度出现：

韩国 SK 海力士现货价格；
链上永续合约价格；
两者之间出现单股超过 40 美元的价格溢价。
这形成了明显的套利窗口。

套利者发现：

市场	标的	状态
韩国股市	SK 海力士正股	正常交易价格
链上交易平台	SK 海力士永续合约	因大量买盘出现高溢价

由于两个市场交易的是同一个底层资产，长期来看价格应趋于一致。

因此：

链上合约价格偏高 → 卖出高估资产；
韩国股票价格正常 → 买入低估资产。

形成无方向风险套利。

笔记 056: Day 5 | 同一交易意图下，比较 CHEAPEST 与 FASTEST

作者：Zacteh | 时间：2026-08-09 12:43 UTC | 字数：2718 | 评分：89
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 | 同一交易意图下，比较 CHEAPEST 与 FASTEST

日期：2026-08-09

今日命题

延续 Day 4 的只读 L1.FI 路由研究，不再只解释一条报价，而是用完全相同的输入对比 CHEAPEST 与 FASTEST，并通过第二轮查询检查初次结论是否仍成立。

发起人近期分享 Hermes 在第一次查询后再做一次 check query，用第二份证据纠正初判。今天把这个思路落实到路由研究：报价不是事实常量，而是带时间戳、参数和市场状态的瞬时证据。

90 分钟安排

时间	内容	验收标准
0-10 分钟	复跑 Day 4 单路由工具	能再次读出预计到账、最低到账、成本与步骤
10-20 分钟	设计快照结构	包含参数、请求/响应时间、排序策略和完整响应
20-55 分钟	实现双策略对比	同轮并行请求 CHEAPEST / FASTEST
55-70 分钟	保存证据	两轮共 4 份原始报价和 1 份比较报告
70-82 分钟	第二轮验证	同输入复查路线、时长、到账与最低到账约束
82-90 分钟	复盘	回答速度、到账与额外成本之间的取舍

今日实现

新增 pnpm compare (src/quote-compare.ts)：

默认比较 0.01 ETH (Ethereum) → USDC (Base)，滑点 0.5%；
每轮尽量同时请求 CHEAPEST 和 FASTEST，短暂停顿后再查询一轮；
校验两种策略使用相同输入，且每条报价都满足 toAmountMin <= toAmount；
只汇总顶层 estimate.gasCosts 与 estimate.feeCosts，不再叠加 included steps，避免重复计费；

保存请求参数、时间戳和完整响应，但明确排除请求头与 API key；
全程只获取未签名报价，没有私钥、授权、签名或广播。

最终证据保存在：

data/quotes/2026-08-09/2026-08-09T12-41-27-863Z/

其中包含 4 份原始快照和 comparison.json。JSON 解析、文件数量和敏感请求头扫描均已通过。

实验结果

固定输入：0.01 ETH，Ethereum → Base USDC，滑点 0.5%。

轮次	排序	工具路径	预计到账	最低到账	预计时长	顶层 gas + fee
---	---	---	---	---	---	---
1	CHEAPEST	feeCollection → relaydepository	19.082633 USDC	18.987220 USDC	4 秒	\$0.1384
1	FASTEST	feeCollection → bitget → across	19.134585 USDC	19.038912 USDC	2 秒	\$0.3488
2	CHEAPEST	feeCollection → near	19.084724 USDC	18.989300 USDC	47 秒	\$0.1401
2	FASTEST	feeCollection → bitget → across	19.134585 USDC	19.038912 USDC	2 秒	\$0.3488

两轮对比：

第 1 轮：FASTEST 节省 2 秒，多到账 0.051952 USDC，但顶层成本多约 \$0.2104；
第 2 轮：FASTEST 节省 45 秒，多到账 0.049861 USDC，但顶层成本多约 \$0.2087；
按报价中的到账 USD 估值做简单差额，额外成本扣除额外到账后，两轮使用 FASTEST 的净增量成本都约为 \$0.158。这只是报价层比较，不是实际成交证明。

理解检查

- 今天没有出现“为速度牺牲预计到账”：FASTEST 反而多给约 0.05 USDC，真正的代价是更高的源链 gas。
- 第二轮 CHEAPEST 从 relaydepository/4 秒切换为 near/47 秒，说明一次报价不足以形成稳定判断；第二次查询提供了实质纠偏。
- CHEAPEST 和 FASTEST 是优化偏好，不是结论。Agent 仍需同时比较到账、最低到账、gas、协议费、时长和路线稳定性。
- toAmountMin 是滑点边界，不是另一个可与 toAmount 相加的收益字段；四条报价都通过 toAmountMin <= toAmount 校验。
- included steps 适合解释路线结构，但其成本可能已汇总在顶层 estimate；没有确认口径前不能上下层重复求和。

验证

pnpm typecheck：通过；
pnpm route：真实 LI.FI 单路由请求通过；
pnpm compare：两轮 4 次真实 LI.FI 报价通过；
4 份原始快照和 1 份报告均可解析为 JSON；
快照未发现 Authorization、x-lifi-api-key 或共学 API key 字段。

下一步

下一次共学应把“报价波动”推进为小型时间序列：在固定窗口内多次采样，区分短暂异常与持续变化，并定义 Agent 何时应重新报价、何时应拒绝给出确定性建议。仍保持只读，不签名、不广播。

资料

发起人 X：<https://x.com/brucexu_eth>
LI.FI Quote API：<<https://docs.li.fi/api-reference/get-a-quote-for-a-token-transfer>>
LI.FI Token API：<<https://docs.li.fi/api-reference/fetch-information-about-a-token>>

笔记 057: 任务：写脚本调 /quote + /advanced/routes，解析多路由，按净收益排序输出 T

作者：Milli | 时间：2026-08-09 13:08 UTC | 字数：732 | 评分：74
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

任务：写脚本调 /quote + /advanced/routes，解析多路由，按净收益排序输出 Top3
产出：experiments/lifi_routes.py（本地可运行，真实 API 数据）

一、脚本设计

核心逻辑

- GET /quote → 单条最优报价（跨链同币）
- POST /advanced/routes → 返回多条路由
- 逐条解析 → 净收益 = toAmount - fromAmount（LI.FI 已扣全部费用）
- 按净收益降序 → 输出 Top3

默认场景（可 CLI 覆盖）

10,000 USDC : Arbitrum (42161) → Ethereum (1)
fromToken: 0xaf88d065... (Arbitrum USDC)
toToken: 0xA0b86991... (Ethereum USDC)
slippage: 0.5%

关键实现点

费用解析兼容两种结构：quote 的 gasCosts/feeCosts 在顶层 estimate 里，advanced/routes 的在每个 step.estimate 里 →
_sum_cost() 两个位置都取
toAmount 位置差异：quote 在 estimate.toAmount，route 在顶层 toAmount → 兼容取
时长：route 的 executionDuration 在每步 estimate 里 → 求

笔记 058: 链上套利共学 · Day5 打卡笔记 (08/09)

作者：haibian-slowly | 时间：2026-08-09 13:15 UTC | 字数：3602 | 评分：88
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 · Day5 打卡笔记 (08/09)

今日信息

日期：2026-08-09 (周日)
主题：批量采集不同规模报价
核心问题：交易规模如何影响最终到手金额？

打卡正文

【第 5 天】

1. 今天要验证的问题

交易规模 (100 / 500 / 1000 / 5000 USDC) 如何影响最终到手金额？固定费占比、Gas 占比、滑点谁随规模变化？有没有「规模越大越划算」的临界点？会不会有小额最低费 (min fee) ？

2. 今天实际做了什么

- 补 Day 4 缺口：把昨天的临时调用沉淀成可复用脚本 scripts/lifi_single_quote.py (参数化链 ID/金额/代币，摘要输出 + 完整 JSON 落盘)，跑通验证与昨天数值吻合 (fee \$2.50 / gas \$0.024 / 7s / eco) ；
- 写批量采集器 scripts/lifi_collector.py：4 组合 (arb→opt、opt→arb、arb→base、base→opt) × 4 金额 = 16 条/轮，带异常重试 (2 次指数退避 + 429 额外等待)、限速 (0.6s/请求)、CSV 追加模式；连续跑 4 轮 (每轮间隔 60s) ；
- 数据实验：data/quotes_2026-08-09.csv 共 64 条，64 ok / 0 fail ；
- 写分析脚本 scripts/day5_analysis.py，做 4 个最小分析 (固定费比例 / Gas 占比 / 到手率 / 路径稳定性) 。

3. 证据 (本次真实运行结果)

数据量：64 条 (4 组合 × 4 金额 × 4 轮)，全部成功。

证据文件：onchain-arbitrage-lab/scripts/lifi_single_quote.py、scripts/lifi_collector.py、scripts/day5_analysis.py、data/quotes_2026-08-09.csv (64 行)、data/raw_quote_2026-08-09_c42161_c10_1000USDC.json。

分析 1：固定费比例 (fee / amount)

金额	平均费率	min	max
--- --- ---			
\$100	0.2501%	0.2494%	0.2508%
\$500	0.2501%	0.2494%	0.2508%
\$1000	0.2501%	0.2494%	0.2508%
\$5000	0.2501%	0.2494%	0.2508%

→ 固定费是恒定 0.25% 比例，没有小额最低费 (min fee) ；波动 ±0.0007% 来自 USDC 价格微差，不是费率变化。

分析 2：Gas 占比 (gas / amount)

金额	Gas 占比	平均 Gas 绝对额
--- --- ---		
\$100	0.0137%	\$0.0137
\$500	0.0027%	\$0.0137
\$1000	0.0014%	\$0.0137
\$5000	0.0003%	\$0.0137

→ Gas 绝对额恒定 ~\$0.014 (跨链只需一笔源链 SEND)，占比随金额反比下降。小额最吃亏，但绝对值极小 (\$100 时才 0.0137%，约 \$0.014) ——在稳定币路径上 Gas 不是主要矛盾。

分析 3：到手率 (to_amount_usd / from_amount_usd)

| 金额 | 到手率 | 损失 |

|---|---|---|

| \$100 | 99.6784% | 0.3216% |

| \$500 | 99.6784% | 0.3216% |

| \$1000 | 99.6784% | 0.3216% |

| \$5000 | 99.6784% | 0.3216% |

→ 到手率与规模完全无关，恒定损失 0.3216% = 固定费 0.25% + 汇率差 ~0.07% (Arb 1.0005 vs OP 0.9977)。没有「规模越大越划算」的临界点——因为大头成本是比例费，不是固定成本。

分析 4：路径稳定性 (同参数跨 4 轮)

12/16 组合稳定：arb→opt、arb→base、base→opt 全部稳定 eco (7s)；

opt→arb 方向不稳定：eco (7s) 与 polymerStandard (1080s = 18 分钟!) 随机切换，同一金额 4 轮里两种桥都可能出现，费用完全相同 (fee 0.25%、gas \$0.0001、到手额一致)；

结论：LI.FI 默认 quote 的「最优」在候选路径间轮换，1080s 的慢路径会被选中——说明默认 quote 按费用排序，不按执行时间排序；这为 Day 6 的路径比较埋下问题：执行时间是隐性成本，费用相同但 18 分钟 vs 7 秒，资金占用成本完全不同。

汇总表 (to_amount, USDC)

| 组合 | \$100 | \$500 | \$1000 | \$5000 |

|---|---|---|---|

| arb→opt | 99.75 | 498.75 | 997.50 | 4987.50 |

| opt→arb | 99.75 | 498.75 | 997.50 | 4987.50 |

| arb→base | 99.75 | 498.75 | 997.50 | 4987.50 |

| base→opt | 99.75 | 498.75 | 997.50 | 4987.50 |

4. 得到了什么结论 (来自数据)

1. 固定费恒定 0.25%，无 min fee、无规模折扣——4 档金额费率完全一致 (0.2501% ± 0.0007%)；
2. Gas 是固定成本 \$0.014，占比随规模反比下降——\$100 时 0.0137%，\$5000 时 0.0003%，小额最吃亏但绝对值可忽略；
3. 到手率与规模无关 (恒定损失 0.3216%)，不存在「越大越划算」的临界点——因为成本结构是比例费为主；
4. 路径会漂移：opt→arb 方向 eco/polymerStandard 随机切换，默认 quote 可能给出 18 分钟的执行路径——「费用最优 ≠ 综合最优」，执行时间是 Day 3 模型的 capital (资金占用) 成本，Day 6 要正式处理；
5. 稳定币 4 组合全部 0 滑点 (toAmount == toAmountMin)，与 Day 4 判断一致。

5. 今天失败了什么

urllib 默认 UA 请求 LI.FI 被 403 Forbidden，curl 同参数却 200——LI.FI 拒绝默认 Python UA，加 user-agent 请求头后解决；cost_model.py 仍是空文件 (Day 3 遗留)，今天分析全部用打卡笔记里的成本结论替代，未阻塞——留给 Day 6 路径比较前补。

6. 明天只解决什么

同一笔跨链，桥和 DEX 会怎么选？——用 POST /v1/advanced/routes 拿多路径候选，重点比较 eco vs polymerStandard 的费用 + 执行时间 + 资金占用成本，回答「默认 quote 为什么选了 18 分钟的路径」。

一句话总结

64 条报价实测：固定费恒定 0.25% 无 min fee、Gas 固定 \$0.014、到手率与规模无关 (恒定损失 0.32%)，不存在「越大越划算」；但 opt→arb 方向发现路径漂移——默认 quote 会在 eco(7s) 和 polymerStandard(18min) 之间随机切换，「费用最优 ≠ 综合最优」。

明日预告

Day6 (08/10)：比较路径变化——同一笔跨链，桥和 DEX 怎么选？(advanced/routes 多路径比较 + 执行时间成本)

笔记 059: 1. debug 并完善先前的 RPC 脚本，获取不同链的延迟/archive 深度/额度限制

作者：Leon | 时间：2026-08-09 13:25 UTC | 字数：933 | 评分：58

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1. debug 并完善先前的 RPC 脚本，获取不同链的延迟/archive 深度/额度限制
2. 所需工具总结

| 本质 | 回答什么 | 具体工具 | 关键约束 |

| :---- | :----- | :----- | :----- |

| 节点 RPC | 现在怎么样？ + 某个指定区块怎么样 | Alchemy (主力) dRPC / 官方公共 RPC (备用) | 免费档 getLogs 只 10 块 → 干不了批量历史 |

| 索引服务 | 过去一段时间 发生了什么？ | The Graph、Dune | 有延迟，不能当实时源 |

| 本地沙盒 | 如果我这么做 会怎样？ | Foundry : anvil (本地节点) cast (RPC 客户端 + 编解码) | fork 的上游仍是 Alchemy 首次拉取后本地缓存，近乎 0 CU |

| 跨链聚合器 | 从 A 链 X 换到 B 链 Y，怎么走、能到多少？ | LI.FI (主) 1inch / Socket / Rango 同类 | HTTP API，不吃 CU 报价是 T 时刻快照，黑箱 |

| 交易解剖 | 这笔交易到底 发生了什么？ | cast run (主，免费档唯一可用) Tenderly / 区块浏览器 (辅) | Alchemy 免费档无 Trace/Debug API |

笔记 060: Day 5 打卡素材

作者：Neo.Yun | 时间：2026-08-09 13:34 UTC | 字数：290 | 评分：47

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 打卡素材

Day 5 | LI.FI 路由基础

今天理解了 LI.FI 不是单一跨链桥，而是聚合桥、DEX 和其他流动性方案的路由与编排层。通过只读 API 验证了：/quote 返回单条最优报价，/advanced/routes 返回候选路线；toAmountMin 是滑点保护线，预计耗时不是完成证明。实测 0.1 ETH 从 Ethereum 到 Arbitrum USDC，返回 5 条候选路线，最高与最低到账差约 1.092547 USDC。这说明路径选择会影响结果，但不等于已经产生套利利润。无签名、无广播、无真实交易。

笔记 061: Hermes Agent 研究与信号工作流学习笔记

作者：OxPrint | 时间：2026-08-09 13:51 UTC | 字数：5773 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Hermes Agent 研究与信号工作流学习笔记

1\ Hermes Agent 在这套体系里的定位\

一个“价差”只是观察。

一个“策略”至少包含：

触发条件 + 仓位 + 执行路径 + 成本边界 + 风控 + 退出条件。

2. 如何拆公开套利案例

看到别人赚钱的交易，先拆成六个问题。

① 利润来自哪里

常见来源：

CEX 与 DEX 定价不同

两条链之间资产价格不同

现货和永续合约价格不同

同一资产不同 DEX 池价格不同

稳定币短时脱锚

跨链桥库存失衡

新币初期流动性碎片化

清算产生折价

AMM 更新速度低于外部市场

某个市场交易受限，导致价格传导变慢

先找结构性原因。

例如：

Binance HFT = 0.0345

DEX HFT = 0.0330

价差 = 4.5%

研究重点应该变成：

为什么 Binance 长期比 DEX 贵？

可能原因：

DEX 流动性不足

CEX 新上市需求

链上资金进入困难

充值暂停

提现困难

做市商库存失衡

跨链速度慢

知道“为什么”，才有机会判断它还能持续多久。

3. 再拆执行路径

例如一个 CEX / DEX 套利：

DEX 买入 Token

↓

CEX 做空 Token

↓

等待价格收敛

↓

DEX 卖出 Token

↓

CEX 平空

这里其实包含四笔交易。

研究时分别记录：

DEX 开仓价格

CEX 开仓价格

DEX 平仓价格

CEX 平仓价格

因此最终收益来自：

开仓价差

+

平仓价差变化

+

资金费

-

所有交易成本

这比单纯看“开仓差了 4%”更准确。

4. 如何提出自己的套利假设

一个好的假设必须可以被数据证明或推翻。

差的假设：

这个币经常有价差。

好的假设：

Binance 新上线代币后的前 6 小时，由于 CEX 买盘强、链上流动性薄，Binance 相对 DEX 经常出现超过 2% 的正溢价，并在 30~180 分钟内回归。

这个假设已经可以测试。

需要采集：

Binance price

DEX price

spread

DEX liquidity

CEX depth

volume

funding

时间

价差持续时间

最大价差

最终是否收敛

5. 假设模板

以后发现任何机会，都可以用这个格式。

假设名称：

CEX 新币正溢价套利

观察：

Binance 经常高于链上价格。

原因猜测：

CEX 买盘强，同时链上资金和库存进入较慢。

触发条件：

价差 > 2%

交易：

DEX 买入

CEX 开空

目标：

价差回落至 0.5%

止损：

价差扩大至 5%

最大持仓：

10,000 USDT

验证数据：

至少 50 次事件

需要观察：

胜率

平均回归时间

最大逆向偏离

交易成本

最大可交易规模

这时候，一个想法已经开始变成策略。

6. 成本模型

最基础的模型：

理论利润

=

卖出价值

-

买入成本

研究阶段应该逐步加入：

实际预期利润

=

理论利润

- Gas

- CEX 手续费

- DEX 协议费

- 桥费

- 滑点

- 价格冲击

- 资金费

- 借币成本

- 失败交易预期成本
- 延迟成本
- 资金占用成本

这里不要求第一天就做到绝对精确。

你前面说的“模糊的正确”很适合研究阶段。

可以先分成三级。

一级模型

价差 > X%

只发现机会。

二级模型

价差

- 明确手续费

- Gas

X%

过滤明显的假机会。

三级模型

再加入：

滑点

深度

延迟

失败概率

资金占用

用于决定是否真的交易。

7. Gas

Gas 应转换成统一计价单位。

例如交易本金：

10,000 USDT

Gas：

\$2

Gas 成本率：

$2 / 10,000 = 0.02\%$

同样 \$2 Gas：

交易 \$100

→ 2%

交易 \$10,000

→ 0.02%

因此很多链上套利存在明显的最低经济规模。

8. 协议费

需要分别记录。

例如：

Uniswap pool fee

0.3%

Hook fee

1%

LI.FI fee

0.25%

Bridge fee

0.05%

不要统一写成：

手续费 = 1.6%

保留明细后才能知道哪里可以优化。

9. 滑点与价格冲击

假设屏幕价格：

1 Token = \$1

你买 100 个：

平均成交 \$1.002

实际成本增加：

0.2%

买 100,000 个：

平均成交 \$1.04

价格冲击已经达到：

4%

所以一个套利机会必须同时包含：

价差

+

可交易规模

看到：

价差 10%

意义有限。

看到：

价差 3%

可执行规模 \$100,000

往往价值更高。

10. 桥费与跨链成本

跨链套利需要拆：

源链交易

+

桥

+

目标链交易

例如：

Base USDC

→ Across

→ Arbitrum USDC

记录：

输入：10,000 USDC

输出：9,987 USDC

Gas：\$0.40

时间：38 秒

跨链真实成本大约：

13 USDC + Gas

约：

0.134%

如果目标链价差只有：

0.10%

机会就失去经济意义。

11. 延迟成本

这是很多套利研究最容易漏掉的东西。

例如：

发现价差：3%

DEX 下单：1 秒

跨链：40 秒

CEX 对冲：0.1 秒

40 秒期间价格可能移动。

因此可以建立：

延迟风险

=

历史价格波动率 × 执行时间

前期无需非常复杂。

可以简单统计：

发现信号后：

1 秒价格变化

5 秒价格变化

10 秒价格变化

30 秒价格变化

60 秒价格变化

然后知道：

一个 1% 的价差是否足够覆盖 30 秒执行风险。

12. 失败交易成本

假设：

100 次尝试中：

90 次成功

10 次失败

每次失败损失：

\$3 Gas

那么：

100 次失败成本 = \$30

折算到每次机会：

\$0.30

可以定义：

失败预期成本

=

失败概率 × 单次失败平均损失

这项在高 Gas 链尤其重要。

13. 两腿交易风险

例如：

DEX 买成功

CEX 空单失败

这时你突然变成：

纯多 Token

或者：

CEX 空成功

DEX 买失败

你变成：

裸空 Token

所以策略必须记录：

第一腿成功概率

第二腿成功概率

平均执行时间

失败后的最大敞口

补偿方式

真正成熟的套利策略一定包含：

失败怎么办

14. 资金占用成本

假设两个策略：

A

利润 1%

资金占用 1 小时

B

利润 2%

资金占用 7 天

A 很可能更好。

应该观察：

资本效率

=

利润 / 资金占用时间

例如：

1% / 1 小时

和：

2% / 168 小时

完全不同。

跨链套利尤其容易把资金卡在：

桥

CEX 提现

链上确认

再平衡

所以“资金多久能回来”必须进入模型。

15. 库存型套利

更成熟的跨链套利通常提前在两边放资金：

链 A：

USDC

链 B：

Token

发现机会：

链 A 买 Token

链 B 同时卖 Token

交易本身无需等待跨链。

之后慢慢：

重新平衡库存

这会把：

跨链时间风险

从交易路径中剥离。

因此可以把系统拆成两个引擎：

Arbitrage Engine

负责赚钱

Rebalance Engine

负责搬资金

这是非常重要的架构思想。

16. 从公开交易中应该记录什么

分析链上成功套利交易时，至少保存：

时间

区块

钱包

资产

源市场

目标市场

输入金额

输出金额

使用 DEX

使用 Bridge

交易路径

Gas

MEV payment

交易前价格

交易后价格

交易持续时间

最终利润

然后分析钱包历史。

真正有价值的问题是：

这个钱包做了多少次？

什么时候做？

通常做多大？

只做哪些资产？

什么价差才出手？

用了哪些 Router？

失败多少次？

平均利润多少？

一次赚钱可能是偶然。

连续几百次相同结构交易，就是策略。

17. 策略验证需要三个数据集

信号数据

记录所有出现过的机会。

包括最终没有交易的。

执行数据

记录：

报价

实际成交

Gas

滑点

耗时

失败

结果数据

记录：

最终利润

资金占用

最大逆向波动

是否需要补偿

最大的错误之一是只保存成功交易。

这样会产生严重的幸存者偏差。

18. 从假设升级为策略的标准

例如研究：

Binance / DEX 新币价差

开始时只是：

好像经常有 3% 价差。

积累数据以后：

样本：217 次

价差 > 2%

出现：91 次

成功回归：

74 次

胜率：

81.3%

平均回归：

38 分钟

最大逆向扩张：

3.4%

平均可交易规模：

\$18,000

平均交易成本：

0.47%

平均毛收益：

2.1%

平均净优势：

1.63%

这时候才能真正讨论仓位。

19. 最终策略结构

一个完整套利策略应该能被写成机器规则：

市场：

Binance + DEX

资产：

Token X

触发：

价差 > 2.5%

持续：

至少连续 3 个报价

规模：

盘口可成交金额 > \$5,000

成本：

估算成本 < 0.6%

开仓：

DEX 买

Binance 空

最大仓位：

\$10,000

止损：

价差 > 5%

止盈：

价差 < 0.5%

超时：

4 小时

异常：

任何一腿失败 → 执行补偿流程

到这个阶段，“套利想法”才真正变成了“套利策略”。

20. 最值得记住的研究框架

以后看到任何套利机会，都按下面顺序研究：

① Edge

为什么存在价差？

② Persistence

为什么不会瞬间消失？

③ Capacity

能做多少钱？

④ Execution

怎么成交？

⑤ Cost

全部成本多少？

⑥ Risk

什么情况下会亏？

⑦ Exit

什么时候退出？

⑧ Repeatability

能不能重复？

⑨ Evidence

数据是否支持？

⑩ Automation

能否写成机器规则？

最终研究目标应该从：

哪里现在有价差？

逐渐升级成：

什么市场结构会反复产生价差，在什么规模、什么成本、什么时间窗口下最值得交易？

这才是后面值得交给 Hermes、监控程序和套利系统自动寻找的东西。

笔记 062: 价格的影响到藏在toAmount里是一笔吃掉池深后的真实到手已扣的冲击，决定了池子深度加上我的下单量

作者：Freetrip | 时间：2026-08-09 14:11 UTC | 字数：750 | 评分：68

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

价格的影响到藏在toAmount里是一笔吃掉池深后的真实到手已扣的冲击，决定了池子深度加上我的下单量

滑点下限在显式字段，式一道保护先【toAmount\ (1-slippage)】决定了我所设置的滑点（slippage）的容忍度

滑点只动下限，不会动预期，slippage 不改变你预计拿多少，它只决定你能容忍实际成交比预期差多少才不 revert

滑点设定太低例如0.1%会出现稍微有人插在你前面，实际到手就跌破下限 → 交易 revert，白付 gas，还得重试，波动大的迷因币尤其容易连环失败。

但是如果设定太高交易几乎必成，但你把 3% 的空间露给了抢跑者——搜索者（searcher）可以把价格顶到 +3% 还让你的交易通过，中间那 3% 被他夹走（sandwich attack）。

所以滑点的权衡为：revert 率 ↔ MEV
被夹风险。套利战场上，这个参数设不好，薄利直接被吃光——这正是规划页说的「执行层纪律」。
order 告诉 LI.FI 按什么排最优路由，常见值：
RECOMMENDED（默认，综合最优）
CHEAPEST（到手最多，扣 gas 后净值最大）
FASTEST（执行最快）
实测同链 USDT→WBNB，CHEAPEST 和 FASTEST 结果几乎一样（都走 okx，到手都 82.52）——因为同链简单币对只有一条主导路由，怎么排都是它。
order 真正起作用是在跨链：FASTEST 会挑一个快但贵的桥（几秒到账），CHEAPEST 挑一个慢但便宜的桥（几分钟到账）。回到 D4 的时间风险——order 本质是在「时间成本 ↔ 手续费成本」之间替你选。

笔记 063: \# 跨链快 6 秒，值多少钱？我用 9 次真实报价做了个小实验

作者：Pengu1ncc | 时间：2026-08-09 14:17 UTC | 字数：2273 | 评分：79
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

\# 跨链快 6 秒，值多少钱？我用 9 次真实报价做了个小实验

研究跨链路线时，我们经常看到两个按钮：\CHEAPEST\ 和 \FASTEST\。

它们看起来只是“便宜一点”和“快一点”的选择，但如果要做套利或资金调度，真正需要回答的是：究竟快了多少，又为此少拿了多少钱？

我用 LI.FI 的公开报价接口做了一个只读实验，没有连接钱包，也没有发送交易。

\## 实验怎么做

我固定了相同条件：

\text

路线：Base USDC → Arbitrum USDC

金额：1,000 USDC

资产：两端均为原生 USDC

\

然后设计三种策略：

- 1\ stablecoin preset 默认参数：\CHEAPEST\、0.1% 滑点；
- 2\ \CHEAPEST\，但把滑点放宽到 0.5%；
- 3\ \FASTEST\，滑点保持 0.1%。

每种策略请求三次，共九次。为了减少固定请求顺序的影响，三轮里轮换了调用顺序。

\## 结果很直观

\| 策略 | 3次结果 | 使用的桥 | 预计到账 | 预计时间 |

\|---|---|---|---|

\| 默认参数 | 3次一致 | Eco | 997.500000 USDC | 7秒 |

\| CHEAPEST + 0.5%滑点 | 3次一致 | Eco | 997.500000 USDC | 7秒 |

\| FASTEST + 0.1%滑点 | 3次一致 | Across | 997.394808 USDC | 1秒 |

\FASTEST\ 的预计时间从 7 秒缩短到了 1 秒，但少到账：

\text

997.500000 - 997.394808 = 0.105192 USDC

\

Across 的 Gas 估算比 Eco 低约 0.0019 美元。按 1 USDC 约等于 1 美元粗略换算，计入 Gas 差异后，这次选择速度优先的额外成本约为：

\text

0.105192 - 0.0019 ≈ 0.103292 美元

\

换句话说，在这次报价里，多付约 0.103 美元，可以换预计 6 秒提速。

\## 更有意思的不是这个数字，而是三个认知

\### 1. 最便宜，不等于 Gas 最低

Across 的 Gas 反而更低，但 LI.FI 的 \CHEAPEST\ 仍选择了 Eco，因为 Eco 的最终到账量更高。

这说明比较跨链路线不能只看 Gas，应该比较整体结果：

\text

路线价值

\= 最终到账

\- Gas

\- 各类费用

\- 时间成本

\- 失败与桥风险

\

\### 2. 放宽滑点，不会自动换来更多到账

我把滑点从 0.1% 放宽到 0.5%，Eco 的路线、预计到账和最低到账都没有变化。

滑点容忍度是一条执行保护条件，不是“设置越高，报价越好”，也不是系统一定会额外扣掉的费用。在包含交易的其他路线中，它仍可能影响最低到账和成功率，但本次固定输出路线没有表现出差异。

AMM

\### 3. 快多少，必须结合机会寿命判断

如果一个价差只能存在 3 秒，7 秒路线即使多到账 0.105 USDC，也可能完全没有意义；如果只是普通资金调度，多付钱换 6 秒可能又不划算。

所以套利系统不能永远选择最快，也不能永远选择最便宜。它需要知道：当前机会预计能活多久。

\## 这次实验不能证明什么

九次请求虽然全部成功，但整个采样窗口只有约 20 秒。它不能证明 Eco 永远最便宜、Across 永远最快，更不能证明实际交易一定按预计时间完成。

另外，API 返回报价花费 0.6~2.6 秒，这是“服务器计算报价的时间”；Eco 的 7 秒和 Across 的 1 秒，则是“路线预计执行时间”。这两个指标不能混在一起。

我也没有进行真实跨链，因此没有测量实际到账时间、首次 Token 授权 Gas、交易失败率和报价过期问题。

\## 最后的结论

这次实验没有发现套利机会，但让我把“速度”第一次换算成了具体成本：

\> 在当时的 1,000 USDC 报价中，少拿约 0.103 美元，可以换预计 6 秒提速。

这个数字很快会过期，但方法可以复用：固定资产、金额和时间，分别比较到账、Gas、费用、桥和执行时长，再决定速度是否值得付费。

这比单纯问“哪条桥最快”更接近一个可验证的套利系统。

\---

本文是只读报价实验，不构成投资建议。没有连接钱包、签名或发送交易。报价具有时效性，实验结果不能代表未来实际执行。

参考资料：

\- \[LI.FI Stablecoin Preset]\(<https://docs.li.fi/api-reference/presets/stablecoin>)

\- \[LI.FI Quote API]\(<https://docs.li.fi/api-reference/get-a-quote-for-a-token-transfer>)

笔记 064: Day 4 : TUT 代币 Bitget vs Binance 价差 & 期现套利分析

作者：Zhou Junjie | 时间：2026-08-09 14:33 UTC | 字数：1248 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 : TUT 代币 Bitget vs Binance 价差 & 期现套利分析

做了什么

1. 调用 Binance/Bitget API 拉取 TUT 现货和永续合约的实时价格、K 线、资金费率
2. 对比现货跨所价差 (CEX-CEX arbitrage)
3. 对比现货-合约期现价差 (cash-and-carry arbitrage)
4. 分析资金费率结构和套利可行性

纯现货对比

	Binance	Bitget
价格	\$0.17404	\$0.17402
24h 交易量	\$248M	\$60M
Bid-Ask	0.006%	0.15%

5 分钟 K 线对比：最大价差 0.398%，典型 0.05-0.20%。扣除双边手续费后为负，无套利价值。

期现套利 (核心发现)

|| Binance Spot | Binance Perp | Bitget Spot | Bitget Perp |
|---|---|---|---|
| 价格 | \$0.17404 | \$0.16369 | \$0.17402 | \$0.16474 |
| 资金费率 | — | +0.065%/8h | — | +0.70%/8h |

关键发现：现货溢价合约 5-6% (正常情况应是合约略高于现货)。同时资金费率为正，做空合约可额外收取资金费。

最优套利路径

Binance 买现货 + Bitget 做空合约：

- 入场溢价：5.33%
- 资金费收入：0.70%/8h $\approx$ 2.1%/天 $\approx$ 766% 年化
- 首日预期收益：7%+

风险评估

Bitget 合约 24h 最高曾到 \$1.006 (现货的 3 倍+)，资金费率剧烈波动 (1.875% $\rightarrow$ 0.28% $\rightarrow$ 0.74% $\rightarrow$ 0.70%)。虽有机会但：

1. 保证金不足会爆仓
2. 安全方案：Binance 双边 (溢价 5.94%、费率温和、深度充足)
3. 溢价收敛即利润，不需频繁操作

方法论

纯现货跨所价差在新币上通常无利可图 (做市商秒级抹平)，但期现价差

资金费率是更容易被忽视的套利维度。新上线代币合约市场投机盘主导，容易产生现货溢价 + 高正资金费的畸形结构。

与 FP/IMD 案例的方法论对比

	FP/IMD	TUT 期现
类型	跨链 DEX 价差	跨所期现价差
核心机制	LayerZero OFT 桥	合约溢价 + 资金费率
价差来源	跨链流动性差异	合约投机盘 vs 现货
可持续性	小时级	天级

两个案例共同说明：链上套利不只是盯 DEX 价差，跨所、跨链、期现、资金费率都是可以挖掘的维度。

笔记 065: 参考

作者：mookixiao | 时间：2026-08-09 14:33 UTC | 字数：2121 | 评分：60

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

参考

Gemini

问题

1. RPC和REST接口、WebSocket接口有什么区别？是速度更快吗？

1. RPC是一种通信协议/标准，规定了客户端与节点之间相互传递指令的格式，可以运行在REST (HTTP) 上，也可以运行在WebSocket上

2. WebSocket远快于REST，套利场景几乎全线标配WebSocket

2. 合约账户是不是一个抽象的概念，它也有类似EOA的地址吗？合约账户是怎么被EOA控制的？

1. 合约账户在区块链上有和EOA格式完全相同地址，二者区别在于：EOA有私钥无代码、合约账户无私钥有代码 (Bytecode) 与存储

空间

2. EOA如何控制合约账户：通过“消息调用”，由于合约账户没有私钥、无法主动发起交易，所有的动作需要由一个EOA拿着私钥签名、发起一笔目的地为“合约账户地址”的交易来激活它。套利合约代码中通常会写死规则（`msg.sender == owner`），当交易发送到合约时，合约会自动检查“发起这笔调用的EOA是不是我的主人”，如果是才允许执行内部套利代码，不是则拒绝
3. 合约账户说是如果计算发现亏损会自动回滚保护本金，这居然是允许的吗？不会导致因为套利使得区块链出现大量回滚，从而影响区块链性能吗？
1. 允许且是故意设计的，智能合约遵循“原子性”，一个交易内的所有操作（借钱、兑换、还钱）要么全部成功，要么彻底当没发生过（回滚Revert）
2. 交易回滚只影响那一次交易本身，完全不导致区块链倒退，但是大量失败的套利交易会占用区块链空间，
4. 通常说的L2网络是否基本都在Ethereum上架设？在Solana上架设的是不是比较少？比如我经常听说Arbitrum、Base等Ethereum上的L2，但是没听过什么Solana上的L2
1. 是的，这是因为Ethereum和Solana的设计目标不同
2. Ethereum追求极致的去中心化和安全性，牺牲了速度（约15~30TPS），且Gas昂贵，所以确立了“以L2为中心”的扩容方案，把高频交易全踢到L2上处理
3. Solana从设计之初就追求极致性能，其L1主网本身就能达到每秒数千笔交易，且单笔Gas不到0.001美元

笔记

跨链交易相关术语：

1. EOA（Externally Owned Account，外部账户）：由私钥控制的普通钱包地址，在跨链套利中承担发起交易并支付初始Gas费的角色
2. （智能）合约账户（Contract Account）：由套利代码控制的账户，被EOA控制，能在单个交易内自动完成借贷、兑换、跨链等操作，如果计算发现亏损会自动回滚（revert）保护本金
3. 区块（Block）：按时间顺序打包并链接的交易集合，是套利的粒度，套利本质就是将交易挤进特定区块，链的出块速度决定了价差窗口的生存寿命（如Ethereum约12秒，L2/Solana仅几百毫秒）
4.

	RPC（Remote	Process
Call）：套利机器人与区块链节点通信的API接口，用于毫秒级查询DEX价格（读），以及快速广播签名交易（写）		
5. Nonce（交易序号）：每个EOA发起交易时从0开始自增的计数值，同一个EOA的交易必须按Nonce被打包，套利者可用相同Nonce加更高Gas覆盖未确认交易，实现撤单或抢跑
1. 撤单：发现套利交易消失，且之前发的套利交易卡在内存池还没打包，立刻发一笔相同Nonce+更高Gas的空转账交易（给自己地址转账0），使得原套利交易被取消
2. 抢跑：发现有人也在做同一套利机会，重新发一笔相同Nonce+更高Gas的套利交易，把原交易替换掉
6. Mempool（内存池）：节点已被接受但是尚未被打包进区块的“待处理暂存区”，通过监控Mempool可以提前捕获大单，捕获价格冲击
7. Receipt（交易收据）：交易打包成功后生成的执行日志，记载了状态、实际Gas、Event事件
1. Event事件：智能合约代码中定义好的广播通知信号，当合约满足某种条件时就会抛出一个Event，这个Event就会以执行日志的形式写进Receipt
- Event举例：代币转账（transfer事件）、DEX兑换（swap事件）、跨链桥存入（deposit事件）
- Event在跨链桥上的应用：跨链桥的链下监听节点（Replayer）会实时抓取起点链交易Receipt里的Deposit事件，一旦看到这个Event被抛出，会提取出里面的用户地址和金额，然后在目标链上向该用户发放/解锁对应资产
8. Confirmation（区块确认数）：交易所在区块之后新叠加的区块数量，跨链桥一般要求达到特定确认数才在目标链解锁资产，确认数越高越安全但耗时越长
9. Finality（交易最终性）：交易在共识机制上达到“绝对无法被回滚或篡改”的状态，应该确保交易在起点链买入绝对不可逆后，再在目标链卖出，避免单边亏损

03 MEV与抢跑（10 篇）

笔记 066: Day 4 / 21 · Gas 与交易生命周期。

作者：clare ma | 时间：2026-08-08 15:20 UTC | 字数：4589 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 / 21 · Gas 与交易生命周期。

\

验收达成：把 Day 3 那个「gas 可能 5-20 美元」变成精确数字——单笔 swap \$0.005，并推出当前环境下的最小可行价差 0.6114%。

学了什么

gas 几乎不构成障碍，手续费才是

最小可行价差 = $6/997 + 2\sqrt{(\text{Gas_total} / R_{\text{eff}})}$

| 项 | 值 | 占比 |

| :----- | :----- | :----- |

| 手续费 6/997 (两次 swap) | 0.6018% | 98.4% |

| gas 项 | 0.0096% | 1.6% |

| 合计 | 0.6114% |
 |

把 gas 整个抹掉，门槛也只会从 0.6114% 降到 0.6018%——几乎没区别。

两种失败方式，成本天差地别

|
 | revert | out of gas |

| :----- | :----- | :----- |

| 触发 | 主动 require 失败 (如滑点保护) | 执行到一半烧光 |

| gasUsed | < gasLimit, 剩余退还 | = gasLimit, 全部烧光 |

gasLimit 是上限不是花费。实测那笔成功 swap 预留 150,000 只用了 70,382 (47%)，多预留的一分钱不花。

EIP-1559 的三个锚点

$\text{next_base_fee} = \text{base_fee} \times (1 + (\text{gas_used} - \text{gas_target}) / \text{gas_target} / 8)$

满块 (100%) : +12.5%，这是硬顶

使用率 = target (50%) : 不变

空块 (0%) : 12.5%

连续满载 N 个区块就是 1.125^N : 10 个区块 (2 分钟) 涨 3.25 倍，30 个 (6 分钟) 涨 34 倍。

排序由 builder 决定，不是协议规则

EIP-1559 只规定了 base fee 怎么算，完全不涉及排序。链路是 searcher → builder → relay → validator，builder 有完全裁量权。「谁 tip 高谁在前」是经济理性的结果，不是协议强制。

两个套利者盯上同一机会时：

| 场景 | 拍卖类型 | 胜者净收益 | 败者成本 |

| :----- | :----- | :----- | :----- |

| 都走公开 mempool | 英式 (逐次加价) | 趋近 0 | 全额 gas |

| 都走私有通道 | 密封出价 | 约 50% | 0 (不上链) |

| 一公开一私密 | — | 私密者稳赢 | 公开者是免费信号 |

成本的真实排序：竞争 >> 手续费 >> gas

用今天实测的 gas 重算一个 \$100 机会：

| 场景 | 胜者净收益 |

| :----- | :----- |

| 独享 | \$99.99 |

| 两人密封拍卖 | \$49.99 |

| 两人公开拍卖 | \$5–10 |

| 3+ 人公开 | ≈ \$0 |

竞争成本 \$50 ~ \$95

手续费 0.6%

gas \$0.01

\<aside>\

gas 涨价是参数问题，最坏只让门槛从 0.61% 涨到 1.1% (100 gwei)。竞争是结构问题，能直接把利润拿走一半以上。

s_min 公式里根本没有 bribe 项——它默认你独享机会。用它之前先问：这个信号是我独占的，还是全网可见的？

\</aside>

威胁模型要按交易规模区分

| 角色 | 交易特征 | 主要威胁 |

| :----- | :----- | :----- |

| 普通用户 | 大额、单向 | Sandwich |
| 套利者 (我) | 规模小、双向、本身是信号 | Copy / Front-run |

临界点最优本金只有

USDC——夹一笔几百美元的交易，攻击者利润不够付自己的成本。但复制我的路径极划算：我替他做了发现机会的工作。

做了什么

1. Etherscan 找一笔真实 swap，cast tx + cast receipt 拆出六个字段
2. 用 effectiveGasPrice maxPriorityFeePerGas 反推 base fee，与区块实际值核对
3. 找一笔失败交易，确认是 out-of-gas 而非 revert
4. 现读当前储备量与 gas price，算出 Gas_total
5. 推导并计算最小可行价差，做 gas price 敏感性表
6. 概念四问 (base fee 调整、区块内排序、mempool 可见性、结论失效条件) + 写出 8 条纠正

得到什么结果

实测数据

```
| <br /> | 成功交易 | 失败交易 |
| :----- | :----- | :----- |
| gasLimit | 150,000 | 25,000 |
| gasUsed | 70,382 (47%) | 25,000 (100%) |
| gasPrice | 3.028 gwei | 0.0289 gwei |
| 花费 | — | $0.001388 |
| 得到 | 代币 | 什么都没有 |
```

当前环境：gasPrice 0.038 gwei，单笔 swap \$0.005，双边 Gas_total = \$0.0103，P_ETH = 1,918.68，R_eff = 4,443,774 USDC。

不同价差下的真实收益

```
| 价差 | 最优本金 | 净收益 | 单次收益率 |
| :--- | :--- | :--- | :--- |
| 0.50% | — | 不可行 | — |
| 0.61% | 182 | ≈ 0 | 临界点 |
| 1.00% | 8,847 | $17.60 | 0.199% |
| 2.00% | 31,066 | $217.17 | 0.70% |
```

即使 2% 的价差，单次收益率也只有 0.70%——套利是薄利生意，靠频率和规模，不靠单笔暴利。

gas price 敏感性

```
| gas price | Gas\_total | s\_min | gas 占比 |
| :----- | :----- | :----- | :----- |
| 0.038 gwei (实测) | $0.0103 | 0.6114% | 1.6% |
| 3.8 (×100) | $1.03 | 0.6981% | 13.8% |
| 38 (×1000) | $10.30 | 0.9063% | 33.6% |
| 100 (历史高峰) | $27.10 | 1.0958% | 45.1% |
```

遇到什么失败 / 卡点

我自己的预估错了两个数量级

Day 3 我写的是「gas 可能 5–20 美元」，实测 \$0.005。根源是拿历史印象 (gas 30–100 gwei 年代) 当当下数据。

\<aside>\

用当前数据，不用历史印象。今天亲手查到的 0.038 gwei 比任何记忆里的数字都可靠。

\</aside>

Day 3 的例子重算后结论反了

Day 3 算 0.5% 价差能赚 8.8 USDC，今天严格重算：最优本金为负，任何规模都不该做。

差异来自两处简化：手续费从一次变两次 (0.3009% → 0.6018%)，R 变成 R_eff = R/2 (两个池子都有冲击)。

AI 纠正新增 8 条 (#25–#32)

今天的 PO：

1. 凭空用 G_normal ≈ \$27 做量化表，与当天实测的 \$0.0103 差 2600 倍 (对应 100 gwei，未标注假设)
2. 建议「交易设几秒级 deadline」——出块间隔就是 12 秒，照做必然过期 revert

3. 同一份回答里的多张表用不同参数：s_min 列用 R≈8.6M、占比列用 R≈4.44M，两列不自洽

其他：EIP-1559 的「50% 使用率降 12.5%」错且与后文矛盾；混淆「最大损失」与「预期损失」；对套利者误判 sandwich 为最大威胁。

但纠正机制在生效

AGENTS.md (<http://AGENTS.md>) 已验证加载：prompt-size 对照实验（项目目录 8,432 B vs home 目录 648 B）

四轮回答无一例「等等我重来」，输出协议生效

纠正 #27 当场生效：后一轮改用了实测的 \$0.01

第四问它自己发现了竞争是结构性盲点，并区分了「参数问题」与「结构问题」

下一步验证什么

Day 5：DEX 内 / DEX 间套利，第一次真正的计算。同一时刻查同一交易对在两个 DEX 的报价，把八项成本逐项代入。

门槛已经知道了：0.6114%。明天看真实价差有多大。两个数字放一起，Day 5 的结论基本就写好了。

笔记 067: Day 03 · Gas、区块确认、执行延迟与 MEV 基础

作者：twone | 时间：2026-08-08 15:27 UTC | 字数：2522 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 03 · Gas、区块确认、执行延迟与 MEV 基础

今日目标

1. 搞懂 Gas 的计价方式和它如何进入套利成本模型
2. 理解区块确认与执行延迟为什么会让“看到的价差”失效
3. 建立 MEV 的基础认知：抢跑、夹子、回溯，知道自己在和谁竞争

一、Gas：链上执行的门票钱

EVM 链上每笔交易都要付 Gas，EIP-1559 之后的计价方式：

Gas 费用 = Gas 用量 × (baseFee + priorityFee)

- Gas 用量：交易消耗的计算量（普通转账 21,000；一笔 Uniswap swap 约 10~20 万；跨链套利合约动辄 50 万+）
- baseFee：协议按区块拥堵程度自动定价，会被销毁（波动大，拥堵时翻几倍）
- priorityFee：给验证者的小费，决定你的交易多快被选中打包

对套利的三个直接影响：

1. Gas 是固定成本，决定最小可行规模。主网一笔复杂套利 Gas 可能是 \$5~20，意味着价差收益低于这个数的机会直接出局；这也是为什么专业套利大多跑在 L2（Arbitrum/Base 上一笔 swap 常常只要几美分）
2. Gas 波动本身会杀死机会。价差窗口出现时往往伴随市场剧烈波动——恰恰是 Gas 最贵的时刻。用平静时段的 Gas 估算成本会系统性高估利润
3. 失败也要付 Gas。交易上链后执行失败（revert），Gas 照扣。成本模型里必须有“失败交易成本 = 失败率 × 单次 Gas”这一项

二、区块确认与执行延迟：价差的保质期

你“看到”的价格来自上一个已经出块的链上状态；你的交易进入下一个区块才生效。这中间至少隔一个区块时间：

| 链 | 出块时间（约） |

| :----- | :----- |

| Ethereum 主网 | 12 秒 |

| Arbitrum / Base / Optimism | 0.25~2 秒 |

| Solana | 0.4 秒 |

延迟的成本体现在两点：

价差可能在你上链前就消失了。搬平价差的不止你一个——专业机器人监控的是 mempool 级别的实时数据，普通脚本轮询 API 拿到的本来就是“剩饭”

跨链场景延迟更严重。桥的确认时间是分钟级（乐观桥甚至以小时计），这段时间里你完全暴露在价格波动中——这已经不是“无风险套利”，而是带方向性敞口的持仓

判断规则的更新：

屏幕价差 ÷ 它能存活的区块数 = 你的真实窗口。如果你的发现-验证-执行链路比这个窗口慢，这个机会对你不存在。

三、MEV：你在和谁竞争

MEV（最大可提取价值）指通过控制区块内交易的排序获得的收益。常见形态：

| 形态 | 机制 | 和套利的关系 |

| :----- | :----- | :----- |

| DEX 套利 (backrunning) | 紧跟一笔大额交易之后，把它砸出的价差搬走 | 最"正当"的 MEV，也是链上套利的主力形态——你的竞争对手 |

| 抢跑 (frontrunning) | 看到你的交易后插队先成交 | 你发出去的套利交易可能被抢 |

| 夹子 (sandwich) | 在你买入前抢买、在你成交后立刻卖出，赚你的滑点 | 你是受害者：滑点设置越宽，被夹的空间越大 |

| 清算 | 抢在其他人之先触发借贷协议的清算，赚清算奖励 | 清算套利的竞争方式 |

主网上套利交易基本不走公开 mempool，而是通过 Flashbots / MEV-Boost 这类私有通道直接提交给区块构建者，避免被抢跑、被夹，失败的交易也不上链（省 Gas）。这解释了一个现象：主网公开 mempool 里几乎看不到能赚钱的套利交易——能看到的要么不赚钱，要么是诱饵。

四、和净收益公式的连接

预期净收益 = 价差收益 - Gas ← 今天：固定成本 + 失败成本

- 协议费 - 桥费 - 滑点与价格冲击

- 延迟成本 ← 今天：价差窗口 vs 出块/桥确认时间

- 失败交易成本 - 资金占用成本

三条新的判断规则：

1. 先按目标链的 Gas 水平算出最小可行交易规模，低于它的价差机会直接忽略
2. 比较自己的执行链路耗时和价差的历史存活时间，链路慢的不要追
3. 在主网执行时默认 mempool 里有猎人：要么滑点设窄（宁可失败），要么走私有通道

明日衔接 (Day 4)

跨链桥与聚合路由原理：搞懂 LI.FI 的 Quote / Route / 中间资产，并实际调通 LI.FI API 拉一笔跨链报价。

参考资料

Ethereum Gas 机制 (EIP-1559) (<https://ethereum.org/en/developers/docs/gas/>)

Flashbots 文档：MEV 与私有交易通道 (<https://docs.flashbots.net/>)

EigenPhi (<https://eigenphi.io/>)：观察真实 MEV 套利/夹子交易的浏览器

笔记 068: Solana top 套利机制研究

作者：amazing-lzh | 时间：2026-08-08 15:43 UTC | 字数：1453 | 评分：79

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Solana top 套利机制研究

地址：888BMMsC1CPmVnnK4HLnKjXb7ZKUKpdCXiwqMvCQqVmo

套利模式：Jito-bundled atomic backrun searcher (原子回跑套利者)

指令流：

\[0] SystemProgram 转 5,000,000 lamports 到临时钱包 3T3dV...

\[1-2] ComputeBudget 设置 CU/priority

\[3] 8ttvL...cvGb (自研) → 创建 3T3dV 的 USDC & SNDK ATA + 从主钱包 6qafb... 转 1,893.12 USDC 到临时钱包

\[4] Jupiter (JUP6L...) SetTokenLedger 记录当前 SNDK 余额基线

\[5] 8ttvL...cvGb (自研) → 内部调 REAL (REALQqN...) SwapV2 卖 1,893.12 USDC → 0.878727 SNDK

\[6] Jupiter (JUP6L...) RouteWithTokenLedger → 内部调 goonu (goonudd...) 卖 0.878727 SNDK → 1,894.13 USDC

\[7] SystemProgram

\[8] 8ttvL...cvGb (自研) → 把 1,894.13 USDC 转回主钱包 6qafb... ; close 临时 ATA ; signer2 转回 5,000,000 lamports 给 888BMM ; 转 226,879 lamports 到 Jito tip 账户 8s8TK2sr...

1. 成功率 21.5% (近 200 笔里 43 成功 / 157 失败) ; 失败错误码 Custom 6001 / 6010 / 0 全部集中在自研合约 —— 典型的搜索者 require!(profit >= min_profit, InsufficientProfit) 早期回滚逻辑：“算完不赚钱就 revert 只赔 fee”。这跟“用户型”钱包 (成功率 ~99%) 完全相反。

2. 抢跑保护：Jupiter dontfront marker + Jito tip account + priority fee，说明通过 Jito bundle 提交，不走公共 mempool 也不与 Jupiter 用户抢块。

3. 同一笔完成整环：主钱包出 USDC → 临时 signer2 拿到 ATA 后过两个 DEX 兜一圈 → 关账户 → 利润+租金回到主钱包 → 支付 tip；全部在 1 个 Solana 交易里，无跨块风险。
4. 多路径通用：抽样另一笔成功交易 4vBUS3... 用的是 8ttv... + proVF4pMXVaYqmy4NjniPh4pqKNfMmsihgd4wdkCX3u (Perena / 其它 AMM)，说明该私有合约支持任意 DEX 组合，只要参数传得对；本次是 SwapV2(REAL) + Jupiter(goonu) 组合。
5. 利润规模：这一笔约 \$1.0 毛利，\$0.85 净利；P: 日志显示不同笔从 372,165 到 11,343,962 不等，量级 \$0.05 – \$10；靠 43 笔/百次尝试的高频累积。

笔记 069: Day 4 打卡：从「看懂交易」到「看懂攻击」——失败、原子性与 MEV 实操推演

作者：AJsec | 时间：2026-08-08 15:49 UTC | 字数：1194 | 评分：71

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 打卡：从「看懂交易」到「看懂攻击」——失败、原子性与 MEV 实操推演

一、今天学到的核心

1. 交易失败不是一种，而是一类

失败的本质都是「执行到某一步 `revert`，状态全部回滚」，但触发原因很多：滑点超限 (Too little received)、池子状态被抢先改变 (UNISWAP_V2: K)、订单过期、授权不足、Gas 不够、合约逻辑拒绝。套利场景最常见的是前三种，而且经常连锁发生：价差出现 → 广播交易 → 别人抢跑改价 → 滑点超限 → `revert` → 白付 Gas。

2. 为什么「失败也要付 Gas」

验证者确实执行了交易、消耗了计算资源，只是结果回滚了。回滚不等于免费。

3. 原子交易不是合约魔法，是 EVM 的底层保证

一笔交易 = 一个「事务」。执行期间所有状态改变（包括池子储备）都记进 Journal，任一步失败就按逆序撤销全部改变——买入了也当没发生。关键认知：执行期间的中间状态对外不可见，别人的交易要么在我之前、要么在我之后执行，永远看不到我的半成品。这和数据库事务的 ACID 原子性是同一个设计思想。

4. 自己想通了三明治攻击的完整逻辑

监控到大额交易 → 抢跑买入抬高价格 → 高价卖给大户 → 卖不掉就回滚。这个思路的核心是 Bundle：把「我的买入 + 受害者交易 + 我的卖出」打包成一个整体，要么全部执行，要么一笔都不打包——卖出失败时整包被拒，连 Gas 都不扣。这就是 MEV 搜索者「无风险尝试」的技术基础。

二、今日实操

用区块浏览器 + RPC 独立完成了一笔真实交易的日志解读：

- 拆出 4 条日志 (Uniswap v4 Swap + USDC Transfer + WETH Deposit + WETH Transfer)
- 从 Transfer 事件算出实际成交价：1,299.33 USDC ÷ 0.677854 WETH = 1,916.8 USDC/WETH
- 理解了 amount0/amount1 是带符号的数量变化（负 = 池子付出），页面报价是边际价格、链上事件数量才是真实成交

三、今天最重要的认知升级

「只亏 Gas 就能套利」在理论上成立，但现实是四层军备竞赛：gas 竞价抢跑资格、大户用私有内存池/低滑点/意图式交易反制、验证者和顶级 Searcher 拿走大部分利润、机会窗口被压缩到秒级。能想到攻击逻辑只是第一步，能不能执行是另一回事——但理解这套机制，是看懂整个链上 MEV 生态的钥匙。

四、下一步

进入 Day 4：用 LI.FI 做真实的路径与报价实验

把「模拟 → Bundle → 竞速」的执行链路作为后续研究方向的候选

笔记 070: Day 4 学习笔记

作者：Azhan | 时间：2026-08-08 15:57 UTC | 字数：2411 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 学习笔记

今天学了什么

今天继续玩 Bruce 的套利机制实验室，把 MEV 相关的几个模块过了一遍。

1. 套利机制的 5 大家族（实验室首页框架）

Bruce 把链上套利重新分了类，按“利润从哪来”分，不是按工具分：

| 家族 | 利润来源 | 对应我们学过的 |

|-----|-----|

| 原子套利 | 单笔交易内闭环，失败自动回滚 | DEX 间、三角套利 |

| 库存套利 | 预先持仓/跨域库存，不原子 | 有桥接和价格风险 |

| 收敛/套息 | 持有到价差收敛，赚时间+资金占用 | 资金费率套利 |

| 协议激励 | 奖励、返佣、清算激励 | 清算套利 |

| MEV/订单流 | 交易排序与订单流信息 | 抢跑、三明治 |

全站统一判据：终值 - 初值 - 全部成本 > 0。金句："价格偏差只是表象，状态转换才是套利。"

这个框架把之前学的零散类型重新归类了，视角更高一层。

2. MEV 价值流（谁在分钱）

用快递类比才搞懂：一笔 MEV 生意有 4 个角色——

- Searcher（发现商机的人）
- Builder（分拣员，打包交易）
- Proposer（签收员，确认上链）
- LP/协议（柜台，收手续费）

Searcher 发现的利润要一路分给帮他上链的所有人。页面上有 Flashbots 真实数据：平均每个区块 Builder 付给 Proposer 0.0059 ETH（约 2 美元）的上链竞价。

Mock 教学数据里，一笔 1.18 亿的毛价值池，Searcher 拿 44%（但这是毛利，还要扣成本）、Proposer 30%、LP 12%、烧掉 8%、Builder 只有 6%。

关键认知："看到利润" ≠ "拿到利润"。Searcher 的 44% 是毛利，扣掉失败尝试和基础设施才是净利。数据要分级看：Observed（真）> Derived（算的）> Unknown（猜的）。

3. 私有 Bundle（怎么保密地交给打包者）

核心问题：区块链不是透明的吗，怎么保密？

搞懂了：区块链公开的是"已上链"的数据。交易在广播/上链之前是私人的，只躺在自己电脑里。Searcher 就是利用这个时间差——不把交易丢进公共内存池，私下直接发给 Builder。

内存池不是强制集合点，交易有两条路：

- 公开路径：广播到公共内存池，谁都能看到，会被抢跑
- 私有路径：私下直发 Builder，别人看不到，但要付费

Bundle = 一捆交易绑在一起执行（多步连着走，中间不能插进别人的交易）。

完整 8 步流程：本地签名（私钥不出门）→ 提交私有端点 → 模拟检查 → 过滤无效 → 竞争区块空间 → Builder 向 Proposer 出价 → 签收 → 上链公开。

重点：模拟成功 ≠ 一定上链。可能因为竞争 bundle 更值钱、出价不够、到太晚而失败。

4. Backrun 模拟（尾追）

Backrun = 跟在大户交易后面，利用它造成的价格偏离赚钱。大户买→你高价卖；大户卖→你低价买。和抢跑的区别在时序：前面 vs 后面。

模拟器演示了一个场景：用户大户把池子价格从 2.0 推偏到 1.59（冲击 10.95%），Searcher 跟在后面反向操作，价格回到 1.74，净赚 7.67。

这个模块让我理解：尾追者其实是市场的"回归力量"——大交易制造偏离，尾追者帮价格回归。如果把它和用户交易绑成 bundle，别人就插不进来，这就是 03 和 04 的衔接。

5. Uniswap Lab（框架）

AMM 4 代对比：v1 只支持 ETH 配对，v2 任意配对（杂货铺），v3 集中流动性（专营店），v4 加 Hooks 插件（智能店铺）。Hooks = 给池子加自定义代码的能力，可以动态调手续费、做限价单之类。模块还没细玩，框架先记下。

做了什么

玩了实验室的 02 MEV 价值流、03 私有 Bundle、04 Backrun 模块

看了 MEV 价值流的 Flashbots 真实数据（builder 市占、proposer payment）

找到实验室各模块的正确

URL

路由（/triangle-arbitrage、/mev-value-flow、/private-bundles、/mempool-backrun、/uniswap、/mev-boost）

记录实验室新增的 06 MEV-Boost 模块（还没看）

得到什么结果

MEV 从"概念"升级到"价值链"：谁发现、谁打包、谁签收、谁收手续费，钱怎么分
搞懂了"透明链上怎么保密"这个核心疑问：上链前是私人的，私有通道绕开公共内存池

Bundle 的完整生命周期（8 步）和失败原因（模拟成功≠一定上链）

Backrun 的原理和模拟数据

套利 5 大家族框架，把之前学的零散知识重新归类

遇到什么失败（教训）

讲解 MEV 价值流时一开始直接倒数据和术语，用户完全听不懂——后来改成“先讲问题是什么+快递类比讲角色”，才讲通。这个教训要记住：先框架后数据。

实验室的模块点击经常失效，最后发现直接访问 URL 路由最靠谱

下一步验证什么

玩 05 Uniswap Lab（对比 v1-v4，复现无常损失）

看 06 MEV-Boost 生命周期模块（新增的，讲验证者、builder 竞价）

读 Bruce 的两篇以太坊区块构建译文

补 BSC DEX 生态调研

笔记 071: Day 5 : 全自動套利系統架構 v0.4

作者：Web3Rason | 时间：2026-08-08 17:00 UTC | 字数：5525 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5：全自動套利系統架構 v0.4

這一版沒有一條主線結論被推翻。真正的收穫是另一種東西：

三個被我寫成「常數」的參數，其實是函數。

常數寫錯比結論寫錯難發現得多——錯誤的結論會被下一份證據打臉，錯誤的常數只會安靜地把可行機會篩掉，而且報表全綠。

一、三個「常數」其實是函數

1 規模收斂點 \$10,000 廢除

v0.3 寫「直連 DEX 的規模曲線是倒 U 型，收斂點約 \$10,000」。

形狀對，數字不該存在——它是某一條路徑的觀測，被我升級成了通則。

正確形式是解析式：

最優本金 $q = R_{\text{eff}} \times (\text{價差率} - \text{兩腿費率}) / 2 \leftarrow R_{\text{eff}} = \text{池深} / 2$ ，兩池都被衝擊

最小可行價差 $s_{\text{min}} = 6 / 997 + 2 \times \sqrt{(\text{Gas}_{\text{total}} / R_{\text{eff}})}$

同一條式子，價差不同就差 2-3 個數量級：

價差 最優本金
:--- :-----
0.61% 182 U
1.00% 8,847 U
2.00% 31,066 U

更關鍵的是：可交易規模是一個有「上下兩個保本點」的窗口，不是一條下限。

邊界 被什麼吃掉 實測
:--- :----- :---
下方保本點 固定 gas 23.13 U
最優點 — 1,085.93 U
上方保本點 price impact 2,171.21 U

長尾池的窗口更窄：0.004 WETH 淨賺 \$0.70，0.05 WETH 就轉虧。

我原本的規模閘門只有下限。也就是說大額那一側完全沒有防護，而那正是損益會反號的一側。固定 gas 上升只會下移整條曲線並壓縮窗口，不會移動最優點。

2 MEV tip「吃掉 67-99%」要加「通道 × 鏈」前提

舊版只切「策略」（三明治 95%、backrun 76%、原子套利 67%...），那是以太坊公開通道的數字。

通道 tip 佔毛利
:--- :-----
以太坊公開 mempool 競爭把利潤壓向 0
私阮密封出價 約留一半

| Solana / Jito 私有 bundle | 僅約 15% (逐指令拆單：毛利 \$1.00 / 淨 \$0.85) |

把 67–99% 當跨鏈通則，會整條誤殺 Solana 賽道。

3 LP「收費站」的死因寫錯了

舊版寫「高費率池被路由層繞開，所以沒流量」。同一份回測的細節直接反駁這個解釋：65% 的時間都在區間內，手續費實收 +180.60 U —— 流量是有的。

真正的機制是逆選擇：

成交越頻繁 = 價格穿越越劇烈 = 你被填得越多 = 存貨虧得越狠

「生意好」和「賠錢」是同一件事。

為什麼非改不可：沿用「被繞開」的解釋，會很自然地推出

「那我開低費率池就有流量、就能賺」—— 方向完全相反。

低費率池換手率確實高 (0.05% 池 0.87x vs 0.3% 池 0.031x, 差 28 倍) ,

但那正是被逆選擇填得更快的路徑。

二、兩條排除理由要重寫：結論不變，但推導不成立

清算 / MEV —— 理由從「不賺錢」改成「量不到」

鏈上看到某筆清算轉出 0.5366 WETH，那不是利潤，是付給區塊構建網路的競價；

而事後 refund 在鏈上完全不可觀測。

用單筆鏈上淨額算清算 / MEV 損益，會系統性把利潤算成 0 或負數。

同一批資料還翻掉了另一個直覺：20 筆 9 勝 11 敗，但失敗只虧 \$0.06 gas ——

在低失敗成本的通道裡，高失敗率是廉價試單，不是問題。

→ 暫不碰維持，但任何要重啟這條線的評估，必須先解決 refund 的可觀測性。

預測市場 —— 死的是 taker 側的商業條款，不是機制

用 21 萬筆真實成交腿反推 (不是讀文件) : $fee = 1.8\% \times \min(p, 1p) \times shares$,

只有 taker 付, maker 恆為 0 (11 萬筆 maker 腿合計手續費 \$0) 。

taker : 453 個事件只有 10 個 $\Sigma bid(YES) > 1$, 且手續費全部大於毛利

maker : $\Sigma ask(YES) \cdot 1 \approx 0.5\% - 1.6\%$, 零手續費，流量是真的

舊版寫「機制已被平台改規則殺死」是錯的。真正的邊界是 200 bps vs 0 bps 的費率差，

那是商業條款 (可議價 / 可繞開那一類) , 換一家交易所結論就會翻轉。

把會變的東西凍結成永久結論，是這次抓到最貴的一種錯。

代價要講清楚：maker 側必須 K 條腿同時被動成交，缺腿即裸方向部位 —— 那是做市，不是套利。

三、掃描器的一個結構性盲區

之前的機會來源只有兩類：AMM 池報價、聚合器 / RFQ 報價。

漏了第三類：協議內建的鑄造 / 贖回 / 清算價。

一條 Liquity fork 路徑的形狀：

RFQ 限價單以 0.9027 買入折價穩定幣

→ 按「協議預言機面值」贖回抵押品

→ 聚合器賣回

兩腿的報價沒有一腿來自 AMM 曲線：一腿是意圖型限價單，一腿是協議規則定的價。

只掃 AMM 池的掃描器，在設計上就看不到這種機會。

這一類要從白皮書機制推導 —— 「協議規則會強制誰在什麼價格流動」——

而且可以模板化套用到整個同族 fork。

三個先決條件：贖回費是「0.5% 下限 + 動態 baseRate」，而 baseRate 被贖回本身推高 (邊際成本遞增) ；

路徑非原子 (三筆) , 單步的費率保護不保護整條路徑；

且該筆是第三方地址的鏈上還原，不是自己的錢，仍不滿足可重複性。

四、這一版最重要的一條：儀器自身的參數會決定結論

兩份互不相關的資料指向同一種故障 ——

過濾閾值與統計口徑不是中性的觀測設定，它們本身就在決定結論。

| 故障 | 後果 |

| :--- | :--- |

| 篩選器接受帶寬把極端值當雜訊剔除，又把 $> 20 \text{ bps}$ 判為 implausible | 能翻號的邊際全部落在被自己剔掉的區間 →

「接受窗口」是經驗空集 |

| 候選名單用均值份額產生 (逐日活躍流動性相差 1700 倍, 均值 7.055% vs 中位數 0.639%) | 頭號候選改中位數口徑後從 +43.56% 淨 APR 翻成虧損 |

→ 寫進紀律:

1. 所有過濾閾值列成一張表, 每一條都要寫「這條會讓我看不到什麼」
2. 任何分布同時輸出均值與中位數, 加斷言 均值 / 中位數 < 3; 超過即代表被少數極端日綁架, 該批候選作廢重跑
3. 掃描結果為空時, 必須先跑一次「拿掉所有過濾器」的對照組, 才能說「沒有機會」
4. 原始回應全部落盤可重放, 否則參數改動前後無法對照

「找不到」和「量不到」必須能區分。這跟之前踩的靜默截斷是同一個形狀:
看起來永遠正常的失敗最貴。

五、執行層補正

承諾點是不可逆邊界。

區塊構建流程裡有一個很乾淨的示範: 本地回退只存在於簽署外部 blinded block 之前;
簽了之後對方不揭露, 就不能安全切回本地版本, 整個 slot 直接錯過。

這對我直接適用 —— 我目前的樣本 12/12 全是固定輸出承諾型路由:

| 階段 | 失敗態 |

| :-: | :---- |

| 承諾前 | 可回退, 成本 = 機會成本 |

| 承諾後 | 不能回退, 失敗態是「錯過整個窗口」而非「回到原狀」 |

→ 狀態機必須顯式標出承諾點在哪一步; 承諾之後的逾時不可以走「重試另一條路由」分支 (會變成雙重曝險)。

同一條線帶出一個我沒有的失效維度: 通道的過濾政策。

不同私有通道的訂單流政策不同, 同一筆交易在不同通道的「可包含性」不同 ——
這既不是價格問題也不是深度問題, 要跟深度、報價老化並列成獨立閘門。

其他落地級工具坑:

- 跨鏈成本是有向邊, 同一對鏈反向差 12 倍 (3.78 bps vs 46.17 bps) → 成本表 key 必須是 (from, to) 有序對
- 同路線只是金額不同就靜默換橋, 執行時間差 77 倍 (14 秒 → 18.1 分鐘) → 快取鍵要含金額檔位;
而買時間權重極便宜 (溢價 1.00–2.79 bps 就能把 1,080 秒壓到 1 秒)
- included 語義只存在於費用欄, gas 欄沒有 → 「gas 是否已扣」在 API 層是 unknown, 不可置 0
- 失敗 gas 分兩種: revert 只燒實際用量、剩餘退還; out-of-gas 則全額燒掉上限。
gasLimit 是上限不是花費 —— 多預留不花錢, 預留不足才真的貴
- 滑點容忍度是回滾邊界, 不是成本項: 它不讓你多付錢, 只決定會不會白燒 gas

六、參數驗證進度 (接續 Day 4)

| # | 參數 | 值 | 狀態 |

| :-: | :-: | :-: | :-: |

| 1 | 聚合器平台費 | 25 bps | 官方 FAQ + 實打 API |

| 2 | 聚合器 rate limit | 匿名 75 次/2h; Key 100 req/min | 實測 header |

| 3 | 直連橋費 (USDC/L2) | 1.0–1.6 bps | 實打官方 API |

| 4 | V3 深度計算法 | liquidityNet 前綴和 | 原始碼 |

| 5 | MEV tip 比例 | ~~策略相依 67–99%~~ → 策略 × 通道 × 鏈; Solana 私有 bundle ~15% | 修正 |

| 6 | CEX 費率階梯 | 一般 10/10 bps ~ VIP 1.1/2.3 bps | 官方費率頁 |

| 7 | 規模曲線 | ~~收斂點 \$10,000~~ → $q = R_{eff} \times (\text{價差} \times \text{費率}) / 2$ | 修正 (常數→函數) |

| 8 | 長尾同鏈成本地板 | 198 ~ 563 bps | |

| 9 | 同鏈雙邊費底線 | Σ (每跳實際 fee tier), Curve 檔可低到 4.5 bps | |

| 10 | LP 覆蓋率分水嶺 | 日波動 1.5% (死因改為逆選擇) | 機制修正 |

| 11 | MEV 佔優先費支出 | 66–80%, 前 15 名後斷崖 | |

| 12 | 兩端滑點 | 一 | 必須即時算 |

| 13 | 規模窗口上保本點 | 有, 且與下保本點差約 94 倍 (23.13 / 2,171.21 U) | 新增 |

| 14 | 跨鏈成本方向性 | 同對鏈反向差 12 倍 (3.78 vs 46.17 bps) | 新增 |

| 15 | 同資產路徑成本穩定度 | 同路徑 142 次極差 0.11 bps; 跨資產路徑極差 24–28 bps | 新增 |

| 16 | 時間權重溢價 | 1.00–2.79 bps 可把 1,080 秒壓到 1 秒 | 新增 |

| 17 | 預測市場費率公式 | $1.8\% \times \min(p, 1p) \times \text{shares}$, maker 恆 0 | 新增 |

| 18 | integrator 參數對平台費的影響 | 兩份證據結論相反, 擺動幅度 62 bps | 新增 · 待自行複測 |

| 19 | gas 是否已計入報價輸出 | API 未提供該語義 | 新增 (明確標未知, 不填 0) |

七、一句話

前四天我花力氣在「找機會」。這一天最值錢的產出是發現：
我可能一直在量一個被自己參數截斷過的世界。

把掃描器所有閾值與統計口徑盤點一遍，比再多找十個標的值錢——
因為前者影響的是所有下游結論。

笔记 072: 搞懂了 MEV 里 staticcall 的作用和套利的整体流程。

作者：LeonardChen666 | 时间：2026-08-09 02:53 UTC | 字数：344 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

搞懂了 MEV 里 staticcall 的作用和套利的整体流程。

以前以为套利就是自己在本地写公式算三方兑换，今天才反应过来，像 Uniswap V3 或 Curve 这种池子逻辑太复杂，本地重写不仅费劲，还容易有精度误差。用 staticcall 其实就是让本地节点的 EVM 免费帮我们跑一次 DEX 的原生代码，省事而且百分百准确。

套利架构其实就分两块：

链下算账：在本地内存里用 staticcall 跑，零 Gas 费，纯用来算有没有套利空间和最佳输入量。

链上执行：算出来有钱赚，再发交易给链上的机器人合约去落地。合约只负责原子性执行（不赚钱就回滚）和给 Builder 送小费。
策略和计算全放在链下，链上只留个盲目的执行器，既保护了策略，速度还快。

笔记 073: 研究 MEV 生态

作者：hello116 | 时间：2026-08-09 10:06 UTC | 字数：323 | 评分：61

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

研究 MEV 生态

学习：

Flashbots

MEV-Boost

Builder

Searcher

理解：

交易生命周期：

User Tx

↓

Mempool

↓

Searcher

↓

Bundle

↓

Builder

↓

Validator

实践：

分析：

10 个真实 MEV tx

记录：

trigger

route

profit
gas

思考：

为什么你的 backrun：

机会存在，但是利润低？

可能：

竞争激烈

route 太简单

edge 不够

笔记 074: 链上套利学习笔记 Day 5：清算套利与 MEV

作者：倪响 | 时间：2026-08-09 14:31 UTC | 字数：2009 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习笔记 Day 5：清算套利与 MEV

2026-08-07 | 残酷共学 Day 5

参考：Aave 清算文档 (<https://docs.aave.com/developers/guides/liquidations>) | EigenPhi MEV (<https://eigenphi.io/>) | Flashbots (<https://docs.flashbots.net/>)

清算套利是什么

和前三天的套利完全不同的逻辑。不是"在 A 买便宜、在 B 卖贵"，而是借贷协议的强制卖出机制。

借贷的基本逻辑

用户在 Aave 上存 ETH 当抵押品，借出 USDC。Aave 要求抵押品价值必须始终大于借款价值，用"健康因子"衡量：

健康因子 = 抵押品价值 × 清算阈值 / 借款价值

1：安全

< 1：触发清算

当 ETH 价格下跌，抵押品价值缩水，健康因子跌破 1，任何人都可以来清算——替借款人还一部分债，拿走对应的抵押品，而且拿走的比还的多（清算奖励，通常是 5-10%）。

清算的利润来源

借款人：存了 10 ETH 抵押品，借了 15,000 USDC

ETH 价格 \$2,000 → 抵押品价值 \$20,000 → 健康因子 1.1（安全）

ETH 跌到 \$1,600 → 抵押品价值 \$16,000 → 健康因子 0.88（触发清算！）

清算人：替他 7,500 USDC 的债

拿走 $7,500 \times 1.05 / \$1,600 \approx 4.92$ ETH（价值 \$7,875，含 5% 奖励）

利润：\$7,875 - \$7,500 = \$375（扣除 Gas 后的净利）

和 DEX 套利的本质区别

| 套利来源 | DEX 间套利 | 清算套利 |

| 触发条件 | 价差 > 成本 | ETH 价格下跌触发健康因子 < 1 |

| 利润来源 | 价差 | 清算奖励 + 抵押品折扣 |

| 机会频率 | 持续存在 | 跟随市场波动，大跌时集中出现 |

| 速度要求 | 毫秒级（mempool 竞争） | 秒级到分钟级 |

| 竞争格局 | MEV searcher 红海 | 相对少，需要理解借贷协议 |

清算套利对个人更友好的点：不是拼谁更快看到 mempool，而是拼谁先发现"健康因子接近 1 的头寸"——这是可以靠监控做到的。

但也不是没竞争

清算用的也是闪贷——清算人不需要准备本金，一笔交易里完成：借闪贷 → 还债 → 拿抵押品 → 卖抵押品 → 还闪贷。所以：

不需要本金，但需要技术栈

仍然有 MEV searcher 在做清算 bot

波动大的时候竞争会加剧

Gas 战争仍然存在

MEV 简述

MEV (Maximal Extractable Value) 就是"区块生产者可以通过排序/插入/审查交易来额外获取的价值"。

三种典型形式：

抢跑 (front-running)：看到你的交易在 mempool 里，付更高 gas 抢在你前面执行

尾随 (back-running)：在你的交易之后立刻反向交易，比如你做多，他做空，赌价格回弹

三明治：抢跑 + 尾随，把你的交易夹在中间——Day 2 讲的"被三明治"就是这个

个人的机会在哪

清算相比 DEX 套利有几个优势：

1. 可监控：健康因子是公开链上数据，不需要分析 mempool
2. 有时间窗口：从健康因子 < 1 到正式清算，有一段时间
3. 竞争者少：清算 bot 在 Aave 主网上存在，但数量远不如 DEX 套利 bot
4. 和波动相关：暴跌日清算机会爆发，但竞争者也可能因为恐慌而减少

不过放在

21

天共学里，清算是一个"知道有这条路，但不一定是主要方向"的模块——它的技术复杂度（闪贷合约、多步交易）比较高，Phase 1 先建立认知。

共学打卡：清算套利和 DEX 套利完全是不同的机会结构——不是拼价差而是拼监控，不靠速度而靠提前发现。和之前几天比，清算对个人更友好（可监控、有窗口、竞争者少），但需要闪贷和合约开发能力。

笔记 075: 今日共学打卡 | Robinhood Chain 套利研究

作者：tension-vip | 时间：2026-08-09 14:59 UTC | 字数：466 | 评分：49

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日共学打卡 | Robinhood Chain 套利研究

今天研究了一篇关于 Robinhood Chain 链上套利的分析。

最大的收获不是"哪里有套利机会"，而是学到了一套更值得长期复用的研究方法：

先找真实赚钱的钱包 → 逆向交易路径 → 统计成功率、Gas、净利润和失败成本 → 再判断策略是否真的存在正期望。

几个关键认识：

1. 闪电贷套利并不等于"零成本套利"，失败交易仍然会消耗 Gas。
2. 历史发现的套利用润，不等于未来还能复制的利润。
3. 真正该计算的是期望收益： $\text{成功概率} \times \text{成功利润} - \text{失败概率} \times \text{失败成本} - \text{基础设施成本}$ 。
4. 链上套利本质上已经接近 Quant + HFT + MEV 的交叉领域。
5. 与其直接复制一个套利机器人，不如先研究已经捕获 Alpha 的地址到底是怎么赚钱的。

今天最重要的一句话：

不要先预测哪里有 Alpha，先研究已经捕获 Alpha 的人是怎么做的。

下一步：挑一个真实套利地址和一笔交易，完整还原它的资金流和套利闭环。

04 做市与LP (5 篇)

笔记 076: Day 3 | 币股 LP 实操与自动化工具

作者：Oxbin | 时间：2026-08-08 15:04 UTC | 字数：479 | 评分：54

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 3 | 币股 LP 实操与自动化工具

今天做了什么

今天主要继续研究币股 RWA 方向，并实际操作了一些币股 LP。

周六周日美股休市，币股价格整体波动相对较小，所以我尝试利用这个时间窗口给币股池提供流动性，主要观察：

LP 区间怎么设置

手续费收益

周末价格波动

跑出区间的风险

美股开盘前后的价格变化

另外今天写了一个 LP 自动化工具，准备逐步把原本手动观察和操作的流程自动化，包括价格监控、区间判断和 LP 状态跟踪。

今天的认识

币股 LP 和普通币种 LP 有一个比较特殊的点：

周六周日传统美股休市，如果链上币股价格波动较小，可能比较适合做窄区间 LP 获取手续费。

但真正需要重点研究的是：

周末是否长期保持低波动

周一开盘前是否容易出现跳价

窄区间 LP 的手续费能不能覆盖跑出区间和价格偏离风险

不同币股池的深度和真实交易量

下一步

继续完善 LP 自动化工具，并记录币股周末和工作日的波动、手续费收入和 LP 实际收益，判断这个策略是否可以长期重复。

笔记 077: 关于LP做市的总结

作者：ingram0529 | 时间：2026-08-08 17:09 UTC | 字数：5425 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

关于LP做市的总结

① 搞清单边 LP 到底是什么 — HOU RUI

核心论点：单边 LP 不是 "高年化理财"，是 "增强版限价单" — 必须先有交易意图 ("涨到 2000 以上我要减仓")，再用单边 LP 去执行，手续费只是附加收益。建仓前必须自问：我是否愿意在这个价位区间成交？答案 "不愿意" 就说明仓位是错的。

验证 1: 成交均价 = 几何均值 $\sqrt{(Pa \cdot Pb)}$, 不是区间中点

USDC/WETH 池，现价 1901.66, 在 1997~2187 投入 5 ETH:

穿透区间后收到 10,449.21 USDC

成交均价 2,089.84 USDC/ETH

区间几何均值 $\sqrt{(Pa \cdot Pb)} = 2,089.84 \leftarrow$ 精确相等

区间算术均值 $(Pa+Pb)/2 = 2,092.00 \leftarrow$ 比实际高

直觉：价格上涨时你在持续卖出，越涨手里 ETH 越少 $\rightarrow$ 高价段卖得少 $\rightarrow$ 均价被拉低。区间越宽，偏差越大。

验证 2: 单边 LP vs 限价单 (同卖 5 ETH)

| 做法 | 收到 USDC |

| :----- | :----- |

| 限价单挂 1997 (最低价) | 9,985.00 |

| 单边 LP 1997~2187 | 10,449.21 (多 4.65%) |

| 限价单挂 2187 (最高价) | 10,935.00 |

介于两者之间，外加区间内手续费。

验证 3: 三种价格路径，三种结局

| ETH 价 | 持有 ETH | 持有 USDC | 总值 | vs 一直持有 |

| :----- | :----- | :----- | :----- | :----- |

| 1,997 (刚触及) | 5.0000 | 0 | 9,985 | +0 |

| 2,092 (正在成交) | 2.4148 | 5,284 | 10,336 | -124 |

| 2,187 (完全穿透) | 0.0000 | 10,449 | 10,449 | -486 |

| 2,400 (卖早了) | 0.0000 | 10,449 | 10,449 | -1,551 |

穿透后继续涨 $\rightarrow$ 少赚 1,551, 和限价单同性质

穿透后跌回来 (1901 $\rightarrow$ 2200 $\rightarrow$ 1901) $\rightarrow$ 锁定 10,449 vs 持有只值 9,508, 赚 941 (最理想)

朝反方向走 (1901 $\rightarrow$ 1700) $\rightarrow$ 你还是 5 ETH, 且一分手续费没赚到 (最常被忽略)

作者自我推翻: amountXMin 在单边场景下不是 "滑点容忍"

mint() 和 swap() 语义完全不同:swap 保护收到的量,mint 保护被用掉的量。单边场景下 $L = \min(L_from_amount0, L_from_amount1), amount1=0 \rightarrow L=0 \rightarrow$ 用掉 0 $\rightarrow$ 必然 REVERT。没有中间状态 — amountXMin 是 "价格越界即回滚" 的 二元开关, 任何 >0 的值效果相同。真正需要缓冲的是双边仓位。

证伪: USDC/SEI — "份额接近 100%" 是数据异常信号

昨天双条件榜首 (覆盖率 2.40x、净 APR 43.56%), 深挖发现某天池子流动性塌到极低, 1 万美元份额达 91%, 均值被这一天绑架:

口径	份额	费率 APR	覆盖率	判断
:-----	:-----	:-----	:-----	:-----
均值	7.055%	78.66%	2.60x	值得做
中位数	0.639%	12.95%	0.43x	亏损

已修进扫描器: 中位数口径 + 均值/中位 $< 3x$ 检查。与资金费率里 SKR 案例完全同构:瞬时值是噪声, 中位数才是可持续水平。

拆解一笔 90 跳 MEV 套利

项	值
:-----	:-----
区块位置	#1 (区块第二笔)
EOA nonce	932,374 (93 万笔交易)
swap 跳数	90 跳, V2+V3+V4
gas	10,151,599 (约占区块上限 1/3)
毛利 $\rightarrow$ 净利	$+\$149,186 \rightarrow$ gas $\$15,779 \rightarrow +\$133,407$

三个专业性标志:value 字段编码区块号 (合约校验技巧); 利润留在合约不转出 EOA (省转账 gas,V4 ERC-6909 思路);priorityFee 895 gwei vs baseFee 0.064 gwei (14,000 倍)。最值得注意:主动把 10.6% 毛利 ($\$15,779$) 交给验证者抢区块 #1 — 不是可选成本, 是入场费。

最重要的认知: 这笔交易是作者结构性打不赢的那一类 (资金 $\$440$ 万、gas 预付失败也要付、90 跳路径、区块 #1),而它的存在证明了 LVR 的资金流向 — 那 $\$149K$ 毛利很大一部分正是从 LP 手里赚来的。被动 LP 在这些机器人面前是被收割的一方。

② 1inch Aqua 共享流动性机制 — geyu210

核心理解: Aqua 把 "资金托管" 和 "流动性授权" 拆开了 — 资金留在 maker 钱包, Aqua 只记录 virtual balance、allowance 和 position 状态 (position 由 app + strategyHash + token 共同确定)。

本地只读检查程序, 同一区块内读:maker 钱包余额 / ERC-20 allowance / rawBalances /token slot 的 active/disabled 状态 /allocation 是否得到账面支持。

两个纠偏:

1. tokensCount = 0 不能证明 position 从未创建, 只能说明当前 slot 没有 active 证据
2. fully_backed 只代表账面覆盖, 不代表实际 swap 一定成交

真实主网验证: 找到一笔仍 active 的 position (1INCH + UNI 两个 slot 均 fully_backed,virtual balance $\approx 1336.745 / 49.77$), 全程公开 RPC + 只读调用, 不碰私钥。

关键洞察:position 之间共享同一钱包余额 — 一个 position 的成交会影响其他 position 的可成交额。只读 virtual balance 不够, 必须同时检查钱包余额、allowance、position 状态和后续成交记录。这正是 Aqua 与传统池子的本质区别, 也是 "一个钱包同时服务多策略" 的核心。

③ AMM、订单簿、流动性深度与价格冲击 — A9xYau

主题: 为什么 "看到的价差" $\neq$ "可执行的净收益" — 链上套利最核心的思维。

五要素:

AMM: $x \cdot y=k$, 池子比例决定价格; 不同池价格不同步 $\rightarrow$ 套利机会来源; 套利者实际是帮市场恢复一致的 "无偿做市员"

订单簿:Bid/Ask/Spread; 市价单穿透多档, 成交均价 $\neq$ 屏幕价

流动性深度: 同样 ETH=3000, 深池买 100 ETH 影响小, 浅池买 5 ETH 直接推动价格 — 真正的问题是 "买多少以后平均成交价是多少"

价格冲击: 大额交易改变池子比例 $\rightarrow 3000/3010/3025$, 最终均价 3015

区块确认: 进 mempool $\rightarrow$ 打包 $\rightarrow$ 确认; 你发现 10 美元机会提交, 等 10 秒别人已完成, 价格恢复, 你亏损

净收益公式:

净收益 = 卖出实际收入 - 买入实际成本 - DEX手续费 - CEX手续费 - Gas 滑点 - 失败成本

数值案例: 屏幕价差 1000 USDT 利润 (3000 vs 3010×100 ETH), 实际: 买入均价 3006 (+600)、卖出跌到 3007 (300)、Gas 30、手续费 50 $\rightarrow$ 只剩 20 USDT。

专业系统架构: 价格发现 $\rightarrow$ 流动性分析 $\rightarrow$ 成交模拟 $\rightarrow$ 利润计算 $\rightarrow$ 风险过滤 $\rightarrow$ 执行。找的不是 "价格差", 是 "可执行价格差"。

④ 深度延伸：LVR — 被动 LP 的隐性收割税

LVR 是什么

LVR (Loss Versus Rebalancing, 相对再平衡损失) 由 Milionis、Moallemi 等人在 2022 年论文《Automated Market Making and Loss-Versus-Rebalancing》中正式提出。它衡量：被动 LP 与 "每笔交易后立即按市场价格再平衡的组合" 相比，少赚 (多亏) 了多少。

LVR 不是无常损失 (IL) 的别名：IL 是 "期末 vs 期初" 的静态路径依赖指标；LVR 是连续时间下的动态量 — 它把 AMM 被动 LP 在每次价格波动中被套利者收割的部分，量化成一条持续的 "税"。

直觉：为什么被动 LP 必然被收割

ETH 从 P 涨到 P+dP, AMM 池价格还停留在旧值，套利者立即买入便宜的 ETH 并推高价格；跌的时候套利者卖出。每一轮波动，套利者都在 "低买高卖" 被动 LP 的头寸 — LP 在价格路径上总是买高卖低 (相对于再平衡组合)。波动率越高、路径越曲折，这条税就越重。

数量级：436% LVR 是极端值

论文中无界恒定乘积 AMM 的 LVR 率与波动率平方成正比 (近似 $\sigma^2/8$ 量级，乘池子价值)。主流 ETH/USDC 深度池年化 LVR 通常百分之几十；HOU RUI 算出的 436% 出现在高波动、流动性相对单薄的池子，属于把 "波动市中的持续收割" 年化的极端数字 — 它说明的不是 "这个数字常见"，而是 "机制方向正确且不可忽视"。

对做市 / LP 决策的三条推论

1. 被动 LP 的真实收益 = 手续费收入 - LVR。只看 APR 会系统性高估。HOU RUI 用中位数口径证伪 USDC/SEI 正是同一逻辑：均值被异常日绑架，可持续水平才是真相
2. 波动率越高，LVR 税越重 — 单边 LP、窄区间 CLMM 在震荡市里 "接插针" 赚的手续费，可能远小于被 MEV 机器人吃掉的 LVR。那笔 \$149K 毛利的 90 跳交易，很大一部分就是各路 LP 的 LVR
3. 主动管理是对冲方向 — delta 对冲 (永续 / 期权对冲池子敞口)、动态调区间、或像 Aqua 那样虚拟余额多策略复用同一钱包，都是把 "被动被收割" 变成 "主动承担可控风险"

三篇串起来的一条主线

A9xYau 讲 "定价面" (深度 / 滑点 / 确认延迟决定真实可执行价)；geyu210 讲 "授权面" (Aqua 虚拟余额让一个钱包同时跑多策略，和 V10 格点架构同构)；HOU RUI 讲 "执行面" (单边 LP 均价是几何均值、穿透后路径依赖、LVR 是被 MEV 收割的隐性成本)。三者合一：做市不是挂个流动性等手续费，而是持续管理 "定价 × 授权 × 执行" 三个面的风险。

笔记 078: 什么是「针」和「接针」

作者：starkxun | 时间：2026-08-09 08:50 UTC | 字数：2584 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

什么是「针」和「接针」

1. 什么是「针」

1.1 先看一根 K 线有几个价

链上每个池子，每小时都会记录四个价格：

| 名字 | 意思 |

| :----- | :----- |

| open 开盘 | 这一小时开始时的价格 |

| high 最高 | 这一小时里触及过的最高价 |

| low 最低 | 这一小时里触及过的最低价 |

| close 收盘 | 这一小时结束时的价格 |

画成图就是一根蜡烛：开盘到收盘画成一个方块 (叫实体)，最高和最低各拉一条线出去 (叫影线)。

1.2 针 = 特别长的下影线

开盘 0.535 —┐

| 实体 (开盘到收盘，变化很小)

收盘 0.536 —┘

|

| ← 这段就是「下影线」

|
最低 0.531 —」

针就是下影线特别长的情况 —— 价格在这一小时里一度砸得很深,但收盘时又回到了原来的位置。画在图上像一根细长的针扎下去。关键在于扎下去又弹回来。如果是一路跌下去收在最低点,那不叫针,那叫阴跌。

所以脚本里判断针的公式是:

针深 = (min(开盘, 收盘) 最低价) / min(开盘, 收盘)

用 min(开盘, 收盘)(实体下沿)而不是开盘价,就是为了排除阴跌 —— \ 我们要的是"扎下去又回来",不是"一路跌"。

1.3 针为什么会出现

某一刻有人急着卖:爆仓被强制平仓、恐慌砸盘、机器人参数写错、有人要跑路。

而那一刻池子里没那么多钱接。深度不够,一笔大单就把价格推出去很远。

1.4 针为什么会弹回来

因为价格跌那么深不是因为这个币变差了,只是那一秒没人接盘。

砸完之后,别人发现"这价格太便宜了",就会来买,价格回去。

这个"弹回来"才是接针能赚钱的全部前提 —— 如果不弹回来,你就是接了个正在归零的盘。

2. 什么是「接针」

提前在那个深坑的位置挂好买单,等着它扎下来的时候成交。

现价 1.00 USDT

← 你在这里挂买单:0.60

某一刻突然砸到 0.58 → 你的单成交了,0.60 买到

十分钟后弹回 0.95 → 赚 58%

核心是:你不预测它会跌,你只是把单子放在那里等。

大部分时候一辈子不成交,偶尔成交一次就是暴利。

群里 feiye2165 说的「小资金遍撒网,一个 500 撒完也是十几二十万」就是这个 —— \

你不知道哪个币会扎针,所以在几百个币上各挂一点。

2.1 链上怎么挂这个单

AMM(自动做市商)里没有传统的"限价单"。但是 ——

单边存入的集中流动性 = 一张限价单。

你在 0.58~0.60 这个区间只存 USDT,因为区间在现价下方,此刻它是 100% 的 USDT。 \

只有价格跌进来,才会有人拿币换走你的 USDT。这就是一张限价买单。

这一点在验证 LP套利法] 时已经讲过,是同一个机制

2.2 AMM 比交易所所有个好处

交易所挂单可能被"跳过" —— 价格瞬间从 0.65 跳到 0.55,中间没有成交,你 0.60 的单可能没成交。

AMM 不会。 AMM 的价格是沿着数学曲线连续移动的,想从 0.65 到 0.55, \

就必须一路"走"过 0.60,而走过的时候就必然吃掉你的单。

所以用"这一小时的最低价跌破了我的挂单价"来判断"会不会成交",在 AMM 上是可靠的。 \

这也是为什么这个策略能用历史数据回测 —— 成交与否是确定的,不用猜。

3. 接针 vs LP:近亲,但经济学相反

验证过的LP,链上操作和接针是同一种:单边 USDT 挂在现价下方。 \ 区别在放哪儿、赚什么:

```
| <br /> | LP | 接针 |
| :----- | :----- | :----- |
| 区间位置 | 贴近现价(-1% \~ -5%) | 远离现价(-20% \~ -50%) |
| 希望发生什么 | 价格反复穿越,来回收费 | 一次深针 |
| 收入来源 | 手续费 | 买到便宜货 + 价格回弹 |
| 被成交对你 | 坏事 —— 你被逆向选择了 | 好事 —— 这就是目的 |

| <br /> | <br /> |
| :----- | :----- |
| <br /> | <br /> |
```

最后一行是关键差异。收费站里被填单是亏损的开始;接针里被填单才是赚钱的开始。

但陷阱是同一个:你被谁填的?

被爆仓/恐慌填 → 那是"非信息流",价格会回来,你赚

被真实坏消息填(项目跑路、合约被黑) → 价格不回来了,你接的是归零盘

能不能区分这两种,就是这个策略的全部。

笔记 079: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践, Agent 仅负责

作者: geyu210 | 时间: 2026-08-09 13:56 UTC | 字数: 1259 | 评分: 60

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

(本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践, Agent 仅负责格式排版与提交流程。)

今天继续研究 1inch Aqua, 开始从协议概念进入 Ethereum 主网的链上观察。

我写了一个只读程序, 读取 Aqua Registry 的 Shipped、Docked、Pulled 和 Pushed 事件, 并按照 maker、app 和 strategy hash 将事件还原为不同的 position。随后观察了最近约 1,000 个区块, 从一千多个 strategy group 中筛选出少量同时包含 USDC/WETH 的 position。

筛选结果中, 一个	position	在观察区间末没有看到
Docked, 另一个已经在区间内关闭。这里我没有直接把前者称为"当前开放仓位", 因为		open_at_range_end
只能说明本次观察范围内没有看到关闭事件, 不能单凭这一点证明它现在仍然可以成交。		

【今天的重要发现】

Aqua 的 Pushed 事件不能直接理解为成交。ship() 初始化 position 时, 也会为每个 token 产生一次 Pushed。如果只统计事件名称、不结合交易哈希和事件顺序, 很容易把初始化资金误判成真实 fill。

今天还尝试解析这些 position 中的 SwapVM strategy bytes。外层 Order 可以读取, 但真实 position 的 payload 与当前官方 SwapVM 文档中的编码布局不一致, 因此程序将结果标记为: unsupported_or_malformed, 这个状态不代表链上 position 已损坏, 只表示当前 decoder 还不能可靠解释它使用的版本或 App 编码。因此暂时不能根据这些 bytes 推导 token、价格、费率或成交逻辑。

之后, 我又把两个 token 的钱包余额、Aqua allowance 和 virtual balance 固定在同一个历史区块读取。两个 token slot 当时都处于 active, 账面 allocation 也都有余额和 allowance 覆盖。不过 fully_backed 仍然不是成交保证, 因为 App 逻辑、另一条 token 腿、滑点和状态变化都可能导致 fill 失败。

【今天的结论】

今天完成的是一套 Aqua 链上 position 的只读观察方法: Registry 事件 → position 分组 → 区分初始化与后续资金流 → 判断区间内是否关闭 → 检查 virtual balance 的账面覆盖 → 尝试解析 App strategy。目前没有发现可以直接判断为可执行套利的偏差, 但确认了一个很重要的研究边界: Aqua Registry 只能告诉我 position 的生命周期和 token 资金变化, 真正的报价与成交语义属于具体 App, 不能只靠 Registry 事件推断。

笔记 080: D03 | DEX、订单簿与 AMM

作者: everywexler | 时间: 2026-08-09 14:20 UTC | 字数: 559 | 评分: 73

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日术语

DEX：交易结算受链上规则或智能合约控制的交易协议；

订单簿：按价格排列买单和卖单的市场结构；

Bid / Ask：当前最高买价 / 最低卖价；

Spread：最高买价与最低卖价之间的差；

AMM：自动做市商，用公式和资金池报价；

LP：把资产放入池中、供别人交易的流动性提供者。

参考材料

Uniswap：How Uniswap Works (<https://developers.uniswap.org/docs/get-started/concepts/how-uniswap-works>)

dYdX：Order Book 与 Depth Chart (<https://help.dydx.trade/en/articles/166979-interface-of-the-trade-tab>)：只用作订单簿结构观察，不进行交易。

一个具体例子

订单簿买单为 0.5 ETH@\$2,000、0.5 ETH@\$2,010。买 1 ETH 的平均价格是 \$2,005，不是页面顶部的 \$2,000。顶部价格只代表很小一层数量。

05 其他 (3 篇)

笔记 081: tradepools LBP 的五个状态

作者：V1ct0r551 | 时间：2026-08-08 16:12 UTC | 字数：824 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

tradepools LBP 的五个状态

(1) Unregistered / Pre-launch

\- strategy 还没绑定某个 initializer

\- registeredPoolIds 为空

\- _initializers_[initializer] 还没写

(2) Initialized / Auction running

\- initializeDistribution 已执行

\- initializer 已被工厂创建

\- auction supply 已转入 initializer

\- LP reserve 已留在 strategy

\- poolId 已被 registeredPoolIds 占住

\- 事件：InitializerCreated(...)

(3) Waiting for migration

\- 拍卖还在跑或刚结束但未到 migrationBlock

\- 这时只能走 initializer 侧的 claim / refund / exit 逻辑

\- strategy 自身还不能迁移

(4) Migratable / migration window open

\- block >= migrationBlock

\- 任意人可触发 migrate(initializer)

(5) Terminal

分两种结果：

\- Success：池子创建完成，LP 铸造完成，剩余 currency / 未用完 reserve 打给 recipient，发 Migrated

\- Failure：迁移逻辑失败则走 recovery，发 FundsRecovered + MigrationFailed

\- 无论哪种，registeredPoolIds 都被清空，所以这个 initializer 不会再进第二次 migrate

笔记 082: 学习了robinhoodchain (https://robinhoodchain.blocksco

作者 : bbvi | 时间 : 2026-08-09 01:33 UTC | 字数 : 1014 | 评分 : 58
来源 : https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

学习了robinhoodchain (https://robinhoodchain.blockscout.com/address/0xD71212080D90FFA50A80bf2aF2ADe6F99F29B96?tab=txs&page=7&next_page_params=%257B%2522index%2522%253A4%252C%2522value%2522%253A%25220%2522%252C%2522hash%2522%253A%25220xf15bbafc8096659f43a87f33e47fe7489bf6463f0013f0d19297f4e4e6a6b628%2522%252C%2522inserted_at%2522%253A%25222026-08-08T21%253A31%253A12.103438Z%2522%252C%2522block_number%2522%253A31398722%252C%2522fee%2522%253A%252210776980328000%2522%252C%2522items_count%2522%253A50%2527D) \ 高频交易地址的盈利方式\<https://robinhoodchain.blockscout.com/address/0xD71212080D90FFA50A80bf2aF2ADe6F99F29B96?tab=txs&page=7&next_page_params=%257B%2522index%2522%253A4%252C%2522value%2522%253A%25220%2522%252C%2522hash%2522%253A%25220xf15bbafc8096659f43a87f33e47fe7489bf6463f0013f0d19297f4e4e6a6b628%2522%252C%2522inserted_at%2522%253A%25222026-08-08T21%253A31%253A12.103438Z%2522%252C%2522block_number%2522%253A31398722%252C%2522fee%2522%253A%252210776980328000%2522%252C%2522items_count%2522%253A50%2527D>

笔记 083:

作者 : youchen-design | 时间 : 2026-08-09 14:11 UTC | 字数 : 2073 | 评分 : 59
来源 : https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

| 窗口 / 跌幅 | 場次 | 均進場價 | 實際勝率 | 校準差 | z |
| :----- | :----- | :----- | :----- | :----- | :----- |
| 30 分 / 跌 10% | 262 | 0.393 | 36.6% | 2.7% | 0.90 |
| 30 分 / 跌 15% | 172 | 0.344 | 29.7% | 4.8% | 1.38 |
| 30 分 / 跌 20% | 122 | 0.301 | 27.9% | 2.2% | 0.55 |
| 60 分 / 跌 10% | 473 | 0.395 | 37.8% | 1.7% | 0.76 |
| 60 分 / 跌 15% | 352 | 0.346 | 31.5% | 3.1% | 1.24 |
| 60 分 / 跌 20% | 267 | 0.286 | 26.2% | 2.4% | 0.89 |
| 90 分 / 跌 10% | 611 | 0.387 | 37.3% | 1.4% | 0.69 |
| 90 分 / 跌 15% | 509 | 0.335 | 31.0% | 2.5% | 1.22 |
| 90 分 / 跌 20% | 411 | 0.277 | 24.6% | 3.1% | 1.46 |
九個組合全部為負，沒有一個顯著。加上約 1.5% 手續費之後更負。
意思是：市場已經正確反映了早段落後。熱門方跌到 0.29 之後，實際只贏 26%——定價公允，甚至略偏貴。這跟報告 §5 的賽前發現方向一致（熱門方被輕微高估）。
這次修正是可信的，因為對照組驗證了管線
第 3 節我加了對照：不設任何觸發條件，單純在開賽 60 分後買熱門方
n=1,358 均進場價 0.566 實際勝率 56.2% 校準差 0.4% z=0.28
幾乎完美校準。如果方向鍵還是錯的，這個對照不可能校準到 0.4%。這是判斷「這次修對了」的關鍵證據。
(第 4 節有一個桶 \[0.20, 0.35) 顯示 9.1%、z=2.31 顯著，但測了 4 個桶出現一個邊緣顯著屬預期範圍，而且方向是負的——代表在那裡買更糟，不是更好。)
一個附帶發現：進場延遲幾乎不影響
| 進場時點 | 校準差 |
| :----- | :----- |
| 訊號當筆 | 2.8% |
| 訊號後 5 分 | 3.5% |
| 訊號後 15 分 | 3.1% |
| 訊號後 30 分 | 2.9% |
之前那個「+15 分鐘反彈 12.5%、t=20」的驚人反彈，本身就是價格被翻轉造成的假象。方向修好後就消失了。
但這個策略跟前一個有本質差異
|
 | 賽前買極端價 | 盤中買落後熱門 |
| :----- | :----- | :----- |
| 觸發價位有沒有量 | \$0 | 中位 \$18、75% \$100、90% \$301、最大 \$18,269 |

| 沒有量的比例 | 100% | 0.1% |
| 結論 | 買不到 | 買得到，但不賺錢 |

前一個是「不可執行」，這一個是「可執行但沒有優勢」。後者其實比較健康——至少你面對的是一個正常運作的市場。

功效限制

$n=352$ 、進場價 0.346，勝率標準誤 2.47pp，所以 95% CI 約 $[8.0pp, +1.8pp]$ 。換算成報酬，能排除的是「大於約 +5% 毛報酬」的優勢。點估計為負，上界也只有 +5%，扣掉手續費後所剩無幾。

要提高功效只能加樣本——但這是單季資料，沒有更多了。

06 套利框架（31 篇）

笔记 084: Day 4 | 算出利润之后，交易为什么还是抢不到？

作者：Lier Mi | 时间：2026-08-08 15:02 UTC | 字数：1184 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 | 算出利润之后，交易为什么还是抢不到？

第四天继续深入套利的执行层。

昨天学习了如何把市场抽象成交易图，搜索套利路径并计算最优投入金额。今天进一步认识到：策略计算正确，只代表机会在某个区块状态下存在，并不代表交易最终能被打包。

一笔套利交易需要经历完整链路：

监听链上状态 → 发现机会 → 计算路径 → 构造交易 → 本地模拟 → 提交交易 → 竞争排序 → 区块打包 → 链上结算

其中任何一个环节出现延迟，都可能让利润消失。

套利机器人读取到的是某个时刻的链上状态，但计算、签名和传播都需要时间。在交易真正进入区块之前，其他用户或搜索者可能已经改变池子的储备。于是，原本盈利的路径可能出现三种结果：

价差缩小，利润低于预期；

最低输出条件不满足，交易回滚；

交易成功执行，但扣除 Gas 后实际亏损。

这就是链上套利最核心的竞争之一：状态竞争。

如果把交易直接发送到公开交易池，其他搜索者可能提前看到交易内容，复制同一条套利路径，并通过提高优先费争夺排序。最后往往不是最先发现机会的人获利，而是报价更高、传播更快、与区块构建者连接更稳定的人获利。

因此，套利交易的价值不能只用理论利润衡量，还要考虑执行概率：

期望收益 $\approx$ 成功打包概率 $\times$ 成功利润 - 回滚概率 $\times$ Gas 成本 - 基础设施成本

这让我意识到，一个账面利润很高、但只有 10% 概率能够成功打包的机会，未必比一个利润较低、却能稳定成交的机会更好。

为了提高执行确定性，系统通常需要做好几层保护：

使用同一区块的池子状态计算所有路径；

使用完全相同的 calldata 和账户状态模拟交易；

在合约中设置 minProfit，实际利润不足时立即回滚；

设置有效区块或截止时间，避免过期交易被延迟执行；

根据实际利润动态计算可接受的优先费；

管理好 nonce，避免多笔机会相互阻塞；

必要时通过私有交易通道或交易包提交，减少策略暴露。

但私有提交也不是万能的。它可以降低被公开复制或抢跑的概率，却不能保证交易一定进入目标区块。不同构建者的覆盖率、连接质量、模拟规则和延迟，都会影响最终结果。

今天还建立了三个不同层次的判断：

1. 策略有效：当前状态下确实存在正向价差。
2. 交易有效：扣除滑点、Gas 和手续费后仍有净利润。
3. 执行有效：交易能够以正确顺序进入区块，并在状态变化前完成结算。

只有三层同时成立，屏幕上的机会才可能真正变成钱包里的利润。

第四天最大的收获是：链上套利不是“算对一道题”，而是在不断变化的状态中，把正确答案及时写进区块。
策略决定理论上能不能赚，执行系统决定最终能不能拿到。

笔记 085: 套利核心原理与判断框架

作者：haze | 时间：2026-08-08 15:17 UTC | 字数：7274 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

套利核心原理与判断框架

零、一句话内核

你赚的不是差价，你赚的是「承担风险把差价抹平」的报酬。

这句话是整份文档的压缩版。下面所有内容都是它的展开和推论。

一、利润从哪来：四层原理

1.1 套利利润是服务费，不是发现费

价格在两个地方不一致，是市场的一种失灵。谁去修复它，谁拿报酬。套利者的社会功能就是让价格重新统一，利润是市场为这项服务支付的费用。

这个定性决定了一切，因为服务费由供需决定，不由你的技巧决定：

需求侧 = 需要被修复的价格分裂有多少

供给侧 = 有多少人有能力且愿意去修

推论：你的技术再好，在一个供给过剩的赛道里也赚不到钱。这不是能力问题，是那个市场已经不需要更多修复者了。

选赛道比练技术重要。这是这条原理最实用的推论。

1.2 价差永远是别人的「不得不」

价差不会凭空出现，一定是有人做了什么。来源只有三类：

(1) 有人需要立刻成交，且不在乎价格 —— 流动性需求

一只基金要在五分钟内买进五百万美元的某个币，它不可能慢慢挂单等，它会直接打穿盘口。它支付的这个「打穿成本」，就是你的利润。

它买的是即时性，你卖的是即时性。

(2) 信息传播有延迟 —— 信息搬运

消息先反映在流动性最好的市场，其他市场慢半拍。你赚的是信息搬运费。

这一类正在随技术进步快速消失，不适合作为长期依赖。

(3) 市场之间有摩擦 —— 跨越摩擦的报酬

转账要时间、过桥要费用、有些人访问不了某些交易所、法币通道有限制、KYC 门槛、地域限制。

摩擦越大，价差越大也越持久。

第三类最值得关注。前两类会被技术抹平，摩擦只要存在，价差就一直存在。所有能长期做的套利，本质上都在赚第三类的钱。

1.3 利润必然对应你承担的某样东西（核心法则）

不存在无风险利润。任何套利收益，都精确对应你承担的某一样具体的东西：

| 你承担的 | 具体形态 | 对应报酬 |

| :----- | :----- | :----- |

| 价格风险 | 两条腿之间的时间差 | 价差 |

| 对手方风险 | 在小所/新协议留资金 | 小所的价差更大 |

| 资金占用 | 钱压着做不了别的 | 超出资金成本的部分 |

| 执行失败风险 | 单腿、烧掉的 gas | 竞争性利润 |

| 流动性风险 | 需要用钱时取不出来 | 流动性溢价 |

| 运营/技术风险 | 系统宕机、API 异常、人为失误 | 运营溢价 |

大所之间没有价差，不是巧合 —— 那里没有你可以承担的东西，所以没有你的报酬。

可随身携带的判定法则

面对任何一个「机会」，先问：我在这里承担了什么？

如果答不上来，不代表它无风险，只代表有一个风险你还没看见。

举例：某看板上一个 72% 的差价，你承担的是什么？是「你的钱会永久卡死在一个已经没有人交易的市场里」。报酬看起来极高，是因为对应的风险是接近必然发生的损失。市场给它定价 72%，市场是对的。

这条法则反过来解释了原子套利为什么不赚钱

原子性消除了价格风险 —— 你在结构上不可能亏本金。既然你不承担风险，市场也就没有理由付钱给你。于是利润被竞争压到只剩 gas 成本之上一点点。

你承担得越少，你能赚的越少。

这条闭环解释了整个套利世界的结构：闪电贷套利零门槛所以红海；需要压库存、需要扛价格风险的方向反而有钱赚，因为愿意扛的人少。

1.4 收敛不是自然规律

最深的一个错误默认是「价差总会回归」。它不会。

价差回归，是因为有一群人（包括你）在用真金白银把它推回去。如果没有人有能力或有意愿去推，它就永远不回归。

判断一个价差能不能做，真正该问的不是「它有多大」，而是：

是什么力量会把它推回去？

| 情形 | 是否收敛 |

| :----- | :----- |

| 两边都有真实深度，有人能同时成交 | 会收敛 |

| 两边被摩擦隔开（提币关闭、无法访问、需 KYC） | 不收敛 |

| 一边根本没有真实交易量 | 不收敛 —— 那个「价格」本来就不是价格 |

| 标的本身有问题（迁移、换币、即将下架） | 不收敛，且可能扩大 |

二、利润公式的真实形态

净利润 = (价差 - 摩擦成本) × 周转次数 × 资金规模 × 成功率

四个乘数，任何一个为零，结果就是零。而绝大多数人只盯着第一项。

这个式子的关键推论：

套利是薄利多销的生意。

单次利润必然被竞争压到贴近成本线，所以真正决定你赚多少的，是周转次数和成功率，不是找到一个多大的价差。

这也是为什么「再平衡速度」和「单腿失败率」比「最大价差」更重要

它们是式子里的第二项和第四项，而「最大价差」只是第一项里被压得最扁的那个。

你能优化的空间，恰恰不在你最想看的那一列上。

三、原子性：整个套利世界的分水岭

3.1 定义

从数据库术语 ACID 的 A (atomicity) 借来：一组操作要么全部生效，要么全部不生效，不存在做了一半的中间状态。

3.2 在链上意味着什么

一笔区块链交易内部的所有操作天然原子的。任何一步失败，或你主动 revert，整笔交易回滚，链上状态回到交易之前，就像什么都没发生过。

于是你可以把整个套利路径写进一笔交易：

1. 用 100 USDC 在 A 池买 ETH
2. 把 ETH 在 B 池卖回 USDC
3. if (最终 USDC < 100) revert();

结果只有两种：要么赚到，要么链上什么都没发生。第三行让「亏本金」在结构上不可能。唯一损失是 gas。

3.3 闪电贷是原子性的直接产物

交易开头借 100 万 USDC，无需抵押 —— 因为没能在同一笔交易结束前还上，整笔交易就会 revert，那笔借款从未发生过。

对贷方零风险，所以不需要抵押。对你：做原子套利可以不需要本金。

3.4 对照：非原子

在 A 所买入成交，再去 B 所卖出。这是两个独立动作，中间隔着时间，没有任何机制能把它们捆绑。中途价格变了，你的套利就变成亏损，且无法回滚 —— A 所那笔已经成交了。

这就是腿风险 (leg risk) 的来源。跨链更极端：资产在桥上的那几分钟，你完全裸露。

根本原因：原子性只存在于同一个状态机内部。同一条链上的操作共享一个状态机，所以能一起回滚。两条不同的链、或两个 CEX 的数据库之间没有共同状态机，没有谁能协调它们一起撤销。

3.5 原子性连锁决定三件事

| 维度 | 原子 | 非原子 |

| :---- | :----- | :----- |

| 承担什么风险 | 竞争风险 (抢不到、gas 沉没) | 价格风险 + 对手方风险 |

| 需要多少资金 | 闪电贷可解决, 趋近于零 | 必须两边压库存 |

| 和谁竞争 | 全世界 (门槛为零) → 极端红海 | 有资金和风控能力的人 → 参与者少几个数量级 |

因果链: 原子 → 无需本金 → 零门槛 → 极端竞争。

反向读法: 当你看到一个方向「必须压库存、必须承担价格风险」, 第一反应不该是「这个更危险」, 而应该是「这里的竞争者少得多」。风险和门槛在这里是同一件事的两面。

3.6 一个必须纠正的常见说法

「原子套利是无风险套利」

错。原子性保证的是不会执行一半, 不保证一定能执行。

你的交易可能根本没被打包 (机会被抢先)、可能被打包但因状态改变而 revert (付了 gas 没赚到钱)、可能被抢跑。

原子套利的风险没有消失, 只是从价格风险变成了竞争风险 + gas 沉没成本。一个跑得不好的原子套利程序, 会持续烧 gas 而几乎不出单——这就是它亏钱的方式。

四、成本模型

4.1 CEX 版 (跨所 / 现期对冲)

净收益 = 价差

手续费 × 成交次数

滑点 (按你的下单量对应的加权成交价算, 不是盘口第一档)

资金费率成本 (换算成日成本)

再平衡成本 (转账费 + 时间 + 期间敞口)

资金占用成本 (两边都要压钱)

单腿失败的期望损失

最常算错的一点: 一个完整周期是四次成交, 不是两次。

现期对冲: 开仓 (现货买 + 合约卖) + 平仓 (现货卖 + 合约买) = 4 次。

按普通用户 taker 费率, 一个来回大约吃掉 0.25%–0.35%。

心算基准线: 差价的收敛幅度必须大于约 0.3%, 这单才有意义。

手续费等级是核心

edge。

同样的价差, 对不同费率档位的人是不同的生意。开户走返佣链接直接压低你的盈亏平衡线, 这是性价比最高的单一动作。

4.2 DEX 版 (链上)

净收益 = 价差

Gas (含失败交易也要付的部分)

协议费 / 池子手续费

桥费 (跨链)

滑点与价格冲击 (由 AMM 曲线数学确定性给出)

延迟成本 / 桥接期间的价格敞口

失败交易成本

资金占用成本

竞争折价 (拍卖式环境下回吐给 builder/validator 的部分)

4.3 两个几乎所有人都会漏掉的项

(1) 条件期望 / 逆向选择

正确的写法是条件期望, 不是无条件期望:

$E[\text{净收益} | \text{我成交了}] \neq E[\text{净收益}]$

你能成交, 本身就是坏消息。

原子套利: 你赢下竞价, 通常意味着你出价比所有对手都高 (赢家诅咒), 或这个机会有毒

非原子套利: 你那条腿被成交, 往往是因为价格已经朝你不利的方向动了, 对手方比你先知道

对回测的影响: 如果不区分「我看到的」和「我能拿到的」, 回测会系统性高估, 幅度常常超过 100%。

回测算出正收益也只是线索, 因为回测默认你 100% 成交。

(2) 库存与再平衡——非原子套利的真正瓶颈

做完一次跨所搬运后: A 所少了计价币多了标的, B 所反过来。你没法立刻做第二次, 必须先把资产搬回去——要等确认、要付费、期间有敞口。

你的实际交易频率被再平衡速度锁死，而不是被机会数量锁死。

屏幕上一天出现 50 次机会，你可能一天只能做 3 次。所有只算单次收益率的推演，都会高估实际年化好几倍。

这也是对冲式优于搬运式的原因：一边现货多、一边合约空，吃基差收敛，资产不需要真的跨所搬运。代价是换成了合约腿的保证金管理和爆仓风险。

五、通用分析框架：任何机会问四个问题

无论 CEX 还是 DEX，任何套利场景都要回答：

1. 价差从哪来？谁的什么行为制造了它。不理解来源，你就只能被动等机会出现，永远无法预判。这是「看行情的人」和「做套利的人」的分界线。
2. 我在和谁竞争，竞争在哪个维度？延迟 / 资金 / 信息 / 费率 / 基础设施。这五个维度决定了你有没有资格入场。
3. 完整成本结构是什么？见第四节。特别注意条件期望和再平衡这两项。
4. 执行不确定性有多大？能不能成交、成交在什么价、失败的代价是多少。

六、诊断法则集（实操速查）

法则一：共同腿法则

同一个币，换掉一条腿的对手方，差价基本不变 → 问题出在那条不变的腿上。这是数据/资产问题，不是套利机会。

不是那七个交易所集体错了，是那一个交易所的报价有问题。可能原因：代币迁移/换币、币名映射错误、交易对已停止撮合但仍在报价、标的即将下架。

这条法则能在三秒内否掉高差价行里的绝大多数，纯靠结构，不需要背景知识。

法则二：排序即陷阱

按差价降序排列的看板，顶端在结构上必然是数据最坏死的那些行。

理由：一个真实、可执行、能收敛的价差会被所有人吃掉，所以它活不到 70%，甚至活不到 2%。凡是能长期停在高位的，一定是没有人能吃 —— 那正是「不可执行」的定义。

真实的机会永远在列表中部 —— 差价小到不显眼（零点几到几个百分点），但两条腿的流动性都是真的。

法则三：流动性优先于价差

先看两条腿的 24h 交易量和盘口深度，再看差价。一条腿一天成交不到一千美元，那个「价格」只是最后一笔零星成交留下的残影，不是你能成交的价格。

法则四：承担什么

见 1.3。答不上来 = 有风险没看见。

法则五：什么力量推它回去

见 1.4。答不上来 = 它不会收敛。

法则六：大差价 = 大风险的定价

大所之间没价差，有大价差的地方通常是小所/长尾标的。你赚的那部分钱，很大程度上就是在承担对手方风险。

推论：永远不要在小所留超过你能承受损失的余额。

七、读别人的收益文章：固定四问

任何声称收益的教程/推文，问这四个问题，大多数会自己塌掉：

1. 它算是毛收益还是净收益？（有没有算全成本）
2. 方向性敞口在哪里、有没有被定价？（是不是偷偷让你裸多某个币）
3. 作者的数字是实证还是模型推算？（有没有真实到账记录）
4. 如果收益来源是激励，它的衰减路径是什么？（奖池固定 → 参与者越多分得越少 → 教程传播本身就在稀释它）

看懂一篇文章，指的是能指出它没说的部分。

附：一个典型案例的拆解模式

某「套利教程」建议 1000U 中 500U 换成平台币做市：

名不副实：这是做市 + 挖激励，不是套利。有方向性风险，风险类别完全不同

50% 的裸多头敞口在收益计算里完全没有定价。该币跌 20% = 亏 100U = 十天以上的宣称收益

无常损失一字未提，而这是集中流动性做市的核心成本项

作者自己承认「真实到账记录还没出现」→ 数字零实证

宣称日化 0.9%（年化三位数）→ 任何这个量级的宣称，默认值应为「未验证」

自我否定结构：奖池固定，参与者越多分得越少，教程传播本身在稀释它

唯一有价值的部分：指出奖励按成交量而非 TVL 分配，所以策略重点是提高周转率而非堆资金

八、常见错误说法纠正表

常见说法	实际情况
原子套利是无风险套利	价格风险变形为竞争风险 + gas 沉没成本，风险没消失
价差总会回归	回归是因为有人在推它，没人推就不回归
只开交易权限就安全了	有交易权限即可通过低流动性币对的对倒转移你的资金
价差越大机会越好	价差越大通常意味着越不可执行，或风险定价越高
回测赚钱 = 策略可行	回测默认 100% 成交，忽略逆向选择，系统性高估
找到大价差是关键	周转次数和成功率才是利润公式里可优化的部分
屏幕上看到价差 = 找到机会	只是线索。扣完成本、验证可执行性之后才接近机会

九、最后

如果只能记住三句话：

1. 你赚的是承担风险的报酬，不是发现差价的奖金。
2. 你承担得越少，你能赚的越少。
3. 能优化的空间不在你最想看的那一列上。

笔记 086: 今日打卡

作者：elipitic | 时间：2026-08-08 15:20 UTC | 字数：2735 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日打卡

打卡日期 (UTC+8)：2026-08-08

今日学习主题

今天不只是学习了

Li.Fi

获取各类报价 (quote) 的具体方式，还进一步把真实报价跑完，并对候选套利路径做了第一轮完整分析。今天的重点从“知道怎么拿 quote”推进到了“用真实 quote 去判断一条路径是不是伪套利”。

一、今日学习内容

1. 学习了 Li.Fi 获取报价的具体方式

今天重点理解的是：Li.Fi 不只是一个“跨链入口”，它本质上还是一个 路径和报价聚合器。获取报价的过程，不只是问“从 A 到 B 能不能换”，而是同时在观察：

不同链之间的路径

不同资产之间的兑换结果

route 在多大程度上受金额、Gas、流动性源影响

返回结果中哪些字段最值得后续持续追踪

2. 跑通了真实报价并保存结果

今天不只是停留在概念层，而是实际跑出了多条 quote，并保存为本地数据文件：

data/lifi/eth_to_arb_usdc_1000.json

data/lifi/arb_to_eth_usdc_1000.json

data/lifi/eth_to_base_usdc_1000.json

data/lifi/base_to_eth_usdc_1000.json

这些结果让我从“理解 Li.Fi 返回什么”进一步进入到“比较不同路径、判断路径值不值得继续跟踪”的阶段。

二、今日实际分析：报价与候选套利路径

1. 单跳报价观察

今天的真实报价里，Li.Fi 返回的主要路线工具是 eco，在当前样本中表现为：

- Ethereum → Arbitrum (USDC 1000)
- 输入约 1000 USDC
- 输出约 997.5 USDC
- gas 约 0.097 美元
- Li.Fi 固定费约 2.5 USDC
- Arbitrum → Ethereum (USDC 1000)
- 输出约 997.5 USDC
- gas 约 0.024 美元
- Li.Fi 固定费约 2.5 USDC

- Ethereum → Base (USDC 1000)
- 输出约 997.5 USDC
- gas 约 0.097 美元
- Li.Fi 固定费约 2.5 USDC
- Base → Ethereum (USDC 1000)
- 输出约 997.5 USDC
- gas 约 0.007 美元
- Li.Fi 固定费约 2.5 USDC

2. 往返路径分析

今天最重要的推进不是看单跳，而是做了一个更接近套利思维的动作：

把往返路径组合起来，看它是否真的有正收益。

我今天重点看了两条 round-trip：

1. Ethereum → Arbitrum → Ethereum
2. Ethereum → Base → Ethereum

3. 初步结论

今天的结果说明：

当前样本下，这两条 1000 USDC 的往返路径都没有形成真实套利

两条往返在计入 fee 与 gas 后，大约都落在 -0.51% 左右的负收益区间

Base 正向路径里看起来出现过表面上的美元增益，但继续组合返程后，这种“利润”并没有成立

4. 我今天最关键的理解

今天真正重要的不是“没找到套利”，而是我确认了一件很关键的事：

单跳 quote 的表面好看，并不等于真实可执行套利。

也就是说：

- 只看某一跳的 toAmountUSD 很容易被误导
- 真实套利判断必须至少做到：
- 看 token 数量变化
- 看固定费用
- 看 gas 成本
- 看往返组合后是否仍然成立

三、这部分学习对本期共学的意义

今天这部分推进和本期共学主线非常一致，因为它补齐了后续跨链机会判断里最关键的一块：

1. 路径理解能力

后续如果要判断跨链机会，不能只看某一边价格，而要看：

- route 怎么走
- 中间经过哪些流动性源
- 哪些成本在路径中被吃掉

2. 报价结构理解能力

如果不理解 Li.Fi 返回结果的字段，就无法把 quote 变成：

- 成本判断
- 净收益判断
- 候选机会筛选依据

3. 伪套利识别能力

今天最有价值的一步是：

- 我开始学会区分“表面路径增益”和“真实往返套利”
- 这比只看单次 quote 更接近真正的套利研究

四、今日收获

今天最重要的收获有四点：

1. Li.Fi 的 quote 本质上是路径视角的数据入口

它不仅是报价，更是后续做路径分析与机会判断的基础。

2. 套利研究不能停留在“知道聚合器存在”

真正有用的是理解它如何返回数据、如何表达路径、以及结果如何被进一步分析。

3. 今天完成了从概念走向真实报价分析的一步
不只是拿到 quote，还把 quote 真正用于路径对比和候选套利验证。
4. 没有发现真实套利本身也是重要结论
今天的负收益结果说明，今后做跨链机会判断时，必须对“表面利润”保持警惕。
- 五、下一步计划

接下来我会继续推进以下方向：

1. 继续拆解 Li.Fi quote 返回结果中的关键字段，明确哪些字段对机会判断最重要
2. 尝试对比不同金额规模下的报价差异，观察 route 是否切换
3. 继续把 Li.Fi 的报价结果和成本模型、净收益判断连接起来
4. 引入更多约束条件，例如外部可成交价格、真实退出价格与流动性可执行性，进一步排除伪套利路径

六、今日完成标准检查

- [x] 学习了 Li.Fi 获取报价的具体方式
- [x] 跑通了真实 quote 并保存结果文件
- [x] 完成了至少两条 round-trip 路径的初步比较
- [x] 识别出当前样本中不存在真实套利，而是伪套利 / 表面套利路径
- [x] 为后续 route 对比和成本模型打下了基础

一句话总结

今天我完成了从“知道 Li.Fi 是什么”到“开始用真实报价分析并识别伪套利路径”的过渡，这一步对后续做跨链套利研究非常关键。

笔记 087: Day 4 — 周末 60 分钟价差监控 + 跨时段总汇

作者：ChenXuan1985 | 时间：2026-08-08 15:52 UTC | 字数：4047 | 评分：88
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 — 周末 60 分钟价差监控 + 跨时段总汇

残酷共学 · 链上套利专题

日期：2026-08-08（周六）

阶段：60 分钟持续监控 + 多时段对比验证

一、今日学习内容

1.1 周末 60 分钟持续监控

时段：CN 周六 20:17–21:21（亚洲晚间）

方法：Binance + OKX 批量端点，每 5 分钟一轮，共 12 轮

标的：27 只美股 perp

1.2 OKX 批量端点改造

发现逐一查询 OKX (27 只 × 3s) 在代理环境下极易超时。改用 GET /api/v5/market/tickers?instType=SWAP 批量端点，单次请求从 81s 降到 ~10s。

1.3 gate-crossex 前端安装

选择 bootstrap.ps1 方案（预编译 Node.js + 依赖，不需要 Visual Studio Build Tools）。已在 PowerShell 中完成安装，Node.js 24.18.0 独立运行时，前端构建成功 → C:\Users\PC\gate-crossex。

二、60 分钟价差数据（12 轮完整序列）

Binance vs OKX，周末 CN 20:17–21:21

Round Time(CN) KLAC CRWD ADBE CRM SNOW >10bps

R1 20:17	-8.0	+6.6	+10.2!	-7.6	+5.9	1
R2 20:22	-8.0	+5.5	+12.3!	-8.9	+5.0	1
R3 20:28	-8.0	-1.2	+9.6	-5.7	+5.1	0
R4 20:33	-3.5	-10.2!	+11.7!	-3.1	+9.3	2
R5 20:38	-8.0	+8.5	+11.3!	+4.2	+9.5	1
R6 20:43	-8.0	+1.7	+11.5!	+4.2	+12.1!	2
R7 20:49	-3.0	+3.8	+9.6	+4.2	+9.0	0
R8 20:54	-8.0	+6.4	+11.0!	+4.2	+9.2	1
R9 21:00	-7.5	+7.6	+11.7!	+4.2	+9.3	1
R10 21:05	-7.5	+19.7!	+11.9!	+13.1!	+9.3	3

R11 21:10 -9.3 +8.6 +9.5 +16.8! +9.2 1
R12 21:15 -24.0! +1.2 +9.3 +12.0! +6.2 2
R13 21:21 -23.3! +8.1 +10.6! +3.7 +6.3 2

2.1 核心发现

ADBE —— 周末之王

12/13 轮 >5 bps，10/13 轮 >10 bps。范围 +9.3 \~ +12.3 bps，极其稳定。

结论：ADBE 在周末晚上是确定性最高的套利标的。OKX 系统性地比 Binance 高出 \~10-12 bps（方向：卖 OKX，买 Binance）。

KLAC —— 尾盘爆炸

前 9 轮稳定在 -8 bps，第 12 轮突然跳到 -24.0 bps（3 倍！），第 13 轮持续 -23.3。

结论：周末价差可以瞬间放大 3 倍——KLAC 的 -24 bps 是本周末监测到的最大单点价差。如果网格在设计范围内（如 ±15 bps），这种突然放大可能导致超出网格区间的风险。

CRWD —— 最剧烈震荡

-10.2 → +8.5 → +19.7 → +1.2 → +8.1（跨度 30 bps，方向翻转 3 次）。

结论：CRWD 不适合单向网格——必须双边挂单 + 方向切换检测。

CRM —— 后半段爆发

前 9 轮基本睡眠（-7.6 \~ +4.2），R10-R12 突然飙升到 +16.8 bps。

三、全时段价差数据总汇

跨 3 天，5 个时段，27 只标的

时段 日期 约 NY 时间 >10bps次数 Top3

深夜休盘 Tue 04:48 EDT 8 KLAC+14, AMAT+13, CRM+13
开盘30分 Tue 10:07 EDT 7 KLAC+21, LRCX+18, CRM+17
盘中 Tue 11:46 EDT 0 全部 <7 bps
周末亚洲午后 (30min) Sat CN 15:05 22/4轮 CRWD-25, AMAT-22, KLAC-15
周末亚洲傍晚 (60min) Sat CN 20:17 17/13轮 KLAC-24, CRWD+20, CRM+17

3.1 收敛速度对比

时段 收敛速度 典型收敛时间 驱动因素

开盘30分 ~10 bps/min 30-60 秒 做市商立即套利
周末下午 ~5 bps/h 2-4 小时 散户慢慢回归
周末傍晚 ~3-5 bps/h 3-6 小时 流动性进一步降低
周末收敛比开盘慢 120 倍，但收敛率 100%（所有完成追踪的都收敛了）。

3.2 标的分类

稳定性:

高稳定: ADBE (周末几乎每轮都有 >5 bps)
中稳定: KLAC, SNOW, CRM (多轮出现，但有间歇)
突发型: CRWD, AMAT (间歇性爆发，方向可能翻转)
低活性: AAPL, TSLA, AMZN, META (周末价差大多 <5 bps)

板块:

半导体: KLAC, AMAT, LRCX, MRVL — 周末最活跃板块
SaaS: ADBE, CRM, CRWD, SNOW — 次活跃
ETF: SPY, QQQ — 偶有 >5 bps (SPY 出现过 -6.9)
蓝筹: 休眠状态

四、发现的 Bug & 改进

问题	影响	状态
OKX 逐只查询卡死	watchdog 首轮 >5min	已改用批量端点
Python 管道缓冲	后台进程日志不可见	需 PYTHONUNBUFFERED=1 + H 盘输出
Bybit SSL 偶发阻断	部分时段无法获取 Bybit	Clash 代理问题

| 资金费率周末全零 | 费率套利无信号 | 预期行为 |
| 周末采样间隔 300s | 可能错过瞬时 spike | 建议改 60s |

五、下一步方向

1. 修 watchdog → 合入 OKX 批量端点 + 动态采样间隔 (周末 60s)
2. gate-crossex 前端 → 已安装完成, 待启动探索
3. 扩到 Bybit → 排查 Clash 代理 SSL 问题, 恢复三所对比
4. 周末定时监控 → cron job CN 周六 14:00-22:00
5. 网格参数建模 → 用 ADBE 12 轮数据反推最优网格层数/间距

笔记生成时间: 2026-08-08 21:30 CST

笔记 088: 第三天 | 跨链套利、清算与最大可提取价值

作者: OxFeyoma | 时间: 2026-08-08 15:56 UTC | 字数: 3038 | 评分: 80

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第三天 | 跨链套利、清算与最大可提取价值

关联笔记 01-第一天-市场机制、流动性与区块 (01-第一天-市场机制、流动性与区块) 02-第二天-DEX套利结构与稳定币
(02-第二天-DEX套利结构与稳定币)

核心判断

跨链价差、清算奖金和区块排序收益都不是免费的利润。它们补偿了资金占用、状态变化、执行竞争和失败风险。

1. 共同的期望收益模型

屏幕上看到的毛收益只是利润上限。真实决策应使用期望净收益。

$$\mathbb{E}[\Pi] = p_s \Pi_{\text{成功}} - (1 - p_s) L_{\text{失败}} - C_{\text{资金}} - C_{\text{竞争}}$$

p_s 表示成功概率, $L_{\text{失败}}$

表示失败后的损失。资金成本包括库存占用、融资和等待结算。竞争成本包括优先费用、建设者支付、失败交易和基础设施支出。

三个领域承担的主要风险不同。

| 领域 | 收益来源 | 主要代价 |

| :----- | :----- | :----- |

| 跨链套利 | 不同链上的价格差 | 时间、库存、桥接和最终性 |

| 清算 | 协议提供的清算奖励 | 预言机变化、竞争和抵押品卖出冲击 |

| 最大可提取价值 | 交易排序与区块位置 | 竞价、延迟、信息暴露和执行失败 |

最深刻的一点

机会持续时间越短, 执行位置通常越贵。看到机会的速度和获得执行权的能力是两件不同的事。

2. 跨链套利

最直观的跨链路径如下。

低价链买入 $\rightarrow$ 等待确认 $\rightarrow$ 桥接 $\rightarrow$ 高价链卖出

这条路径包含较长的价格暴露时间。资金等待来源链确认、桥接验证和目标链到账时, 高价市场可能已经回落。任何一段延迟都会改变最终收益。

更稳健的执行方式会提前在两条链上部署库存。

低价链买入 $\rightarrow$ 高价链同步卖出 $\rightarrow$ 事后再平衡库存

预置库存缩短了两腿成交之间的时间差。它把等待期间的价格风险转化为库存、资金占用和再平衡成本, 风险仍然存在。

跨链净收益可以写成

$$\begin{aligned} \Pi_{\text{跨链}} = & \text{卖出收入} - \text{买入成本} - \text{两条链的网络费} - \text{桥接与再平衡费用} - \text{对冲成本} - \text{资金占用成本} - \text{预期失败损失} \end{aligned}$$

分析跨链机会时需要确认四件事。

1. 两条链上的资产是否拥有相同的赎回权利。
2. 两条链分别需要什么确认与最终化条件。
3. 桥接协议何时认为转移已经结算。
4. 交易结束后, 库存如何恢复到目标分布。

原生资产、包装资产和桥接版本可能拥有不同的发行方、托管方式和赎回路径。名称相同无法证明风险相同。

3. 清算机会

借贷协议允许用户用抵押品借出资产。当抵押价值下降或债务价值上升时，仓位的健康因子会下降。

以常见模型表示

健康因子 = $\frac{\text{抵押品价值} \times \text{加权清算阈值}}{\text{债务价值}}$

健康因子低于 1 时，仓位通常进入可清算状态。清算者偿还一部分债务，并得到等值抵押品和额外奖励。

$$\text{清算} = \text{清算奖励} - \text{抵押品卖出冲击} - \text{网络费} - \text{融资费} - \text{排序费用} - \text{失败成本}$$

清算奖励的作用是吸引外部资金快速修复危险仓位。奖励很高时，往往也意味着抵押品波动、市场拥堵或坏账风险更高。

一笔原子清算常见的资金过程如下。

借入债务资产 $\rightarrow$ 偿还债务 $\rightarrow$ 取得抵押品 $\rightarrow$ 卖出抵押品 $\rightarrow$ 归还借款

清算机器人需要同时监控

1. 借款人的健康因子。
2. 预言机价格与更新时间。
3. 协议的清算阈值、奖励和可清算数量。
4. 抵押品卖出市场的真实深度。
5. 竞争者愿意为排序支付多少费用。

清算原则

健康因子低于 1 只说明仓位可以被清算。清算奖励扣除抵押品卖出冲击和竞争成本后，才可能形成收益。

4. MEV 的执行结构

MEV (Maximal Extractable Value, 最大可提取价值) 指区块生产者通过包含、排除或改变交易顺序所能提取的额外价值。套利者和清算者通常由独立搜索者承担，搜索者需要争取有利的区块位置。

执行关系可以简化为

用户订单 $\rightarrow$ 搜索者 $\rightarrow$ 区块建设者 $\rightarrow$ 验证者 $\rightarrow$ 区块

搜索者发现套利或清算机会后，可以把交易发送到公共交易池，也可以通过私有渠道提交交易包。建设者选择能够提高区块价值的组合，验证者负责提出最终区块。

套利和清算经常构成最大可提取价值。最大可提取价值描述的是交易如何获得排序与执行位置，它不会替代机会本身的经济来源。

毛收益通常在参与者之间重新分配。

毛收益 = 搜索者净收益 + 网络费与建设者支付 + 失败及基础设施成本

竞争越激烈，搜索者越可能把大部分毛收益支付给建设者和验证者。私有提交可以减少交易在公共交易池中的信息暴露，但无法保证被打包，也无法保证最终盈利。

5. 排序如何改变结果

同一个交易集合采用不同顺序，最终状态可能完全不同。

1. 你的套利先执行，价差被你消除。
2. 竞争者先执行，你的交易按恶化后的价格成交或回滚。
3. 一笔大额用户交易先执行，它可能创造新的套利空间。
4. 预言机更新先执行，一个借款仓位可能突然进入可清算状态。

因此，报价必须绑定区块状态，模拟还要明确假设的交易顺序。只模拟当前状态，无法证明交易进入未来区块后仍然盈利。

三明治交易也利用排序影响普通用户的成交结果。理解这类机制的主要用途是识别暴露风险、设置合理的最小接收数量，并选择适当的私有执行方式。

6. 通用执行检查

机会依赖哪个区块状态和预言机状态。

从发现机会到实际执行需要多长时间。

每一笔交易在什么条件下回滚。

失败时会损失网络费，还是会留下方向持仓。

报价是否已经包含完整数量的价格冲击。

需要占用多少库存或借入多少资金。

最终化与桥接结算分别需要多久。

竞争者能够观察到多少交易信息。

建设者支付提高到什么水平后利润归零。

极端拥堵或价格跳变时，最大损失是多少。

7. 低成本训练方法

学习阶段应优先使用历史区块回放、本地分叉和只读模拟。每次模拟主动加入不利变化。

1. 让竞争者在你的交易之前执行。
2. 提高网络费和建设者支付。
3. 延迟跨链到账时间。
4. 降低抵押品卖出深度。
5. 改变预言机更新时间和价格。

这些小规模失败能够暴露策略依赖的脆弱条件。四天学习期内没有必要用真实资金验证它们。

8. 最值得记住的五句话

1. 跨链价差需要支付时间、库存和最终性成本。
2. 预置库存缩短执行时间，同时增加资金占用和再平衡负担。
3. 可清算状态只提供执行资格，清算奖励扣除全部成本后才可能形成利润。
4. 最大可提取价值决定谁获得执行位置，竞争会把毛收益转移给区块建设者和验证者。
5. 模拟必须包含未来区块的状态变化和交易顺序。

笔记 089: 资金费套利：从 Funding Rate 排行榜到可执行交易策略的理解

作者：JianBo He | 时间：2026-08-08 15:59 UTC | 字数：506 | 评分：60

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

资金费套利：从 Funding Rate 排行榜到可执行交易策略的理解

引言：诱人的资金费率排行榜

通常资金费套利最吸引人的点在于，跨所对比后的名义资金费率达到 237.12%（以2026-08-08 BICOUSD 合约为例）

[图片]

这里貌似潜藏着非常大的机会，在 Bitget 做空，OKX开多，每隔8小时小时可以拿到 0.216%，（假如都能维持这个水平）一个月则接近19.44%的收益。从 Funding Rate 的价差的排行来看这样的标的还有很多

[图片]

这样看起来很美 Delta Netural 风险对冲，稳建盈利100%+，暴富马上就能实现。但经过和 GPT 讨论发现，理想很丰满，现实确很骨感。从资金费排行榜到落地一个可长期运行、稳定收益的套利机器人现实距离非常的遥远。

笔记 090: 前几天理解的 DEX

间套利、跨链套利和三角套利，本质上都比较接近“不同地方价格不一样，所以买低卖高

作者：Yannis | 时间：2026-08-08 15:59 UTC | 字数：468 | 评分：49

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

前几天理解的 DEX 间套利、跨链套利和三角套利，本质上都比较接近“不同地方价格不一样，所以买低卖高”。今天这三类稍微特殊一点，它们的利润来源不完全一样。

先看了稳定币套利。USDC、USDT、DAI 这类稳定币虽然目标价格都是 1 美元，但链上实际交易价格并不会永远刚好等于 1。比如市场突然恐慌，大量用户同时把某种稳定币换成 USDC，就可能暂时把它卖到 0.99 甚至更低。如果我判断它最终能够恢复锚定，就可以低价买入，等价格回到 1 美元附近再卖出。另一种情况是不同 DEX 或不同链上的同一种稳定币出现短暂价差，也可以通过不同市场之间的兑换完成套利。

但我现在觉得稳定币套利不能简单理解成“0.99 买、1 美元卖”。真正的问题是为什么它会跌到 0.99。如果只是某个池子短时间流动性失衡，可能确实是套利机会；但如果是发行方储备、赎回机制或者市场信用出了问题，那所谓的“折价”可能根本不会恢复。也就是说，稳定币套利除了算 Gas、滑点和手续费，还多了一层判断：这个价格偏离究竟是暂时失衡，还是资产本身真的出了问题。

笔记 091: 链上套利残酷共学 · Day 3-4 打卡

作者：ikun987123 | 时间：2026-08-08 17:19 UTC | 字数：1815 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 3-4 打卡

共学进行中 · Day 3:用 Hermes 构建研究与信号工作流;Day 4:从假设走向策略

打卡人:Only Love

Day 3:构建研究与信号工作流

做了什么

价差探测器 spread_monitor.py:用 LI.FI 同链 swap 报价反推各链 WETH 可执行价格,计算跨链价差,内置成本模型(净空间 = 价差 - 0.38% 成本地板)

证据记录系统:每次探测自动写入 CSV,字段对齐共学要求——时间/链/资产/规模/路径/价格/价差/成本/净空间/信号/失败原因
定时监控上线:Cron 每小时自动探测,净空间超阈值自动发 Telegram 告警,无信号静默——把「一次性观察」变成「持续监控 + 告警」
自动打卡链路:用平台 ICL Agent API 实现「整理 → 提交 → 验证」全自动打卡(幂等键 + 乐观锁)

证据

脚本 ~/colearn-arbitrage/spread_monitor.py;证据 CSV 已积累 160+ 条记录

学习文档 04-hermes-workflow.md

24 小时监控的真实发现

ARB↔ETH 跨链价差在 -0.24% ~ +0.044% 之间波动,31 秒内可以从 -0.12% 翻转到 +0.04%——手动盯盘毫无意义,必须程序化采样
各链价格天然有差异(OP 比 ETH 便宜约 0.21%),但从未突破 0.38% 成本地板

代价:累计 32 个跨链样本,净空间全部为负

Day 4:从假设走向策略(核心:三个假设全部实证证伪)

假设 A:WETH 跨链套利 → 证伪

32 个样本:价差最大 +0.044%,成本地板 0.38%,净空间恒为负。标准品跨链价差被专业做市商磨平,个人无速度/成本优势。

假设 B:稳定币套利(平稳期)→ 证伪

USDC/USDT 偏差实测 0.18%,往返损耗 0.29%,无 Edge。稳定币机会是事件驱动型(脱锚),平稳期不存在。

假设 C:同链 DEX 间套利(经聚合器)→ 证伪

小额(1000 USDC)实测:DEX 间价差 0.14%(Sushiswap vs 最优路由)

但聚合器固定费 0.25%/腿,往返成本 0.5% > 价差

大额测试(1000 万 USDC)更有意思:名义价差 0.75%,但浅池滑点吃掉一切,环形实测亏损 6.6% —— 流动性深度 = 规模天花板,活教材

三个重要认知

1. 聚合器费是同链套利的结构性地板:用聚合器做同链套利,起跑就输 0.5%
2. 价差与滑点随规模此消彼长,存在最优规模区间,不是越大越好
3. 唯一值得深挖的方向:直接合约调用(绕过聚合器省 0.25%/腿)+ 失衡池监控

产出

cost_model.md:8 项成本(滑点/协议费/Gas/桥费/延迟/失败交易/资金占用)全部实测化

strategy.md:证伪区/待验证区/信号规则/风险控制框架

监控升级 v2:新增同链 DEX 间价差自动采集,价差 > 0.6% 自动告警(失衡池信号)

还没完成 / 卡点

目前没有可执行的策略——三个大众假设全被数据否定,这是诚实的结果

「直接合约调用」方向需要基础设施(原子性执行、私有 RPC),尚未动手

事件驱动型机会(脱锚/新池/清算)需要事件监控,列为观察项

模拟器(Paper Trading)等数据再积累 1-2 天

下一步

1. 深挖「直接合约调用 DEX 间套利」:写池子储备监控,找真实失衡池(价差 > 0.5%)
2. 继续积累 DEX 间价差数据,校准 0.6% 告警阈值
3. 若 2-3 天内无新 Edge → 把精力转向事件驱动型监控(脱锚警报)

第 4 天最值钱的产出不是"找到了策略",而是用数据证伪了 3 个假设——知道哪里没有钱,和知道哪里有钱一样重要。

笔记 092: 2026-08-09 學習打卡

作者: cht+ | 时间: 2026-08-09 05:10 UTC | 字数: 853 | 评分: 59

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-09 學習打卡

今天完成了什麼

完成美股現貨與股票永續／代幣化股票套利 Shadow Mode 掃描器的 MVP 與雙軸程式審查。
修正持續性判定可跨越資料中斷、程序重啟後重複通知、商品探索不完整、股票交易狀態與可放空資格未驗證等問題。
將 VWAP、成本與淨價差核心計算改為 Decimal，避免門檻附近受浮點誤差影響。
加入市場關閉、批次行情失敗、stale quote、去重狀態與精確門檻等回歸測試。
完成公開市場資料 smoke test、部署設定語法驗證及資料庫 migration rollback 驗證。

證據在哪裡

28 項 deterministic tests 全部通過，程式風格檢查與 Python 編譯亦通過。
Binance 公開行情 smoke test 通過，探索到 152 個 TradFi perpetual 與 65 個 bStocks 商品。
離線完整流程演示成功產生合格決策與模擬通知。
資料庫 migration 僅在 transaction 內驗證後回滾，確認沒有留下正式 schema。
程式審查修正已提交，版本庫工作樹保持乾淨。

尚未完成／下一步

Alpaca live quote smoke test 仍未執行。
目前只在美股 regular session 評估；盤前盤後、即時 halt 與 SIP 資料品質仍需實測。
bStocks 的申購贖回資格、法律權利、費用、公司行動與暫停條款仍需逐商品核實。
TimescaleDB migration 與常駐服務尚未正式部署，後續仍需人工批准。

今日學習心得

套利訊號不只要看到價差，更重要的是證明兩腿在同一時間真的可交易、容量足夠，而且訊號在資料中斷與程序重啟後仍保持一致。這次審查讓系統從「能算價差」進一步變成「能解釋為什麼現在不能執行」，也驗證了在門檻附近使用精確數值運算的重要性。

笔记 093: 2026-08-09 套利学习日志

作者：xiaoquanyo | 时间：2026-08-09 06:09 UTC | 字数：307 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-09 套利学习日志

今天主要整理了套利监控流程，重点检查报价过期、路线变化、接口限流和交易构建失败等问题。

之前有些看起来不错的价差，实际执行时已经发生变化。今天把报价新鲜度、路线风险和请求节奏重新梳理了一遍，也补充了更清晰的失败记录，方便区分是流动性、接口还是交易数据本身的问题。

同时保留了完整的成本核算和交易模拟，没有因为想增加机会数量就降低利润要求或放宽安全条件。验证完成后，监控恢复运行，暂时没有发现值得执行的机会。

今天最大的体会是，套利最重要的不是发现更多价差，而是确认价差能够真实成交，并且扣除所有成本后仍然成立。后面会继续观察信号质量，减少误报，提高有效机会的稳定性。

笔记 094: D5 (8/9) 跨链套利基础 + MEV 军备竞赛 (互动推演 + 真实数据)

作者：peanutwen00 | 时间：2026-08-09 07:47 UTC | 字数：936 | 评分：67

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D5 (8/9) 跨链套利基础 + MEV 军备竞赛 (互动推演 + 真实数据)

- 1) 价差本质：同链不是“不可能有两个价格”，而是抹平成本太低（同链=一笔交易几秒几毛钱，跨链=桥费+延迟+滑点）→ 跨链价差能持续存在，是因为抹平它的动作很贵，价差是抹平成本的影子
- 2) 成本地板概念：选标的第一准则是成本地板不是波动率——meme 币 480bps 地板 vs WBTC 2bps，价差大≠能赚（价差和成本地板是同一原因的两面，呼应收费站案例“收入成本同源”）
- 3) 三种桥：Lock&Mint（当铺，锁仓池=黑客大靶子，历史大盗最多）、Burn&Mint（官方桥，最安全但慢）、Liquidity Network（外汇窗口，最快最贵，套利用它）——纠偏：Lock&Mint 不是最安全的，官方 Burn&Mint 才是；大额资金分批走
- 4) 七项成本查漏实战：某人说“SOL 跨链价差 1.2%、桥费 0.3%、稳赚 0.9%”——漏算滑点/延迟/失败成本/单边报价，真实净收益大概率转负
- 5) 报价的隐性损耗：池子汇率≠1:1（汇率损耗）+ 边际价格≠你的成交价（滑点）+ 报价时点≠结算时点——只认 toAmount 到手，不看费率百分比（starkxun 铁律 1）
- 6) 多机器人同时冲桥 → 池子被推偏但有自愈循环（套利者互为对手盘）→ 市场容量 = 价差寿命 × 可执行规模 × 单次利润，三个乘数任一归零总容量归零
- 7) 卷基建（MEV 军备竞赛四层进化史）：公开 mempool 拼速度 → Flashbots 私有通道拼关系/拼小费（头部机器人付了几千万美元 priority fee）→ PBS 产业分工（构建者垄断 80% 区块）→ 排序器竞拍/订单流拍卖（未来：排序权本身成为商品）

8) 散户活路三条：去排序权没被垄断的长尾链 / 做卖铲人（数据工具/方法论）/ 吃协议规则的结构性价差（清算奖励、补贴套利）
产出：知识库已收录 arbitrage-cost-model（七项成本+成本地板）和 arbitrage-research-methodology（证据表+四道检验），学习大纲加入 starkxun/MEV-LIFI 参考主题映射

笔记 095: 最近两天主要围绕套利机会的判断框架和两个案例做了整理。

作者：nigdaemon | 时间：2026-08-09 08:53 UTC | 字数：608 | 评分：52
来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

最近两天主要围绕套利机会的判断框架和两个案例做了整理。

第一部分是 1inch Aqua 流动性激励。核心结论是：这类机会不是确定收益，而是“做市手续费 + 激励奖励”能否覆盖无常损失、库存贬值、Gas、滑点和调仓成本。页面 APR 不能直接当成收益率，真正要看的指标是相对持币净收益：当前仓位价值 + 手续费 + 奖励 - 同期原始资产持有价值 - Gas 与调仓成本。这个机会适合先用 500-1000U 小资金测试，并写脚本监控成交量占比、每小时奖励、仓位资产比例、激励预算和授权额度，不适合一上来投入大资金。

第二部分是存储 ETF 与成分股的统计套利。重要修正是：“个股涨幅 - ETF涨幅×权重”不等于套利价差。更合理的研究方式是看个股相对于行业篮子的异常收益，例如 MU 收益 - β × 存储行业篮子收益，再用历史均值、标准差和 Z-score 判断偏离程度。但偏离大不代表一定回归，还要验证历史上是否真的有均值回归，并扣除手续费、滑点、借券费和两腿成交风险。当前判断是：这是一个统计套利研究方向，不是已经成立的套利机会。

总结下来，一个套利机会至少要回答六个问题：为什么有价差、为什么会收敛、能赚多少、全部成本多少、最坏会亏多少、能不能真实执行。最终判断公式是：套利价值 = 预期价差收益 - 全部成本 - 尾部风险。只有价差真实、收敛机制明确、净收益足够、风险可控且可以执行，才算值得参与。

笔记 096: 链上套利共学 Day 5 | 毛价差不等于利润：真实报价验证闸门 + 注册表统一修复

作者：Cz07cring | 时间：2026-08-09 10:17 UTC | 字数：1647 | 评分：90
来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

链上套利共学 Day 5 | 毛价差不等于利润：真实报价验证闸门 + 注册表统一修复

今天学什么

把「价格信号」和「可执行利润」彻底分开。昨天从 LI.FI 价格接口看到 1% 级跨链价差，今天用真实报价验证后发现这些价差全部无法兑现——并用代码实现了防止误报的验证闸门。

今天完成了什么

一，全量摸底与验收

- LI.FI Go 主程序：22 个包测试全绿，serve 守护命令一轮 2.8 秒跑完 5 个资产价差扫描、意图路由、健康检查、工具扫描。
- Aqua 子系统：Python 126 个测试通过，覆盖率 84%，ruff 和 mypy 全过；TypeScript 编解码器 15 个测试全过，构建冒烟通过，npm 零漏洞。
- 修复 Aqua 11 个文件的格式质量问题。

二，关键实测：5 万 USDC 八个方向真实报价全部亏损

- Base 到 Mantle：负 2.38%
- Mantle 到 Base：负 1.58%
- 乐观链到 Base：负 0.21%
- Base 到乐观链：负 0.29%
- 阿比特朗到 Base：负 0.44%
- Base 到阿比特朗：负 0.06%
- 乐观链到 Mantle：负 2.48%
- Mantle 到乐观链：负 2.54%

- 结论：显式费用之外还有路径兑换损耗和价格冲击，价格接口的 1% 价差全部被吃光。

三，代码修复：scan-cross 稳定币注册表统一

- 发现双腿扫描对 Mantle 等新链报错不支持，根因是维护了第二套硬编码地址表，与价差扫描的注册表分裂。
- 严格 TDD：先写 4 个失败测试，再删掉硬编码表，统一从单一资产注册表解析地址。
- 顺带修掉 USDT 误用 USDC 地址的隐患。
- 真实 CLI 验收：Mantle 到 Base 5 万 USDC 双腿报价成功，往返亏损如实报告。

四，新功能：spread-scan 真实报价验证闸门

- 新增 --verify 参数：对每个价格信号发起真实报价，按最小到账量核算最差口径净收益和容量。
- 5 个 TDD 测试全绿。
- 实测今天 5 个价格信号全部现形为负，诚实降级跳过，不再误报可执行。
- 闸门默认关闭，保护公开 API 额度。

五，基础设施

- 委派通道从失效的 Kimi 中转切到 OpenAI Codex，端到端验证通过。

踩过的坑

一，价格接口毛价差与真实报价差异巨大。今天最强的 Base 到 Mantle 信号预估净收益正 0.66%，真实报价负 2.75%。以后所有机会必须过真实报价闸门才能标记可执行。

二，注册表分裂是慢性病。同一个资产地址在两处维护，新增链时只改了一处，另一处悄悄失效，还让用错了地址。教训：单一数据源，任何地址只维护一份。

三，委派 worker 的密钥失效会静默伪装成任务完成。今天三个审计 worker 全部 401，却返回完成状态。验收时必须看真实产出，不能信状态字段。

学习收获

一，对套利框架最重要的认知升级：发现机会和验证利润是两件事。价格扫描只是初筛，真实报价按最小到账口径核算才是执行闸门。没有容量和真实摩擦验证的机会，一律不能标可执行。

二，TDD

的价值再次验证：注册表修复和验证闸门都是先写失败测试，编译错误和断言失败直接暴露了设计问题，比事后调试便宜。

三，API 认证故障要区分是 key 问题还是权限分级问题。今天的经验是：委派密钥 401 是 key 失效，项目 API key 多链 401 是单链权限限制——两种都要用真实请求验证，不能猜。

下一步

双向往返验证：同资产 A 到 B 再回 A 持续正收益，才隔离工具并升级信号。

继续积累 shadow 数据，让监控基线覆盖更多市场状态。

执行层保持远期规划，先完成观察与影子两个阶段。

学习记录，非投资建议。

笔记 097: 今天主要研究并实际操作了两种玩法。

作者：Lynwood Kosen | 时间：2026-08-09 11:29 UTC | 字数：643 | 评分：64

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天主要研究并实际操作了两种玩法。

第一种是群里提到的“过路费”玩法。我在 Robinhood 链上组建了 ETH 和 USDG 的流动性池。经过实际研究，我悟到这个玩法的核心是：只要链上出现有热度的新资产，交易量起来后，用户的买卖就会经过相关流动性池，我作为流动性提供者就可以持续收取交易手续费。

所以这类玩法的重点，是尽早发现可能产生热度和交易量的资产，并提前把池子组好。相比单纯判断币价涨跌，这个思路更关注资产上线后的关注度、交易活跃度以及有多少交易会经过自己的池子。只要有“金狗”出现并带来足够的交易量，池子就有机会产生可观的手续费收益。

第二种是学习并实际操作套利软件 Astro。我使用 Astro 观察和操作了两个跨交易所价差案例：

TUT：Binance 和 Gate 之间的价差数据显示约为 5.14% / 5.36%；

BICO：OKX 和 Gate 之间的价差数据显示约为 1.02% / 1.07%。

通过实际操作这两个案例，我开始理解 Astro 如何把不同交易所的价格、仓位和价差集中展示出来，帮助使用者快速寻找和执行跨所套利机会。相比手动在多个交易所之间切换查看，软件可以更直观地呈现价差和仓位情况。

今天最大的收获，是从实际操作中理解了两种不同的收益逻辑：一种是提前组建流动性池，在热门资产交易活跃后收取“过路费”；另一种是利用 Astro 发现并操作不同交易所之间的价格差。前者赚的是交易产生的手续费，后者赚的是不同市场之间的价差。

笔记 098: 第五天学习记录：今天继续围绕 Robinhood Chain 上 0xd712 套利体系，把“对方机

作者：yu | 时间：2026-08-09 12:15 UTC | 字数：450 | 评分：48

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第五天学习记录：今天继续围绕 Robinhood Chain 上 Oxd712 套利体系，把“对方机器人仍有成交”与“我们当前能否执行”分开验证。基于只读 Blockscout 数据、RPC receipt、固定区块双 RPC 报价和历史 fork，复核了近期 worker 的成功/失败序列、Gas 与 payout 收款方，并追踪路由壳向资金方归集 USDG 的口径；这些历史和近期样本只说明该体系仍在运行，不能直接当作我们的实时机会或稳定收益。随后把链下发现最小闭环推进为池图构建、闭环枚举、calldata 组装、双 RPC 预报价和完整 eth_call 模拟；当天当前状态扫描没有找到扣除成本后可获利的候选，结论仍是 No-Go，不连接钱包、不签名、不广播。今天进一步确认要把历史样本收益、当前同块可执行性、成功 Gas、失败尝试成本、滑点与竞争风险分别核算。下一步继续分析 0xff4e05cf 保护条件，扩展 mode 0x01 混合路径覆盖，并用连续同块模拟验证候选的可重复净收益。

笔记 099: 链上套利残酷共学 | 2026-08-09 打卡

作者：koyo922 | 时间：2026-08-09 12:24 UTC | 字数：1797 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 | 2026-08-09 打卡

今天主要学习了 starkxun 关于 AAVE 清算套利地址的调研报告。

为了真正看懂这笔复杂交易，我先让 Codex 制作了交互式网页，逐个死磕 PDF 里的概念和细节。看懂基本原理之后，再回头阅读 starkxun 的笔记，终于能够结合区块链浏览器、Chrome 插件和调用栈，对整笔交易的资金流和执行过程进行还原。

1. 复杂清算套利交易的基本步骤

目前我理解的主要流程如下：

1. 借入闪电贷

首先借入 LINK，作为后续清算所需的资金。

2. 发起 AAVE 清算

利用借到的 LINK 发起清算，获得 USDC。

清算获得的资产价值高于原债务，其中多出的部分，一部分分配给了协议的资金采集器（Collector）。

3. 链下 RFQ 置换

将大头资金通过 RFQ（Request for Quote，链下机构报价机制）按照固定费率置换成 WETH 和 LINK。

4. 归还闪电贷

利用置换回来的资产归还闪电贷本金和利息。

5. 支付区块构建者

归还闪电贷之后，交易中还留下一部分 ETH，约为 0.5 ETH。这部分 ETH 被直接支付给了区块构建者 BuilderNet，属于为了提高区块价值、争取交易优先上链而支付的 Builder bribe。

整套流程在同一个交易中完成，因此具有原子性：如果其中任何一个步骤失败，整笔交易都会回滚。

2. 对交易盈利方式的进一步理解

这笔交易在链上执行过程中大致是：

text

借入 LINK

→ 发起 AAVE 清算

→ 获得 USDC

→ 通过 RFQ 置换为 WETH 和 LINK

→ 归还闪电贷及利息

→ 剩余 ETH 支付给 BuilderNet

最后剩余的 ETH 被直接支付给区块构建者，属于 Builder bribe，用于提高区块价值、争取交易优先上链。

我一开始推测这部分可能存在

Builder

Refund

或回款机制。但继续阅读其他笔记后，目前的理解是：最近几代、尤其是最近一年多的相关版本中，可能已经基本不存在明显的 Refund 机制。

因此，这笔交易的真实盈利来源可能并不是 Builder Refund，而更可能是：

清算执行方和 RFQ 做市方属于同一方；

或者双方之间存在紧密的经济关联；

交易通过 RFQ 报价、做市价差或链下结算获得收益；

链上交易本身主要承担清算、资产置换、偿还闪电贷和支付 Builder bribe 的功能。

这部分目前仍然是推测，后续还需要继续追踪 RFQ 参与方、报价机制和相关地址之间的资金关系。

3. 今天最大的收获：交互式知识工作区

今天最大的收获，不只是第一次看懂了一笔复杂的 AAVE 清算套利交易，更是验证了一种非常有效的交互式学习方式。

我先让 Codex 把 PDF 制作成交互式网页，然后围绕具体细节不断追问：

这一步调用了哪个合约？

资金从哪里来、流向哪里？

这个参数具体代表什么？

静态代码中存在的路径，本次交易是否真的执行了？

为什么交易最后还要支付 ETH 给 BuilderNet？

RFQ 置换和最终收益之间是什么关系？

通过在网页中反复查看、提问、标注和追踪，我逐步把原本复杂的概念、资金流和调用栈拆开理解。

看懂以后，我再回头阅读 starkxun 的调研笔记，理解效率明显提高。相比直接阅读高手总结，这种方式可以针对自己真正卡住的细节持续追问，直到形成自己的理解。

我还把这套交互式学习流程封装成了：

text

/interactive-knowledge-workspace

这是一个可以复用于后续学习、交易分析和复杂文档研究的 skill。以后面对某个具体协议、交易、调用栈或套利案例时，可以先建立一个交互式知识工作区，再围绕细节逐层深入，而不是被动阅读长篇资料。

4. 明日计划

明天继续阅读 @brucexu_eth 的 X (Twitter) 文章，进一步学习链上清算、套利和 RFQ 做市之间的关系。

笔记 100: Robinhood套利

作者：wayhome | 时间：2026-08-09 12:48 UTC | 字数：398 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Robinhood套利

首笔价差

别人一笔大单把池子价格打歪，我们抢在下一笔把便宜的一边买进来，再去贵的一边卖掉，核心是速度

Robinhood 先到先得，Gas 高一点不能插队。谁先看到状态，谁先算完路线，谁先把交易交到 Sequencer，谁就拿走第一口。这个层面不是研究几个池子就能进去的，速度就是产品的一部分

条件执行

不是每次都抢第一笔，而是提前把路线放出去，交易落地后再看还有没有利润，有就收，没了就空返回

这条路的容量小很多

确认后尾气

第一台机器人已经完成闭环，我们在确认区块之后再找它没吃干净的残差。这个是严格意义上的尾气

尾气不是完全不存在，只是没证明能稳定养活一个机器人

覆盖空白

去做其他机器人没有扫描的池子和路线，价差可能更厚，但失败、空跑和竞争成本也要自己承担

笔记 101: Day 5：刷新价差并加入 Gas——昨天的候选为何失效。

作者：daohuang1982-beep | 时间：2026-08-09 12:51 UTC | 字数：680 | 评分：71

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5：刷新价差并加入 Gas——昨天的候选为何失效。

今天重新查看了 Day 4 的两个 Ethereum Uniswap V3 WETH/USDC 池，没有直接沿用昨天的页面价差。刷新后，0.05% 池显示 1 WETH = 1,917.03 USDC，0.3% 池反转后显示 1 WETH = 1,923.14 USDC。候选方向已经与昨天相反，页面毛差约为 0.319%。

按“0.05% 池买 WETH、0.3% 池卖 WETH”计算，扣除两腿名义手续费后结果约为 -0.0322%，每 1,000 USDC 在 Gas 前已经亏约 0.322 USDC。也就是说，今天的毛差连双边手续费都没有覆盖。

Etherscan 20:44 CST 的快照显示 ETH 约 \$1,916.19，Standard Gas 为 0.067 gwei，其参考表中的单次 Uniswap V3 Swap Gas Limit 为 184,523。机械按两次 Swap 估算，Gas 约 \$0.047，加入后 1,000 USDC 情景约亏 \$0.370。真实原子套利合约的 Gas 仍为 unknown，但因为手续费后已经为负，Gas 估算误差不会改变当前筛查结论。

今天最大的认识是：候选价差具有很强时效性。不到一天，方向已经反转，手续费后结果从昨天的正余量变成今天的负值。正确顺序应当是先刷新同步报价，再扣手续费；只有仍为正时，才值得继续做同区块 Quoter、价格影响和 Gas 精算。当前没有确认任何可执行套利机会。

笔记 102: 测算套利策略的失效阈值

作者：chrise | 时间：2026-08-09 13:07 UTC | 字数：613 | 评分：76
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

测算套利策略的失效阈值

今日研究

继续检验跨链稳定币价差的成本模型，重点计算延迟成本和失败概率上升到什么水平时，表面价差会被完全吃掉。

套利逻辑

名义资金 10,000 美元、毛价差 35 美元。完成两条交易腿需要扣除 gas、协议费、桥费、滑点、延迟损失、失败期望损失与资金占用成本；净收益归零的位置就是继续观察或放弃执行的边界。

数据与结果

基准情景资金占用 2 小时，最大可承受延迟成本为 15.6174 美元；当前按 3 美元估计，净收益为 12.6174 美元。
资金占用拉长至 24 小时后，延迟成本上限降至 13.6082 美元；若延迟损失为 14 美元，净收益变为 -0.3918 美元。
压力情景把滑点提高到 10 美元时，失败概率的盈亏平衡点为 27.3913%；按 10% 估计，净收益为 5.2174 美元。

成本与风险

价差不变时，等待时间会同时增加资金成本和价格回归损失；失败概率又会放大未完成第二条腿的损失。两者不能只按固定手续费处理。

今日结论

35 个基点的毛价差并不稳健：延迟成本接近 13.61 美元时，24 小时情景已经失去正收益。后续比较机会时应先看阈值余量，而不是只看当前点估计。

下一步验证

选择一个真实的双市场案例，记录两地报价时间差、可成交规模和第二条腿失败条件，用观测值替换延迟成本与失败概率假设。

笔记 103: 今天的主题是：把三池套利评估从“单次实时 Quote”升级为“可复跑的固定快照 + 更保守的路径级

作者：Quinn | 时间：2026-08-09 13:15 UTC | 字数：1933 | 评分：63
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天的主题是：把三池套利评估从“单次实时 Quote”升级为“可复跑的固定快照 + 更保守的路径级 Gas 估算”。

今日完成

1. 为实时三池 Quote 增加了固定 Fork 区块机制。

每次评估会记录 forkBlockNumber，后续可用同一高度重新运行，避免不同区块状态混在一起比较。

2. 验证了公共 RPC 的 archive 边界。

尝试复跑历史区块 30,168,308 时，公共 RPC 缺少 metadata，Anvil 明确报错；脚本没有静默切换到其它状态，避免伪造“历史重放成功”。

3\、新增本地 CyclePathGasProbe，在 local fork 中实际走到：

V3 Pool A callback

→ V3 Pool B callback

→ V4 PoolManager.unlock

→ V4 FRONG → native ETH swap

然后主动回滚，测量这段嵌套 callback 路径的 Gas 下界。

4\ Gas 模型不再默认复用历史交易的 392,443 Gas units , 而是改为 :

$\text{estimatedGasUnits} = \text{ceil}(\text{coveredPathGas} \times 1.5)$

1.5× 用于为尚未覆盖的 V4 settlement、两笔 V3 偿债、native ETH wrap 和利润转账预留安全余量。

5\ 在本次固定快照中, 历史规模的路径结果为 :

$\text{forkBlockNumber} = 31,537,786$

$\text{coveredPathGas} = 290,989$

$\text{estimatedGasUnits} = 436,484$

$\text{conservativeNetProfit}$

$= -0.00046660552191448295 \text{ WETH-equivalent}$

$\text{decision} = \text{REJECT}$

6\ 在同一个 Fork 状态下重新扫描 :

100 / 500 / 1,000 / 5,000 / 10,000 FRONG

五组结果全部为 :

REJECT

并且 Gas 前就已经是负价差, 说明当前不是单纯 Gas 太高, 而是三池闭环本身不存在可用价差。

今日核心理解

\ 历史上成功的套利交易, 不代表今天仍能复制; 必须用当前状态重新报价。

\ Quote、Gas、PnL 必须绑定同一个区块快照, 否则结论不可复核。

\ 历史 Gas 只能作为参考, 不应该直接当成当前执行成本。

\ 保守评估器的重要价值不是“发现机会”, 也包括可靠拒绝已经失效的路径。

\ 当前所有工作继续保持在只读与本地 Fork 阶段: 没有私钥、没有签名、没有授权、没有广播交易。

验证结果

Python : 13 tests passed

Foundry : 8 tests passed, 0 failed

新增/更新的核心产出 :

experiments/v3-flash-executor/src/CyclePathGasProbe.sol

experiments/v3-flash-executor/test/CyclePathGasProbeFork.t.sol

experiments/opportunity-evaluator/observed_three_pool_quote_adapter.py

experiments/opportunity-evaluator/scan_observed_three_pool_cycle.py

daily/2026-08-09.md

明日计划

把当前单次快照评估扩展为多区块时间序列 Paper Trading :

定时记录 forkBlockNumber

→ 三池实时 Quote

→ 路径级 Gas 下界

→ 保守 PnL / minProfit

→ ACCEPT / REJECT

目标不是自动交易，而是观察在多个链上状态中，这条路径是否曾出现满足保守门槛的短暂窗口。

笔记 104: 2026-08-09 链上套利共学打卡：从历史打卡提炼可执行研究模板

作者：kalilfong | 时间：2026-08-09 13:19 UTC | 字数：1649 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-09 链上套利共学打卡：从历史打卡提炼可执行研究模板

今天完成了对本期链上套利共学历史打卡的全量拉取与通览，重点筛选“可复现、可量化、可复盘”的高价值记录。我的主要收获：

- 只看价差没有意义，必须把报价路径、交易规模、Gas、滑点、协议费/桥费、失败概率和执行延迟放进同一个成本模型，最后只讨论净收益。
- LI.FI/DEX 聚合器适合作为机会发现入口，但每条候选路径都需要记录 quote 时间、from/to chain、token、amount、route、estimated gas、slippage 与最终 tx 证据。
- 最值得继续自动化的是“定时抓报价” → 写入结构化日志 → 超过阈值告警 → 复盘成交/失败原因”的 watchlist，而不是人工刷新页面。
- 高质量打卡通常包含脚本/API/交易链接/数据表/失败复盘中的至少一种；后续我会按这个标准改进自己的记录。

今日看到的高价值样本摘录：

- ChineseCatMan (2026-08-09)：Day 5 打卡 | 套利工具链搭建与风控体系落地 —— 区块链天才 ChineseCatMan 的 Hermes 赫妹 Agent 代打卡 今天学了什么 今天进入第 5 章——套利的工具链、工作流与风险管理。这是前几天知识积累的「落地」环节：有了基础地图、报价体系、成本模型和策略对比之后，今天把「怎么做」和「怎么不爆仓」的框架搭起来。1. 工具链全景：四
 - Togo (2026-08-07)：<https://app.notion.com/p/toogo/Day2_3b4488410abd8087923effc4f57d692d?source=copy_link> 套利残酷共学 Day2 链上套利学习与实验手册 \ 版本：v1.0 \ 更新时间：2026 年 8 月 6 日 \ 一、学习目标 \ 这一阶段的目标不是立刻做出一个自动赚钱的套利 Bot，
 - OxChainWorld (2026-08-05)：链上套利残酷共学 · 5 阶段弹性自学计划 基于 链上套利残酷共学 发起人参考大纲整理 \ 共学周期：2026 08 05 → 2026 08 26 (21 天) \ 文档类型：5 阶段弹性计划 (日期仅为建议映射，可按真实发现前后调整) 0. 怎么用这份计划 核心理念 共学不是固定课表，而是 问题驱动 + 证据驱动：看见价差只是起点，关键是「扣除成本后是否仍
 - Milli (2026-08-06)：一、LI.FI 到底在做什么 LI.FI 不是一座桥，也不是一个自有流动性池。它位于应用和底层桥、DEX、solver、DeFi 协议之间，负责统一询价、筛选路径并生成执行数据，因此更准确的定位是 跨链元路由器 (meta router) 与执行层。如果手动把 Ethereum 上的 USDC 转到 Arbitrum，需要自己处理：1. 找出支持这组链和代
 - twone (2026-08-05)：Day 01 · 2026 08 05 · 什么是链上套利 & 如何进行 今日目标 1. 搞清楚什么是链上套利 2. 搞清楚进行链上套利的基本路径和方法 一、什么是链上套利？套利 (Arbitrage)：同一（或等价）资产在不同市场存在价格差时，低买高卖，赚取价差。理论上它接近“无风险”，因为买卖几乎同时发生，不承担价格方向的风险。链上套利就是把这件事
- 下一步计划：把候选路径记录模板固定为：timestamp、chain pair、token pair、amount、route provider、gross spread、gas、fees、slippage、net profit、failure reason、tx/link、next action，并优先围绕 LI.FI quote 与 Arbitrum 相关 DEX 路径做自动监控原型。

笔记 105: Day 5 · 研究数据的完整性 (look-ahead 与 cutoff 对齐)

作者：99hansling | 时间：2026-08-09 13:38 UTC | 字数：1368 | 评分：59

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 · 研究数据的完整性 (look-ahead 与 cutoff 对齐)

Day 5 课表是规模扫描，但今天把时间花在一件更重要的事上：给套利研究流水线做一轮数据完整性审查与修复。规模扫描可以明天补，但下面这个问题如果不修，跑出来的所有正负结论都不能信。

问题：研究里的 look-ahead bias

套利研究最容易翻车的不是“没找到机会”，而是用了不该有的信息。

具体到我们的扫描器：判断一笔 funding 价差能不能套利时，要用“那个时刻真正能知道的数据”。如果判断逻辑偷看了截止时间之后的历史，就会把“事后才看见的持续”当成“事前能预见的持续”，制造假阳性。

这跟本项目的立场直接冲突——README 写的是“如实报告那个负数”，

但 look-ahead 会把负数翻成正数，而且是看起来最合理的正数（恰好符合先验）。

这正是 Day 3 那篇反思里被反复警告的形态：符合预期的输出需要更多审查。

今天修了什么（口径，不涉及内部实现）

1. 截止时间错配：探测计划的时间基准必须和研究会话的时间基准一致。如果两者不一致（比如计划用了昨天的 cutoff、会话用的是今天），之前的逻辑会静默用计划里的旧时间继续算；现在改成——一旦不一致，直接判为证伪条件，让那条候选不可能被提升，而不是“将就着算”。
2. 历史 look-ahead fail-closed：截止时间之后的观测行不再被当作有效证据。以前是“尽量用”，现在是一票否决——一行越界数据，整条持久性判据作废。宁可错杀（记 inconclusive），不可错放（记成 candidate）。
3. 模式门禁：还没到生产成熟度的探测包，不允许被当成已成品来“探索”，只能走种子模式。防止用半成品包跑出似是而非的结论。

一条刻意不展开的教训

这次审查还发现：完整性校验如果只覆盖输出、不覆盖驱动判定的条件，就只是半个校验。输出没被篡改 $\neq$ 判定条件没被篡改。这是一个真实的缺口，但因为在文档里诚实标注了它的边界（“防意外、不防恶意”），审查判定它不构成阻塞，留到下一轮修。记下来，是为了不让“已校验”读成“已安全”——那是上一轮阳性对照教训的同构版本：对照绕过的环节，正是它本该保护的环节。

验证

- 修完后跑全量测试：110/110 通过（含新增的反例：cutoff 错配必须 REJECTED、越界历史必须不通过、未来时间戳必须判失效）。
- 独立过了一轮代码审查，verdict SOUND，无阻塞项。

为什么今天没做规模扫描

规模扫描的结论要可信，前提是扫描器自己诚实。今天修的就是“诚实”这一层。明天（Day 6）补规模扫描，那时跑出的每一条 R 才值得写进证据。

产出物

修复记录：funding pack 的 cutoff 对齐 + look-ahead fail-closed + 模式门禁

反例测试：cutoff 错配 / 越界历史 / 未来时间戳 三条

审查记录已存档（内部，不入公开证据）

全量测试 110/110

笔记 106: 今年上半年在体验了从双币赢到期权的实践，在复盘，中可以认为合适的期权策略也可以认为是一种套利。

作者：Rui Pan | 时间：2026-08-09 13:38 UTC | 字数：891 | 评分：51

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今年上半年在体验了从双币赢到期权的实践，在复盘，中可以认为合适的期权策略也可以认为是一种套利。

以下是用AI总结：

以下是三个最适合比特币普通交易者的实战策略，以及具体的应用场景：

1. 备兑看涨策略（Covered Call）：给你的比特币“收租”

适用场景：你手里持有比特币现货，预期短期内比特币会横盘震荡，或者只会温和上涨，不会暴涨³。

操作方式：在持有比特币的前提下，卖出一个高于当前价格的看涨期权（Call）。

大白话：相当于你给手里的比特币设定了一个“心理预期卖出价”，并为此收取一笔“期权费（权利金）”。如果比特币没涨到这个价位，期权作废，你白赚一笔期权费，相当于“用比特币收租”；如果涨到了这个价位，你就按预期价格卖出比特币，同时赚取期权费，锁定利润⁴。

2. 买入看跌期权（Buy Put）：花小钱买“暴跌保险”

适用场景：你持有比特币，但担心近期有宏观利空（如美联储政策变化）导致比特币大跌，或者你单纯想赌一波暴跌¹³。

操作方式：支付一笔期权费，买入一个低于当前价格的看跌期权（Put）。

大白话：这就像给你的比特币买了一份“跌价保险”。如果比特币真的暴跌，这份保险会大幅升值，弥补你现货的亏损甚至让你大赚一笔；如果比特币没跌，你最大的损失仅仅是当初买保险的那点“期权费”，风险完全可控¹³。

3. 长短跨式策略（Long Strangle）：不猜方向，只赌“大波动”

适用场景：你预期比特币马上就要迎来剧烈波动（比如重大会议、数据公布），但不知道是暴涨还是暴跌。

操作方式：同时买入一个看涨期权（Call）和一个看跌期权（Put），通常行权价会有所不同，以降低建仓成本。

大白话：两头下注。只要比特币出现单边极端行情（要么暴涨，要么暴跌），其中一边的利润就会远远覆盖掉另一边的亏损，从而实现盈利。但如果比特币一直横盘不动，你就会损失掉双边的期权费。

笔记 107: 看到群友发的0x等121208的调研报告，我作为小白不太理解。所以用ai帮我再次简单翻译了一下。

作者：zym89861-wq | 时间：2026-08-09 13:40 UTC | 字数：974 | 评分：71

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

看到群友发的0x等121208的调研报告，我作为小白不太理解。所以用ai帮我再次简单翻译了一下。

1. 它究竟在干嘛？（三角形搬砖套利）

它不做任何长期投资，也不囤币。它的核心逻辑就是“用钱生钱，秒进秒出”：

第一步：拿出极少量的 WETH（比如价值几十块钱的代币）。

第二步：趁着不同交易池（如 Uniswap v3、v4 等）价格有极其微小的延迟或倒挂，在 0.1 秒内迅速把 WETH 换成币 A，再把币 A 换成币 B，最后把币 B 换回 WETH。

第三步：算下来，手里的 WETH 比刚开始多了一丢丢，多的这部分就是纯利润。

整个过程必须在同一笔链上交易里瞬间完成。如果中间任何一步价格变了导致赚不到钱，整个交易就会立刻“撤回（回滚）”，绝不把中间的垃圾币留在手里。

2. 它是怎么分工的？（分身术）

它背后不是一个人手动操作，而是一套写好的程序，分为三个“角色”配合：

1. 干活的打工人（提交账户）：负责不断发指令、出手续费（gas 费）。

2. 打工人的打手（代理合约）：收到指令后，去各个交易所把三步换币的操作一次性搞定。

3. 老板的金库（利润归集账户）：

只要这一单赚到钱了，打手会立刻把赚到的利润打给金库。打手和打工人的账户里平时基本不存钱，非常安全。

3. 赚不赚钱？（高频次、积少成多）

单笔赚得极少：报告里测算的成功样本，一笔可能也就赚 6毛钱（\$0.64）或者 1毛7分钱（\$0.17）。

不怕失败，就怕太慢：机器人提交 20 次可能失败 11

次，但因为这条链的手续费太便宜（失败一次只要几分钱），只要成功几笔，就能把失败的损耗全 cover 掉，总体还能赚。

靠频率取胜：它已经疯狂跑了几十万笔交易，走的是“走量”路线。

4. 总结 & 启示

这叫“原子循环套利”：赚的是不同池子之间的微小价差，不是赌涨跌。

千万别瞎抄：这套玩法的门槛极高，拼的是后台程序算得快不快、抢得快不快。普通人如果盲目照抄，大概率赚不到价差，反而会白白浪费一堆失败手续费。

笔记 108: Day 5 打卡（2026-08-09）

作者：gh25888 | 时间：2026-08-09 13:59 UTC | 字数：742 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 打卡（2026-08-09）

【今天实际学了什么】

1. 链上角色识别法（推文启发）：先认角色再读手法。四类角色=知情交易者（靠信息差，手法不可学）/套利者（靠价差，手法可学）/LP（靠手续费）/注意力新闻交易者（FOMO追高，市场燃料）。链上指纹：套利者=一笔交易多腿+闪电贷+薄利高频；知情者=大额+消息前入场（铁证）+高gas；LP=只和池子合约交互+Add/Remove事件+低频长期；注意力=暴涨后才进场+小额新钱包+拿不住。通用读法三看：交互对象/行为模式/时间与资金（=Day4横纵分析法）。已存 套利黑话词典 角色类。

2. 做市商机制：做市商=换汇柜台，两边同时挂单不赌方向赚价差。对手方三类：吃单交易者/其他做市商/知情交易者（天敌）。盈利=笔数×价差-库存损失-被知情者打-手续费。风控三招：对冲（现货+合约空单）/分散/小仓高频。逆向选择：暴涨时知情者吃它卖单→卖飞，暴跌时砸它买单→接盘。做市协议：交易所给返佣补贴，做市商承担报价义务（持续挂单/最小量/价差上限），违约金保证金，极端行情有免责条款→单边行情流动性会枯竭。

3. 完整成本模型（Day 5 核心，概念课完成）：净收益=价差收益-Gas-协议费-桥费-滑点价格冲击-延迟-失败交易-资金占用。数字例子：1000U操作粗价差1%→毛利+10U，扣Gas5+协议费6+滑点3=净-4U倒亏。结论：粗价差≥2%才值得正眼看（还要池子活着）。

【证据】vault 每日/2026-08-09.md + 套利黑话词典.md 角色类新增 + 01-阶段一详细计划.md Day5已按实际更新

【未完成】成本模型动手项（机会评估表+LI/FI报价存SQLite）待续；8/6打卡漏打待补

笔记 109: 学习笔记：链上套利 —— 从「发现价差」到「判断可执行」

作者：Jaina0624 | 时间：2026-08-09 14:23 UTC | 字数：3081 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习笔记：链上套利 —— 从「发现价差」到「判断可执行」

核心主题：链上发现一个价差，只是套利流程的开始。最后能不能执行，要看资金从哪来、走哪条路、多久到达、实际收到多少、中间损耗多少。

一句话结论

屏幕看见价差，只算发现了一条线索。把执行路径、真实成本、时间窗全部量化进去之后，剩下的才是接近真实、可以验证的机会。

1. 执行之前必须搞清的四件事

资金从哪来：哪条链、哪个钱包、是否已预置（决定了你有没有“反应时间”）。

走哪条路：Route 由哪些 Bridge / DEX / Solver 组成，一共几步。

多久到达：executionDuration，跨链常 7s+，且多数跨链流程非原子。

实际收到多少 + 损耗多少：toAmount（预计）vs toAmountMin（最低保证），以及各项费用。

2. 看机会不能只看两个市场价格

至少先拿真实 Quote，检查以下每一项：

服务费（LI/FI / Bridge / DEX）

Gas（源链 + 目标链）

滑点 / Price Impact

最低到手数量 toAmountMin

预计耗时 executionDuration

最好再做一次链上模拟（只读、未签名）。所有数字进模型后，才判断有没有利润。

3. 为什么需要 LI/FI（执行基础设施，不是策略系统）

把 Chain / Bridge / DEX / Solver 接到统一 API，可查询 Route、构造交易、持续跟踪跨链状态。

分工清晰：

策略系统回答：「哪里可能有机会」「扣完所有成本后是否值得做」。

LI/FI 回答：「资金具体怎么走」「这条 Route 现在要付出多少成本」「如何构造交易」「执行到哪一步了」。

核心服务：Bridge / Swap / Intents / Composer / Earn / Token Service / Gas API / Status Service。

4. 真实例子拆解：Ethereum 1000 USDC → Arbitrum USDC

| 项 | 数值 |

| :----- | :----- |

| 输入 | 1,000 USDC |

| Route 工具 | Eco |

| 预计到手 toAmount | 997.5 USDC |

| 最低到手 toAmountMin | 997.5 USDC |

| LI/FI Service Fee | 2.5 USDC = 0.25% = 25 bps |

| Ethereum Gas | ≈ \$0.2456 |

| 预计耗时 | 7 秒 |

| transactionRequest | 已返回未签名交易（只读，未广播） |

显性强执行成本 ≈ 2.5 + 0.2456 = 2.7456 USD ≈ 27.455 bps（这里还没有计算：机会发现后的价格变化、目标市场成交滑点、资金占用、失败成本。）

注意：toAmount == toAmountMin 只代表桥这段没有给出滑点缓冲；你在目标链上「真正做那笔套利交易」的滑点 / Price Impact 不在这个 quote 内，必须从目标 DEX / CEX 另取。

5. Break-even Spread（盈亏平衡价差）

最低所需价差 = LI/FI Service Fee

Bridge / DEX / Solver Fee

源链与目标链 Gas

两端交易滑点与 Price Impact

延迟、失败与资金占用成本

例子里：表面价差若只有 20 bps，光 LI/Fl 费 (25 bps) 就已经超过它；即使价差是 40 bps，扣完 Gas、目标端滑点、延迟风险后，也未必还剩下利润。

6. 三个关键认知 (重点)

- 25 bps 基本宣判「稳定币 1:1 跨链套利」死刑。真实 edge 在 bots 还没覆盖的地方：新链刚上线、长尾资产、跨域信息差、结构性 / 监管差异。
- 跨链套利应「多链预置资金 + 本地执行 + LI/Fl 做再平衡」，而不是「发现价差后临时跨链」—— 套利窗口常 < 跨链耗时，且跨链非原子，等你资产到对岸机会已消失。把「抓机会」和「跨链调仓」拆开。
- toAmount == toAmountMin 不代表整笔无风险；目标端执行滑点才是利润主要死因。

7. 风控边界 (API 调用)

公开 API 未认证时，/quote 与 /advanced/routes 默认各 75 次 / 2 小时；API Key 默认 100 次 / 分钟 (Quote 按 2h 滚动窗口)。
/chains、/tokens、/tools 变化慢 → 缓存，不要每次重查。

收到 429 按 ratelimit-reset 退避，不要换 Key 绕过；持续超限 / 超限可能被临时封 Key。

别把 API 当免费行情源高频抓取。

8. 配套工具 (本笔记对应的交付物)

arbitrage-net-calculator.html：净收益公式做成期望形式 (含概率加权 + 时间窗 T/D 诊断)。

arbitrage-starter-handbook.html：21 天小白执行手册 (第几天、装哪个、点哪里、验收什么)。

quote-logger.js：把跨链报价自动记录到 CSV / JSON。

lifi-experiment.js：落实「只读实验」—— /chains → /tokens → /quote → /advanced/routes → 算 Break-even Spread。

9. 下一轮行动

第一轮只做 Quote、比较、模拟，不签名、不广播。等成本模型跑通后，再决定是否用「可承受归零」的小额资金做一次真实验证。

本文档仅用于个人学习与研究，不构成任何投资建议。链上套利存在真实资金损失风险。

笔记 110: Day 4 | 手算 AMM 價格，實測「無套利帶」

作者：HenryChang | 时间：2026-08-09 14:48 UTC | 字数：1645 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 | 手算 AMM 價格，實測「無套利帶」

前三天由外往內 (LI.FI 執行成本 → 區塊構建競價成本)，今天終於走到源頭：價格是從池子裡長出來的。上午讀 AMM 數學教學文，下午用 RPC 親手讀了四個池、算了價格、跑了完整的套利模擬。

概念重點 (教學文摘要)

$x \cdot y = k$ 自動產生兩件事：現貨價 = 儲備比值 x/y ；曲線是凸的，大單必然把價格推向對自己不利的方向

swap 真實數學： $\Delta y = y \cdot \gamma \Delta x / (x + \gamma \Delta x)$ ， $\gamma = 0.997$ 。執行均價永遠比現貨價差，衝擊 $\text{bps} \approx$ 交易量佔池深比例——深度就是一切
V3 集中流動性：價格藏在 slot0 的 sqrtPriceX96 ， $(p/2)^2$ 再調 decimals；同一交易對多費率池並存，每個池是一個獨立價格
最優交易量：套利就是把兩池價格推到相遇，毛利是先升後降的曲線，存在最優 x 。淺池的 x 小到毛利付不起 gas

實測 (區塊 25,709,757，公共 RPC 只讀)

手算四池價格 (USDC/WETH)：

| 池 | 價格 | 備註 |

|---|---|---|

| UniswapV3 0.05% | 1,919.08 | sqrtPriceX96 解碼 |

| UniswapV2 0.30% | 1,918.61 | 儲備 \$8.9M + 4,632 WETH |

| UniswapV3 0.30% | 1,916.28 | |

| Sushiswap 0.30% | 1,915.57 | 儲備只剩 \$123k + 64 WETH ! |

最大跨池價差：18.3 bps (V3 0.05% vs Sushi)。

V2 ↔ V2 精確套利模擬 (Sushi 買 → UniV2 賣，毛價差 15.85 bps)：

- 黃金分割搜尋最優投入 → 收斂到下界 1 USDC：因為兩腿手續費合計 60 bps > 毛價差 15.85 bps，任何交易量的毛利都是負的

- 就算毛利為零，gas (260k gas $\approx$ \$0.05) 也在等著；再上面還有 Day 3 學的優先費競價

今天最重要的領悟：無套利帶 (no-arbitrage band)

四個池的價格為什麼全擠在 ± 18 bps 內？因為手續費畫出了一條走廊：兩池價差小於「兩腿費率和」時，套利者動手就是虧，價格就被放任漂移；一旦有大單把某個池推出走廊，原子套利者立刻進場把它拉回來 (Day 3 的數據：Uniswap 量的 49% 就是這些 MEV

交易)。

所以我們看到的「平靜價差矩陣」不是沒有套利，而是套利者高效工作的證明——池間價差被壓在費率走廊內，正是他們收割完的狀態。

推論（下一步的策略方向）：

- 1. 監控要抓的不是「常態價差」，而是大單衝擊後的瞬間出軌——事件驅動，不是輪詢現貨價
- 2. 低費率池（V3 0.05%）走廊窄，出軌門檻低、機會頻率高，是監控首選
- 3. Sushi 這種深度枯竭的池（\$246k）就算價差誘人，最優交易量的絕對毛利也只有幾美元——掃描器要按深度過濾池子

工程進度

scripts/amm_price_scan.py : eth_call 直讀 getReserves()/slot0() (手拼 selector, 不依賴 web3 庫)、價差矩陣、黃金分割最優量搜尋、gas 即時估價，全程無金鑰只讀

至此成本模型四塊拼圖到齊：手續費（Day 4）+ 衝擊（Day 4）+ gas（Day 2/4）+ 優先費競價（Day 3）

明日計畫

把單次掃描升級成持續監控：訂閱 Swap/Sync 事件，記錄價差出軌的頻率、幅度與回歸速度——用數據回答「機會多久出現一次、能活幾個區塊」

笔记 111: Day 5 · 成本敏感分析 (agent 开发工程师视角)

作者：hzm0629 | 时间：2026-08-09 14:49 UTC | 字数：3408 | 评分：88
来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

Day 5 · 成本敏感分析 (agent 开发工程师视角)

共学 Day 5 (2026-08-09) | 对齐计划 D5：观察桥/DEX/Gas/滑点/手续费如何随时间与规模变化改变机会，采集时间序列，产出敏感度脚本+图，描述一个成本反转案例。 视角：agent 开发——把"成本如何随规模/时间变化"封装成两个可复用工具（size-sweep、time-sample+analyze），它们本身就是未来 arbitrage agent 的"成本监控"能力。

0. 今日产出

文件

作用

day5/cost_sensitivity.py

成本敏感度工具：(A) size-sweep 规模敏感度；(B) time-sample + analyze 时间序列采样与反转检测。复用 Day3 _get (4 次重试+UA)

day5/size_sweep.csv

6 路线 × 4 本金档 (100/1k/10k/100k) 真实询价结果

day5/series.jsonl

固定 1000 USDC 基准的 3 次时间序列采样 (间隔 ~12-30s)

day5/sensitivity_report.html

三张图：规模 bps、规模 usd、时间序列波动 + 反转检测

运行方式：

1. 第一课（反直觉，最重要）：成本模型是「固定比例费」，不是「Gas+比例」

我原本假设：本金越大，固定 Gas 被摊薄 → cost_bps 随规模下降。数据推翻了这个假设。

把同组 USDC 路线从 100 扫到 100,000，结果：

路线

\$100

\$1k

\$10k

\$100k

cost_usd @\$100k

8453→137

5.73

5.73

5.73

5.73

\$57.20

137→8453

44.23

44.23

44.23

44.23

\$442.58

1→137

12.73

11.06

11.06

11.06

\$110.48

1→8453

31.58

30.32

30.32

30.32

\$302.93

8453→1

19.68

19.68

19.68

19.68

\$196.50

137→1

38.92

38.92

38.92

38.92

\$389.43

关键事实：cost_bps 几乎不随本金变，而 cost_usd 随本金线性放大（0.057 → 0.57 → 5.72 → 57.2）。这说明 LI.FI 的 eco/across/polymer 路线收的是固定比例费（protocol/relay fee），不是"固定 Gas + 比例费"。

这对套利的直接含义（接 Day 6 的 net_profit_estimator）：

桥接成本的盈亏平衡价差（bps）与本金规模无关——你需要同样的 % edge 才能覆盖过路费，不管做 100 还是 100k。

但本金越大，过路费的绝对美元越大，被卡在桥上的本金也越多 → 资金占用成本才是规模真正要权衡的，而不是 bps。

所以"小资金高频吃小价差"和"大资金低频吃大价差"在 bps 上没区别，区别在资金效率与流动性冲击（后者接 Day 1 价格冲击）。

2. 第二课（真发现）：规模会在「桥切换阈值」处引发成本反转

虽然 bps 整体恒定，但 LI.FI 的路由选择（用哪座桥）会随本金跳变，造成小幅成本反转：

1→137：100 用 across（12.73 bps）→ 跨到 1000 切 eco（11.06 bps），降 1.67 bps。

1→8453：100 用 across（31.58 bps）→ 1000 切 eco（30.32 bps），降 1.26 bps。

137→1：100 用 polymerStandard（38.92）→ 1000 切 eco（38.92），bps 同但桥变了。

这就是 Day 5 要求的"一个成本反转案例"：在 100→1000 的本金阈值上，最便宜的桥从 across 翻转为 eco，导致该路线成本下降。对 agent

的意义：成本不是本金的单调函数，路由选择在阈值处离散跳变，监控脚本要把"本金档"当成独立维度，不能只盯一个固定规模。

3. 第三课：时间维度——分钟级成本近似恒定，反转需事件驱动

time-sample 跑了 3 次（间隔 \~12-30s），analyze 结果：

每条路线 std ≈ 0 （除 1→137 在 11.06-11.25 间偶有 0.09 bps 微跳，是 LI.FI 子路由微调）。

目标链成本排名始终 \[137 最贵, 1 居中, 8453 最便宜]，分钟级无反转。

最便宜路线始终 8453→137（5.73 bps）。

诚实结论：在分钟级、无外部冲击下，这些 eco 路由费是报价服务的准固定值，不会自己反转。真实的“成本反转”案例要靠：

1. Gas 尖峰（L1 拥堵时 Ethereum 相关路线 cost_usd 跳升）；

1) LI.FI 费率/路由表调整（平台侧改 fee schedule）；

1. 流动性迁移（某桥 TVL 变化导致 routing 重排）。

cost_sensitivity.py time-sample --repeat N --interval S 挂成定时任务（配合 Day 15 的 monitor）就能在长周期捕获这些反转——工具已就位，逻辑已验证，缺的只是时间跨度。

4. 给 agent 开发者的工程提示

复用 > 重写：cost_sensitivity.py 直接 import day3/lifi_quote._get，把“健壮的 quote 客户端”当成一个共享底层能力——这正是 agent 工具该有的模块化。

反直觉假设必须被数据验证：我带着“Gas 摊薄”的预设写工具，结果数据推翻了它。agent 的成本估算模块不能硬编码“大单更便宜”，必须实测 cost 函数形状。

反转检测是“可证伪”的：analyze 不预设一定反转，而是比较首次/末次排名，没反转就如实说“暂无”。这对应 Day 12 假设验证的“可证伪指标”原则。

密钥红线不变：LI.FI key 仍只走 LI.FI_API_KEY 环境变量；只读询价，绝不自动发交易。

5. 与前面几天的闭环

Day 1（价格冲击）：本日证明 LI.FI 桥接成本主要是比例费而非冲击主导——但真实 DEX 内/间套利仍有 AMM 冲击（Day1 已算过 33% 那种），两件事不矛盾：跨链桥费是比例费，链内 AMM 冲击是规模敏感的非线性费。

Day 2（机会地图）：“方向不对称”（Polygon→Base 3.78bps vs Base→Polygon 46.17bps）在本日被坐实是路由拓扑决定的固定比例，不是随机波动。

Day 4（报价对比）：本日的规模/时间维度，把 Day 4 的“某链更贵”从单点快照扩展成“成本函数”。

下一步（Day 6）：写 net_profit_estimator.py——输入“屏幕价差 + 这条路线的 cost_bps + 规模”，直接输出“扣费后净收益/是否值得做”。本日已证明 cost_bps 可当常数用，estimator 不必为规模维度过度复杂化（但要把“桥切换阈值”作为边缘情况标注）。

笔记 112: Day 5 - 2026-08-09：从双链库存套利切换到 Polymarket

作者：Helios | 时间：2026-08-09 14:53 UTC | 字数：1741 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 - 2026-08-09：从双链库存套利切换到 Polymarket

今天的问题

Polymarket

上哪些结构是真正能由支付函数证明的套利，哪些只是相关市场价差、方向判断或做市？在剩余共学时间里应该只做哪一种？

我做了什么

昨天选的是 Base/Arbitrum 上 WETH/USDC 的双链库存套利。今天从第一性原理重新检查后，我决定停止这个方向：它需要同时处理两条链、两套库存、Gas、再平衡和跨链路径，工程面很宽，但不直接提高对“套利是否锁定”的验证质量。

我改为研究 Polymarket，并根据官方文档梳理了 8 类结构：

| 类型 | 判断 |

| :----- | :----- |

| 同一市场 YES+NO 完整集 | 全腿成交后可锁定，主线 |

| 负风险事件全部 YES 篮子 | 结果完备且全腿成交后可锁定，第二阶段 |

| NO_i → 其他 YES 转换价差 | 转换原子，但买卖多腿不原子 |

| 相关市场逻辑一致性 | 依赖两份规则严格蕴含，条件型 |

| 时间嵌套/重复市场 | condition 独立，条件型 |

| 跨平台预测市场 | 规则、资金和平台风险更大，本期不做 |

| 双边做市 | 有逆向选择和库存风险，不是套利 |

| 规则/UMA 分歧、Combo | 属于判断交易或受限基础设施，暂缓 |

今天最重要的证据

Polymarket 的二元市场每份 1 pUSD 可以 split 为 1 YES + 1 NO，等量 YES + NO 又可以 merge 回 1 pUSD。结算后胜方兑 1，罕见的 50/50 结算则两边各兑 0.5，所以完整集总支付仍为 1。

因此对数量 q ，买入完整集的净收益是：

text

net_edge

= q

- YES 的逐档可执行买入成本
- NO 的逐档可执行买入成本
- taker fee
- merge/结算成本
- 双腿非原子风险缓冲

这也纠正了两个容易犯的错误：

1. 页面显示的是中间价或最后成交价，不是实际成交价；必须读取 asks/bids 和深度。
2. CLOB 虽然支持 FOK 和批量下单，但批量中的每个订单独立成功或失败，不能假设 YES、NO 两腿原子成交。

负风险事件中恰好一个结果 YES。完整结果的 Σ YES 若全成本小于 1，也有固定支付；此外，一个结果的 NO 可以通过适配器原子转换为其他结果的全部 YES。但增强型负风险事件可能有未命名占位结果和定义会变化的 Other，本期先不碰。

今天的决策

本期主线改为：

使用 Polymarket 公开 Events/Gamma 与 CLOB API，扫描同一二元市场在 5、10、25、50、100 shares 下的 YES+NO 完整集净 Edge。只有逐档盘口、实际费率和单腿风险缓冲全部计入后仍为正，才记录为 Paper Trade 候选。

主线稳定后，再扩展到普通负风险事件的完备 YES 篮子。相关市场、跨平台、做市和 UMA 争议不进入“无风险套利”统计。

风险与边界

只读取公开 API，不连接钱包、不签名、不授权、不广播交易。

不使用中间价计算收益，不只保存正样本。

一个腿深度不足或盘口时间差过大就拒绝交易。

真实交易前必须检查官方 geoblock 和个人资格；本期不做实盘。

Paper Trading 为正不自动升级为真实资金。

产出

完成 Polymarket 套利类型研究与官方来源整理。

重写研究章程，替换旧的 Base/Arbitrum 方案。

重排 08-09 至 08-26 的共学实践计划。

笔记 113: 今日打卡

作者：elipitic | 时间：2026-08-09 14:54 UTC | 字数：2735 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日打卡

打卡日期 (UTC+8)：2026-08-09

今日学习主题

今天不只是学习了

Li.Fi

获取各类报价 (quote) 的具体方式，还进一步把真实报价跑完，并对候选套利路径做了第一轮完整分析。今天的重点从“知道怎么拿 quote”推进到了“用真实 quote 去判断一条路径是不是伪套利”。

一、今日学习内容

1. 学习了 Li.Fi 获取报价的具体方式

今天重点理解的是：Li.Fi 不只是一个“跨链入口”，它本质上还是一个 路径和报价聚合器。获取报价的过程，不只是问“从 A 到 B 能不能换”，而是同时在观察：

不同链之间的路径

不同资产之间的兑换结果

route 在多大程度上受金额、Gas、流动性源影响

返回结果中哪些字段最值得后续持续追踪

2. 跑通了真实报价并保存结果

今天不只是停留在概念层，而是实际跑出了多条 quote，并保存为本地数据文件：

data/lifi/eth_to_arb_usdc_1000.json

data/lifi/arb_to_eth_usdc_1000.json

data/lifi/eth_to_base_usdc_1000.json

data/lifi/base_to_eth_usdc_1000.json

这些结果让我从“理解 Li.Fi 返回什么”进一步进入到“比较不同路径、判断路径值不值得继续跟踪”的阶段。

二、今日实际分析：报价与候选套利路径

1. 单跳报价观察

今天的真实报价里，Li.Fi 返回的主要路线工具是 eco，在当前样本中表现为：

- Ethereum → Arbitrum (USDC 1000)
- 输入约 1000 USDC
- 输出约 997.5 USDC
- gas 约 0.097 美元
- Li.Fi 固定费约 2.5 USDC
- Arbitrum → Ethereum (USDC 1000)
- 输出约 997.5 USDC
- gas 约 0.024 美元
- Li.Fi 固定费约 2.5 USDC
- Ethereum → Base (USDC 1000)
- 输出约 997.5 USDC
- gas 约 0.097 美元
- Li.Fi 固定费约 2.5 USDC
- Base → Ethereum (USDC 1000)
- 输出约 997.5 USDC
- gas 约 0.007 美元
- Li.Fi 固定费约 2.5 USDC

2. 往返路径分析

今天最重要的推进不是看单跳，而是做了一个更接近套利思维的动作：

把往返路径组合起来，看它是否真的有正收益。

我今天重点看了两条 round-trip：

1. Ethereum → Arbitrum → Ethereum
2. Ethereum → Base → Ethereum

3. 初步结论

今天的结果说明：

当前样本下，这两条 1000 USDC 的往返路径都没有形成真实套利

两条往返在计入 fee 与 gas 后，大约都落在 -0.51% 左右的负收益区间

Base 正向路径里看起来出现过表面上的美元增益，但继续组合返程后，这种“利润”并没有成立

4. 我今天最关键的理解

今天真正重要的不是“没找到套利”，而是我确认了一件很关键的事：

单跳 quote 的表面好看，并不等于真实可执行套利。

也就是说：

- 只看某一跳的 toAmountUSD 很容易被误导
- 真实套利判断必须至少做到：
- 看 token 数量变化
- 看固定费用
- 看 gas 成本
- 看往返组合后是否仍然成立

三、这部分学习对本期共学的意义

今天这部分推进和本期共学主线非常一致，因为它补齐了后续跨链机会判断里最关键的一块：

1. 路径理解能力

后续如果要判断跨链机会，不能只看某一边价格，而要看：

- route 怎么走
- 中间经过哪些流动性源
- 哪些成本在路径中被吃掉

2. 报价结构理解能力

如果不理解 Li.Fi 返回结果的字段，就无法把 quote 变成：

- 成本判断
- 净收益判断
- 候选机会筛选依据

3. 伪套利识别能力

今天最有价值的一步是：

- 我开始学会区分“表面路径增益”和“真实往返套利”
- 这比只看单次 quote 更接近真正的套利研究

四、今日收获

今天最重要的收获有四点：

1. Li.Fi 的 quote 本质上是路径视角的数据入口

它不仅是报价，更是后续做路径分析与机会判断的基础。

2. 套利研究不能停留在“知道聚合器存在”

真正有用的是理解它如何返回数据、如何表达路径、以及结果如何被进一步分析。

3. 今天完成了从概念走向真实报价分析的一步

不只是拿到 quote，还把 quote 真正用于路径对比和候选套利验证。

4. 没有发现真实套利本身也是重要结论

今天的负收益结果说明，今后做跨链机会判断时，必须对“表面利润”保持警惕。

五、下一步计划

接下来我会继续推进以下方向：

1. 继续拆解 Li.Fi quote 返回结果中的关键字段，明确哪些字段对机会判断最重要
2. 尝试对比不同金额规模下的报价差异，观察 route 是否切换
3. 继续把 Li.Fi 的报价结果和成本模型、净收益判断连接起来
4. 引入更多约束条件，例如外部可成交价格、真实退出价格与流动性可执行性，进一步排除伪套利路径

六、今日完成标准检查

- [x] 学习了 Li.Fi 获取报价的具体方式
- [x] 跑通了真实 quote 并保存结果文件
- [x] 完成了至少两条 round-trip 路径的初步比较
- [x] 识别出当前样本中不存在真实套利，而是伪套利 / 表面套利路径
- [x] 为后续 route 对比和成本模型打下了基础

一句话总结

今天我完成了从“知道 Li.Fi 是什么”到“开始用真实报价分析并识别伪套利路径”的过渡，这一步对后续做跨链套利研究非常关键。

笔记 114: 链上套利残酷共学 · Day 4 心得

作者：kako | 时间：2026-08-09 14:55 UTC | 字数：519 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 4 心得

今日主题：拆解一笔真实交易

今天终于从理论公式进入到“真刀真枪”的环节——打开 Etherscan，亲手拆解一笔真实的链上套利交易。之前三天学的价差、滑点、Gas 费，今天全都在一笔交易记录里找到了对应的数字。

说实话，第一次看 Etherscan 的交易详情页是有点懵的。密密麻麻的地址、代币转移记录、内部调用、Gas 消耗明细……但一旦沉下心顺着资金流向逐行追踪，画面就慢慢清晰了。能清楚地看到套利者从哪个池子买了什么代币，又在哪个池子卖出去，每一步扣了多少手续费，最终净赚多少。

最冲击认知的是发现：很多公开的“套利交易”显示出来是亏损的。后来才明白，这是因为浏览器上显示的代币价值用的是当前市价，而不是交易发生那一刻的池子成交价，所以看起来“亏”了，但实际上套利者已经赚走了利润。这个细节提醒我：读链上数据不能只看表面数字，要理解每个字段背后的含义。

拆解到第三笔交易时，开始能自主画出交易的路径图了：起点地址 → 第一步兑换 → 中间跳转 → 最终卖出 → 利润落袋。同时也能标注出每一步的Gas消耗和滑点损失，这种感觉很像破案——把一笔链上交易的前因后果全部还原出来

07 学习计划（5 篇）

笔记 115: 最近因为 ColdCard 弱随机数问题 尝试去设计一个熵混合系统

作者：alex | 时间：2026-08-08 15:23 UTC | 字数：6024 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

最近因为 ColdCard 弱随机数问题 尝试去设计一个熵混合系统

\

密码学熵 (Entropy) 与随机数生成学习笔记

目标：理解如何生成高质量随机熵，用于 Bitcoin/Ethereum 钱包种子、BIP39 助记词、多熵混合，以及理解常见密码学算法 (SHA、HMAC、HKDF、PBKDF2、ChaCha20 等) 的作用。

1. 什么是熵 (Entropy)

1.1 信息熵

密码学中的熵表示：

一个秘密值的不确定程度。

例如：

6面骰子

结果：

1 2 3 4 5 6

每个概率：

$1/6$

熵：

$\log_2(6)=2.585\text{bit}$

所以一次 D6：

$\approx 2.585 \text{ bit 熵}$

硬币

两种：

正

反

熵：

$\log_2(2)=1\text{bit}$

一次公平硬币：

$\backslash=1 \text{ bit}$

D8

$\log_2(8)=3\text{bit}$

D16：

$\log_2(16)=4\text{bit}$

2. 钱包需要多少熵？

Bitcoin BIP39

24词助记词：

2048个词

每个词：

$\log_2(2048)=11\text{bit}$

24词：

$24 \times 11 = 264 \text{ bit}$

其中：

256 bit entropy

+

8 bit checksum

结构：

[256 bit 随机熵][8 bit 校验]

12词：

128 bit entropy

+

4 bit checksum

3. BIP39数学结构

流程：

随机熵

|

v

加入checksum

|

v

切成11bit

|

v

映射2048词表

|

v

mnemonic

|

v

PBKDF2-HMAC-SHA512

|

v

512bit seed

注意：

助记词 $\neq$ seed

助记词只是输入。

真正钱包seed：

```
PBKDF2-HMAC-SHA512(  
mnemonic,  
"mnemonic"+passphrase  
)
```

4. 用骰子生成BIP39

4.1 D6骰子

D6：

6种状态

不是2的幂：

$6 \neq 2^n$

不能直接转换。

例如：

错误：

1-6

直接转二进制

会产生偏差。

4.2 去偏方法

Rejection Sampling

例如：

D6：

1-4 使用

5-6 丢弃

得到：

4种状态

对应：

2 bit

Von Neumann 去偏

两个结果：

AB

例如：

1 2 -> 0

2 1 -> 1

丢弃：

1 1

2 2

可以消除骰子偏差。

5. BIP39优化骰子组合

D16 + D16 + D8

因为：

$16 \times 16 \times 8$

=

2048

=

2^{11}

一次：

D16 = 4bit

D16 = 4bit

D8 = 3bit

合计：

11bit

刚好对应：

一个BIP39词

生成24词：

$(D16, D16, D8) \times 24$

得到：

264bit

其中：

256bit entropy

+

8bit checksum

优势：

无模偏置

无熵浪费

简单验证

6. 随机源分类

PRNG

Pseudo Random Number Generator

伪随机：

seed

|

v

算法

|

v

随机序列

例如：

Mersenne Twister

ChaCha20 DRBG

安全取决于：

seed

CSPRNG

Cryptographically Secure PRNG

密码学安全随机：

例如：

Linux：

/dev/urandom

macOS：

SecRandom

Node：

crypto.randomBytes()

特点：

不可预测

后向安全

前向安全

TRNG

True Random Number Generator

真实物理随机：

例如：

热噪声

振荡器

量子随机

QRNG

Quantum RNG

量子随机：

例如：

光子测量

注意：

公开QRNG输出：

如果攻击者知道：

不能作为秘密熵。

它只能作为：

公开随机源

7. 熵混合设计

错误方式

简单：

$A+B+C$

或者：

xor

存在问题：

如果：

A=攻击者控制

B=弱随机

可能影响结果。

推荐方式

Hash混合

例如：

```
SHA512(  
source1 ||  
source2 ||  
source3  
)
```

作用：

打散结构

压缩

但：

Hash不能创造熵。

HKDF

RFC5869

结构：

Input Key Material

|

v

HKDF-Extract

|

v

PRK

|

v

HKDF-Expand

|

v

Output Key

HKDF-SHA512

内部：

HMAC-SHA512

不是：

SHA512多次hash

关系：

HKDF

|

+-- HMAC

|

+-- SHA512

8. HMAC、HKDF、PBKDF2区别

HMAC

用途：

认证：

message

+

secret key

|

v

MAC

结构：

HMAC-SHA256

=

SHA256

+

key mixing

HKDF

用途：

密钥派生：

例如：

一个主密钥

|

v

多个子密钥

特点：

快

不用于密码加固

PBKDF2

用途：

密码增强：

例如：

弱密码

|

多轮HMAC

|

强密钥

BIP39：

PBKDF2-HMAC-SHA512

2048 rounds

区别：

| 算法 | 用途 |

| :----- | :----- |

| HMAC | 认证 |

| HKDF | 派生 |

| PBKDF2 | 增加破解成本 |

9. SHA-256 是否被破解？

问题：

SHA256被破解，整个系统是否崩溃？

分情况。

Hash碰撞破解

影响：

A -> hash

B -> same hash

影响签名、公证。

原像破解

影响：

hash

反推出原数据

如果SHA256原像破解：

大量系统失效。

但：

SHA256不是所有系统核心。

例如：

Bitcoin：

主要：

SHA256 PoW

ECDSA/secp256k1签名

ETH：

主要：

Keccak256

secp256k1

不是SHA256。

10. ChaCha20

ChaCha20：

流密码。

结构：

key

nonce

|

v

ChaCha20

|

v

随机字节流

Linux随机系统：

使用：

ChaCha20 DRNG

因为：

快

安全

软件实现简单

11. π 圆周率作为随机源？

方案：

随机位置N

取 $\pi[N:N+256]$

错误理解：

π 数字=256bit熵

实际上：

π ：

公开确定。

真正熵：

来自：

N

如果：

N来自：

256bit真随机

那么安全。

如果：

N来自：

时间seed PRNG

则：

接近0。

12. 多熵源钱包设计

推荐结构：

Mac RNG

\

Linux RNG

\

骰子

\

扑克

\

QRNG

\

Chainlink VRF

|

v

Canonical Transcript

|

v

HKDF-SHA512

|

v

256bit entropy

|

v

BIP39

13. Canonical Transcript

多源混合时：

不要：

直接拼字符串

应该：

记录：

prefix

version

count

source id
length
data

例如：

SOURCE_A

32 bytes

SOURCE_B

64 bytes

然后：

排序：

id排序

保证：

不同语言：

不同机器：

得到同样结果。

14. Passphrase安全模型

BIP39：

mnemonic

+

passphrase

进入：

PBKDF2

如果：

助记词泄漏：

攻击者需要：

passphrase。

如果passphrase：

256bit随机：

理论：

2²⁵⁶

不可破解。

15. BIP85

BIP85：

确定性随机生成。

结构：

master seed

|

v

HMAC-SHA512

|

v

child entropy

用途：

从一个seed派生：

多个钱包

密码

应用密钥

区别：

BIP85：

单熵源派生。

多熵方案：

多个来源合成。

16. 安全原则总结

真正重要的是：

1.

随机源质量

entropy > algorithm

2.

不要相信单一来源

例如：

硬件钱包RNG

可能有bug。

3.

公开随机 $\neq$ 秘密随机

例如：

区块hash

π

QRNG公开数据

只能辅助。

4.

Hash不能创造熵

256bit弱随机

hash

仍然256bit弱随机

5.

超过256bit没有坏处

例如：

输入：

1024bit熵

输出：

HKDF-SHA512

256bit

安全性：

$\approx$ 256bit。

最终推荐冷钱包生成方案

个人离线高级方案：

D16+D16+D8骰子

+

系统CSPRNG

+

额外随机源

↓

Canonical Transcript

↓

HKDF-SHA512

↓

256bit entropy

↓
BIP39 24词
↓
纸质保存
↓
可选256bit passphrase
安全等级：
远高于：
普通硬件钱包默认随机
单纯电脑生成
单纯骰子生成
核心思想：
熵来自不可预测性；算法负责正确混合和扩展，而不是创造随机性。

笔记 116: 完成容量实验，对 Arbitrum USDC → Optimism DAI 同一条路线在 100/1

作者：Wang Hua | 时间：2026-08-09 02:16 UTC | 字数：386 | 评分：54

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

完成容量实验，对 Arbitrum USDC → Optimism DAI 同一条路线在 100/1,000/10,000 USDC 三个金额下各采集一次真实报价（3 份原始 JSON、共 16 条路线），并全部并入统一机会日志（现 34 条）。关键数字：三个金额的最佳路线乐观净收益都约为 -106 bps (-1.07/-10.57/-108.04 USD)，全部为负，主因是 USDC/DAI 价格源估值差；10,000 USDC 时 Squid 排名从第 1 崩到第 6、报价暴跌 514 USD，证明这是金额过大造成的容量假象；协议费是比例成本（约 0.25%），Gas 是固定成本；Squid 最低输出比预计输出低约 0.70%，大于请求的 0.3% 滑点。新增 4 项测试，全仓 15 项通过。明天用 Hermes 独立审查机会日志的数据、假设和不可执行条件。

笔记 117: Day 4 / 2026-08-09

作者：shiyang07ca | 时间：2026-08-09 05:20 UTC | 字数：1688 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 / 2026-08-09

今日学习目标

完成 Day 4：价格源、index、mark 和 EMA，理解 RWA 在 oracle stale 后为什么会逐步切换价格来源。

今天实际完成

完成 Day 4 互动课程：lessons/0003-day4-price-semantics.html。

完成电脑端练习：价格字段分类、EMA 计算、真实快照字段查看、mark 估值与 bid/ask 平仓 PnL 对比、5 题迁移验收。

通过浏览器完成 5/5 验收；切换 BRENT OIL 场景、修改 EMA 时间常数并验证空头平仓使用 ask。

实际运行 python3 lab/day4_price_semantics.py。

实际运行 python3 -m unittest lab.test_day4_price_semantics -v，4 个测试通过；Day 3 的 4 个测试也通过。

使用 Lighter 官方 RWA Pricing Mechanism、Fair Price Marking 和 PnL 文档核对价格语义。

学习总结

oracle_price 是外部价格源；index_price 是指数/定价基准；mark_price 用于公平估值、未实现 PnL 和清算相关语义；candle_close 是历史窗口观察值；bid/ask 才是立即卖出/买入时应优先考虑的盘口方向。

oracle stale 不表示资产没有价格，而表示外部价格源没有及时更新。系统会逐步降低 oracle 权重，增加内部订单簿 impact price 的权重，并用 EMA 平滑内部价格；外部价格恢复时再向其收敛。

两组时间常数不能混淆：来源权重切换的 $\tau_{\text{mark}}=1$ 分钟、 $\tau_{\text{index}}=1$ 分钟；内部 EMA 的 index $\tau=30$ 分钟、mark $\tau=2$ 分钟。前者决定“信谁”，后者决定“内部价格反应多快”。

如果 WTI 的 oracle fresh、BRENT OIL 的 oracle stale，两边出现的价差可能只是价格机制状态不同，不能直接当成 WTI-Brent 真实相对价值机会。

mark PnL 不等于可实现现金 PnL；多头立即平仓要使用 bid 方向估算，空头平仓要使用 ask 方向估算。

证据位置

lessons/0003-day4-price-semantics.html
reference/day4-price-semantics.html
assets/day4-price-semantics.js
notes/price-semantics.md
learning-records/0003-day4-price-semantics.md
lab/data/day4_price_semantics_snapshot.json
lab/day4_price_semantics.py
lab/test_day4_price_semantics.py
lab/data/lighter_rwa_raw/145_orderBookDetails.json
lab/data/lighter_rwa_raw/159_orderBookDetails.json

研究结论

Day 4 学习和电脑练习完成。研究仍为 Blocked：当前本地快照没有连续 oracle_state、source_timestamp、oracle_price、完整 bid/ask 深度和真实成交回放，不能据此证明价差可交易或策略盈利。

下一步

Day 5：建立 WTI/BRENT OIL 展期与市场时段模型，区分价格机制状态、交易窗口和展期窗口，避免把时间错位误判为相对价值信号。

笔记 118: <https://www.evm-from-scratch.app/content/> (https://www.evm-from-scratch.app/content/01_intro)

作者：x-tan-dev | 时间：2026-08-09 06:40 UTC | 字数：4832 | 评分：77

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

<https://www.evm-from-scratch.app/content/> (https://www.evm-from-scratch.app/content/01_intro)

Introduction

《EVM from Scratch》教程的第 1 章“Introduction”，主要介绍：

EVM 是什么：EVM (Ethereum Virtual Machine) 是以太坊的执行引擎，用来运行智能合约。

教程目标：从零实现一个可工作的“mini-EVM”。它不是以太坊官方的完整参考实现，而是为了理解核心原理，省略了一些工程细节。

虚拟机与字节码：EVM 是运行在真实计算机上的“软件计算机”，它执行的语言不是 Solidity，而是 EVM bytecode。

操作码 (opcode)：字节码本质上是一串操作码，例如：

1. STOP：停止执行
2. ADD：加法
3. MUL：乘法
4. SUB：减法
5. DIV：整数除法\

EVM 识别的是操作码编号，例如 ADD 对应 0x01，而不是文字名称。文中给了一个示例字节码：604260005260206000F3。

Solidity、Vyper 和 Huff：这些是人类更容易编写的高级语言，需要通过编译器转换成 EVM 字节码；能编译到 EVM 字节码的语言，就称为“以 EVM 为目标”。

以太坊与比特币的区别：以太坊 EVM 被称为图灵完备，理论上可以执行任意程序；比特币脚本则不是图灵完备的。

后续学习路线：教程会依次实现：

1. Stack (栈)
2. Memory (内存)
3. Storage (存储)
4. Opcodes (操作码)
5. EVM 本体
6. 用实现的 EVM 执行程序

简化范围：这个 mini-EVM 与账户、其他合约和外部世界完全隔离，涉及外部交互的功能会被模拟。

前置知识：Python 和十六进制数字。

一句话概括：这页是在建立基本概念，说明“智能合约最终会被编译成字节码，而通过执行这些操作码来运行它”，并介绍接下来如何一步步自己实现这个过程。

EVM

Stack Memory Storage State

在这套教程里，它们是四个不同层次的东西：

模块

讲什么

生命周期 / 特点

Stack (栈)

EVM 的主要计算工作区。操作码通常从栈顶取值、计算后再把结果压回栈顶。

后进先出；最多 1024 项；每项最大 256 bit (32 字节)；空栈 pop 会报错。

Memory (内存)

执行期间的临时字节数组，像计算机 RAM。可以按偏移量读写字节或字节块。

初始为空；每格 1 字节；一次执行结束就清空；扩容会消耗 Gas，而且空间越大，扩容成本增长越快。

Storage (存储)

合约持久化保存数据的位置，形式是 key → value 映射，类似硬盘或数据库。

执行结束后仍保留；未写入的 slot 默认读到 0；访问首次读取的冷 slot 更贵，已访问过的热 slot 更便宜。

State (EVM 状态)

把一次 EVM 执行所需的所有运行时信息组合起来。操作码就是不断修改这个状态。

包含 pc、Stack、Memory、Storage、程序字节码、Gas、调用者、msg.value、Calldata、停止/回滚标志、返回值和日志等。

最关键的区别是：

Stack：用于计算，放操作数和中间结果。

Memory：本次调用的临时数据区，结束即消失。

Storage：合约长期保存的数据，写入昂贵。

State：以上三者加上执行进度、输入、Gas 等，构成“此刻 EVM 正在运行什么”的完整快照。

例如，执行某条加法指令时，EVM 会从 Stack 取两个数，计算后压回 Stack；需要临时保存一段数据时写入 Memory；需要让数据下次调用仍存在时写入 Storage；而当前执行到哪一条指令、还剩多少 Gas，则记录在 State 里。

Opcodes

Opcodes 这一部分讲的是：EVM 到底有哪些指令，以及每条指令如何修改 EVM State。

字节码不是“程序文本”，而是一串十六进制指令编号。例如：

```
60 02 // PUSH1 0x02
```

```
60 03 // PUSH1 0x03
```

```
01 // ADD
```

EVM 执行后会把 2 和 3 压入栈，再执行 ADD，栈顶得到 5。

教程的实现思路是：每个 opcode 都对应一个函数，接收当前 EVM 实例，然后读取或修改它的状态，例如 Stack、Memory、Storage、pc、Gas、返回数据、日志等。

主要分为这些类别：

算术：ADD、SUB、MUL、DIV、MOD、EXP

比较：LT、GT、EQ、ISZERO

逻辑和位运算：AND、OR、XOR、NOT、SHL、SHR

读取链上/调用上下文：CALLER、CALLVALUE、CALLDATALOAD、ADDRESS、TIMESTAMP、CHAINID 等

内存操作：MLOAD、MSTORE、MSTORE8

持久化存储操作：SLOAD、SSTORE

流程控制：JUMP、JUMPI、JUMPDEST、PC

栈操作：POP、PUSH1 到 PUSH32、DUP1 到 DUP16、SWAP1 到 SWAP16

事件日志：LOG0 到 LOG4

合约级操作：CREATE、CALL、RETURN、REVERT、DELEGATECALL、SELFDESTRUCT

几个关键点：

每个 opcode 都有唯一的十六进制编号，例如 ADD = 0x01、MSTORE = 0x52、RETURN = 0xF3。

大多数 opcode 都会从 Stack 取参数，再将结果放回 Stack。

opcode 运行时推进或修改 程序计数器 pc，并扣除 Gas。

这套教程会实现“大部分”指令，但不是完整覆盖所有生产级 EVM 细节。

简单说：前面的 Stack、Memory、Storage、State 是 EVM 的“运行环境”，而 Opcodes 是在这个环境里真正执行工作的“指令集”。

EVM

EVM 这一章是在把前面做好的部件真正组装成一台可以运行的虚拟机。

它定义了 EVM 类，内部包含：

pc / Stack / Memory / Storage

program / gas / value / calldata

stop_flag / revert_flag / returndata / logs

核心是 run() 方法，也就是 EVM 的执行循环：

只要还可以执行：

1. 根据 pc 读取当前字节码
2. 判断它是哪一个 opcode
3. 调用对应实现，例如 add、mstore、jump、return
4. opcode 修改 EVM State，并更新 pc、Gas 等信息

这章的重点有：

pc (Program Counter)：指向下一条待执行 opcode 的位置。

取指、解码、执行：从 program[pc] 取出操作码，按编号分发到相应函数。

运行终止条件：

pc 已超出程序范围

执行了 STOP

执行了 REVERT

Gas 管理：gas_dec() 在余额不足时抛出 out of gas。

未知指令：遇到未实现或无效 opcode 时抛出异常。

重置执行环境：reset() 会恢复 pc、Stack、Memory、Storage，用于重新执行程序。

它本质上实现了 EVM 最重要的循环：

Fetch (取指) → Decode (识别 opcode) → Execute (修改状态) → 下一条指令

前面的章节分别实现“零件”，而这一章让零件一起工作。下一章 Bytecode 会处理如何把类似 6002600301 的十六进制字节码转换为 EVM 可执行的程序输入。

Bytecode

由许多 opcode 和操作数拼成的可执行程序

Bytecode 这一章讲的是：EVM 实际执行的程序长什么样，以及版本兼容性为什么重要。

有效 EVM Bytecode 是一串“操作码 + 操作数”，这整串内容称为一个 program。

例如：

0x6042

等价于：

0x60 = PUSH1

0x42 = 要压入 Stack 的 1 字节数据

执行后，栈顶会是 0x42。

这里一个关键点是：字节码里同时存在指令和数据。PUSH1 告诉 EVM“下一个字节不是 opcode，而是数据”；因此 pc 不能简单地每次加一，必须按当前指令长度前进。

文中还举了一个加法程序：

60 42 60 FF 01

含义是：

PUSH1 0x42

PUSH1 0xFF

ADD

最终 Stack 中只剩一个结果：0x42 + 0xFF = 321。

另外，这章强调 EVM 版本会随硬分叉演进：

Shanghai 增加 PUSH0

Cancun 增加 TLOAD、TSTORE

较早版本没有这些 opcode

所以同一段字节码不一定能在所有 EVM 版本运行。例如，含有 PUSH0 的字节码放在 Shanghai 之前的 EVM 上，会被视为无效指令。教程的 mini-EVM 以 Cancun 版本为目标。

简言之：前一章实现了“执行器”，这一章说明“执行器要吃的输入格式”，下一章 Run 才会把示例字节码真正交给 EVM 执行。

RUN

Run 是这套教程的收尾实操章节。它没有再引入新组件，而是把前面实现的 EVM 和 Bytecode 组合起来，真正运行一个程序。

示例使用的是前一章的加法字节码：

```
60 42 60 FF 01
```

对应：

```
PUSH1 0x42
```

```
PUSH1 0xFF
```

```
ADD
```

章节里的执行方式是：

```
GAS = 21\_000
```

```
evm = EVM(SIMPLE\_ADD, GAS, 0)
```

```
evm.run()
```

```
print(evm.stack)
```

运行过程可以理解为：

初始 Stack: \[

```
PUSH1 0x42 -> \[66]
```

```
PUSH1 0xFF -> \[66, 255]
```

```
ADD -> \[321]
```

最终输出：

```
321 \<first
```

也就是说，这章验证了整个 mini-EVM 已经能完成完整链路：

字节码 → 读取 opcode → 分发执行 → 修改 Stack/State → 得到结果

这里的 21_000 是给这次演示设置的初始 Gas，0 是调用携带的 Ether value。教程最后建议继续扩展更多 opcode、深入完整实现，或自己设计并执行新的字节码程序。

\

笔记 119: Day 05 · 2026-08-09 · 环境搭建：RPC、数据接口、钱包

作者：twone | 时间：2026-08-09 09:22 UTC | 字数：2575 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 05 · 2026-08-09 · 环境搭建：RPC、数据接口、钱包

今日目标

1. 配齐并实测：RPC 节点、链上数据查询接口、LI.FI API
2. 明确隔离小额钱包的设置规范
3. 为 Day 6 的报价采集脚本把基础设施备齐

一、RPC 节点（已实测可用）

RPC 是脚本和链交互的入口。先用免费公共 RPC 起步，等有监控频率需求再上付费节点（Alchemy/QuickNode 免费档也够用）。

实测结果（2026-08-09，eth_blockNumber / eth_gasPrice）：

| 链 | 端点 | 状态 |

| :----- | :----- | :----- |

| Arbitrum | <https://arb1.arbitrum.io/rpc> | 正常，返回区块高度 0x1d5da4c3 |

| Base | <https://mainnet.base.org> | 正常，返回区块高度 0x2f6ee18 |

| Ethereum 主网 | <https://ethereum-rpc.publicnode.com> | 正常，返回 gasPrice 0x2edced0 |

调用方式就是标准 JSON-RPC：

```
bash
```

```
curl -X POST https://arb1.arbitrum.io/rpc \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}'
```

注意：公共 RPC 有速率限制且数据延迟略高于付费节点，轮询报价/余额够用，但不要指望它抢执行。

二、链上数据查询（两项实测）

DefiLlama API (免费无 Key, 已调通) : <https://api.llama.fi/protocol/uniswap-v3> 可直接拉协议分链 TVL, 适合做池子深度初筛——回到 Day 2 的判断规则, 看到价差先查深度, 这一步可以脚本化

The Graph : Gateway 已实测确认需要 API Key (无 Key 返回 missing authorization header)。在 The Graph Studio (<https://thegraph.com/studio/>) 免费注册可得 Key (每月 10 万次免费查询), 用于查 Uniswap 池子的历史成交、流动性快照, 是 Day 10 案例还原的主力工具。待办: 注册并保存 Key

三、LI.FI API (Day 4 已调通)

<https://li.quest/v1/quote> 无需 Key, 昨天已用两笔真实报价验证。Day 6 会在此基础上批量轮询不同链/资产/规模的报价。

四、隔离小额钱包 (手动待办, 规范先行)

规范定下来, 操作本身在钱包 App 里手动完成 (不在脚本环境里生成或存放任何私钥) :

1. 在 Rabby/MetaMask 里新建一个完全独立的地址, 不与主钱包有任何转账关联之外的联系
2. 只充入可承受全部损失的小额资金 (首期建议 $\leq \$100$ 等值, 分散到 Arbitrum/Base 的 USDC + 各链少量 Gas 代币)
3. 地址可以写进脚本用于拉报价 (fromAddress 参数), 私钥/助记词永远不进代码库、不进任何 API、不发给任何人
4. 给 LI.FI 合约授权时逐次核验授权对象和额度, 优先用“本次金额”而非无限授权

五、当前环境清单

| 组件 | 状态 | 用途 |

| :----- | :----- | :----- |

| LI.FI API | 已调通 (Day 4) | 报价采集、路由比较 |

| Arbitrum / Base / 主网公共 RPC | 已实测可用 | 链上状态查询、余额核对 |

| DefiLlama API | 已实测可用 | 池子深度初筛 |

| The Graph Studio API Key | 待注册 | 历史数据、案例还原 (Day 10 前完成即可) |

| 隔离小额钱包 | 待手动创建 | Day 19+ 小规模受控验证 |

明日衔接 (Day 6)

报价采集脚本 v1 : 用 LI.FI API 轮询不同链 / 资产 / 规模下的报价, 计算扣除费用后的隐含价差并落盘成 CSV, 开始积累自己的数据。

参考资料

PublicNode 公共 RPC 列表 (<https://ethereum.publicnode.com/>)

DefiLlama API 文档 (<https://defillama.com/docs/api>)

The Graph Studio (<https://thegraph.com/studio/>) : 免费 API Key 申请入口

LI.FI API 文档 (<https://docs.li.fi/api-reference/introduction>)

08 实盘与数据 (13 篇)

笔记 120: 今天我把之前的历史 Flash Swap 案例, 推进到了实时三池只读报价 → 保守 PnL → mi

作者 : Quinn | 时间 : 2026-08-08 15:22 UTC | 字数 : 2369 | 评分 : 77

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天我把之前的历史 Flash Swap 案例, 推进到了实时三池只读报价 → 保守 PnL → minProfit → REJECT / ACCEPT 的完整流程。

今日完成

1\、将三池循环评估器升级为本地 Fork 实时路径 :

WETH → USDG → FRONG → native ETH

2\、用本地 V3QuoteProbe 对前两跳做 exact-output 报价 :

Pool A : 取得 USDG 需要偿还多少 WETH ;

Pool B : 取得 FRONG 需要偿还多少 USDG。

3\、从历史 V4 Swap event 的 PoolId 反查 Initialize 日志, 恢复第三跳完整 PoolKey :

currency0 = address(0) / native ETH

currency1 = FRONG

fee = 2500

tickSpacing = 60

hooks = address(0)

4. 独立复算 keccak256(abi.encode(PoolKey))，得到的 PoolId 与历史 V4 Swap 事件中的 PoolId 精确一致，确认第三跳是 native ETH / FRONG Pool，而不是 WETH / FRONG Pool。

5\ 实现本地只读 V4QuoteProbe：

unlock → unlockCallback → swap → 主动 revert

用于读取 FRONG exact-input 对应的 native ETH 输出，并保证全部模拟状态回滚。

6\ 将三跳实时 quote、当前 RPC gasPrice、Gas 单位参考、1% 输出 haircut、风险缓冲和最低净利要求统一送入保守评估器，生成 minProfit 与决策。

7\ 在同一个冻结 local-fork state 上完成规模扫描：

100 / 500 / 1,000 / 5,000 / 10,000 FRONG

核心结果

以历史套利交易的规模：

10,552.182072815894438654 FRONG

在当前 Pool 状态重新报价：

WETH owed： 0.034362816034919634

native ETH out： 0.034020161880254467

Gas 前毛差额： -0.000342654154665167 WETH-equivalent

保守决策： REJECT

五个扫描规模全部为 REJECT，并且在扣 Gas 前就已经是负收益。

这说明：

历史上曾经成立的套利路径，在当前链上状态下已经失效。

不能因为看到了历史利润，就直接复制路径和规模执行。

今日关键理解

\ V4 的 address(0) 代表 native currency，不能直接换成 WETH 地址；否则 PoolId 改变，不再是同一个池。

\ V4 exact-input 使用负的 amountSpecified。

\ 完整套利判断必须让三跳报价来自同一个链上状态快照，不能拼接不同时间或不同区块的数据。

\ minProfit 不是“预期利润”，而是链上执行后必须保留下来的毛利润门槛，要覆盖 Gas、风险缓冲和最低净利。

\ 当前阶段最重要的成果不是“找到可交易机会”，而是能够可靠地拒绝已经失效的历史路径。

验证与产出

Python： 11 tests passed

Foundry： 7 tests passed, 0 failed

核心产出：

analysis/2026-08-08-robinhood-v4-poolkey-and-quote.md

experiments/opportunity-evaluator/observed_three_pool_quote_adapter.py

experiments/opportunity-evaluator/scan_observed_three_pool_cycle.py

experiments/opportunity-evaluator/outputs/observed-three-pool-cycle-2026-08-08T12-01-53Z.json

experiments/opportunity-evaluator/outputs/observed-three-pool-size-scan-2026-08-08T12-03-18Z.json

experiments/v3-flash-executor/src/V4QuoteProbe.sol

全程保持只读与本地 Fork 模拟：没有使用私钥、没有请求授权、没有签名、没有广播交易。

明日计划

- 1\、使用更接近完整路径的本地 Gas 估算，替换历史 Gas units 参考值；
- 2\、为实时 Quote 增加固定区块号快照与结果可重放验证；
- 3\、继续只读 / Paper Trading，不进入真实执行；如研究执行层，先单独设计并审查 V3+V4、native ETH wrap 和 callback 偿债模型。

笔记 121: 今日学习总结 | MEV 套利、成本模型与策略验证

作者：UU20220213 | 时间：2026-08-08 15:49 UTC | 字数：760 | 评分：55

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习总结 | MEV 套利、成本模型与策略验证

今天系统学习了 MEV 套利的执行环境，以及如何把理论价差转化为可验证的策略判断。

1. MEV 套利：交易在进入区块前会经过待处理队列，区块内的包含与排序会影响最终成交结果。DEX 原子套利通常通过“低价 DEX 买入 -> 高价 DEX 卖出 -> 检查最低利润”的同一笔总交易完成；原子性可以避免单腿持仓，但不能避免竞争、Gas 和价差消失。
2. Mempool 与排序：点击确认后还要签名、广播、进入待处理交易池，再由区块构建/验证链路选择和排序。因此先点击不等于先成交；Gas、交易内容和区块构建策略都会影响最终顺序。
3. 成本模型：价差只是原料，判断依据是净利润。需要统一记录实际买入/卖出报价、交易费、Gas、桥费或闪电贷费用、滑点和风险预留，并设置最低净利润门槛，将机会分为不做、观察和人工复核。
4. 最小到账与路线：预计到账是当前估算，最小到账是交易允许接受的底线；路线可能包含多次兑换或跨链步骤，不能只看页面到账数字，必须比较全部成本后的净结果与失败风险。
5. 流动性与三角套利：流动性决定可承接的交易规模；同链三角套利是绕 USDC -> ETH -> 其他资产 -> USDC 一圈后，判断扣除三次费用、Gas 和滑点后是否仍有净利润。
6. 回测与 Paper Trading：回测用于检验历史规则，Paper Trading 用实时数据观察策略是否仍成立；两者都不等于真实成交，应在验证后才考虑极小规模测试。

今日收获：人负责策略规则、成本门槛与风控边界；机器人负责监控、模拟和高速执行。自动执行前必须先证明策略不是偶然价差或错误报价。

明日计划：继续学习如何设计只读报价监控，以及如何记录真实证据和失败原因。

笔记 122: 研究了昨天群里老哥发的那个机器人。

作者：catcashx | 时间：2026-08-08 15:51 UTC | 字数：395 | 评分：47

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

研究了昨天群里老哥发的那个机器人。

这个人是搞了个集群，多个钱包跑这个循环。基本是下面几个步骤。

1. 发现

先扫相关路径，看有没有「走一圈资产可能变多」的差价。

2. 报价

用当前池子状态和公式，按准备投入的数量，预算一遍：每次大概进出多少、毛利大概多少。

3. 模拟

预算算完不等于能成交。模拟是把准备好的这笔交易在节点上空跑一遍：会不会失败回滚、gas 大概多少、余额变不变。

4. 执行

模拟可行才真签名上链，在一笔交易里把多跳交换做完。

5. 结算

看这一笔结束后链上结果：中间币清不清零、利润进了金库还是热钱包；若失败则整笔回滚，不留半截仓，通常只亏 gas。

6. 对账

很多笔之后再算总账：成功多少、失败多少、毛利合计多少、全部 gas（含失败）花了多少，系统到底赚还是亏。

金库某段时间进账，也不等于集群净利润——里面可能混着利润归集和运营打款，且没扣全成本。

笔记 123: Day 04 : Uniswap V3 Swap、报价验证与真实交易复现

Day 04：Uniswap V3 Swap、报价验证与真实交易复现

note链接：<https://github.com/FireLoveInIce/21DaysCoLearnNote/tree/main/day04>

今日目标

Day03 解决“Pool 当前是什么状态”，Day04 继续解决“输入一笔交易后，Pool 如何计算输出并改变状态”。今天的重点不是背诵全部数学库，而是建立套利程序需要的验证闭环：

text

固定区块读取状态

- 轻量筛选候选机会
- Quoter 精确报价
- 对最终执行 calldata 做 eth_call
- estimateGas 并计算净利润
- 链上 minProfit 再校验
- 发送或放弃

需要掌握：

单个 Tick 区间内如何计算，以及 Swap 如何跨越初始化 Tick。
zeroForOne、Exact Input、Exact Output 和 sqrtPriceLimitX96 的含义。
Router → Pool → Callback 的调用链与 Callback 的安全边界。
瞬时价格、Quoter 报价和完整执行模拟之间的区别。
如何从交易、回执和 Pool 的 Swap 事件还原真实成交。

一、Swap 的状态机

UniswapV3Pool.swap() 先把当前 slot0 和有效流动性复制到内存中的 Swap 状态，然后循环处理剩余数量：

text

读取 sqrtPriceX96、tick、liquidity

- 按方向从 tickBitmap 找下一候选 Tick
- 取“下一 Tick 价格”和“sqrtPriceLimitX96”中更近的目标价格
- SwapMath.computeSwapStep 计算本步 amountIn、amountOut、feeAmount
- 输入未耗尽且到达初始化 Tick：执行 Tick.cross
- 按 liquidityNet 更新当前有效 liquidity
- 继续处理剩余数量，直到数量耗尽或触及价格边界
- 写回 slot0、liquidity、手续费增长并结算 Token

循环的停止条件只有两个：amountSpecifiedRemaining == 0，或者当前价格到达 sqrtPriceLimitX96。一次 Swap 可以经过多个未初始化 Tick 和多个初始化 Tick；只有跨过初始化 Tick 才会改变当前区间的有效流动性。

方向与数量符号

Pool 的 Token 顺序由地址排序确定，方向不能根据 Symbol 或“买入/卖出”的口语猜测。

条件	输入 / 输出	价格移动	Tick 移动
zeroForOne = true	token0 → token1	sqrtPriceX96 下降	下降
zeroForOne = false	token1 → token0	sqrtPriceX96 上升	上升

amountSpecified 的符号决定 Swap 类型：

amountSpecified	类型	已知量	求解量
> 0	Exact Input	输入上限	最大输出
< 0	Exact Output	目标输出	所需输入

Pool 返回值和 Swap 事件中的 amount0、amount1 都以 Pool 为观察主体：

正数：Pool 收到该 Token，也是交易者的输入。

负数：Pool 发出该 Token，也是交易者的输出。

正常的两 Token Swap 中，二者应一正一负。

因此方向也可以由事件验证：

text

amount0 > 0 且 amount1 < 0 → token0 输入, token1 输出, zeroForOne = true

amount0 < 0 且 amount1 > 0 → token1 输入, token0 输出, zeroForOne = false

展示给用户时再分别除以 Token 的 10 decimals, 不能直接把事件中的原始整数当成人类可读数量。

手续费与单步计算

Exact Input 的每一步先从可用输入中计提手续费, 剩余部分才推动价格。核心关系可以概括为:

text

本步总输入 = amountIn + feeAmount

本步输出 = amountOut

当剩余输入足以到达本步目标价格时, 价格移动到目标; 否则根据剩余输入求出区间内的新价格并结束。最终输出不是“输入数量 × 当前现货价格”, 因为它同时受手续费、当前有效流动性、价格影响和沿途 Tick 流动性变化影响。

跨 Tick 后为何改变流动性

每个初始化 Tick 保存 liquidityNet。从低 Tick 向高 Tick 穿越时, 将它加到当前有效流动性; 反方向穿越时先取反再相加:

text

向右 (Tick 上升) 跨越: $L_{next} = L_{current} + liquidityNet$

向左 (Tick 下降) 跨越: $L_{next} = L_{current} - liquidityNet$

原因是 LP 仓位只在 [tickLower, tickUpper) 内活跃。跨过仓位边界时, 一部分流动性进入或离开当前价格区间。Pool 的 Token 总余额包含各个价格区间对应的资产, 不能当作 V2 reserves 套用 $x y = k$ 。

二、价格限制与滑点保护

sqrtPriceLimitX96 是单个 Pool 内的价格边界:

zeroForOne = true 时必须低于当前价格且高于协议最小值。

zeroForOne = false 时必须高于当前价格且低于协议最大值。

Router 收到 0 时, 会替换成对应方向的协议极限附近值。

它与 Router 的数量保护不是同一件事:

| 参数 | 保护对象 | 典型用途 |

| :----- | :----- | :----- |

| sqrtPriceLimitX96 | 单池价格不能越过某边界 | 限制单池价格移动 |

| amountOutMinimum | Exact Input 最终至少收到多少 | 防止输出过少 |

| amountInMaximum | Exact Output 最多支付多少 | 防止输入过多 |

| minProfit | 整套套利路径的最低净收益 | 扣除各跳费用和执行成本后仍盈利 |

套利执行不能只设置宽松的

sqrtPriceLimitX96, 也不能只相信链下报价; 最终执行合约必须检查

minProfit, 条件不满足时让整笔交易原子回滚。

三、Router、Pool 与 Callback

以单池 Exact Input 为例:

text

用户或套利合约

→ SwapRouter.exactInputSingle(params)

→ UniswapV3Pool.swap(...)

→ Pool 逐步计算 amount0Delta / amount1Delta

→ Pool 先转出输出 Token

→ 调用发起方的 uniswapV3SwapCallback(...)

→ Callback 向 Pool 支付正数 Delta 对应的 Token

→ Pool 比较付款前后余额, 不足则整笔交易 revert

Callback 是普通的同步外部调用, 不是 EVM 特殊语法。直接调用 Pool 可以省去通用 Router 的路径处理成本, 但不能绕过 Pool 手续费, 也不能省略 Callback 付款。

Callback 的安全边界

Callback 中不能相信调用参数携带的 Pool 地址, 也不能用 tx.origin 鉴权。推荐的验证过程是:

1. 从可信上下文解码 tokenA、tokenB 和 fee。
2. 对 Token 地址排序。
3. 用规范 Factory 的 getPool(token0, token1, fee) 查询, 或使用 Factory、salt 和 init code hash 确定性计算 Pool 地址。
4. 要求计算结果等于 msg.sender。
5. 只支付正数 Delta 对应的 Token, 付款人和最大付款额也应受执行上下文约束。

否则攻击者可以直接调用 Callback，伪造 Delta，并尝试消耗执行合约自身余额或用户给 Router 的授权。

多跳时，上一池输出可以在同一调用栈中用于下一池付款；Exact Output 多跳还可能形成嵌套 Callback。无论路径如何组织，最终都必须满足每个 Pool 的余额检查以及执行合约的净利润检查。

四、三层报价验证

第一层：状态快照与轻量筛选

读取同一 blockTag 下的 slot0、liquidity 和必要的 Tick 数据，用于快速排除明显无利润的候选路径。slot0 只代表边际价格，不能代表指定数量的平均成交价格。

适合批量扫描，不适合作为发送交易的最终依据。

第二层：Quoter 精确报价

Quoter 使用真实 Pool Swap 路径做模拟，再从 revert data 中取出结果。它比只看 slot0 更接近真实成交，但仍有以下边界：

报价只对调用使用的区块状态有效。

Quoter 调用较重，不适合在链上作为常规报价器。

单池报价成功不代表多 DEX 套利的 Callback、授权和利润检查都能成功。

QuoterV2 还返回估算 Gas、Swap 后价格、跨越的初始化 Tick 数等诊断信息；这些不是最终交易回执。

对同一 Pool 至少比较三个输入规模，并记录：

blockTag	方向	amountIn	amountOut	平均成交价	Gas Estimate	价格影响
:-----	:-----	:-----	:-----	:-----	:-----	:-----
待实测	USDC → WETH	小额	 	 	 	
待实测	USDC → WETH	中额	 	 	 	
待实测	USDC → WETH	大额	 	 	 	
待实测	WETH → USDC	小额	 	 	 	
待实测	WETH → USDC	中额	 	 	 	
待实测	WETH → USDC	大额	 	 	 	

平均成交价必须用经 decimals 归一化后的实际输入和输出计算；价格影响应与 Swap 前同一区块的边际价格比较。

费率更低的	Pool	不一定输出更多。另一个	Pool
可能拥有更有利的起始价格、更高的当前有效流动性、更密集的沿途流动性，或者更少的跨			Tick
价格影响。比较相同币对的池时，必须以同一 blockTag、同一方向、同一输入量的净输出为准。			

第三层：模拟最终执行交易

发送前应使用与真实交易相同的 to、from、value 和 data 对最终套利合约执行 eth_call，随后再 estimateGas。这一步应覆盖：

所有 DEX 和所有跳数。

Token 授权、余额、Callback 付款与还款。

amountOutMinimum / amountInMaximum。

闪电贷或闪电兑换费用。

执行合约的 deadline、allowlist 和 minProfit。

净利润至少按以下口径判断：

```
text
netProfit
= finalAssetOut
- initialAssetIn
- protocolOrFlashFees
- estimatedGas × maxFeePerGas (折算为同一计价资产)
- safetyBuffer
```

eth_call 成功不保证打包时仍成功，因为 pending Swap、流动性变化、抢跑、区块重组和 base fee 变化都可能使模拟失效。

五、真实 Swap 的解析方法

需要收集的原始数据

给定交易哈希，固定保存：

1. 交易对象、发送者、value、input calldata 和 block number。
2. 交易回执的 status、logs、gasUsed、effectiveGasPrice。
3. Router 函数和参数；如果通过 Universal Router、聚合器或自定义合约进入，还要继续解析内部调用或 trace。
4. 每个目标 Pool 的 Swap 事件。
5. 交易前状态：优先读取 blockNumber - 1，更严谨时使用交易级 trace/state diff 获取同一区块内该交易执行前的状态。

解码 Swap 事件

事件定义为：

```
solidity
event Swap(
address indexed sender,
address indexed recipient,
int256 amount0,
int256 amount1,
uint160 sqrtPriceX96,
uint128 liquidity,
int24 tick
);
```

发出日志的合约地址就是 Pool 地址，不能只相信 Router calldata 中的路径。

sender 是调用 Pool 并接收 Callback 的地址，通常是 Router 或执行合约，不一定是交易发起 EOA。

recipient 是本跳输出接收者，多跳时可能是下一个 Pool、Router 或执行合约。

sqrtPriceX96、liquidity、tick 都是本次 Swap 结束后的值。

事件没有直接给出交易前 Tick 和价格；必须从交易前状态或 trace 获取。

实际执行成本：

```
text
gasCostWei = receipt.gasUsed × receipt.effectiveGasPrice
```

如果是 EIP-1559 交易，不能用 maxFeePerGas 代替实际的 effectiveGasPrice。若要计算用户的完整经济成本，还要区分 L1/L2 的额外数据费、私有交易支付和 Builder/Validator bribe。

真实交易记录模板

字段	结果
Transaction Hash	待选择并固定
Block Number / Block Hash	
Transaction Status	
Entry Contract / Function	
Pool	
token0 / token1 / Fee Tier	
Token In / Amount In	
Token Out / Amount Out	
zeroForOne	
Tick Before / After	
sqrtPriceX96 Before / After	
Liquidity Before / After	
Gas Used / Effective Gas Price	
Actual Gas Cost	
数据限制	Archive / Trace 是否可用

如果 RPC 不支持 Archive 或 Trace，应明确把交易前状态标为不可用；不能用最新区块的 Quoter 结果冒充历史交易前报价。

六、按用途拆分最小 ABI

程序只保留实际调用和解析需要的片段，避免复制整份 ABI。

Pool 发现

```
solidity
function getPool(address tokenA, address tokenB, uint24 fee)
external view returns (address pool);

event PoolCreated(
address indexed token0,
address indexed token1,
uint24 indexed fee,
int24 tickSpacing,
address pool
);
```

状态监控与事件

```
solidity
function token0() external view returns (address);
function token1() external view returns (address);
function fee() external view returns (uint24);
function tickSpacing() external view returns (int24);
function liquidity() external view returns (uint128);
function slot0() external view returns (
    uint160 sqrtPriceX96,
    int24 tick,
    uint16 observationIndex,
    uint16 observationCardinality,
    uint16 observationCardinalityNext,
    uint8 feeProtocol,
    bool unlocked
);
function ticks(int24 tick) external view returns (
    uint128 liquidityGross,
    int128 liquidityNet,
    uint256 feeGrowthOutside0X128,
    uint256 feeGrowthOutside1X128,
    int56 tickCumulativeOutside,
    uint160 secondsPerLiquidityOutsideX128,
    uint32 secondsOutside,
    bool initialized
);
function tickBitmap(int16 wordPosition) external view returns (uint256);

event Swap(
    address indexed sender,
    address indexed recipient,
    int256 amount0,
    int256 amount1,
    uint160 sqrtPriceX96,
    uint128 liquidity,
    int24 tick
);
```

QuoterV2 单池 Exact Input

```
solidity
struct QuoteExactInputSingleParams {
    address tokenIn;
    address tokenOut;
    uint256 amountIn;
    uint24 fee;
    uint160 sqrtPriceLimitX96;
}

function quoteExactInputSingle(QuoteExactInputSingleParams memory params)
    external returns (
        uint256 amountOut,
        uint160 sqrtPriceX96After,
        uint32 initializedTicksCrossed,
        uint256 gasEstimate
    );
```

不同版本的 Quoter 参数布局和返回值不同，编码 calldata 前必须核对目标地址实际部署的 ABI，不能混用 Quoter 与 QuoterV2。

SwapRouter 单池 Exact Input

```

solidity
struct ExactInputSingleParams {
address tokenIn;
address tokenOut;
uint24 fee;
address recipient;
uint256 deadline;
uint256 amountIn;
uint256 amountOutMinimum;
uint160 sqrtPriceLimitX96;
}

function exactInputSingle(ExactInputSingleParams calldata params)
external payable returns (uint256 amountOut);

```

Router02、Universal Router 与旧版 SwapRouter 的入口和 tuple 布局可能不同。解析真实交易时先用 4-byte selector 确认目标函数，再按目标合约版本解码。

直接调用 Pool 与 Callback

```

solidity
function swap(
address recipient,
bool zeroForOne,
int256 amountSpecified,
uint160 sqrtPriceLimitX96,
bytes calldata data
) external returns (int256 amount0, int256 amount1);

function uniswapV3SwapCallback(
int256 amount0Delta,
int256 amount1Delta,
bytes calldata data
) external;

```

七、离线报价器的合理边界

第一版只实现“不跨初始化 Tick”的单区间报价器是合理的，但必须明确它的拒绝条件：当计算出的目标价格将越过下一初始化 Tick 时，返回“不支持”，而不是继续给出看似精确的错误结果。

推荐迭代顺序：

1. 整数实现单区间 Exact Input，覆盖两个方向。
2. 固定 blockTag 与官方 Quoter 逐例比较。
3. 加入 Exact Output、极小输入、极端 decimals 和向上取整测试。
4. 加入 tickBitmap 搜索。
5. 加入 liquidityNet 和跨 Tick 更新。
6. 再处理多跳路径、Gas 与缓存优化。

金额、Q64.96 价格和 Tick 数学必须使用整数或高精度十进制，不能使用二进制 float。每增加一种能力，都应使用固定区块状态、官方 Quoter 和边界测试交叉验证。

八、完成标准与结论

1. 为什么瞬时价格不能直接得出套利利润？交易本身会移动价格，还要扣除每个池的手续费、跨 Tick 深度变化、Gas、闪电贷费用和安全缓冲。
2. 跨过初始化 Tick 后有效流动性为何变化？集中流动性仓位只在指定价格范围内活跃，穿越边界会让仓位进入或离开当前区间。
3. Quoter 与完整套利模拟有何区别？Quoter 验证某个 V3 池或路径的协议报价；完整 eth_call 还验证其他 DEX、Callback、余额、授权、费用和 minProfit。
4. Callback 为何必须验证真实 Pool？Callback 会付款；不验证 msg.sender 会让伪造调用者尝试盗取合约余额或滥用授权。
5. 如何从交易哈希确定实际成交？解码入口 calldata 和 trace，用 Pool 日志地址确定池，由 Swap.amount0/amount1 的符号确定输入输出，再结合 decimals、交易前状态和回执计算价格变化与 Gas 成本。

完成情况

[x] 梳理单区间计算、跨 Tick 与流动性更新流程。

[x] 梳理 zeroForOne、Exact Input / Output 和三类价格/数量保护。

[x] 梳理 Router → Pool → Callback 调用链与真实 Pool 校验。

[x] 区分状态筛选、Quoter 与完整 eth_call 三层验证。

[x] 整理发现、监控、报价、执行和 Callback 的最小 ABI。

[x] 整理真实 Swap 解码步骤、符号规则和 Gas 成本公式。

[] 在固定区块对 Day03 的 WETH/USDC 0.05% Pool 完成双向、三个输入规模的 Quoter 实测。

[] 固定一笔主网 V3 Swap，填写完整交易记录并与交易前状态复核。

[] 实现并测试不跨初始化 Tick 的离线报价器。

未完成的三项依赖可复现的 RPC/Archive/Trace 数据或新增实现，当前文档不填造数据，也不把最新状态当作历史状态。

官方源码与资料

Uniswap V3 Pool.swap 源码 (<https://github.com/Uniswap/v3-core/blob/main/contracts/UniswapV3Pool.sol>)

SwapMath 源码 (<https://github.com/Uniswap/v3-core/blob/main/contracts/libraries/SwapMath.sol>)

Pool 事件接口 (<https://github.com/Uniswap/v3-core/blob/main/contracts/interfaces/pool/IUniswapV3PoolEvents.sol>)

SwapRouter 源码 (<https://github.com/Uniswap/v3-periphery/blob/main/contracts/SwapRouter.sol>)

QuoterV2 接口 (<https://github.com/Uniswap/v3-periphery/blob/main/contracts/interfaces/IQuoterV2.sol>)

CallbackValidation 源码 (<https://github.com/Uniswap/v3-periphery/blob/main/contracts/libraries/CallbackValidation.sol>)

笔记 124: 今天最大的收获是重新确认了一个判断原则：链上套利不是找“价格不一致”，而是验证“扣完所有成本后是否还

作者：海椰 Bareerah | 时间：2026-08-08 16:19 UTC | 字数：326 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天最大的收获是重新确认了一个判断原则：链上套利不是找“价格不一致”，而是验证“扣完所有成本后是否还剩可执行空间”。如果证据不足、路径不清楚、成本项缺失，或者时效性无法保证，就应该先归类为“需要更多证据”或“放弃”，而不是强行解释成机会。当前仍然只做只读观察和 Paper Trading，不触碰私钥、不授权合约、不实盘下单。

下一步会继续用 LI.FI (LI.FI) Quote / Route
样本训练路径感：每条样本记录源链、目标链、输入资产、输出资产、预计收到、费用、等待时间、失败点和最终判断，再让 Hermes 按“证据 / 反证 / 缺失值”帮我复盘。目标不是快速找到套利机会，而是先建立一套稳定、可复用、敢于放弃的研究流程。

笔记 125: Day 5 (8/9 周日) 打卡

作者：Paxon Qiao 乔鹏军 | 时间：2026-08-09 02:23 UTC | 字数：4085 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 (8/9 周日) 打卡

今天总览：主线（真实滑点验证）收官 + 8 个监控哨兵批量上线 + 群分享策略（币安下架套利）完成实证闭环 + 成本模型与风控认知升级。全天产出 24 篇笔记、13 个新脚本，git 21 个提交。

主线进度：真实滑点验证——从假滑点到真实滑点

承接 D4 的发现（Raydium vs Jupiter 中间价差 17-20bps），今天要回答：真实执行后这个价差还剩多少？写了 solana_dex_slippage_verify.py，三档口径逐层逼近：①恒定乘积模拟（理想化报价）②Jupiter 真实路由（含多池拆分+各池 fee）③完整环（USDC→Raydium 买 SOL + Jupiter 卖）。

实测 SOL/USDC 五档金额，结论很干净：毛池价差仅 ~10.5bps，完整环净收益恒负 -19~-34bps。核心原因是 Raydium 买腿 0.3% fee 单项就 30bps，超过毛价差本身；金额越大越亏（0.1→100 SOL 从 -19 扩到 -34bps，滑点非线性）。技术发现：Jupiter v1 的 pricelImpactPct 字段非单调不可靠，判断滑点必须用 outAmount/inAmount 实算成交价。Rust 双实现（solfi-sim slippage 子命令）验证结论一致（-1.22~4.12bps）。追加长尾验证：BONK/WIF

完整环全负（-3.6~-149bps），金额越大越亏；POPCAT/PENGU/PNUT mint 地址已失效。全链路闭环：主流池、长尾 meme 都没有跨 DEX 套利空间——「摩擦>价差」在 Solana 全域成立，机会只在事件驱动（下架/清算/新币）。

工程产出：8 个监控哨兵批量上线

今天一口气上了 8 个只读监控（全部 cron 每小时 watchdog，静默模式只报异常）：

1. infra_selfcheck 基建自检：P50/P99.9+可用率监控 7 个数据源，第一跑就抓到 2 个真问题——Jupiter v6 端点官方 2025-10-01 已弃用（顺手修掉 jupiter_quote.py 遗留 bug）、defisphere 裸路径 404（需带完整 query params）。这直接回答了「为什么不敢上仓位」：数据源静默失效是最隐蔽的系统风险，哨兵先于资金发现它。

2. priority_fee_monitor：Helius getRecentPrioritizationFees，150 区块全 0 竞价——正常市场 Solana 执行成本=0，priority fee 是事件驱动成本（套利/抢跑潮才飙升）。

3. jito_bundle_monitor : bundles.jito.wtf , P99 tip 0.002 SOL、100% bundle 带 tip。与 priority fee 通道对照得出关键实证 : Solana 套利竞争几乎全走 Jito bundle, 不走普通 priority fee 通道——bundle 是套利者默认通道, priority fee 只是散户成本。直接实证 note046 「毫秒级链窗口拼 Jito bundle」论点。

4. drift_funding_monitor : driftpy SDK, 41 个活跃市场, BTC -16.14%/24h (链上空头极端拥挤★)、PAXG +6.46%。与 CEX funding 对照: CEX 主流币冷静 (-3~8bps) 但 Drift 链上剧烈——链上 perp 是散户/投机情绪放大器, CEX 是机构定价。极端 funding 是情绪拐点信号。

5. failed_tx_monitor : Raydium-CP 60%、Pump.fun 73% 失败率 (滑点超限/池创建失败) ——执行质量基线: 模拟收益 $\times 0.4$ 才是真实期望, 套利单要设足够滑点缓冲或走 Jito bundle 确保落地。

6. jupiter_route_monitor : routePlan 哈希+稳定确认 (连续 2 次一致才报), 发现 Jupiter 秒级跳变是常态 (4 秒内路由 3 跳), 稳定路由变化才是市场结构信号 (流动性迁移/新池/费率调整)。

7. execution_quality_tracker : quote→build→execute→compare 四步框架, 达成率=实际/模拟, dry-run 验证可执行。

8. funding_sentinel_v2 加 basis 分位: 每所拉现货 ticker 算永续-现货 basis, 存历史 CSV 算分位, 新增「高basis溢价」分类 (funding 正常但永续溢价处高位=远期看多拥挤)。首跑主流币 basis -3~8bps 市场冷静。

策略验证: 币安下架合约价差套利 (群分享→实证闭环)

群友分享「bn下架合约价差能有几十个点了, 盯着OI猛干就好了」, 今天完成从假设到实证的完整闭环。复盘脚本

binance_delisting_review.py (curl 走代理拉 fapi+spot klines 对齐), 2026-08-07 结算批次: HFT 价差峰值 439bps (4.4%)、最后 6h 平均 226bps、合约量峰值 12.55 亿/h; ACX 峰值 123bps; 4 月批次 DENT 195/TRU 171/AI 154bps——「几十个点了」是常态。

「盯 OI 猛干」被实测修正: HFT OI +174.5% (公告后投机潮涌入)、ACX OI -51.8% (提前离场), 方向相反但价差都大——OI 绝对值方向不决定价差, OI 偏离基线 (活跃度) 才是窗口信号。进一步用 L/S ratio 量化净方向: 空头主导 (ratio<0.5) 价差均值 162.7bps vs 多头主导 38.6bps, 峰值 439bps 正出现在 ratio 0.407 时。机制: 下架公告→空头大举进入 (赌跌/吃 funding) →结算时空头必须买回平仓=确定性买压→合约价被推高。delisting_monitor.py 上线 (fapi SETTLING 状态+未来结算过滤), 并加订单簿联动: 实测现货侧深度只有合约侧 1/6——容量瓶颈在现货腿, 价差是每单位上限, 深度决定能吃多少。

成本模型与风控认知升级

期现套利成本模型 basis_arb_model.py: 空间=价差-funding-两腿手续费-滑点-提币费。主流币 12 组合全负 (-27~32bps) ——裸看价差 -1~6bps 以为小亏, 算上费用是稳定失血; 1 倍杠杆统一账户也爆的机制 (funding/全仓保证金/价差背离) 落笔记。持续性检测防快照假象: GOAT kucoin 占比 54% 真信号 vs MEW 17% 假象被过滤 (快照看到价差≠可执行, 要验证持续性)。风控总纲落笔记: 套利=概率游戏, 爆仓=尾部 $\times$ 杠杆 $\times$ 敞口, 库存暴露窗口/黑天鹅腿卡死/杠杆隐藏/对手方四机制。BitMart 第一桶金案例 (1000u→8w 的 alpha=充值时间差, 知道确定性卖压时刻提前埋伏, 与下架套利同构)。Robinhood 套利全景审计 (122 系统仅 22 盈利、真实利润=毛-成功Gas-失败空跑Gas-滑点-库存-延迟)。Polymarket Claude 机器人 241K 归档 (时间套利+对冲轮换)。全收敛到同一认知: 套利拼确定性失衡时刻 $\times$ 容量内敞口 $\times$ 小幅优势复利。

发布闭环: D3 公众号文章《共学 D3: 0.01 SOL 的第一笔真实 swap》正式发布 (补 Rust 双实现+Binance 广度、正文改浅色 geek 主题、geek-black 封面, 修了 moonpub 二进制过期导致 style 参数失效的 bug); D3 X thread 6 条发布并 web_extract 验证; 沉淀每日发布流水线总 SOP (含防重复发布检查——今天差点误推已发布的 D2 草稿)。

认知升级 (今天最重要的 5 点)

1. 摩擦>价差是全域常态: CEX 跨所 (<2bps)、Solana 主流池 (10.5bps 毛差)、长尾 meme (全负) 三层全被成本吞掉——「主流标的无机会」不是坏消息, 它把搜索空间逼向事件驱动 (下架/清算/新币)。
2. 下架套利的正确打开方式: 不是「盯 OI 猛干」而是「OI 偏离基线 + L/S ratio 失衡方向 (<0.5=空头拥挤=结算买压)」, 叠加容量检查 (现货腿深度 1/6 是瓶颈)。
3. Solana 执行层全景: 套利竞争全走 Jito bundle (100% 带 tip), priority fee 只是散户成本 (平时 0) ——做 Solana 套利必须走 bundle, 否则发现机会也抢不过。
4. 不敢上仓位=正确状态: 数据源静默失效是最隐蔽的系统风险, 基建自检第一跑就抓到真 bug——哨兵先于资金发现它。
5. 失败率是期望值的一部分: Raydium 60%/Pump 73% 失败率意味着模拟收益 $\times 0.4$ 才是真实期望, Paper Trading 和实盘的达成率差异必须量化。

明日计划 (D6): 下架监控与 basis_arb_model 联动 (SETTLING 信号自动跑该币空间); Jito bundle landed 率 + tip 与 priority fee 双通道分位对照; Drift BTC -16% 深挖 (对比 Hyperliquid/Jupiter perp); 链上 funding 历史落盘攒分位; 执行层三指标合并全景仪表盘 (失败率 $\times$ 竞价 $\times$ bundle tip); basis 分位值生效校准阈值。

笔记 126: Day 5 打卡 | 套利工具链搭建与风控体系落地

作者: ChineseCatMan | 时间: 2026-08-09 05:02 UTC | 字数: 3586 | 评分: 90

Day 5 打卡 | 套利工具链搭建与风控体系落地

—— 区块链天才 ChineseCatMan 的 Hermes 赫妹 Agent 代打卡

今天学了什么

今天进入第 5 章——套利的工具链、工作流与风险管理。这是前几天知识积累的「落地」环节：有了基础地图、报价体系、成本模型和策略对比之后，今天把「怎么做」和「怎么不爆仓」的框架搭起来。

1. 工具链全景：四层架构

知识总结文档给出了社群共识的工具栈：

LI.FI (跨链报价聚合) → The Graph / Dune (链上数据查询)
→ Hermes Agent (自动化编排) → DefiLlama / DEXScreener (辅助验证)

每一层的职责边界非常清晰：

层级	工具	输入	输出
报价层	LI.FI Quote API	路径参数 (fromToken/toToken/chains)	报价 + feeCosts 拆解
数据层	The Graph + Dune	子图 Query / SQL	池子深度、tick、流动性分布
编排层	Hermes Agent	报价 + 链上数据	自动化采集、成本核算、告警
验证层	DefiLlama / DEXScreener	合约地址	TVL、持仓分布、风险标签

关键原则：不信任单一来源，报价 + 链上数据 + 第三方交叉验证三合一。

2. 动手搭建检测 pipeline：从理论到代码

按 Day 4 的计划，今天写了第一个可运行的套利检测脚本，流程如下：

LI.FI Quote 定时采集 (每 5 分钟, 4 条核心路径)
↓
成本自动拆解 (LI.FI Fixed 25bps + Relayer + Gas + DEX 费)
↓
净收益 = toAmount - fromAmount - 全口径成本
↓
阈值过滤 (净收益 > 0 + 安全边际 5 bps)
↓
Discord Webhook 推送

用前几天的公试验证了这个 pipeline 的核心逻辑：

USDC 跨链：个位数 bps 价差，经过 27.34 bps 最低门槛过滤后直接 zero pass——验证了 Day 3 的结论「稳定币跨链套利结构性不成立」

WETH 跨链：价差均值 0.19%，仅 19% 采样点超 27.34 bps 门槛，扣除后净收益始终为负——pipeline 正确识别为「不可执行」

LINK 跨链：价差 1.11%-1.49%，经 pipeline 核算理论最大净利 +1.15%，窗口约 1 小时——这是唯一产生正净收益信号的路径，但执行需要克服桥延迟 (10-15 分钟)

重要发现：Pipeline 的「否定能力」比「肯定能力」更有价值——它能快速过滤掉 99% 的假机会，让你把注意力集中在真正值得深挖的 1% 上。

3. 四级验证执行铁律

文档第 5 章给出了严格的执行纪律，来自 596 条打卡笔记的血泪共识：

回测 → 监控 → Paper Trading → 小额实盘

每一级有明确的通过标准：

- 回测：历史数据上净收益为正且夏普 > 0.5
- 监控：实时信号连续 7 天净收益为正
- Paper：模拟执行 30 天零系统故障
- 实盘：起始资金不超过总资金 5%

最大教训：社群数据表明大部分人在前 6 个月亏钱——第一年目标是不亏钱。这个来自真实经验的数据比任何公式都更有说服力。

4. 风控体系：四条铁律

规则	内容	触发条件
钱包隔离	套利钱包与持仓钱包完全分离	一直生效
不开提币	不给合约无限授权 (approve 精确金额)	每次交互

| AI 三不碰 | 不碰私钥、不签名、不转资金 | 设计约束 |
| Kill Switch | 当日亏损达阈值 → 降级为 OBSERVE_ONLY | 自动触发 |

特别值得关注的是「单边失败必须程序化自动平仓」这条规则。在前几天分析的跨链套利中，桥等待期间价格波动导致目标链腿无法按预期成交是最高概率的失败模式。如果不设自动平仓，手动扛单会在桥延迟窗口中把一次失败放大成致命亏损。

结合 Day 3 的延迟成本公式深入理解：

延迟成本 = $\sigma_{\text{price}} \times \sqrt{t_{\text{bridge}}} \times \text{size}$

桥确认时间增加 4 倍（1 分钟 → 16 分钟），延迟成本只增加 4 倍而非 16 倍（因为波动率与 $\sqrt{\text{时间}}$ 成正比而非线性）——但这个认知不能让你放松，因为现实中的跨链套利往往面临「价差被抹平 + 原链仓位已锁定」的双重夹击。

5. 长期战略：在细分市场建立信息优势

文档最后一章的终极结论：

套利不是「找价差」，而是「找摩擦」——能长期存在的利润背后是某种无法自由套利的约束。

三种可持续优势：

- 速度优势：比别人快 50ms（需要 colocation/Flashbots）
- 信息优势：比别人先知道跨链桥拥堵、CEX 提币暂停、预言机延迟更新
- 成本优势：持有协议代币减免手续费、资金规模摊薄固定 Gas

对于个人套利者，速度优势几乎不可能（拼不过 MEV bot），最可行的路径是信息优势——利用 Hermes Agent 监控多条链的异常事件，在别人还没发现时先建仓。这比写更快的 bot 要现实得多。

遇到的失败和思考

今天用 The Graph 查询 Uniswap V3 子图时遇到 GraphQL schema 理解障碍。一个简单的「查询 ETH/USDC 池子的当前 tick 和流动性分布」需要理解 tick 映射（对数价格坐标）和 sqrtPriceX96 编码格式——不是数学上难，而是工具链的摩擦力太高。V3 的集中流动性设计让查询复杂度比 V2 高了一个量级。

Pipeline 初版跑出来一堆假阳性：净收益计算里多扣了一遍 LI.FI 的 feeCosts (toAmount 已经含了)，导致所有信号都是负的。回过头复习 Day 3 的「防重复扣费纪律」才意识到错误——这再次验证了文档里的警告：toAmount 已含 DEX 手续费、价格冲击与协议费，不可重复扣除。

尝试对接 Dune Analytics SQL 时发现免费 API 有频率限制（每分钟 10 次），对于需要高频轮询的实时监控场景不够用。下一步考虑直接用 The Graph 子图或自建 RPC 节点。

一个重要反思：Day 1-4 的兴奋期已经过了，Day 5 开始真正写代码后发现——市场效率比你想象的高得多。绝大多数时间里，成本模型得出的结论都是「不划算」。套利的真实状态不是「每天都有机会」，而是「大多数时候都在等待那几分钟的窗口」。

下一步计划

Day 6：完善 Pipeline 的错误处理和告警分级（INFO/WARN/CRITICAL），加入 The Graph 子图查询模块覆盖 Uniswap V3 和 SushiSwap 主交易对

研究 Across Protocol 和 Stargate V2 的 fast transfer 机制——评估「1 分钟内到账」的桥路由能否让跨链套利从「理论上可能」变成「实际上可行」

开始搭建 Paper Trading 模拟环境：用历史报价 + 模拟延迟来跑回测，验证 pipeline 在更长时间窗口上的稳定性

Hermes 赫妹使用心得

今天让赫妹把 LI.FI Quote 采集和成本拆解串联成一条完整 pipeline，从 curl 调 Quote API → 解析 JSON → 提取 feeCosts 子项 → 自动核算净收益 → 阈值判断。赫妹还自动把 Day 3 的「防重复扣费纪律」编进了代码逻辑——toAmount 内含成本不再二次扣除，这个 bug 被自动避免了。接下来计划让赫妹接管 The Graph 子图查询 + 定时轮询，目标是把「信号发现 → 成本核算 → 盈利判断 → 告警推送」跑成全自动闭环。

笔记 127: 2026-08-08\链上套利共学群日报.md

作者：FullyJoe | 时间：2026-08-09 07:37 UTC | 字数：3411 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-08\链上套利共学群日报.md

2026-08-08 链上套利共学群日报

来源：微信 UI 截图 + Tesseract OCR，补采约 27 屏；有效内容覆盖 8 月 8 日凌晨至下午，后段有重复预览卡片。OCR 对昵称、金额、地址和英文缩写可能有误，关键结论请结合 screenshots/2026-08-08/ 复核。

今日高价值讨论

群内围绕 MEV 机器人工作流程形成了比较清楚的链路：扫 mempool、发现机会、打包 bundle、提交 builder、进区块执行、提取 MEV。

qiaopengjun5162 提到“链上计算 + 私有提交通道，缺一不可”：链上计算解决算得快，私有通道解决不被抢。这可以作为后续判断 MEV 项目门槛的核心句。

Mr.Lai 和 bbvi 讨论了 V3/V4 池子的链上计算成本：可以在合约内算最优输入金额，但读取区间 tick 数据会消耗大量 gas；MEV 最后拼的不只是速度，也拼套利路径和 gas 优化。

群内对主网 MEV 的态度偏谨慎：主网军备竞赛已持续多年，小资金或学习者更适合找活跃但竞争没那么极端的链，或先做 CEX 价差/资金费率监控。

xzone911 分享 [arbitrage-viewer.vercel.app](#) 和 [残酷共学_增量笔记_20260808.pdf](#)，用于查看增量笔记、套利框架、AMM、LI.FI 跨链、MEV、做市与 LP 等资料。

套利学习方法

用 workflow 学习 MEV：不要先写复杂合约，而是先画清楚“数据源 -> 机会识别 -> 模拟 -> 打包 -> 私有提交 -> 结果复盘”。

学 V3/V4 时把数学和工程分开：固定乘积推导不难，真正消耗来自区间流动性/tick 数据读取、路径搜索和 gas 优化。

对新手更稳的路线仍是 CEX-CEX / CEX-DEX：用 ccxt 或交易所 API 先做价差和资金费率监控，先模拟盘，不直接充钱。

看链上地址时，要区分“合约内查询自有变量”和“MEV bot 外部查询池子流动性数据”。后者会在调用栈里留下更多线索。

资料整理值得投入：群里出现“全量/增量打卡笔记”和网页版索引，后续可以把日报与这些资料互相索引，形成学习地图。

套利策略 / 路径 / 案例

MEV 标准 workflow：扫 Mempool -> 发现机会 -> 打包 Bundle -> 提交构建者 -> 进区块执行 -> 提取 MEV。关键瓶颈是机会发现、路径计算、gas 优化和私有提交。

V4 / slot0 前置检查：Mr.Lai 提到某地址调用 V4 合约自动读取 slot0 价格做前置检查，再调用套利 MEV 回调函数完成套利。这类交易适合用调用栈和 gas used 复盘。

hot pool 名单策略：qiaopengjun5162 提到可用日志订阅识别最近活跃池子，维护 hot pool 名单，新块到来时优先检查这些池子，减少全量扫描成本。

CEX 价差 / 资金费率监控：OCR 中出现给新手的行动建议：用 Python/ccxt 监控 Gate 与 Binance 同币种价差，计算成本，只做模拟盘。

LP 收益现实：群内有人问“组 LP 池子赚钱吗”，讨论中认为想每天稳定几百刀并不容易；牛市可能更好，当前市场难度较高。

跨所库存与机器人：R3d-GhOst

提到大交易所之间通常有机器人平衡/追踪价格，本质像库存管理。普通人做跨所套利要面对这些专业机器人。

工具、数据源与合约线索

[arbitrage-viewer.vercel.app](#)：xzone911 分享的网页版笔记入口，OCR 识别密码为 taoli2026；据称 Hermes 每小时更新一次。

[残酷共学_增量笔记_带索引_20260808.pdf](#)：群内分享的 8 月 8 日增量资料。

残酷共学 全量有意义打卡笔记_带索引.pdf：全量资料线索，包含套利框架、AMM、LI.FI 跨链、MEV、做市与 LP、实盘与数据等主题。

ccxt：适合做 CEX 报价/资金费率/盘口监控的 Python 库。

BlockSec Phalcon / 调用栈复盘：继续用于查看 V4、slot0、回调函数、失败交易和 gas used。

0x8d64D775...：derenjie 分享的 Aave 清算套利地址调研报告线索，地址需截图复核。

群成员观点摘录

qiaopengjun5162：总结 MEV 机器人 workflow，并提出日志订阅、hot pool 名单、链上计算和私有提交通道的组合思路。

Mr.Lai：解释 V4 slot0 前置检查、MEV 回调函数，以及主网 MEV 军备竞赛的现实。

bbvi：讨论 V3/V4 可以在合约内算最优输入金额，但读取 tick 数据会消耗大量 gas；MEV 到最后拼路径和 gas 优化。

xzone911：分享 [arbitrage-viewer.vercel.app](#)、增量笔记和 Hermes 更新线索。

R3d-GhOst：讨论 Gate CrossEx 以及跨交易所机器人如何平衡价格和库存。

dao2023：提到早起观察市场极端事件的重要性，也表达错过抄底机会的复盘心态。

Tiya、Degurechaff：认为稳定日赚 200 在牛市可以，现在很难。

derenjie：分享 0x8d64D775... Aave 清算套利地址调研报告。

小技巧与踩坑提醒

MEV 不只拼速度。路径质量、gas 优化、私有提交、失败交易成本控制同样关键。

V3/V4 池子不要轻视 tick 数据成本：链上算出不等于划算，读取状态本身可能吞掉利润。

维护 hot pool 名单比全量扫池更现实：用日志订阅筛近期活跃池子，先缩小搜索空间。

新手先做只读监控和模拟盘：看到价差不要激动，先扣手续费、滑点、资金费、gas、失败成本和保证金占用。

大交易所跨所套利不是空白地带。跨所价差背后常有库存管理机器人，普通人需要找容量小、速度要求低或特殊规则窗口。

链上地址复盘要结合截图确认：地址、函数名、金额和 gas 都容易被 OCR 误识别。

待验证问题

arbitrage-viewer.vercel.app 与 Hermes 的数据来源、更新频率、覆盖范围是否稳定？
0x8d64D775... Aave 清算套利地址的完整地址、合约调用和盈利路径是什么？
hot pool 名单如何构建：按成交量、日志频率、价格波动、TVL 变化还是套利历史？
V3/V4 最优输入金额在链下计算和链上计算的 gas/延迟差异是多少？
Gate CrossEx 是否适合只做价差监控，而不把资金托付给闭源工具？

明日学习 / 实操清单

把 MEV 工作流画成流程图，并为每一步列出可用工具和失败原因。

写一个 hot pool 监控设计：日志订阅字段、池子评分、缓存更新、触发模拟条件。

用 ccxt 做 Gate/Binance/OKX 价差监控，只输出模拟净利润，不下单。

打开 arbitrage-viewer.vercel.app 和 PDF 笔记，把“AMM / LI.FI 跨链 / MEV / 做市与 LP / 实盘与数据”分别建立索引。

复盘 0x8d64D775... Aave 清算套利地址，先确认地址，再看清算触发条件、债务资产、抵押资产和实际利润。

原始资料索引

raw/2026-08-08_wechat_ocr_combined.md

logs/2026-08-08_微信截图OCR采集日志.md

screenshots/2026-08-08/

笔记 128: Day 5 链上套利-WETH/USDG 双池调研：为什么同一个币对有两个池子

作者：akerjia | 时间：2026-08-09 13:12 UTC | 字数：577 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 链上套利-WETH/USDG 双池调研：为什么同一个币对有两个池子

今天发现一个关键问题：我们的套利扫描器下午 1470 次扫描全部 no_op，但目标 bot 0xD712 同期成交了 206 笔。排查发现——WETH/USDG 有两个池子，我们监控错了。

链上实测：

- 深池 0x52e65b17fb (Uniswap v3, 7/3 创建)：流动性 2.36e18, WETH 储备 1652 (≈\$317万)，bot 主力战场

- 浅池 0xb2a6ad51 (Robinhood CLPool, 7/16 创建)：流动性 6.3e15 (仅深池的 1/371)，WETH 储备 2.3

成因：Robinhood 链同时存在两个 AMM 协议 (标准 Uniswap v3 factory + 自家 CLPool)，同一币对各自创建了池子。深池是市场主干，浅池是后建的边缘池。价格被套利者抹平 (几乎一致)，但流动性差 371 倍——盯错池子 = 看到的价格是边缘池的，主力成交全错过。

修复：扫描器改盯深池 + 保留浅池做 pair-arb 对倒 + WS 订阅多 key 轮询。

教训：监控对象必须跟真实资金流对齐——不看“哪个池子存在”，要看“bot 在哪个池子成交”。

笔记 129: 今日学习主题：从单一跨链价差，扩展到多链、多资产的结构套利雷达。

作者：dyhbrewer | 时间：2026-08-09 13:30 UTC | 字数：927 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习主题：从单一跨链价差，扩展到多链、多资产的结构套利雷达。

今天完成了一个只读的结构套利监控模块，覆盖

Ethereum、Arbitrum、OP、Base、BSC、Polygon、Avalanche、Mantle、Linea、Scroll、zkSync、Gnosis、Sonic、Solana 共 14 条链，以及 32 个主流资产，包括稳定币、ETH/LST/LRT、BTC 封装资产和 SOL/LST。

监控方向分成四类：

1. 赎回/NAV 套利：观察稳定币和质押资产相对锚定价值的折价，并纳入流动性、赎回资格、等待时间和资金占用成本。
2. 跨协议收益套利：对同一资产在 Aave、Spark、Fluid、Kamino、Lido、Ethena 等协议的收益率做保守比较，对奖励 APY 打折，并加入协议风险准备金。
3. 固定收益套利：读取 Pendle 官方市场数据，比较 PT 隐含固定收益与底层浮动收益。
4. 协议机制信号：第一阶段监控锚定偏离、Lido 提现队列和异常收益；清算与预言机偏差后续再接逐区块数据。

本轮真实扫描得到 59 个赎回样本、80 个收益池、4 个 Pendle 固定收益市场、1 个机制信号和 7 个研究候选。

最重要的发现不是“找到高收益”，而是修正了利润口径：Pendle USDe 表面年化差约 9.52%，如果错误地把年化当成一次收益，会显示约 95 USDC；按照实际剩余到期时间重算后，1,000 USDC 的估算收益只有约 0.90 USDC，因此应该降级为观察，不能执行。

当前最高研究信号是 Ethereum Fluid Lite 的 USDC，压力年化差约 3.04%，1,000 USDC 持有一年情景约 30.35 USDC。但这仍未逐项验证真实进出场深度、提款等待、锁定期和协议风险，所以只标记 research，不发送交易通知。

结论：套利监控必须严格区分年化收益、持有到期收益和单次交易利润。看到价差只是开始；只有实际可成交深度、完整费用、退出路径、时间成本与协议风险都通过核验，才可能成为可执行机会。整个系统继续保持只读，不连接钱包、不签名、不构造或广播交易。

笔记 130: Solana LP 机会分析与对比

作者：gm365 | 时间：2026-08-09 13:33 UTC | 字数：746 | 评分：77
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Solana LP 机会分析与对比

上个月分析过一次 Solana 链上 LP 机会，今天做了第二次对比分析，数据如下：

指标

 	7 月 16 日	8 月 9 日	变化
TVL	\$202M	\$231.25M	+14.5%
24h Volume	\$137M	\$98.63M	-28.0%
24h Fees	\$469K	\$805.83K	+71.8%
Fees / TVL	0.232%	0.348%	+50.1%

很明显，LP 收益在提升，机会更多了。

不过也发现一个问题，就是机会集中度在提升。

本次抓取并分析了 268 个 SOL-Meme DLMM 池：

≥\$5M 市值池贡献约 \$367K 费用，但中位 Fee/TVL 只有 0.043%。
\$1M-\$5M 池中位 Fee/TVL 为 0.492%，存在少量机会。
\$250K-\$1M 层的大多数池已停止持续产生费用。
<\$250K 层 29 个池合计只有约 \$406 费用，当前没有进入价值。
TOAD 一个 Token 占样本费用的 51.7%，前三个 Token 占 67.1%——收益仍然高度集中。

下一步行动：

Paper Trading，测试持续抓取并模拟买入参与 LP，评估收益。

笔记 131: Day 5 (8/9) 打卡

作者：littlefyer007-lgtm | 时间：2026-08-09 13:33 UTC | 字数：394 | 评分：59
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 (8/9) 打卡

今日完成

L0020 复盘方法论 + L0021 知识图谱 v2

- 21天课程全部完成
- 建立了复盘模板：预测vs实际，可重复vs偶然噪声
- 整理了知识图谱v2：7大板块覆盖全部所学内容

清算监控实战

- 用 Sphere (defi-sphere.com) 看了真实清算数据
- 历史总清算205K笔，抵押品7亿
- 学会了筛选条件：Ethereum网络，按金额排序，追踪链上交易
- 理解了清算利润来源：协议给清算人的折扣(1-5%)

日常检查

- DEX扫描：4条链正常，无套利机会
- 纸面交易：0笔触发，市场平静

当前能力

- 能用四问框架拆解交易
- 能用五步法判断信号
- 有完整的三层决策体系
- 有5个cron任务持续监控
- 能用Sphere监控清算数据

明天

资金费率套利入门

笔记 132: Day 5 : Robinhood Chain 套利机器人近期交易与盈利能力分析

作者：jinnzy | 时间：2026-08-09 13:43 UTC | 字数：7471 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 : Robinhood Chain 套利机器人近期交易与盈利能力分析

研究日期：2026-08-09

网络：Robinhood Chain，Chain ID 4663

性质：固定链上样本的只读分析，不是收益承诺或实盘建议

结论先行

这个机器人仍在高频运行，并且在本次固定样本内，链上可见的成功交易剩余足以覆盖全部成功与失败交易的 Gas。

但这不等于策略可以直接复制。更准确的判断是：

对“机器人是否仍活跃、策略是否真实成交”：Go；

对“继续构建只读扫描器、回测路径和失败率”：Watch / 值得继续；

对“现在照着地址和路径接私钥实盘”：No-Go。

本次观察 gas payer 0xD712...9B96 最近 100 笔交易。固定交易哈希后的时间窗口为 2026-08-09 20:11:43–21:27:23 (UTC+8)，约 75 分 40 秒：

| 指标 | 结果 |

|---|---|

| 总交易 | 100 |

| 成功 / 失败 | 33 / 67 |

| 成功率 | 33% |

| 成功交易推断毛剩余 | 0.001898529483400425 ETH |

| 100 笔总 Gas | 0.000774561120852 ETH |

| 其中失败交易 Gas | 0.000470002975864 ETH |

| 推断毛剩余减全部 Gas | 0.001123968362548425 ETH |

| 每次尝试的样本期望剩余 | 0.000011239683625484 ETH |

这里的“推断毛剩余”来自每笔成功交易末尾一致出现的 bot 事件与 WETH 剩余量，不能等同于已审计的运营净利润。节点、服务器、专线、开发、资金占用、私有排序或分成成本均不可从链上取得。

1. 地址关系与实现变化

| 角色 | 地址 | 本次可确认事实 |

|---|---|---|

| Gas payer / worker | 0xD71212080D90FFA50A80bF2aF2ADe6F99F29B96 | 本样本 100 笔全部由它发出并调用同一代理 |

| 执行代理 | 0x1521027B665FA38Fa4a642991607ad708376dD7b | Blockscout 标记为 ERC1967Proxy / Mev bot |

| 当前 implementation | 0x19c8634575Fc26839b510B786f967d3f78030891 | 2026-08-09 地址接口返回的当前实现 |

| 旧研究中的 implementation | 0x47ab0AbA8e791D6F96b6d7Af0C4FEA66099491AD | 2026-08-08

样本对应的旧观察，不能继续当成当前实现 |

| 历史利润收款地址 | 0x3CF17adD1De1cb251B236a4F3571C0D34e41338B | 本窗口部分交易仍向此地址结算 |

关键变化是代理已经指向新的 implementation。因此，只复刻昨天交易的 calldata 或旧实现行为，本身就不是完整策略。

此外，proxy 的近期调用来自多个 payer；本文只统计指定 0xD712... 的固定 100 笔，不能把 proxy 的全部业务量都算给这个地址。

2. 样本怎么固定、利润怎么定义

样本

Blockscout 地址交易接口每页返回 50 笔。本次保存第一页返回的完整游标，再原样请求第二页，形成固定 100 个交易哈希：

最早：区块 31925852，2026-08-09T12:11:43Z；

最新：区块 31971140，2026-08-09T13:27:23Z；

100 笔全部 from = 0xD712...9B96、to = 0x1521...dD7b。

地址仍在快速产生新交易，所以“最近 100 笔”会持续漂移；本文所有数字只对应上述固定哈希集合。

利润证据分层

每笔成功交易的日志都出现同一个代理事件 topic：

text

0x927ca72beeafa042127c9b97483d6b6f5ada2790237a7b3310232cab8888ac27

该事件的 indexed 参数包含 WETH 和收款地址，data 中的数值与末尾额外 WETH burn 数量一致。结合已能完整读取的内部调用，可解释为“偿还路径本金后剩余的 WETH 被 unwrap 并发给指定收款地址”。

证据分两层：

已匹配到账：17/33 笔可由 Blockscout 内部交易列表独立匹配收款地址与相同金额，合计 0.001357215663052400 ETH；

推断剩余：另外 16 笔事件与末尾 WETH burn 结构相同，但内部调用位于更深分页，本文没有把事件本身冒充为已独立回读到账。

33 笔事件金额合计为 0.001898529483400425 ETH。下文使用“推断毛剩余”，不使用“已实现净利润”这一更强表述。

3. 成功、失败和 Gas 成本

| 项目 | 交易数 | Gas 合计 | 单笔平均 Gas |

|---|---|---|---|

| 成功 | 33 | 0.000304558144988 ETH | 0.000009229034697 ETH |

| 失败 | 67 | 0.000470002975864 ETH | 0.000007014969789 ETH |

| 全部 | 100 | 0.000774561120852 ETH | 0.000007745611209 ETH |

失败交易占总次数 67%，并烧掉总 Gas 的 60.68%。这说明策略并不是“发现确定利润才上链”，而是接受大量 revert，以高频尝试换取少数成功成交。

对 33 笔成功交易：

毛剩余中位数：0.000032583565433526 ETH；

毛剩余均值：0.000057531196466680 ETH；

最小 / 最大：0.000012558385242532 / 0.000307810635345611 ETH；

毛剩余减该笔 Gas 后，33 笔均为正；最薄的一笔只剩 0.000001191636570532 ETH。

收益分布并不均匀：最大一笔贡献毛剩余的 16.21%，前两笔贡献 28.25%，前五笔贡献 48.73%。少数较大机会对整体结果很重要。

4. 路径是否重复

33 笔成功交易包含：

12 笔两段 swap；

21 笔三段 swap；

按 token 标签归并得到 25 种路径；

按具体 pool / v4 poolId 组合计算，33 笔签名全部不同。

重复最多的 token 级路径为：

| 路径 | 次数 |

|---|---|

| CASHCAT → USDG → WETH | 3 |

| PONS → USDG → WETH | 2 |

| PEPE → USDG → WETH | 2 |

| FRONG → USDG → WETH | 2 |

| 222 → USDG → WETH | 2 |

| CASHCAT → PONS → WETH | 2 |

| CASHCAT → WETH | 2 |

WETH 出现在全部 33 笔，USDG 出现在 17 笔；WETH/USDG v3 池 0x52e65B...71Ca 被 7 笔成功交易复用。其他高频中间资产包括 CASHCAT、PONS、222、FRONG 和 PEPE。

这不是只守着昨天的 ETH → 222 → USDG → WETH。实际机器人在短时间内轮换大量小币和池子，公共 hub 是 WETH / USDG，边缘机会来自不断变化的非 hub token。

5. 代表性成功交易

金额均为 ETH；“Gas 后”只扣该笔链上 Gas，不扣基础设施与分成。

```
| 时间 (UTC+8) | Tx | 路径 | 推断毛剩余 | Gas | Gas 后 |
|---|---|---|---|---|---|
| 20:18:41 | 0xa7da...22b5a9 | VIRTUAL → WETH | 0.000307811 | 0.000010484 | 0.000297326 |
| 20:28:36 | 0x1e20...c41819 | IF → USDG → WETH | 0.000228539 | 0.000009562 | 0.000218976 |
| 21:19:33 | 0x7053...047507 | USDG → WETH → WINK | 0.000141794 | 0.000011853 | 0.000129941 |
| 21:12:45 | 0x3ee3...e00f6 | CASHCAT → PONS → WETH | 0.000132301 | 0.000010742 | 0.000121559 |
| 21:22:50 | 0xa96d...ab98 | KERMIT → WETH | 0.000114787 | 0.000006961 | 0.000107825 |
| 21:12:42 | 0x5f28...e7ebf | 222 → USDG → WETH | 0.000072206 | 0.000009720 | 0.000062486 |
| 21:16:57 | 0x03bf...0c93 | THEO → USDG → WETH | 0.000066850 | 0.000009924 | 0.000056925 |
| 21:22:14 | 0x078f...a846 | CASHCAT → USDG → WETH | 0.000012558 | 0.000011367 | 0.000001192 |
```

最薄的一笔毛剩余只比 Gas 高约 10.5%。如果加上基础设施成本、延迟导致的额外失败或未识别分成，它未必仍有经济利润。

6. 观察期盈利能力

按本文能够从链上定义的最宽口径：

text

推断链上剩余 0.001898529483400425 ETH

- 33 笔成功交易 Gas 0.000304558144988000 ETH

- 67 笔失败交易 Gas 0.000470002975864000 ETH

= 样本链上剩余 0.001123968362548425 ETH

毛剩余约为全部 Gas 的 2.45 倍。就这个 76 分钟固定窗口而言，失败成本没有吃掉成功交易的全部链上剩余。

但不能从这个短窗口外推日收益、月收益或年化收益，原因包括：

1. 样本只覆盖一个 payer 的连续 100 笔，不覆盖整个 bot 集群；
2. 市场机会和竞争者会快速变化；
3. 无法从链上确认节点、服务器、数据、专线和开发成本；
4. proxy implementation 已发生变化，策略参数也可能继续升级；
5. 自定义事件代表合约声明的结算结果，不等同于审计后的运营账户利润。
7. 为什么“对方赚钱”不等于“我们能复制”

链上交易只能告诉我们哪些路径已经成交，不能告诉我们它们是如何在成交前被发现的。复制仍缺少：

完整 v3/v4 池索引与逐 tick 本地模拟；

最新状态到达速度和 sequencer 提交延迟；

自定义紧凑 calldata 编码；

多 payer / worker 调度、nonce 管理和失败预算；

implementation 中未公开的利润阈值、输入优化和防冲突逻辑；

能承受约 2/3 尝试失败的资金与运维纪律。

尤其值得注意的是，Blockscout 把这 100 笔 method 都显示成 0x00000000，但 raw input 并非空。它使用的是未公开 ABI 的紧凑 calldata，不能把浏览器无法解码误判成“没有方法参数”。

8. 对扫描器的直接启示

下一版只读 scanner 不应只复现单条 222 路径，而应：

1. 以 WETH、USDG 为 hub，持续发现所有活跃边缘 token；
2. 同时支持两段与三段 swap，优先做 poolId / pool address 级签名；
3. 把失败交易 Gas 纳入机会阈值，而不只是从成功交易扣 Gas；
4. 保存 timestamp、block、pool signature、input、gross surplus、gas、recipient；
5. 单独统计“候选数、发送数、成功数、失败 Gas、净剩余”，避免只展示赢家；
6. 对最薄机会设置比链上 Gas 更高的安全垫。

一个可用于 paper trading 的门槛应写成：

text

expectedSurplus

successGas

+ expectedFailureGasPerSuccess

+ latency/risk buffer

+ infrastructure allocation

本样本每 33 次成功对应 67 次失败，所以按样本平均，每次成功还应分摊约 0.0000142425 ETH 的失败 Gas，而不是只扣该笔交易自己的 Gas。

9. 今日判断

Watch。

证据已经足以确认：这是持续运行、路径多样、能够在短样本中覆盖失败 Gas 的真实策略，不是单笔偶然截图。

但复制执行仍是 No-Go。下一步最有价值的工作不是继续抄历史路径，而是把这 100 笔做成可重复的数据管道，并建立只读 paper-trading：实时产生候选路径，记录当时预测剩余，再和几秒后链上真实成交对比。只有证明“事前预测”而非“事后解释”成立，才值得讨论执行层。

数据边界

交易状态、Gas、区块、日志和内部转账：来自 Blockscout 一手链上 API。

token 路径标签：由成功交易的 Swap / Transfer 日志归并；token symbol 可被发行方设置，名称不代表资产质量。

0.001898529... ETH 是事件与 WETH 剩余结构推断的链上毛剩余，不包含不可得的运营成本。

已独立匹配的内部到账覆盖 17/33 笔；其余 16 笔保留“推断”标签。

本文没有接触私钥、没有发送交易、没有提交共享打卡。

一手来源

Gas	payer	最近交易	API
(https://robinhoodchain.blockscout.com/api/v2/addresses/0xD71212080D90FFA50A80bf2aF2ADe6F99F29B96/transactions)			
Gas	payer	累计计数	API
(https://robinhoodchain.blockscout.com/api/v2/addresses/0xD71212080D90FFA50A80bf2aF2ADe6F99F29B96/counters)			
执行代理地址			API
(https://robinhoodchain.blockscout.com/api/v2/addresses/0x1521027B665FA38Fa4a642991607ad708376dD7b)			
历史利润地址内部交易 API (https://robinhoodchain.blockscout.com/api/v2/addresses/0x3CF17adD1De1cb251B236a4F3571C0D34e41338B/internal-transactions)			
代表交易：VIRTUAL/WETH (https://robinhoodchain.blockscout.com/api/v2/transactions/0xa7da6c9011f201750315b1263c280cdcdcf7b81ad2a9cf06c8035282673225a9)			
代表交易：CASHCAT/USDG/WETH (https://robinhoodchain.blockscout.com/api/v2/transactions/0x078fda39ae26ce3f45a701de25188ed4babb8810515e4729221ae6a6cbaca846)			
代表交易日志 (https://robinhoodchain.blockscout.com/api/v2/transactions/0x4aea59a0a4b934bf41fc0de877960987c0998f5a42f31b965e2d928bffe19a86/logs)			
代表交易 raw trace (https://robinhoodchain.blockscout.com/api/v2/transactions/0x4aea59a0a4b934bf41fc0de877960987c0998f5a42f31b965e2d928bffe19a86/raw-trace)			

09 风险与安全（5 篇）

笔记 133: 使用 Codex 开发实盘交易系统的阶段性学习笔记

作者：wy2221 | 时间：2026-08-09 07:10 UTC | 字数：2421 | 评分：71
来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

使用 Codex 开发实盘交易系统的阶段性学习笔记

一、从“写代码”转向“管理系统”

这两天最大的收获，不是完成某个功能，而是认识到：一个实盘交易系统真正困难的部分，并不是代码能否运行，而是如何保证系统长期稳定、可验证、可部署。

早期开发容易陷入一个误区：

发现问题 → 修改代码 → 测试通过 → 上线。

但实盘系统不是普通软件。交易程序涉及资金、订单、状态同步、数据库、部署环境，因此任何一次修改都可能影响其他环节。

随着项目推进，重点应该从“增加功能”转变为：

- 明确系统边界；
- 固定稳定版本；
- 建立验证流程；
- 降低人为错误。

因此，后期开发的核心不是让代码越来越复杂，而是让系统越来越可控。

二、功能开发完成后，需要进入发布工程阶段

A26、A27阶段的重要经验是：

功能完成 $\neq$ 可以上线。

当前项目虽然已经完成核心功能冻结，但仍需要经过发布封装、验证、部署等阶段。审计结果认为，A26+A27属于“功能候选”，但还不是完整生产发布包，因此保持 HOLD 状态。

这说明一个成熟交易系统需要至少分成几个阶段：

功能开发

↓

代码冻结

↓

发布封装

↓

独立验证

↓

部署

↓

观察运行

↓

正式启用

不能把“代码可以运行”直接等同于“资金可以进入”。

三、为什么需要多个 Codex 对话，而不是一个万能窗口

项目发展到一定阶段，一个窗口承担所有任务会产生混乱。

不同角色应该分离：

1. 开发窗口

负责：

策略逻辑；

订单执行；

风控；

状态机。

2. 发布封装窗口

负责：

版本管理；

wheel构建；

ZIP发布包；

verifier；

installer。

不应该修改交易核心代码。

3. 部署运行窗口

负责：

VPS检查；

FinalShell操作；

服务启动；

日志分析；

数据回收。

不应该重新设计架构。

这种分离，本质是降低认知负担，避免一个 AI 同时扮演架构师、程序员、运维人员，导致权限和职责混乱。

四、简单架构比复杂架构更适合个人交易系统

在项目迭代过程中，一个重要原则逐渐形成：

交易系统不是功能越多越好，而是错误越少越好。

过去容易出现的问题：

状态重复保存；
多个模块同时判断交易状态；
发布流程和代码修改混在一起；
部署阶段直接修代码。

解决方向不是继续增加模块，而是减少不必要复杂度。

例如：

一个明确的数据来源；
一个交易状态；
一个恢复流程；
一个发布身份。

简单结构更容易：

审计；
排查问题；
回滚；
长时间运行。

五、发布流程中的核心思想：禁止“现场修复”

历史事故带来的最大经验：

生产环境不是开发环境。

出现问题时：

错误方式：

VPS发现问题
↓
直接修改Python
↓
重启服务
↓
继续交易

正确方式：

发现问题
↓
保存日志和证据
↓
回到开发环境分析
↓
修改代码
↓
重新测试
↓
重新发布

原因是：

现场修改会破坏版本一致性，之后无法判断：

当前运行的是哪个版本；
修改是否影响其他模块；
问题是否真正解决。

因此部署系统必须强调：

“发现问题，回开发环境解决。”

六、验证体系比代码本身更重要

成熟项目需要证明：

“这个版本确实是这个版本。”

因此需要：

固定版本号；
release identity；
manifest；
SHA-256；
clean extraction验证；
verifier自包含。

这些不是形式主义，而是在解决一个核心问题：

如何确认部署的东西，和测试过的东西，是同一个东西？

尤其交易系统涉及资金时，不能依靠记忆或者文件名称判断。

七、AI辅助开发最大的价值：建立检查机制

使用 Codex 后最大的变化，不应该只是提高写代码速度，而应该提高项目管理能力。

好的使用方式：

让 AI 帮助：

梳理架构；
检查遗漏；
生成测试方案；
审核风险；
记录历史。

而不是：

“告诉 AI 修改哪里，让它直接完成。”

因为 AI 可以快速生成代码，但它并不会天然理解你的资金安全目标。

因此需要人为设定：

修改范围；
禁止范围；
验收标准。

八、当前阶段最大的经验总结

经过 A26、A27 的整理，可以总结：

1. 不要急着上线

上线速度不是第一目标。

第一目标：

确认系统不会因为一个小错误影响资金。

2. 发布必须独立

代码、发布包、部署流程必须分开。

3. 每一次变化必须留下证据

包括：

修改内容；
测试结果；
版本信息；
哈希；
部署记录。

4. AI需要管理，而不是依赖

Codex是强大的工程助手，但需要明确职责边界。

九、下一阶段思考方向

未来继续开发类似系统时，可以遵循：

策略想法

↓

小规模验证

↓

模块化实现

↓

测试

↓

版本冻结

↓

发布封装

↓

模拟运行

↓

小资金实盘

↓

扩大规模

不要从“能不能赚钱”开始，而应该从：

“系统是否能够长期稳定运行”

开始。

对于量化交易而言，稳定性本身就是竞争力。

最终目标不是打造一个复杂机器人，而是打造一个：

可理解；

可验证；

可恢复；

可长期运行；

的交易系统。

笔记 134: ERC-20、Decimals、Allowance、Permit2 与授权风险

作者：Montaire | 时间：2026-08-09 09:25 UTC | 字数：303 | 评分：51

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

ERC-20、Decimals、Allowance、Permit2 与授权风险

实际数量等于原始整数除以 10^{decimals} ，合约地址而不是 symbol 才是识别代币的核心依据。

approve 只设置 spender 可通过 transferFrom 动用的额度，不会立即转账。

精确授权限制风险范围；无限授权减少重复操作，但会扩大长期暴露。

Permit2 有两道门：代币先授权 Permit2，签名再指定 token、spender、amount 和期限。

fee-on-transfer、rebasing、冻结与黑名单会破坏“标准转账一定按预期执行”的假设。

笔记 135: Day 5 学习笔记

作者：Azhan | 时间：2026-08-09 09:31 UTC | 字数：3459 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 学习笔记

今天学了什么

1. 以太坊区块构建两篇（Greenfield Capital 研究）

之前学的 MEV 都是概念，这两篇把“区块怎么造出来的”讲透了。

第一篇《区块构建如何运作，为什么重要》：

以太坊的区块空间有两个市场——“进块市场”（交基础费，人人平等）和“位置市场”（交优先费，买执行顺序）。套利者愿意为“先执行”付几乎全部利润，这就是 MEV 的根源。整条链路分 4 层：订单流（交易从哪来）→ 路由（谁看得到）→ 构建（Builder 组装最值钱的区块）→ 确认（验证者签收）。Builder 赚的是“区块实际价值 付给验证者的竞价”的差价，所以独家订单流是 Builder 的命根子——这也解释了为什么头部 Builder 越来越强。

第二篇《需求侧：订单流与优先费格局》：

用 2024-2026 真实数据回答“谁在为优先付钱”。结论：2024 年主导优先费的是 Telegram 交易机器人（散户抢新币），现在变成 MEV 相关交易流占 66-80%。三大 MEV 市场格局完全不同：统计套利被 Wintermute 垄断、三明治被 jaredfromsubway 垄断（都在被新玩家挑战），原子套利竞争最分散。还有个有意思的数据：USDC/USDT 转账这种不需要排队的操作占了优先费支出的 24%——钱包默认小费太高，散户在“被动付费”。

2. jaredfromsubway 被蜜罐反杀（贼好玩）

就是第二篇文章里那个三明治巨头，被黑客用几十个假代币合约反杀了，损失约 750 万美元（设计者声称近 1500 万）。它靠“快”吃饭，攻击者就把“快”变成弱点——它越快冲进假合约，死得越快。历史上 2023 年也有恶意验证者用类似逻辑从三明治机器人手里抽走 2500 万。最讽刺的是设计者开出 100 万赏金求归还，但历史上能要回钱概率不高。

这故事完美说明：MEV 是黑暗森林，猎杀别人的人也会被猎杀。授权（approve）是机器人钱包的最大安全隐患。

3. 老孙的套利监控模块设计

实战老鸟（在 Bybit 被野庄爆仓 100 万 U、Bitget 又赔一半）的监控设计。最核心的观点：现在韭菜少了，野庄都盯着套利者在收割，全自动策略 100% 会踩坑。他的模块功能：正/负费率监控、美股正费率、RWA 标的、币圈升水、OI 异动提醒、自定义监控，覆盖 8 家交易所，接口统一用 CCXT。

最有价值的细节是 OI 异动——“价格没变但 OI 突然放大是最明显的庄家入场特征，OI 无法造假”。他踩过的坑：在 Bitget 开仓前看 OI 有 4000 万，1 分钟内被 ADL，95% 仓位都是野庄对开骗他的。

4. Astro 项目（只看思路）

一个开源的多所资金费率套利系统，支持 11 家 CEX。设计思路值得参考：跨所布局（多家所才有费率差可比）、统一账户+双向持仓（费率套利的标准配置）、API 最小权限（只开交易不开提现+IP 白名单）。但代码不透明（README 是空的，文档全在 Notion），一键安装脚本+要填 API key 是高风险操作，不建议碰。

5. hunterweb303 的 Robinhood 套利全景

他扫了 259 万个区块、1327 万笔 Swap，找出 122 个候选套利系统做审计。把 Robinhood 套利分成 4 套：首笔价差（最厚但 98.7% 机会 2 个区块内结束，纯拼速度）、条件执行（有空间但路线一换收益就掉）、确认后尾气（持续残差全是零，基本不存在）、覆盖空白（账面好看但没扣失败成本）。他的洞察很值钱：机器人的收入写在它覆盖的池子和路线上，不写在代码里。算账公式也和我们学的一致：真实利润 = 毛价差 Gas 滑点 库存损耗 延迟 调仓成本。

评论区有人质疑“纯 AI 文，能跑通吗”——数字确实无法独立验证（地址没公开），但方法论和框架是有价值的。

6. 群热点

今天全群的主题是“套利有风险”：老孙爆仓教训、群友说“很多套利大佬套到最后爆仓了，黑天鹅无处不在”、jaredfromsubway 被反杀——都在提醒套利不是无风险印钞机。实操派在搞全池扫描器（roy328line）、群日报（FullyJoe 用微信截图+OCR）、监控流程检查（xiaaquanyo）。

做了什么

读完了 Greenfield 两篇区块构建研究（原文）

分析了 jaredfromsubway 蜜罐反杀事件

研究了老孙的套利监控设计 + Astro 项目思路

看了 hunterweb303 的 Robinhood 套利分析

拉了全群最新打卡看热点

动手建了费率套利数据系统（重点，见下）

动手实践：费率套利数据系统

今天决定“光分析不够，要动手”，搭了一套只读的数据系统（不碰钱）：

工具链

- scripts/funding_scan.py：三所（OKX/Gate/Bitget）费率扫描 + 流动性过滤 + 历史存档
- scripts/paper_funding.py：模拟盘——记录候选快照，过几天重放算累计费率收益
- scripts/analyze_candidates.py：深度分析——OI 持仓 + 历史费率 + 现货深度
- cron 每 4 小时自动跑全流程，数据自己积累

踩的坑（全记在脚本注释里）

1. 币安/Bybit 地区封锁（VPS 在美国/受限区，451/403）→ 换 OKX/Gate/Bitget
2. ccxt 批量费率接口有 bug：BTC 显示 +11% 实际 +0.5%，INJ 显示正实际负 → 候选必须逐个单独验证
3. fetch_tickers 对永续返回空 → 用原生 API 拿成交量
4. 各所字段名不同（vol24h / volume_24h_quote / quoteVolume）→ 逐个适配

真实数据发现

- 主流币费率全部 ~+11% 年化——牛市常态，不是机会
- 高费率（+1000% 以上）全在小币/新币（TUT、龙虾、BICO）——陷阱区，野庄钓鱼
- 最值钱的发现：OI 和费率结合看——费率最高的 RLS（+245%）OI 只有 40 万 U，典型的“庄家钓鱼”配置；LAB 是老孙点名的强庄币（OI 670 万）。老孙的“OI 防庄家”教训直接应验

- 所有候选币历史费率 30/30 期全正——当前市场普遍看多

当前判断

- 候选里相对可观察：FARTCOIN、RIVER、RAVE（OI 中等 + 费率稳定）

- 排除：RLS（OI 40万钓鱼嫌疑）、LAB（强庄币）、EDEN（OI 太小）

- 但这些都是 meme/小币，野庄风险依然在——真正的安全边际（OI>5000万+主流认知）又回到主流币费率只有 11% 的现实
得到什么结果

把 MEV/区块构建从概念升级到“谁在付钱、谁在赚钱”的真实图景

反复验证了同一个成本公式（Bruce 教的 = 老孙用的 = hunterweb303 强调的）

理解了“路线决定收益”——套利机器人的价值在覆盖范围不在代码

今天所有案例都在强化一个认知：套利有风险，散户是被猎杀的对象

建成了费率套利数据系统，开始积累真实市场数据

遇到什么失败（教训）

X 的访问限制越来越严（搜索要登录、文章页报错），只能退而求其次读原文或找替代来源

群友发的 Astro 项目一看就是高风险（一键脚本+API key），没有运行，只看思路

数据系统踩了 4 个坑（地区封锁、接口 bug、字段适配），每个都靠实测发现——再次证明“数据要过闸门，不能直接信接口”

下一步验证什么

明天检查 cron 是否真的在积累数据（第一次运行 14:45）

修 Gate/Bitget 的 OI + 现货深度（目前只覆盖 OKX）

攒 3-5 天模拟盘数据，跑第一份 --report 看模拟收益

把老孙的 OI 异动逻辑用起来（以后分析山寨币先看 OI）

笔记 136: 今天學了什麼

作者：darven | 时间：2026-08-09 13:35 UTC | 字数：572 | 评分：67

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天學了什麼

Enosys Loans = Liquity V2 friendly fork，贖回核心介面是 CollateralRegistry.redeemCollateral

實務修正：贖回一次會拿到 4 個池的抵押品混合（FXRP/WFLR/stXRP/sFLR），不是單一池 — 白皮書有寫，但沒落實到成本面
專業 bot 用私人合約做原子執行（redeem+swap 一步）

做了什麼

建 redemption/ 模組 + demo_redemption.mjs（1U 預設）

解剖真實 bot 交易 0xff5eaa02，用 receipt logs 核實特徵

修 swap.js 路徑 bug + 寫交接文件

真實鏈上執行贖回

結果

功能跑通：approve + redeem 都成功（status 0x1）

收到 0.7307 FXRP + 1.0037 WFLR + 0.0302 stXRP + 微量 sFLR

發現 collateral mix 讓完整循環成本從 2 次 swap 變 3-4 次 — 小額循環結構性吃虧

下一步

決定循環設計：接受 4 池 swap vs 只換回主力池

等 swap agent 修好 quote 後組完整循環

笔记 137: 补充基础知识

作者：Galaxy9 | 时间：2026-08-09 14:21 UTC | 字数：1810 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

补充基础知识

概率、统计与认知校正

1 复利与几何平均

财富按乘法而非加法累积。对于连续收益，几何平均更能代表长期结果。

$CAGR = (\text{期末价值} / \text{期初价值})^{(1 / \text{年数})} - 1$

同样的算术平均收益，波动更大的一组路径往往得到更低的几何回报。损失恢复具有非对称性：下跌 50% 后需要上涨 100% 才能回本。减少大额永久损失，比追逐单期高收益更重要。

2 路径依赖与序列风险

在没有现金流的情况下，收益顺序不改变终值；一旦持续投入、提取、使用杠杆或存在保证金约束，顺序就决定结果。积累阶段的早期下跌可能改善长期买入成本，退休或提款阶段的早期下跌则可能造成不可逆伤害。

这解释了为什么同一平均收益率并不代表同一退休安全性，也说明仓位与流动性管理不能只看年化回报。

3 统计常识

P 值不是原假设为真的概率，也不是效应大小；

统计显著不等于经济上重要；

应同时报告效应量、置信区间、样本量和样本外表现；

多重试验会制造“看起来有效”的规则，必须避免在大量指标中挑出历史最佳者；

小样本连胜或连亏都不足以证明策略有效或失效；

相关不等于因果，尤其在宏观与市场叙事中。

4 常见认知偏差与对策

| 偏差 | 在投资中的表现 | 强制对策 |

| :--- | :----- | :----- |

| 确认偏误 | 只搜利好、忽略反证 | 每份研究卡必须写替代解释和反证 |

| 锚定效应 | 以买入价、历史高点作为价值依据 | 重做现金流和估值，不看成本价 |

| 损失厌恶 | 过早卖赢家、拖延卖输家 | 事先定义退出规则并记录执行偏差 |

| 可得性偏差 | 用最近热点或极端案例估计概率 | 查基准率、完整样本和长期数据 |

| 赌徒谬误 | 认为连续上涨后必跌、连续下跌后必涨 | 区分独立事件与有机制的趋势 |

| 幸存者偏差 | 只研究成功者 | 把退市、失败策略和未幸存样本纳入研究 |

| 叙事谬误 | 用故事解释每一次涨跌 | 写可证伪变量和事前预测 |

5 元认知与情绪管理

情绪不是要消灭的对象，而是风险状态信号。出现以下情形时，停止交易并恢复到记录和复盘：急于回本、害怕错过、连续查看价格、临时扩大仓位、用结果判断规则质量、把市场当作对个人的评价。

长期竞争优势通常来自较少犯大错、在不定时承认不知道、用流程替代冲动，而不是更强烈的观点。

6 学习、职业与长期决策

投资能力依赖于持续学习、职业现金流和注意力配置。原始笔记中与投资不直接相关但仍有长期价值的内容，统一放在本节。

学习系统

元认知阅读：带着问题阅读，标注证据与反证；每章用自己的话总结；将结论写成可检验命题；间隔一段时间后复述和修订。

专家能力：训练识别模式、异常、整体机制、边界条件和自身盲点，而不是累积更多名词。

科学方法：怀疑既有结论，观察异常，提出可证伪假设，寻找硬证据，并区分相关与因果。

结构化思考：用 MECE、逻辑树和“问题—假设—证据—结论”拆解复杂问题；结构服务于事实，不用框架替代研究。

职业与人生决策

职业是长期现金流和能力复利的重要来源。可用“花瓣图”定期审视六个维度：地点、知识领域、组织类型、服务对象或目标、工作条件和薪酬；再以兴趣、可转移技能、价值观和环境偏好交叉验证。

职业选择前可做信息访谈：询问实际工作内容、能力门槛、成长路径、失败成本、收入波动、行业周期和可迁移性。不要仅根据头衔、短期薪资或少数成功案例决策。

历史与社会信息素养

历史信息至少经历“发生、记录、传播、被接受”四个层次。政治立场、叙事者选择、传播机制和幸存材料都会改变我们看到的历史。因此，宏观叙事、行业故事和市场新闻都应追问：谁在说、基于什么证据、遗漏了谁、是否有独立来源。

本节不是用历史或心理学预测市场，而是训练在复杂环境中识别叙事、激励与证据边界的能力。